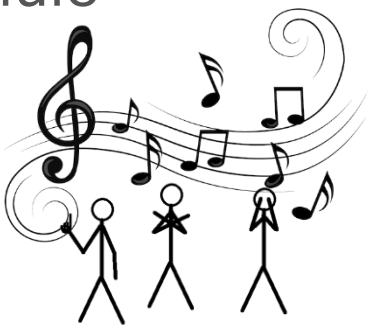Berliner Hochschule für Technik



Thema:

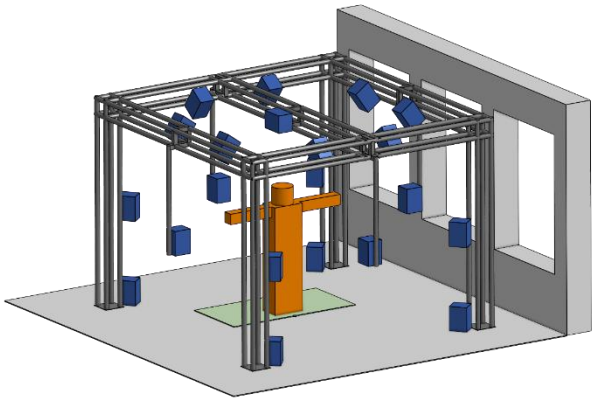
# Kamerabasierte Gestenerkennung zur Steuerung eines virtuellen Orchesters

**Masterarbeit**

Fachbereich VII – Elektrotechnik - Mechatronik - Optometrie
Studiengang: Information and Communications Engineering (M. Eng.)

Betreuer: Prof. Dr.-Ing. Marcus Purat
Zweitprüfer: Prof. Dr.-Ing. André Jakob

vorgelegt von: Jonas Wegener
8 Fachsemester
Matrikelnummer: 946481
13187 Berlin
+49 1578 5273 814
derJonas@Mailbox.org



abgegeben am: 21. Oktober 2026

Projektlink: https://github.com/Eulelarve/Masterarbeit.git

# Inhalt

Formelverzeichnis ....4
Abbildungsverzeichnis ....5
Abkürzungsverzeichnis ....7
1 Einleitung ....11
2 Grundlagen ....12
2.1 Robot Operating System ....12
2.2 User Datagram Protocol ....12
2.3 JavaScript Object Notation ....13
2.4 MediaPipe ....14
2.4.1 MediaPipe Framework ....14
2.4.2 MediaPipe Pose ....16
2.4.3 MediaPipe Hands ....18
2.5 OpenPose ....19
2.6 OpenCV ....20
2.7 Morphologische Transformationen ....21
2.8 Background Subtraction ....26
*2.9* Suzuki Algorithmus ....26
2.10 Farbmodelle zur Hautsegmentierung ....27
2.11 ABI-Kompatibilität ....29
2.12 Intel RealSense Tiefenkamera Treiber ....30
2.13 Convolutional Neural Networks ....31
2.14 Transformer ....33
2.15 Menschliche Auditive Richtungswahrnehmung ....33
2.16 Sichtfeld Berechnungen ....34
2.17 Weitere Mathematische Grundlagen für diese Arbeit ....35
3 Stand der Technik ....37
*3.1 Handgestenerkennung* ....37
*3.2 Visuellen Handgestenerkennung* ....39
*3.3 Deep-Learning-basierte Erkennung dynamischer Gesten* ....40
*3.4 Gestensteuerung in der Robotik* ....41
3.5 Pose-Schätzung ....42
3.6 Klassifikation einer Greifgeste ....43
3.7 Gestensteuerung von Multimediasystemen ....44
3.8 Gestensteuerungen mit MediaPipe ....44
3.9 Gestenerkennung ohne KI ....48
3.10 Gestenerkennung mit Tiefenkamera ....50
3.11 Haltungsanalyse mit OpenPose ....51

3.12 Gestensteuerungen mit Nvidia JetPack ..... 52
3.13 Relevanz für diese Arbeit ..... 53
4 Konzeption der Gestenerkennung ..... 56
4.1 Raumsimulation, Kameraposition und -FOV ..... 56
4.2 Anforderungen an die Projektkomponenten ..... 59
4.3 Kernaufgaben der Gestensteuerung ..... 60
4.3.1 Benutzeroberfläche ..... 60
4.3.2 Positionierung eines Instruments ..... 61
4.3.3 Auswählen eines Instruments ..... 62
4.3.4 Gesten für weitere Funktionalitäten ..... 63
4.3.5 Systemkommunikation ..... 63
4.4 Wahl der Software ..... 64
4.5 Konzeptspezifikation mit KI ..... 65
4.6 Konzeptspezifikation ohne KI ..... 67
5 Implementierung der Gestensteuerung ..... 71
5.1 Wahl der Hardware ..... 71
5.1.1 Kamerawahl ..... 71
5.1.2 Wahl der Recheneinheit ..... 72
5.1.3 Bildschirmwahl ..... 74
5.2 Konzeptwahl und Einstieg ..... 75
5.3 Erkennung der Raumrichtung ..... 75
5.3.1 Hand- und Schulterposition ..... 75
5.3.2 3D-Raum-Koordinaten ..... 76
5.3.3 Zeigerichtung des Arms ..... 77
5.3.4 Raumrichtung ..... 77
5.3.5 Raytracing zur Raumgrenze ..... 78
5.3.6 Sonderfall ..... 78
5.4 Erkennung der Hand ..... 79
5.5 Greifgestenerkennung ..... 80
5.6 Steuergesten ..... 83
5.7 Grafische Benutzeroberfläche ..... 86
5.8 Systemkommunikation ..... 90
6 Test der Gestensteuerung ..... 92
6.1 Testprozedur ..... 92
6.2 Resultate aus dem Test ..... 94
6.3 Programmoptimierungen ..... 96

7 Ergebnisse.....99
8 Resümee.....102
Literatur und Quellen.....104
Glossar……..…….112
Anhang…… .....115

## Formelverzeichnis

Formel 1 - YUV-Transformation [41] ..... 27
Formel 2 - YCbCr-Transformation [41] ..... 28
Formel 3 - H Lab-Transformation [41] ..... 28
Formel 4 - XYZ-Transformation [41] ..... 28
Formel 5 - Hautmodel aus [41] ..... 28
Formel 6 - FOV Umrechnung von Winkel zu Länge [62] ..... 34
Formel 7 - dFOV Winkel aus vFOV und hFOV ..... 35
Formel 8 - vFOV, hFOV Winkel und Seitenverhältnisse ..... 35
Formel 9 - Kamerakoordinaten zu Raumkoordinaten ..... 35
Formel 10 - Winkelberechnung mit Arcustangenz-2 [65] ..... 36
Formel 11 - Winkelberechnung mit Arcuscosinus [66] ..... 36
Formel 12 - Richtungsanteile von Azimut und Elevation im Raumkoordinatensystem .78

## Abbildungsverzeichnis

Figure 1 - Calculator-Graph einer Mediapipe Objekterkennung [8] ..... 15
Figure 2 - Pose-Estimation mit BlazePose (MediaPipe) [14] ..... 17
Figure 3 – MediaPipe BlazePose Körperlandmarks [15] ..... 18
Figure 4 - Handerkennung mit MediaPipe [16] ..... 19
Figure 5 - MediaPipe Handlandmarks [8] ..... 19
Figure 6 - Bilder vor und nach Erosion [32], [33] ..... 22
Figure 7 - Bilder vor und nach Dilatation[32], [33] ..... 23
Figure 8 - Bilder vor und nach Opening [32], [33] ..... 23
Figure 9 - Bilder vor und nach Closing [32], [33] ..... 24
Figure 10 - Einfluss von Erosion, Dilatation, Opening und Closing [32] ..... 24
Figure 11 - Bilder vor und nach Morphologischer Gradient [32], [33] ..... 24
Figure 12 - Bilder vor und nach Top Hat [32], [33] ..... 25
Figure 13 - Bilder vor und nach Black Hat[33] ..... 25
Figure 14 - RGB- und Tiefenbild mit gleicher Positionsreferenz vor und nach der Kalibrierung (Alignment) ..... 30
Figure 15 - Bilder der Intel RealSense D455 Tiefenkamera mit Rauschen im Tiefenbild ..... 31
Figure 16 - menschliche akustische Richtungsauflösung (MAA) in Abhängigkeit der Ambisonics-Ordnung [59] ..... 34
Figure 17 - Robotersteuerung mit Raumkarte und Zeigerichtung (grün), Bewegungszielen (hellblau), Bewegungsroute (pink), Hindernisse (rot) [73] ..... 42
Figure 18 - Landmarks des Motion-Capture-Systems [75] ..... 44
Figure 19 - *Vision Robotic Arm Gesture Recognition* [66] ..... 45
Figure 20 - *GestureBot Vision System* [78] ..... 46
Figure 21 - *Control PIPER Arm with Hand Gestures* [79] ..... 47
Figure 22 - *Mimic Robot* [83] ..... 49
Figure 23 – *Vision-based arm gesture recognition* [86] ..... 50
Figure 24 - Gesture Controlled Robotic Arm [88] ..... 51
Figure 25 - *Sports2D* [89] ..... 52
Figure 26 - *Gestensteuerung eines Roboters auf dem Nvidia Jetson Nano* [92] ..... 53
Figure 27 – Handöffnungsgrad aus [66] ..... 55
Figure 28 – Modell Audiosystem leer und mir 1.80m Mensch und Standzone ..... 57
Figure 29 - Modell Audiosystem mit Aktionsbereich ..... 57
Figure 30 - Modell Audiosystem Armbereiche 207 cm und 140 cm im Sichtfeld 80° x 50° und 30° Kameraneigung ..... 58

Figure 31 - Modell Audiosystem Frontalansicht mit 170 cm, 0° Kamerazeigung und 180 cm Mensch ..... 59
Figure 32 - Azimut- und Elevations-Level im Raum ..... 61
Figure 33 - Handzustände offen und geschlossen ..... 63
Figure 34 – Schemata vom Multithreading im Projekt [83] ..... 70
Figure 35 - Intel RealSense Tiefenkamera D455 [100] ..... 72
Figure 36 - Minisforum UM760 Slim [107] ..... 74
Figure 37 - MediaPipe Pose und Hand Erkennung mit ROIs als grünes Quadrate ..... 80
Figure 38 - Handstatus mit statischer Methode ..... 81
Figure 39 - Handstatus mit dynamischer Methode ..... 82
Figure 40 – Infogeste als Schaubild und in echt ..... 84
Figure 41 - Reset-Geste als Schaubild und in echt ..... 85
Figure 42 - Ansicht-ändern-Geste als Schaubild und in echt ..... 85
Figure 43 - Beenden-Geste als Schaubild und in echt ..... 86
Figure 44 – GUI: Verschieben eines Instruments mit jeder Hand, im 3D-Audioraum ..... 87
Figure 45 – GUI: Lautstärke ändern, im 3D-Audioraum ..... 88
Figure 46 – GUI: Instrumentenmenü, im 3D-Audioraum ..... 89
Figure 47 – GUI: Informationsfenster, im 3D-Audioraum ..... 89
Figure 48 - 3D-Audioraum Kameraposition und -kalibrierung ..... 92
Figure 49 - 3D-Audioraum Bodenmarkierungen für Systemmitte (schwarz) und 15° Schritte ..... 93

## Abkürzungsverzeichnis

| Abkürzung | Bedeutung |
|---|---|
| ABI | Application Binary Interface; binäre Schnittstelle zwischen Softwarekomponenten |
| AI | Artificial Intelligence; englische Bezeichnung für künstliche Intelligenz |
| BGR | Blue, Green, Red; Farbkanalreihenfolge |
| BHT | Berliner Hochschule für Technik |
| CNN | Convolutional Neural Network; faltendes neuronales Netz |
| CPU | Central Processing Unit; Hauptprozessor |
| CUDA | Compute Unified Device Architecture; NVIDIA-Plattform zur parallelen Berechnung |
| DDS | Data Distribution Service; Middleware-Standard für verteilte Kommunikation |
| dFOV | Diagonal Field of View; diagonales Sichtfeld |
| DP | DisplayPort; digitale Bildschnittstelle |
| FPS | Frames per Second; Bilder pro Sekunde |
| FOV | Field of View; Sichtfeld einer Kamera |
| GPU | Graphics Processing Unit; Grafikprozessor |
| GUI | Graphical User Interface; grafische Benutzeroberfläche |

| Abkürzung | Bedeutung |
|---|---|
| HD | High Definition; hochauflösendes Bildformat mit einer Standardauflösung von 1280 × 720 Pixeln |
| HDMI | High-Definition Multimedia Interface; digitale Audio-/Videoschnittstelle |
| hFOV | Horizontal Field of View; horizontales Sichtfeld |
| HSV | Hue, Saturation, Value; Farbmodell aus Farbton, Sättigung und Helligkeitswert |
| IP | Internet Protocol; Protokoll zur Adressierung und Übertragung von Datenpaketen |
| IPM | Inverse Perspective Mapping; perspektivische Bildtransformation |
| JSON | JavaScript Object Notation; textbasiertes Datenformat |
| KI | Künstliche Intelligenz |
| LAN | Local Area Network; lokales Rechnernetz |
| LSTM | Long Short-Term Memory; Architektur rekurrenter neuronaler Netze |
| MAA | Minimum Audible Angle; kleinster wahrnehmbarer Winkelunterschied einer Schallquelle |
| ML | Machine Learning; maschinelles Lernen |
| NPU | Neural Processing Unit; Prozessor zur Beschleunigung neuronaler Netze |
| PC | Personal Computer |
| RAM | Random Access Memory; Arbeitsspeicher |

| Abkürzung | Bedeutung |
|---|---|
| RGB | Red, Green, Blue; additives Farbmodell |
| RGB-D | RGB plus Depth; Farb- und Tiefendaten |
| ROI | Region of Interest; relevanter Bildausschnitt |
| ROS | Robot Operating System; Software-Framework für Robotikanwendungen |
| RTPS | Real-Time Publish-Subscribe; Kommunikationsprotokoll für verteilte Systeme |
| SDK | Software Development Kit; Softwareentwicklungspaket |
| SSD | Solid-State Drive; nichtflüchtiger Massenspeicher |
| TCP | Transmission Control Protocol; verbindungsorientiertes Transportprotokoll |
| TSM | Temporal Shift Module; Verfahren zur Einbeziehung zeitlicher Information in 2D-CNNs |
| UDP | User Datagram Protocol; verbindungsloses Transportprotokoll |
| USB | Universal Serial Bus; Schnittstelle für Datenübertragung und Stromversorgung |
| USB-C | USB-Steckverbinderstandard Typ C |
| VGA | Video Graphics Array; Bildstandard mit einer Auflösung von 640 × 480 Pixeln |
| vFOV | Vertical Field of View; vertikales Sichtfeld |
| WLAN | Wireless Local Area Network; drahtloses lokales Netzwerk |

| Abkürzung | Bedeutung |
|---|---|
| YUV | Farbmodell mit Helligkeits- und Farbdifferenzkomponenten |
| YCbCr | Farbmodell mit Luma- und zwei Chrominanzkomponenten |
| XYZ | CIE-XYZ-Farbraum; standardisierter dreidimensionaler Farbraum |

# 1 Einleitung

Gestensteuerung ist eine Form der Mensch-Computer-Interaktion, bei der Eingaben nicht über klassische Bedienelemente, sondern durch Bewegungen oder Haltungen des Körpers erfolgen. Kamerabasierte Verfahren ermöglichen es hierbei, solche Gesten berührungslos zu erfassen und für die Steuerung digitaler Systeme zu nutzen. Sie kommen unter anderem in der Robotik, in interaktiven Multimediaanwendungen und in immersiven Umgebungen zum Einsatz. Je nach Aufgabe werden einzelne Hand- oder Körperstellungen klassifiziert, Körperhaltungen geschätzt oder menschliche Bewegungen über mehrere Bilder hinweg ausgewertet.

Während klassische Bildverarbeitung Merkmale wie Farben, Konturen und geometrische Beziehungen rein regelbasiert auswertet, verwenden KI-basierte Verfahren trainierte Modelle, um relevante Informationen aus Bilddaten abzuleiten. KI-Modelle können hierbei auch auf den Resultaten klassischer Bildverarbeitung aufsetzen. Andersherum können die Ergebnisse trainierter Modelle auch regelbasiert weiterverarbeitet werden, um bestimmte Gesten oder Verhältnisse in Bildern oder Videos zu erkennen.

Diese Arbeit implementiert und untersucht eine kamerabasierte Gestensteuerung für ein virtuelles Orchester in einem 3D-Audiosystem im Raum B217a der Berliner Hochschule für Technik. Das dort vorhandene Lautsprechersystem ist begehbar und kann Klangquellen räumlich darstellen. Ziel des Projekts ist es, eine Person durch Arm- und Handgesten zum virtuellen Dirigenten werden zu lassen: Eine Greifgeste wählt ein Instrument aus, die Ausrichtung des ausgestreckten Arms bestimmt dessen räumliche Klangrichtung und das Öffnen der Hand beendet die Positionierung wieder. Eine grafische Benutzeroberfläche stellt Instrumente und Rückmeldungen dar. Steuerbefehle können über eine Netzwerkschnittstelle zwischen der Gestensteuerung und dem Audiosystem ausgetauscht werden. Das Audiosystem wird im Rahmen der vorliegenden Implementierung noch nicht mit der Gestensteuerung verbunden.

Nach der Einführung in die Grundlagen und den Stand der Technik beginnt die Arbeit mit der Erstellung eines eigenen Konzepts zur Gestensteuerung. Es werden die räumlichen Randbedingungen, insbesondere Kameraposition und Sichtfeld, untersucht und Anforderungen an die Hard- und Software formuliert. Für die Gestensteuerung wird ein Konzept mit und eines ohne KI-Anwendungen erstellt. Im KI-Konzept werden die Skelettdaten von MediaPipe mit einer Tiefenkamera kombiniert. Dieses Konzept wird auch implementiert und getestet.

Nach der Wahl der Hard- und Software, umfasst die Implementierung die Erkennung von Körper- und Handlandmarks, die Berechnung der Zeigerichtung, die Erkennung einer Greifgeste und weiterer Steuergesten, die grafische Oberfläche zur Systeminteraktion und die Kommunikation über UDP mit JSON.

Technische Funktion und die Bedienbarkeit werden anhand von Videoaufnahmen und Nutzertests evaluiert. Die Ergebnisse werden eingeordnet, Grenzen der Untersuchung benannt und Weiterentwicklungen abgeleitet. Ein Großteil dieser Weiterentwicklungen wird bereits in der vorliegenden Implementierung berücksichtigt.

# 2 Grundlagen

In diesem Kapitel sollen Techniken und Themenbereiche erklärt werden, welche für das Verständnis der vorliegenden Arbeit relevant sind.

## 2.1 Robot Operating System

Das Robot Operating System (ROS) ist ein quelloffenes Software-Framework für die Entwicklung modularer Robotikanwendungen. Die aktuelle ROS-Applikation ist ROS 2 [1]. ROS stellt standardisierte Mechanismen für die Kommunikation zwischen einzelnen Softwarekomponenten bereit und ergänzt diese durch Werkzeuge für Konfiguration, Protokollierung, Simulation und Systemüberwachung. Dadurch können einzelne Komponenten unabhängig voneinander entwickelt, getestet und in unterschiedlichen Robotersystemen wiederverwendet werden. [2]

Die grundlegende Einheit eines ROS-Systems ist der Node (zu Deutsch Knoten). Ein Node übernimmt typischerweise eine klar abgegrenzte Aufgabe, beispielsweise die Verarbeitung von Kameradaten, die Erkennung von Gesten oder die Steuerung eines Aktors. Nodes können über Topics, Services und Actions miteinander kommunizieren. Topics dienen der asynchronen Übertragung kontinuierlicher Datenströme nach dem Publish-Subscribe-Prinzip, während Services für kurze synchrone Anfrage-Antwort-Kommunikationen und Actions für länger andauernde Aufgaben mit Rückmeldungen und Ergebnissen vorgesehen sind. [3]

ROS organisiert die beteiligten Nodes in einem verteilten ROS-Graphen. Die Kommunikation erfolgt über ein Middleware-Konzept, das standardmäßig auf Data Distribution Service (DDS) beziehungsweise dem zugehörigen RTPS-Protokoll basiert. Dadurch können Nodes automatisch gefunden werden und sowohl innerhalb eines Prozesses als auch in verschiedenen Prozessen oder auf unterschiedlichen Rechnern kommunizieren. ROS unterstützt die Anbindung an verschiedene Programmiersprachen, wie C++ und Python, mit entsprechenden Libraries. [3], [4]

## 2.2 User Datagram Protocol

### *Einordnung von UDP*

Das User Datagram Protocol (UDP) ist ein verbindungsloses Transportprotokoll der Transportschicht und baut direkt auf dem Internet Protocol (IP) auf. Es ermöglicht Anwendungen, Daten in Form einzelner Datagramme an einen Zielhost und einen Zielport zu übertragen, ohne zuvor eine Verbindung aufbauen zu müssen. UDP stellt dabei nur einen minimalen Transportdienst bereit und überlässt viele Aufgaben der Anwendungsschicht. [5]

### *Aufbau*

Ein UDP-Datagramm besteht aus einem kurzen Header und den eigentlichen Nutzdaten. Der Header enthält unter anderem Quell- und Zielport, die Länge des Datagramms sowie eine Prüfsumme. Die Ports ermöglichen es dem Betriebssystem, ein empfangenes Datagramm dem richtigen Anwendungsprozess zuzuordnen. Die Prüfsumme dient dazu, Übertragungsfehler zu erkennen. [5], [6]

*Unterschied zu TCP*

Im Gegensatz zu TCP (Transmission Control Protocol) garantiert UDP weder die Zustellung noch die Reihenfolge oder die einmalige Übertragung der Datagramme. Pakete können verloren gehen, verspätet eintreffen oder doppelt empfangen werden. UDP führt außerdem keine automatische Fluss- oder Staukontrolle durch. Falls eine Anwendung Zuverlässigkeit, Reihenfolgesicherung oder eine Bestätigung des Empfangs benötigt, ist TCP geeignet. Diese Funktionalitäten müssten bei einer UDP-Kommunikation auf der Anwendungsebene ergänzt werden. [5], [6]

*Anwendungsbereich*

Der Verzicht auf Verbindungsaufbau und umfangreiche Kontrollmechanismen reduziert den Protokoll-Overhead und ermöglicht eine geringe Übertragungslatenz. Deshalb eignet sich UDP insbesondere für Anwendungen, bei denen aktuelle Daten wichtiger sind als die zuverlässige Übertragung jedes einzelnen Pakets. Dazu gehören beispielsweise Audio- und Videoströme, Sensordaten sowie Echtzeitanwendungen. [5], [6]

## 2.3 JavaScript Object Notation

JavaScript Object Notation (JSON) ist ein leichtgewichtiges, textbasiertes und programmiersprachenunabhängiges Datenformat. Es dient dazu, strukturierte Daten zwischen Anwendungen auszutauschen und für die Verarbeitung zu serialisieren. [7]

*Struktur*

JSON unterstützt Strings, Zahlen, boolesche Werte, `null`, Objekte und Arrays. Objekte enthalten Schlüssel-Wert-Paare in geschweiften Klammern, Arrays enthalten geordnete Werte in eckigen Klammern. Schlüssel und Werte werden durch einen Doppelpunkt, einzelne Einträge durch Kommata getrennt. Objekte und Arrays können einzeln angelegt oder beliebig ineinander verschachtelt werden. [7]

Beispiel eines Objekts:

```
{
  "instrument": "piano",
  "azimuth": 45.0,
  "active": true
}
```

Beispiel eines Arrays:

```
["piano", "violin", "flute"]
```

*Speicherung und Übertragung*

JSON-Daten können als Text, beispielsweise in einer Datei mit der Endung *.json*, gespeichert werden. Zur Übertragung können sie als Text serialisiert und über ein geeignetes Kommunikationsprotokoll versendet werden. Der Empfänger parst den Text, um die enthaltenen Werte wieder als Datenstruktur zu verarbeiten. JSON definiert dabei das Datenformat. [7]

## 2.4 MediaPipe

### 2.4.1 MediaPipe Framework

MediaPipe ist ein von Google entwickeltes Framework zur Erstellung von Verarbeitungspipelines für Anwendungen der maschinellen Wahrnehmung (Perception). Es ist plattformübergreifend (Python, C++, …) und auf mobilen Endgeräten (Handys, Notebooks, …) nutzbar. Das Paper [8] stellt das MediaPipe Framework vor. Es beschreibt dies als Framework zur Entwicklung von Wahrnehmungspipelines für Audio-, Video- und Sensordaten. Es ermöglicht das Bauen von Echtzeitanwendungen durch die Verkettung modularer Verarbeitungskomponenten, sogenannter *Calculator*, in einem gerichteten Graphen.

*Calculator*
Ein Calculator ist die Implementierung eines Knoten-Prozesses durch eine Calculator-Klasse (C++). Calculators bilden die eigentlichen Verarbeitungseinheiten innerhalb eines MediaPipe Graphen. Sie empfangen Daten über Eingangsports, führen eine definierte Verarbeitung aus und stellen die Ergebnisse über Ausgangsports zur Verfügung. Ein Calculator kann dabei sowohl einfache Verarbeitungsschritte als auch komplexere Algorithmen und Aufgaben wie die Ausführung eines maschinellen Lernmodells implementieren. [9]

*Kernkonzept*
Als Kernkonzept beschreiben die Autoren von [8], dass eine MediaPipe-Pipeline als Graph konfiguriert wird, in dem die Knoten (Calculator), die Datenströme (Streams) verarbeiten. Siehe Figure 17. Jeder Knoten empfängt Datenpakete mit Zeitstempeln, führt Berechnungen durch und gibt neue Pakete aus. Die Synchronisation erfolgt über die Zeitstempel, sodass mehrere Ströme, etwa Kamerabilder und berechnete Inferenzergebnisse, zeitlich korrekt zusammengeführt werden können. Datenströme können deterministisch, also in einer reproduzierbaren und vorhersehbaren Reihenfolge, verarbeitet werden. Optional können auch nicht-deterministische Strategien verwendet werden, um beispielsweise Echtzeitanforderungen zu erfüllen, indem zum Beispiel Pakete verworfen oder verzögert werden, um Latenzgrenzen einzuhalten.

*Architektur*
MediaPipe besteht grundsätzlich aus drei Teilen:

- einem Framework zur Inferenz aus Sensordaten
- Werkzeugen zur Leistungs- bzw. Performancemessung
- einer Sammlung wiederverwendbarer Komponenten, den Calculators

Ein Entwickler kann existierende Komponenten neu kombinieren, eigene schreiben und die Pipeline schrittweise erweitern, per Baukastenprinzip, ohne die gesamte Anwendung umschreiben zu müssen. [8]

*Multithreading*

Das Framework verwaltet die parallele Ausführung der Verarbeitungsknoten über einen Scheduler, das ist eine Steuerung für parallele Prozesse. Jeder Knoten kann auf einem eigenen Thread (Parallelprozess) laufen und intern nicht mehr als einen aktiven Thread besitzen. Durch die Aufteilung des Gesamtprozesses auf Threads kann die Rechenleistung, in Form der einzelnen Knoten, effektiv auf verschiedene CPU-Kerne aufgeteilt werden. Zudem unterstützt MediaPipe die Ausführung von Rechenprozessen auf der GPU. Die Zeitstempel der Datenpakete dienen als Synchronisationsschlüssel für die aufgeteilten Prozesse. [8]

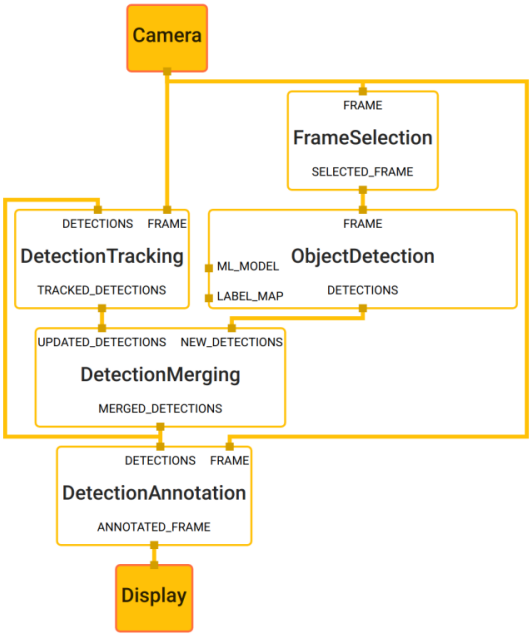



**Figure 1 - Calculator-Graph einer Mediapipe Objekterkennung [8]**

### *MediaPipe Solutions*

MediaPipe Solutions sind vorgefertigte, höher abstrahierte Komponenten in Mediapipe, die bestimmte Wahrnehmungsaufgaben bereits als nutzbare Anwendungen bereitstellen. Sie kapseln die interne Verarbeitung und nutzen verschiedene Calculators. So ermöglichen sie einen direkten Einsatz komplexerer Pipelinesysteme ohne manuelle Graph-Konfiguration. Damit bieten sie einen vereinfachten Einstieg für Standardaufgaben wie Hand- oder Gesichtserkennung oder anderen Implementierungen maschineller Lernfunktionen. Die Solutions bieten dabei weniger Kontrolle über die interne Datenverarbeitung als das zugrundeliegende Framework. [10]

### *Solutions zur Skeletterkennung*

Für die Erkennung menschlicher Körperhaltungen und Handgesten stellt MediaPipe verschiedene vorgefertigte Komponenten bereit. Dazu gehören unter anderem der Pose Landmarker (MediaPipe Pose) zur Erfassung von Körperlandmarken und der Hand Landmarker (MediaPipe Hands) zur Erfassung von Handlandmarken. Diese Komponenten können Bilddaten analysieren und die erkannten Landmarken als strukturierte Ergebnisse zurückgeben.

Der Hand Landmarker erkennt die Positionen charakteristischer Punkte der Hand. verbunden bilden diese eine Art Skelett. Das Ergebnis enthält unter anderem die Handlandmarken in normalisierten Bildkoordinaten, Handlandmarken in

Weltkoordinaten sowie Informationen zur Zuordnung der erkannten Hand als linken oder rechten Hand.

Der Pose Landmarker stellt in ähnlicher Weise Landmarken für den gesamten menschlichen Körper bereit. Neben den Koordinaten werden hier auch Parameter wie Visibilety (zu Deutsch Sichtbarkeit) geschätzt. Genauer beschrieben sind MediaPipe Pose und MediaPipe Hands in den folgenden gleichnamigen Abschnitten dieses Kapitels.

*Landmarks*
MediaPipe unterscheidet zwischen Bild- und Weltlandmarken.
Bildlandmarken beinhalten folgende Informationen [11]:

- x, y: Position des Landmarks im Bild, normalisiert auf Bildbreite bzw. Bildhöhe. Die Normierung verwendet 0 für den linken bzw. oberen Rand und 1 für den rechten bzw. unteren Rand. Es können auch Werte außerhalb dieses Bereichs auftreten, wenn eine Landmarke außerhalb des analysierten Bilds vermutet wird.

- z: Relative Tiefenkoordinate des Landmarks. Kleinere Werte bedeuten dichter an der Kamera. Die genaue Skalierung ist vom verwendeten Modell abhängig.

- Visibility: Gibt an, wie wahrscheinlich die Landmarke sichtbar und nicht durch andere Objekte verdeckt ist. Ein Landmark außerhalb des Bildbereichs gilt dabei ebenfalls als nicht sichtbar. Der Wertebereich ist von 0 für nicht sichtbar bis 1 für klar sichtbar.

- Presence: Gibt an, wie wahrscheinlich die Landmarke grundsätzlich in der Szene vorhanden bzw. innerhalb des gültigen Szenenbereichs liegt. Der Wertebereich ist ebenfalls gleitend von 0 bis 1.

Weltlandmarken beinhalten folgende Informationen [12], [13]:

- x, y, z: Dreidimensionale Koordinaten des Landmarks im Weltkoordinatensystem. Die Zählrichtung der Koordinaten gleicht den der Bildlandmarken. Die Einheit ist hier allerdings Meter. Bei Mediapipe Pose ist der Koordinatenursprung in der Körpermitte zwischen den Hüften.

- Visibility: wie bei den Bildlandmarken

- Presence: wie bei den Bildlandmarken

### 2.4.2 MediaPipe Pose

Das Paper [14] stellt mit BlazePose ein Verfahren zur Echtzeit-Schätzung der menschlichen Körperpose aus RGB-Bildern vor. Das Verfahren wurde auch für mobile Geräte entwickelt und schätzt für eine Person 33 Körper-Landmarks. Diese beschreiben verschiedene Körper- und Gesichtsmerkmale, siehe Figure 3.

Die Pipeline besteht aus einer Personendetektion und einem anschließenden Landmark-Modell. Die Personendetektion bestimmt zunächst den relevanten Bildbereich. Das

Landmark-Modell verwendet eine Verzweigung verschiedener neuronaler Netzwerke. Es nutzt drei Informationen:

- Heatmaps – Wahrscheinlichkeitskarten für jede der 33 Landmarks
- Offsets – Verschiebung der Heatmap zur tatsächlichen Bildposition
- Regression – die direkte Koordinaten-/Wertevorhersage

Heatmaps und Offsets werden nur bei dem Modelltraining verwendet. Bei der späteren Ausführung (Inference) werden diese Ausgabeschichten entfernt. Die Heatmap dient während des Trainings dazu, ein sogenanntes leichtgewichtiges Embedding zu trainieren. Dieses Embedding wird anschließend vom Regressionsnetz verwendet, um die genauen Landmark-Koordinaten zu bestimmen. Dieses Vorgehen resultiert in einer hohen Genauigkeit bei vergleichsweise niedriger Inferencezeit.

BlazePose ermöglicht somit die Schätzung von Körper-Landmarks aus RGB-Bildern und RGB-Videoframes. Das Verfahren bildet u.a. die Grundlage für die Pose-Schätzung in MediaPipe Pose. Es wirbt damit, dass seine Landmark-Schätzung auf mobilen Geräten in Echtzeit möglich ist. [15]



**Figure 2 - Pose-Estimation mit BlazePose (MediaPipe) [14]**

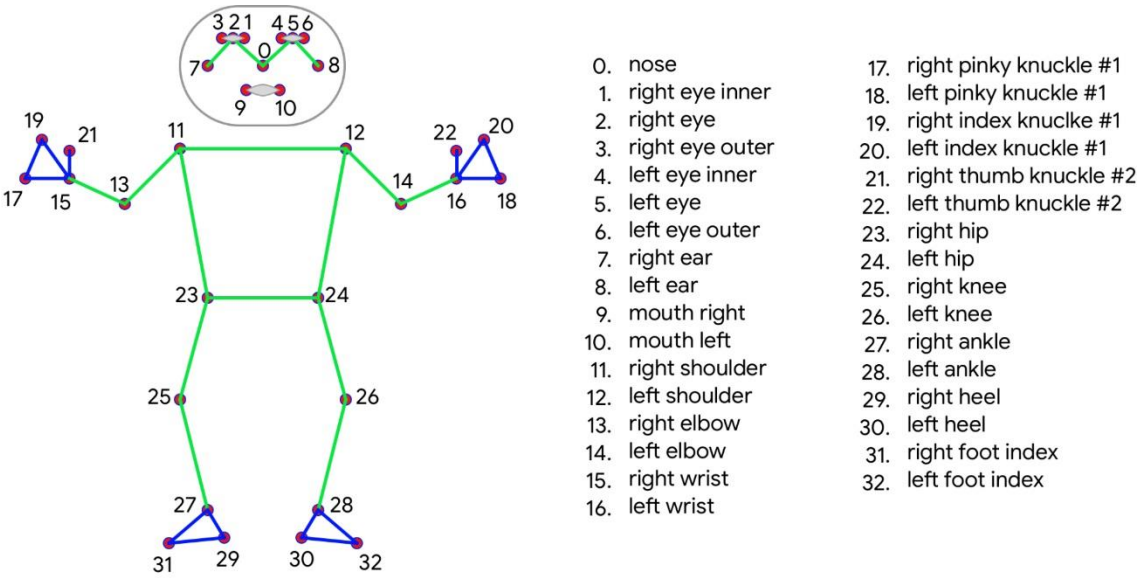



**Figure 3 – MediaPipe BlazePose Körperlandmarks [15]**

### 2.4.3 MediaPipe Hands

Das Paper [16] stellt ein Verfahren zur Echtzeit-Erkennung und Verfolgung menschlicher Hände vor. Das Verfahren bestimmt die Position der Hand sowie ein Skelett aus 21 Handlandmarks. Es basiert genau wie BlazePose auf RGB-Bildmaterial als Eingangsdaten und bietet eine kompakte Darstellung der Hand über deren Gelenkpunkte. Siehe Figure 4 und Figure 5.

Die Verarbeitung besteht aus zwei Modellen. Zunächst bestimmt ein Palm (Englisch für Handfläche) -Detektor die Position der Handfläche im Bild. Anschließend verarbeitet ein Hand-Landmark-Modell den erkannten Bereich und bestimmt die 21 Landmarken der Hand. Die Aufteilung in diese beiden Schritte reduziert die Aufgabe für das Landmark-Modell, da dieses nicht mehr das gesamte Bild untersuchen muss.

Dieses Verfahren ist für die Verarbeitung von Videosequenzen ausgelegt. Denn nach der ersten Erkennung kann die Position der Hand in den folgenden Bildern verfolgt werden. Dadurch muss die Handfläche nicht in jedem Bild erneut vollständig detektiert werden. Der Palm Detektor wird erneut verwendet, wenn die Verfolgung verloren geht. Auf diese Weise wird eine effiziente Echtzeitverarbeitung ermöglicht.

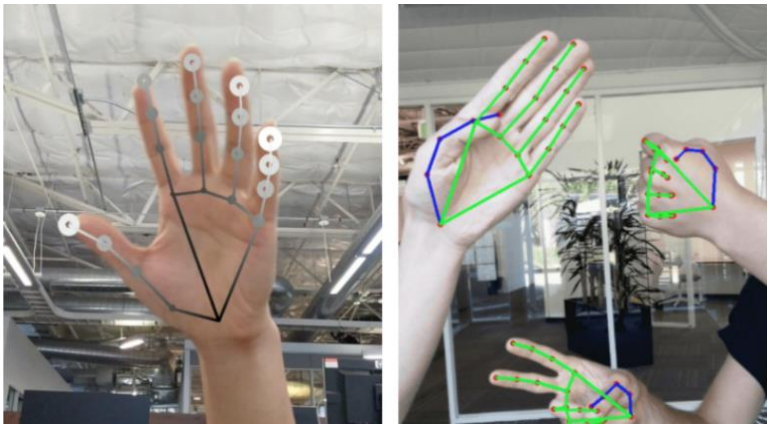
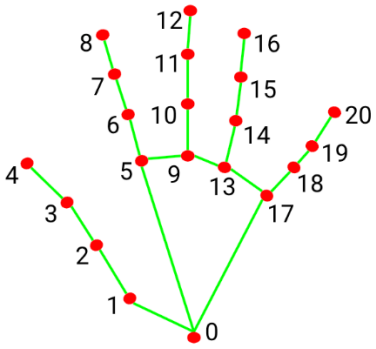


**Figure 4 - Handerkennung mit MediaPipe [16]**




0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

**Figure 5 - MediaPipe Handlandmarks [8]**

## 2.5 OpenPose

OpenPose ist ein von der Carnegie Mellon University entwickeltes Open-Source-Framework zur Echtzeit-Erkennung menschlicher Körperhaltungen aus Bildern und Videos. Es stellt das erste System dar, das Körper-, Hand-, Gesichts- und Fuß-Landmarken mehrerer Personen gleichzeitig in Echtzeit detektieren kann. Das Verfahren wurde in dem IEEE Paper [17] veröffentlicht. Der kostenfreier Preprint [18] ist u.a. auf *arXiv* zu finden. Die eigentliche Software ist quelloffen verfügbar. Diese hat Anbindungen zu C++ und Python.

### *Kernkonzept*

Als zentrales Konzept führt OpenPose sogenannte Part Affinity Fields (PAFs) ein. Dabei handelt es sich um zweidimensionale Vektorfelder, die die räumliche Zuordnung zwischen Körperteilen und den zugehörigen Personen kodieren. In einem Bottom-up-Ansatz werden zunächst alle Landmarken im Bild detektiert und anschließend mithilfe der PAFs den einzelnen Personen zugeordnet. Dieses Vorgehen ermöglicht eine von der Personenanzahl weitgehend unabhängige Laufzeit. [18]

### *Architektur*

Die OpenPose-Pipeline besteht aus zwei Hauptkomponenten. Ein neuronales Netzwerk erzeugt Heatmaps für die Landmark-Positionen sowie die Part Affinity Fields für die Zuordnung der Körperteile. Anschließend werden die detektierten Landmarken über einen Greedy-Matching-Algorithmus zu vollständigen Skeletten zusammengefügt. Das

Netzwerk verwendet mehrere Verfeinerungsstufen, in denen sowohl die Heatmaps als auch die PAFs schrittweise verbessert werden. [18], [19]

*Multithreading / GPU-Unterstützung*
OpenPose verteilt die Verarbeitungsaufgaben zwischen CPU und GPU. Es ist mit seiner CUDA-Unterstützung für NVIDIA-GPU optimiert. Ist eine entsprechende Grafikkarte vorhanden, übernimmt die CPU vorrangig das Einlesen und die Vorbereitung der Bilddaten, die Steuerung der Verarbeitungspipeline sowie die Nachverarbeitung der Ergebnisse. Die rechenintensive Inferenz des neuronalen Netzwerks wird dann auf der GPU ausgeführt, wobei CUDA die Faltungsoperationen auf viele parallele GPU-Kerne verteilt. OpenPose kann mehrere GPUs parallel nutzen und verwendet innerhalb der CPU mehrere Threads. [20]

*Landmarks*
OpenPose unterstützt verschiedene Landmark-Sets. Für den Körper werden 18 oder 25 Skelettpunkte bereitgestellt. Zusätzlich können Hand- und Gesichtslandmarken detektiert werden, wodurch insgesamt bis zu 135 Keypoints pro Person möglich sind. Die Ausgabe umfasst die zweidimensionalen Bildkoordinaten sowie einen Konfidenzwert für jede Landmarke. [21], [20]

*Vergleich zu MediaPipe*
Im Gegensatz zu MediaPipe, das für mobile Geräten und eingebettete Systemen optimiert ist, richtet sich OpenPose primär an Desktop-Systeme mit leistungsfähigen GPUs. Während MediaPipe stark auf Modularität und plattformübergreifende Einsetzbarkeit ausgelegt ist, bietet OpenPose eine spezialisierte Lösung für Multi-Person-Pose-Estimation auf PC-Systemen. [20], [22], [8]

*TF-Pose*
Für leistungsschwächere Embedded-Systeme wurde tf-pose-estimation [23] entwickelt. Es ist ein Nachbau der OpenPose Funktionalitäten basierend auf TensorFlow [24]. Das Ziel war es auch mit weniger Rechenleistung Echtzeitperformanz zu erreichen.

## 2.6 OpenCV

OpenCV (Open Source Computer Vision Library) ist eine quelloffene Softwarebibliothek, die mehrere hundert Algorithmen für die Verarbeitung und Analyse von Bild- und Videodaten beinhaltet.

*Plattformanbindung*
OpenCV ist in erster Linie in C++ implementiert, denn die Kernbibliothek besteht aus optimiertem, portabel geschriebenem C++-Code. Die Anbindung an andere Programmiersprachen wie Python, Java, MATLAB/Octave oder Ruby erfolgt über Library interne Wrapper, auch Bindings genannt. OpenCV läuft plattformübergreifend auf Desktop-Systemen (Windows, Linux, macOS, FreeBSD, OpenBSD), mobilen Plattformen (Android, iOS, Maemo OS) und eingebetteten Linux-Varianten. Unterstützt wird eine breite Palette von CPU-Architekturen (x86, x86_64, ARM, RISC-V, PowerPC, MIPS). [25], [26]

*Verarbeitung einzelner Bilder*

Die Anbindung an Quellen von Bild- und Videomaterial regelt das OpenCV-Modul *videoio* [27]. Dieses ermöglicht die Verbindung zu Kameras und die Verarbeitung von Videoströmen aus verschiedenen Dateiformaten (MP4, AVI, MOV, MKV, WebM). Aus Videodaten werden die Bilder einzeln eingelesen, im Code meistens in einer Loop. Einzelne Bilder werden als mehrdimensionale Matrizen ausgegeben. Für ein RGB-Bild mit einer HD-Auflösung hätte die Matrize die Dimensionen [720, 1280, 3]. Wobei die drei Farbkanäle in der Reihenfolge Blau, Grün, Rot gespeichert werden und je durch einem 8-Bit-Wert zwischen 0 und 255 dargestellt sind. Neben dem Einlesen wird mit *videoio* auch das Speichern von Bildmaterial ermöglicht. [28]

*Klassische Bildverarbeitung*

Das Modul *imgproc* [29] umfasst Funktionen für verschiedene grundlegende Aufgaben der Bildverarbeitung. Es ist das zentrale Modul für pixelbasierte Verfahren. Zu diesen gehören lineare und nichtlineare Filter, Farbraumkonvertierungen, geometrische Transformationen, Histogramme sowie morphologische Operationen. Darauf aufbauend stellt das Modul Verfahren zur Kantendetektion, Konturanalyse und Bildsegmentierung bereit. [28],[29]

Mehr zu klassischer Bildverarbeitung in den folgenden Abschnitten 2.7 bis 2.10.

*Namespace*

Die Funktionen und Klassen aller OpenCV Module sind seid Version 2.0 unter einem einheitlichen Namespace zu finden. Dieser ist `cv` in C++ und `cv2` in Python.

*OpenCV in MediaPipe*

OpenCV ist die zugrundeliegende Library für Bild- und Videoverarbeitung in MediaPipe. Es übernimmt unter anderem die Aufgaben der Video-Erfassung, Dekodierung, Farbraumkonvertierung und geometrischen Vorverarbeitung, bevor die Daten an die ML-Modelle (z. B. Pose-, Hand- oder Gesichtserkennung) weitergegeben werden. Typische Operationen sind das Einlesen von Videostreams (VideoCapture), Konvertierung zwischen BGR und RGB, Skalierung, Cropping und einfaches Drawing (über ein Bild zeichnen) zur Visualisierung von Landmarks. Das sind alles Funktionen, die im *imgproc*- und *videoio*-Modul von OpenCV implementiert sind. [30]

## 2.7 Morphologische Transformationen

Die Morphologie ist ein Zweig der Bildverarbeitung, der sich mit der Verarbeitung von binären Pixelbildern (Rastergrafiken) befasst. Binäre Rastergrafiken sind Bilder, deren Bildelemente (Pixel) nur einen von zwei verschiedenen Farbwerten annehmen können. Basisoperationen in der Morphologie sind Dilatation, Erosion, Vereinigung, Schnittmengenbildung und Mengendifferenzbildung. Aufbauend auf diesen Operationen können weitere Operationen wie Opening, Closing, Verdünnung, Umriss-Extraktion oder beispielsweise die Skelettierung konstruiert werden. [31]

Heutzutage gibt es zuteilen auch Morphologische Verfahren für Grauwertbilder bzw. Farbkomponentenbilder [32].

Auf docs.opencv.org [33] werden sieben der grundliegenden Morphologischen Transformationen erklärt und mit OpenCV Funktionen in Python präsentiert. Auf dieser Dokumentation gestützt, sollen diese sieben Transformationen im Folgenden mit Bildbeispielen erklärt werden. Die Reihenfolge der Transformationsverfahren verdeutlicht anschaulich wie, in der klassischen Bildverarbeitung, einfache Methodiken zu komplexeren Verfahren zusammengesetzt werden können.

### *Erosion*

Die Grundidee der Erosion ähnelt der Bodenerosion: Die Ränder eines Vordergrundobjekts werden abgetragen (dabei wird der Vordergrund üblicherweise weiß dargestellt). Dazu wird ein Kernel (eine kleine 2D-Pixelmatrix) über das Bild bewegt (ähnlich wie bei einer zweidimensionalen Faltung). Ein Pixel im Originalbild bleibt nur dann weiß (Wert 1), wenn alle Pixel unter dem Kernel ebenfalls weiß sind. Andernfalls wird das Pixel entfernt (auf 0 gesetzt). Dadurch werden insbesondere Pixel an den Objektgrenzen entfernt. Je größer der Kernel ist, desto stärker schrumpft das Objekt. Die weiße Vordergrundfläche wird also kleiner. Auch werden Unterbrechungen im Objekt vergrößert und Objekte kleiner als der Kerne verschwinden komplett. Die Erosion eignet sich z.B. für:

- Trennung miteinander verbundener Objekte
- Verkleinerung von Objektregionen

Figure 6 zeigt zwei Beispielobjekte mit vorher (links) und nachher (rechts) Bilder.

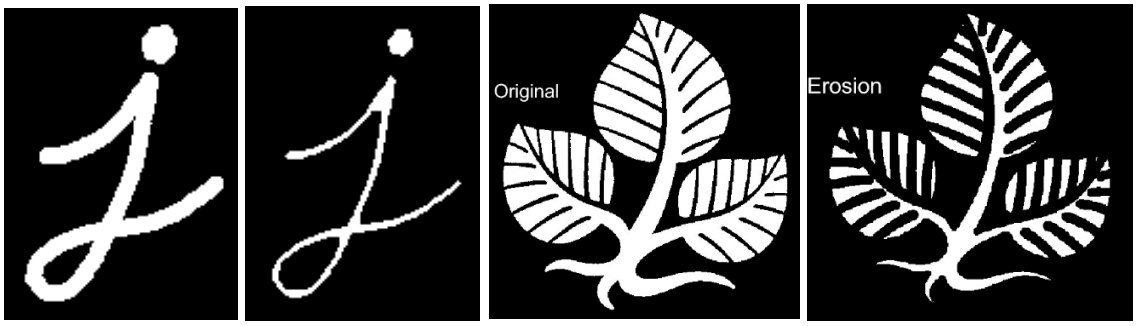



**Figure 6 - Bilder vor und nach Erosion [32], [33]**

### *Dilatation*

Die Dilatation ist genau das Gegenteil der Erosion. Hier wird ein Pixel als 1 gesetzt, wenn mindestens ein Pixel unter dem Kernel den Wert 1 besitzt. Dadurch vergrößert sich die weiße Region im Bild beziehungsweise das Vordergrundobjekt wächst. Öffnungen und Löcher kleiner als der Kernel werden geschlossen. Die Dilatation eignet sich z.B. dafür:

- getrennte Teile eines Objekts zu verbinden
- kleine Unterbrechungen in Objekten zu schließen

Figure 7 zeigt dieselben Beispielobjekte wie zuvor, vor und nach der Dilatation.

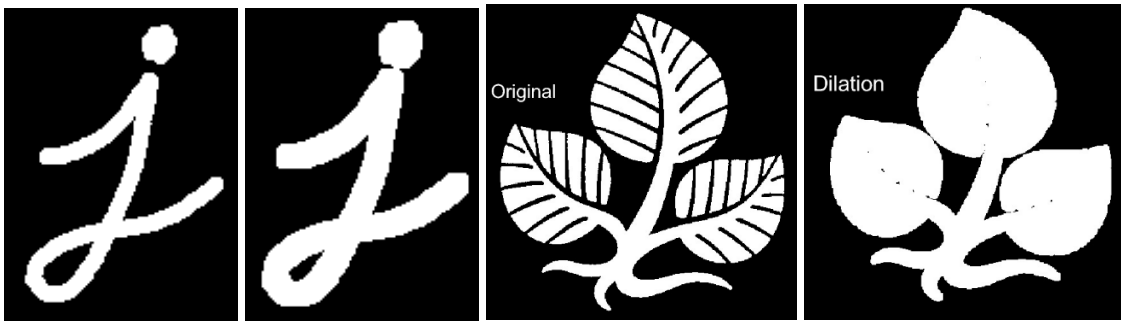



**Figure 7 - Bilder vor und nach Dilatation[32], [33]**

### *Opening*

Opening ist lediglich eine andere Bezeichnung für eine Erosion, gefolgt von einer Dilatation. Bei der Rauschunterdrückung wird häufig zuerst eine Erosion und anschließend eine Dilatation durchgeführt. Die Erosion entfernt zwar kleine weiße Störungen, verkleinert jedoch gleichzeitig das eigentliche Objekt. Durch die anschließende Dilatation wird das Objekt wieder vergrößert. Da das Rauschen bereits entfernt wurde, erscheint es nicht erneut, während die Objektfläche wieder zunimmt. Es eignet sich weiterhin zum:

- Trennen angrenzender Objekte
- Entfernen kleiner Objekte
- öffnen randnaher Objektunterbrechungen

Beispiele sind wieder in Figure 8 zu sehen.

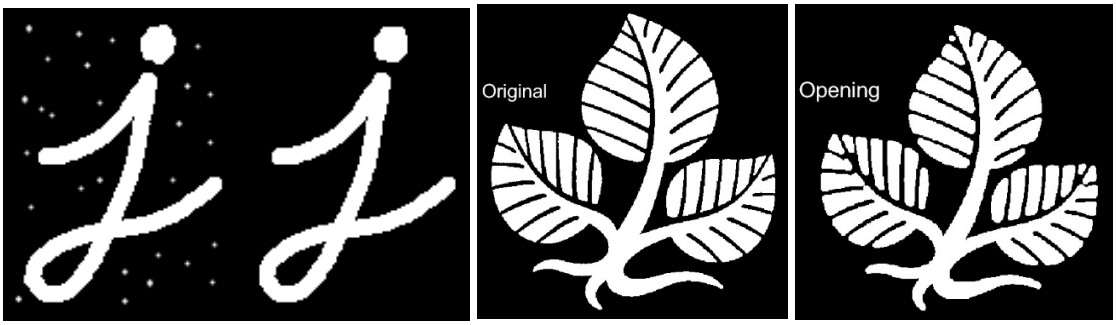



**Figure 8 - Bilder vor und nach Opening [32], [33]**

### *Closing*

Closing ist die umgekehrte Operation zu Opening, also eine Dilatation gefolgt von einer Erosion. Das Closing eignet sich z.B. dafür:

- kleine Löcher und Öffnungen innerhalb von Objekten zu schließen,
- getrennte Objekte zu vereinen

Figure 9 zeigt die Closing Beispiele. Figure 10 fasst die einflösse von Erosion, Dilation, Opening und Closing nochmal anschaulich zusammen.

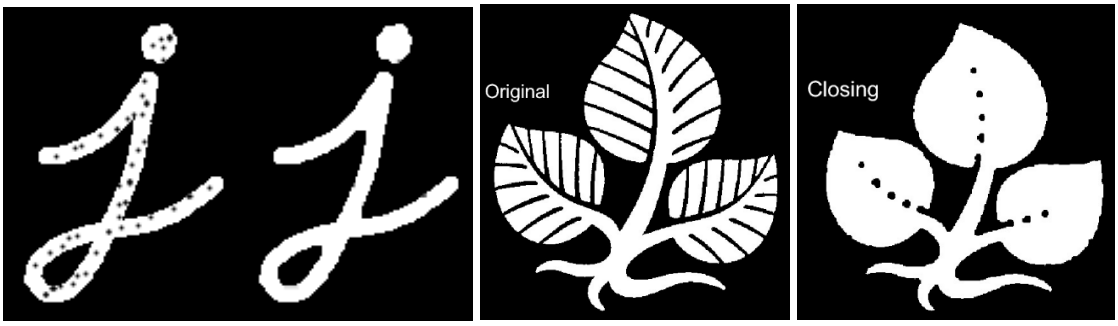



**Figure 9 - Bilder vor und nach Closing [32], [33]**

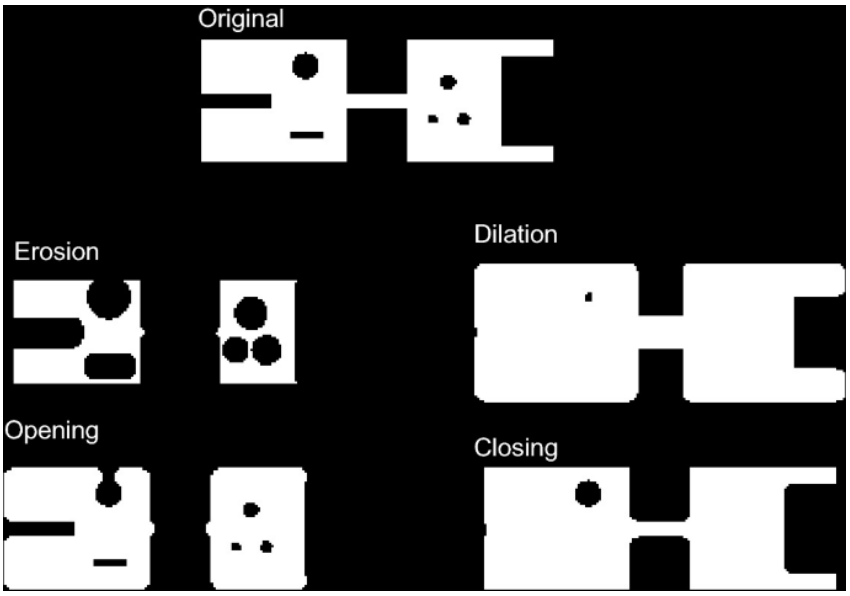



**Figure 10 - Einfluss von Erosion, Dilatation, Opening und Closing [32]**

### *Morphologischer Gradient*

Der Morphologische Gradient wird auch Umriss-Extraktion genannt. Es ist die Differenz zwischen der Dilatation und der Erosion eines Bildes. Zurückbleibt die Umrisslinie beziehungsweise die Kontur eines Objekts, welche die originale Objektgrenze nach innen (Erosion) und nach außen (Dilatation) austrägt. Die Kernelgröße gibt hierbei an, um wieviel die Kontur nach innen und außen ausgetragen wird. Figure 11 zeigt Beispiele.

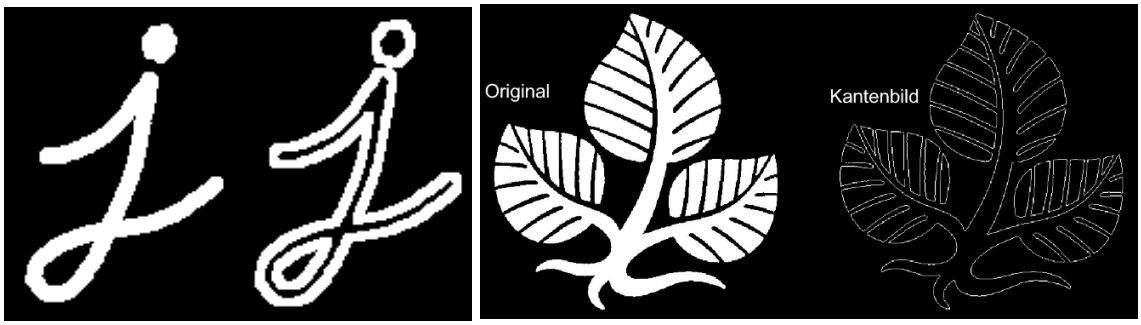



**Figure 11 - Bilder vor und nach Morphologischer Gradient [32], [33]**

### *Top Hat*

Die *Top-Hat-Transformation* zu Deutsch Zylinderhut-Transformation wird auch *white-top-hat-transform* genannt. Sie ist die Differenz zwischen dem Eingabebild und dem Opening des Bildes. Dadurch bleiben vor allem kleine helle Strukturen zurück, welche durch das Opening entfernt wurden. Mit Top Hat lassen sich also Bildbereiche entfernen, welche die Kernelgröße überschreiten. Im j-Bild in Figure 12, wurde ein 9x9-Kernel genutzt, es wurden Flächen rausgeschnitten, welche 9 x 9 Pixel groß oder größer sind.

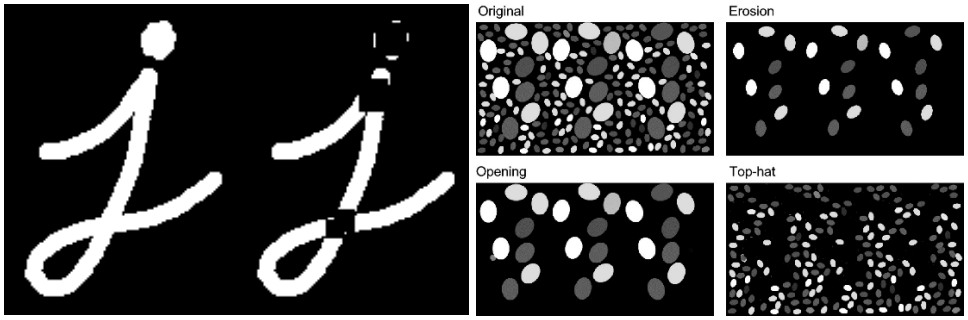



**Figure 12 - Bilder vor und nach Top Hat [32], [33]**

### *Black Hat*

Die *Black-Hat-Transformation* (auch *black top-hat transform* oder *bottom-hat-transform*) ist die Differenz zwischen dem Closing des Eingabebildes und dem ursprünglichen Eingabebild. Dadurch bleiben hauptsächlich kleine Lücken und Löcher erhalten, welche durch das Schließen beseitigt wurden. Figure 13 zeigt die Lücken, welche das Closing geschlossen hat. Mit anderen Worten es wurden alle Zwischenräume extrahiert, welche den Kernel nicht vollständig ausfüllen.

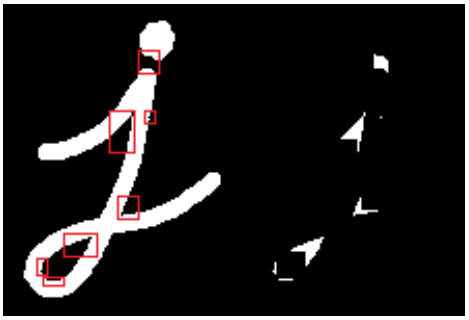


**Figure 13 - Bilder vor und nach Black Hat [33]**

### *Kernels*

Es kann sinnvoll sein einen n x m Kernel nicht vollständig mit Einsen zu füllen, sondern eine speziell geformte Filtermaske zu bilden. Mit einer runden Maske im Kernel, lassen sich beispielsweise runde Konturen formschlüssiger filtern, glätten oder extrahieren. Ein Beispiel ist das Bild rechts in Figure 12, es wurde mit einem runden 16 x 16 Pixel Kernel bearbeitet.

OpenCV bietet vier vorgefertigte Kernelformen an. Hier beispielhaft der Kreuz-Form-Kerne:

```
# Cross-shaped Kernel
>>> cv.getStructuringElement(cv.MORPH_CROSS,(5,5))
array([[0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0],
       [1, 1, 1, 1, 1],
       [0, 0, 1, 0, 0],
       [0, 0, 1, 0, 0]], dtype=uint8)
```

## 2.8 Background Subtraction

Background Subtraction bezeichnet ein Verfahren mit dem bewegte oder veränderte Bildbereiche vom statischen beziehungsweise langsam veränderlichen Hintergrund getrennt werden. Dazu wird ein Hintergrundmodell erstellt, das die erwarteten Pixelwerte des unbewegten Szenarios beschreibt. Der aktuelle Kameraframe wird anschließend mit diesem Modell verglichen. Überschreitet die Differenz eines Pixels einen festgelegten Schwellenwert, wird es dem Vordergrund zugeordnet; andernfalls gilt es als Hintergrund. Das Ergebnis ist typischerweise eine binäre Vordergrundmaske. [34]

*Hintergrundmodell*

Das Hintergrundmodell kann statisch bleiben oder sich im Laufe der Zeit an Veränderungen anpassen. Ein statisches Modell ist einfach umzusetzen, reagiert jedoch empfindlich auf veränderte Beleuchtung oder eine veränderte Kameraposition. Adaptive Verfahren aktualisieren das Modell kontinuierlich und können dadurch beispielsweise langsame Helligkeitsänderungen oder sich verändernde Hintergrundbereiche berücksichtigen. Gleichzeitig besteht die Gefahr, dass langsam bewegte Objekte schrittweise als Teil des Hintergrunds übernommen werden. [34]

*Bewegungserkennung*

Die Bewegungserkennung durch Bilddifferenzierung weist eine ähnliche Grundlage auf. Dabei werden zwei zeitlich aufeinanderfolgende Frames oder ein aktueller Frame und ein zeitlich zurückliegender Referenzframe pixelweise voneinander subtrahiert. Große Helligkeits- oder Farbunterschiede weisen auf Bildbereiche hin, in denen sich zwischen den Aufnahmen etwas verändert hat. Nach einer Schwellenwertbildung entsteht auch hier eine binäre Maske, die potenziell bewegte Bereiche beschreibt. [35]

## *2.9* Suzuki Algorithmus

Die Konturenerkennung in OpenCV basiert auf dem Border-Following-Verfahren nach Suzuki und Abe (1985) [36], das zusammenhängende Ränder in binären Bildern verfolgt, auch *Suzuki's Contour tracing algorithm* oder nur *Suzuki Algorithmen* genannt. Der Suzuki Algorithmus soll hier nur grob erklärt werden. Mehr dazu unter [36], [37].

Der Algorithmus findet Objektgrenzen (Konturen) und unterscheidet zwischen äußeren Grenzen und inneren Grenzen. innere Grenzen sind Grenzen zu Hohlräumen im Objekt. In der Topologie des Algorithmus wird vermerkt, wenn grenzen andere Grenzen umschließen. Zum Beispiel kann topologisch erfasst werden, wenn, eine Lochgrenze

innerhalb einer äußeren objektgrenze ist und sich ggf. ein weiteres Objekt innerhalb dieses Loches befindet. Die Grenzen werden dabei folgendermaßen gefunden:

- das Bild wird Zeile für Zeile, dann Pixel für Pixel durchgegangen, von oben rechts nach unten links
- wird die Pixelfolge $[0, 1]$ erkannt, beginnt eine Außengrenze
- wird die Pixelfolge $[1, 0]$ erkannt, beginnt eine Innengrenze (ein Loch)
- wurde ein Beginn gefunden wird die Grenze, also die Kontur, im Uhrzeigersinn verfolgt, die durchlaufenen Pixel werden gespeichert
- eine Kontur gilt noch nicht als abgeschlossen, wenn die Verfolgung wieder am Startpixel ankommt
- Erst wenn eine Pixelfolge von zwei beliebigen Pixeln erneut durchlaufen wird, gilt die Kontur als vollständig geschlossen.

## 2.10 Farbmodelle zur Hautsegmentierung

In diesem Abschnitt werden ein paar Farbmodelle und ihr Einsatz zur Erkennung von Bereichen menschlicher Haut in Bilder vorgestellt.

*YUV-Farbmodel*

YUV [38] ist nach RGB [39] eines der ältesten Farbmodelle, welche in Zusammenhang mit elektronischer Datenübertragung genutzt wird. Im Gegensatz zum RGB-Raum, ist das YUV-Farbmodell (Englisch *YUV color space*) ein Farbdifferenzmodell, dieses trennt Farbinformationen von der Helligkeit. Es entstehen drei neue Bildkanäle Y für die Helligkeit sowie U und V für die Farbdifferenz, auch genannt Chrominanz. In der Bildverarbeitung wird der YUV-Space oft genutzt, um eine Farberkennung von der Helligkeit zu entkoppeln. Dadurch wird diese robuster gegenüber verschiedenen Lichtverhältnissen. [40]

Die YUV-Transformation lautet wie folgt:

**Formel 1 - YUV-Transformation [41]**

$$\begin{pmatrix} Y \\ U \\ V \end{pmatrix} = \begin{pmatrix} +0.299 & +0.587 & +0.114 \\ -0.147 & -0.289 & +0.436 \\ +0.615 & -0.515 & -0.100 \end{pmatrix} \cdot \begin{pmatrix} R \\ \mathrm{G} \\ \mathrm{B} \end{pmatrix}$$

Die Farbgewichtungen in der ersten Reihe, welche Y ergeben, entsprechen der Helligkeitswahrnehmung eines Menschen. Wir nehmen Grün heller wahr als Rot und Rot etwas heller als Blau. Für die Hautfarbenerkennung werden besonders die Kanäle U und V genutzt. Oft werden diese dazu einfach Pixel für Pixel mit Schwellwerten abgeglichen, Angaben für diese Schwellwerte variieren, je nach Projekt und Paper.

*Neuere Farbmodelle*

Heutzutage gibt es eine Vielzahl an Farbmodellen. Für die Erkennung von Hautformen werden unteranderem YCbCr [42] und das Hunter-Lab-Farbmodell (H Lab) [43] erfolgreich genutzt. Auch diese trennen die Helligkeit von den chromatischen Farbinformationen.[44], [41]

Die YCbCr-Transformation unterscheidet sich von der YUV-Transformation nur etwas in der Gewichtung. Sie ist wie folgt definiert:

**Formel 2 - YCbCr-Transformation [41]**

$$\begin{pmatrix} \boldsymbol{Y} \\ \boldsymbol{Cr} \\ \boldsymbol{Cb} \end{pmatrix} = \begin{pmatrix} \mathbf{+0.299} & \mathbf{+0.587} & \mathbf{+0.114} \\ \mathbf{-0.169} & \mathbf{-0.441} & \mathbf{+0.500} \\ \mathbf{+0.500} & \mathbf{-0.418} & \mathbf{-0.081} \end{pmatrix} \cdot \begin{pmatrix} \boldsymbol{R} \\ \mathbf{G} \\ \mathbf{B} \end{pmatrix}$$

Die H-Lab-Transformation basiert auf dem XYZ-Farbraum und ist durch ihre Wurzelfunktionen rechenaufwändiger:

**Formel 3 - H Lab-Transformation [41]**

$$L = 10 \cdot \sqrt{Y}$$

$$a = 17{,}5 \cdot \frac{1{,}02 \cdot X - Y}{\sqrt{Y}}$$

Wobei der XYZ-Farbraum wiederum aus dem RGB-Raum berechnet werden kann:

**Formel 4 - XYZ-Transformation [41]**

$$\begin{pmatrix} \boldsymbol{X} \\ \boldsymbol{Y} \\ \boldsymbol{Z} \end{pmatrix} = \begin{pmatrix} \mathbf{+0.431} & \mathbf{+0.342} & \mathbf{+0.178} \\ \mathbf{-0.222} & \mathbf{+0.707} & \mathbf{+0.071} \\ \mathbf{+0.020} & \mathbf{-0.130} & \mathbf{-0.939} \end{pmatrix} \cdot \begin{pmatrix} \boldsymbol{R} \\ \mathbf{G} \\ \mathbf{B} \end{pmatrix}$$

*Hautsegmentierungsmethoden mit Farbmodellen*

In Paper [41] werden weiße, gelbe, braune und Schwarze Hautfarben erkannt. Die Basis bildet ein Pool aus 200 Bildern. Insgesamt werden hier elf verschiedene Farbmodelle in zwei Hautsegmentierungsverfahren getestet. Das erste Verfahren funktioniert über einfache Schwellwerte. Die Schwellwerte für den Einsatz des YUV-Modells sind dabei beispielsweise:

$$80 < Y < 180$$
$$20 < U < 100$$
$$100 < V < 160$$

Das zweite Verfahren ist etwas aufwändiger und nutzt ein selbst erstelltes Hautmodel. Dieses besteht aus dem Exponentialterm einer zweidimensionalen gaußschen Wahrscheinlichkeitsverteilung. Formel 5 beschreibt dieses Model in Matrizenschreibweise.

**Formel 5 - Hautmodel aus [41]**

$$P(r,b) = exp[-0.5(x-m)^T C^{-1}(x-m)]$$
$$mit:$$
$$x = (r,b)^T$$
$$m = E[x]$$
$$C = E[(X-m)(X-m)^T]$$
C : Kovarianz Matrix
M : Mittelwerts Vektor
r, b : chromatische Kanäle eines Farbmodells

Nach dem das Modell trainiert wurde – also Werte für $m$ und $C$ definiert wurden, kann ein Graustufenbild erstellt werden, in welches pixelweise die Wahrscheinlichkeit auf Hautfarbe dargestellt wird. Anhand des Graustufenbilds wird dann der Schwellwert bestimmt und auf dasselbige angewendet. Das Ergebnis ist ein Schwarz-Weiß-Bild der Hautbereiche und nicht Hautbereiche, wie es auch bei der einfachen Schwellwert Methode erstellt wird. Der Unterschied besteht darin das der Schwellwert hier dynamisch an die Wahrscheinlichkeiten im Bild angepasst werden kann.

Das Paper [41] beschreibt, dass besonders YCbCr und H-Lab eine hohe Präzision bei der Erkennung von Hautfarbe erzielen können.

*Kombination von Farbmodellen*
Das Paper [40] testet ebenfalls verschiedene Farbmodelle nur Hautfarbenerkennung. beschreibt einen anderen Ansatz. Hierbei werden die beiden Farbmodelle YUV und RGB parallel ausgewertet, wobei jeweils eine einfache Schwellwertmethode zum Einsatz kommt. Durch die Kombination beider Farbmodelle erreichen die Autoren eine höhere Präzision als durch die Verwendung von nur einem Farbmodell.

## 2.11 ABI-Kompatibilität

ABI steht für *Application Binary Interface*. Libraries, insbesondere Gerätetreiber sind Oft nicht in der Programmiersprache geschrieben, in der sie zum Einsatz kommen sollen. Um den Treiber dennoch einzusetzen, kann er in eine Shared-Library (englisch für geteilte Bibliothek) überführt werden, diese ist nicht mehr an die Programmiersprache gebunden. Gängige Dateiformate dafür sind DLL (Dynamic Link Library) unter Windows, oder SO (Shared Object) unter Linux. Wird Der Treibercode fertig kompiliert als Maschinencode in eine solche Datei gepackt, dann ist diese an eine bestimmte Prozessorarchitektur gebunden. Maschinencode für 32 Bit-Systeme, funktioniert nicht auf 64 Bit-Systemen, Code für ARM-Architekturen läuft nicht auf x86-Architekturen. Das nennt sich ABI-Kompatibilität. Mit anderen Worten gibt die ABI-Kompatibilität an, ob die Schnittstellen zwischen Anwendungen binärer Systeme zueinander passen. Darauf ist insbesondere dann zu achten, wenn Geräte mit spezieller Treibersoftware auf weniger gängigen Prozessortypen genutzt werden sollen. Ein Beispiel hierfür wäre die Verwendung einer speziellen Tiefenkamera an einem bestimmten Mikrocomputer.

## 2.12 Intel RealSense Tiefenkamera Treiber

Vorab: die Nutzung des RGB-Sensors der Kamera ist in Windows 10 und 11 ohne zusätzliche Treibersoftware möglich. Auch die Tiefenkamera wurde wärend dieses Projekts teilweise ohne weitere Treiber erkannt.

Für die vollumfängliche Nutzung, mit Kalibrirung und Tifenberechnung ist jedoch die Intel RealSense SDK 2.0 (librealsense) zu empfehlen. Dies ist die offizielle plattformübergreifende Library zur Interaktion mit den RealSense Kameras. Sie ist, wie für Treibersoftware üblich, in C++ geschrieben. Teil dieser Bibliothek sind Wrapper für verschiedene Plattformen, u. a. Python, MATLAB und Unity [45]. Die gängigen Systeme Windows, Linux und macOS werden unterstützt. Auch ARM-Architekturen von Raspberry Pi und NVIDIA JetSon sind kompatibel. So ist es auf der offiziellen GitHub Seite [46] beschrieben. Die Library pyrealsense2 [47] ist die Anbindung an Python, konkreter werden alle Versionen ab Python 3.10 in der aktuellen Version unterstützt.

### *Alignment für 3D-Koordinaten*

Die librealsense-Library stellt Funktionen zur geometrischen Verarbeitung von Farb- und Tiefendaten bereit. Dazu gehört das sogenannte Alignment, mit dem das Tiefenbild anhand der Kamerakalibrierung auf das Koordinatensystem des Farbbildes ausgerichtet werden kann. Das Alignment kann in beide richtungen ervolgen. Die intrinsischen Parameter der einzelnen Kamerasonsoren, bestimmt und verarbeitet die Library dabei automatisch. Darüber hinaus ermöglicht die Library, für eine angegebene Pixelkoordinate die zugehörige Tiefeninformation zu bestimmen. Unter Verwendung dieser Pixelkoordinate und der ermittelten Tiefeninformation kann anschließend ein dreidimensionaler Punkt im Kamerakoordinatensystem berechnet werden. Die Koordinaten dieses Punktes werden dabei in der Einheit Meter angegeben.

Das Alignment ist nötig damit Pixel-Koordinaten aus dem RGB-Bild im Tiefenbild auf den zugehörigen Tiefenwert verweisen können. Figure 14 zeigt RGB- und Tiefenbild, jeweils vor (links) und nach (rechts) dem Alignment. Das kleine Kreuz an der Fingerspitze Zeigt auf die gleichen Pixelkoordinaten. Es lässt sich erkennen, dass erst nach dem Alignment (rechtes Doppelbild) die Fingerspitze an der gleichen Pixelposition dargestellt wird.

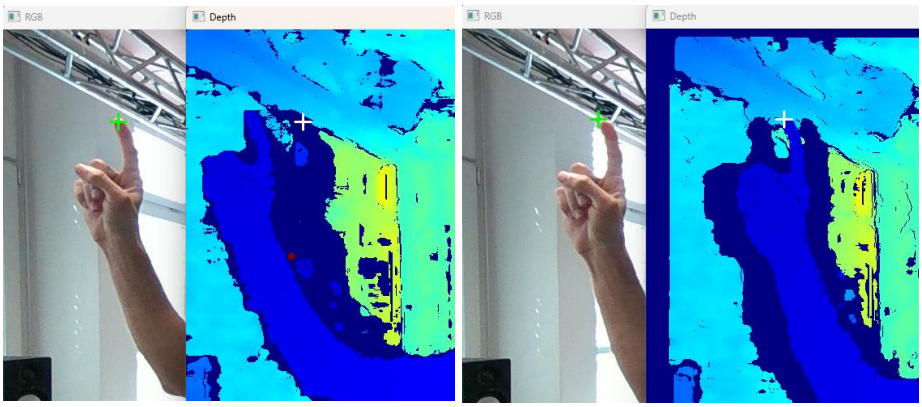



**Figure 14 - RGB- und Tiefenbild mit gleicher Positionsreferenz vor und nach der Kalibrierung (Alignment)**

*Datenaufzeichnung*

Für die Aufzeichnung der Kameradaten stellt die Library eine entsprechende Aufnahmefunktion bereit. Dabei können die von der Kamera erfassten Datenströme, beispielsweise Tiefen- und Farbbilder, zusammen mit Geräte-, Sensor- und Streaminformationen, Metadaten sowie Kalibrierungs- und Extrinsikdaten gespeichert werden. Hierfür existieren das ältere bag-Format [48], das sich am ROS-1-Rosbag-Format orientiert, und das neuere db3-Format [49], das auf dem ROS-2-rosbag2-Format und einer SQLite-Datenbank basiert. Mehr zu ROS in Abschnitt 2.1. Das db3-Format wurde mit librealsense 2.58.1 eingeführt und ersetzt das .bag-Format. Die älteren .bag-Dateien enthalten unkomprimierte und ungefilterte Daten und können dadurch eine Größenordnung von etwa 100 MB pro Sekunde Aufnahmezeit erreichen. Das db3-Format kann kompremiert werden. Das Beispiel [50] zeigt das hier etwa 30 MB pro Secunde möglich sind. Bestehende .bag-Aufnahmen können mit der *librealsense*-Library in .db3-Datein konvertiert werden. [49]

Die Tiefensensoren der RealSense Kameras haben eine bestimmte Distanzspanne (Range) in welcher diese arbeiten können, siehe [51]. Pixel mit distanzen auserhalb dieser Range bekommen den wert 0 zugewiesen, schwarz in Figure 15. Das Tiefenbild in Figure 15 zeigt deutliches Bildrauschen in den bereichen auserhalb der Sensorrange. Zu erkennen ist dies an dem bunten Punkten in dem ansonsten schwarzen Bereichen.

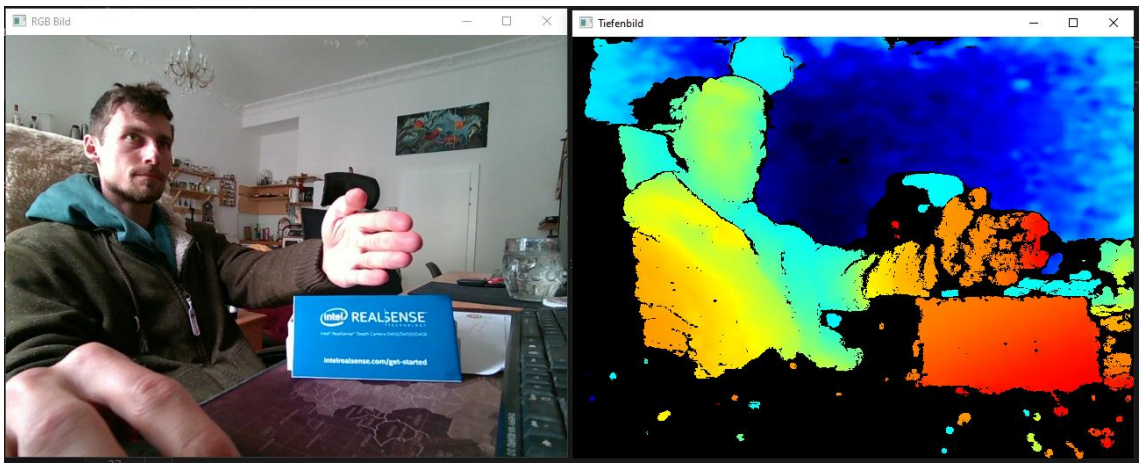



**Figure 15 - Bilder der Intel RealSense D455 Tiefenkamera mit Rauschen im Tiefenbild**

*Persönliche Anmerkung zu DB3*

Beim Alignment der Farb- und Tiefenbilder trat bei der Verwendung von Aufzeichnungen im DB3-Format Probleme auf. Aufgrund einer fehlerhaften Speicherung beziehungsweise Wiederherstellung der Kamera-Intrinsics im DB3-Datenformat kann das Alignment mit pyrealsense2 ab Version 2.58 nicht korrekt durchgeführt werden. Daher wird in diesr Arbeit pyrealsense2 in der Version 2.55.1.6486 verwendet, die noch das ältere BAG-Format nutzt.

## 2.13 Convolutional Neural Networks

*Einordnung von CNNs*

Convolutional Neural Networks (CNNs) sind eine spezielle Form künstlicher neuronaler Netze. Neuronale Netze bestehen aus miteinander verbundenen Verarbeitungseinheiten,

die aus Eingabedaten schrittweise eine Ausgabe berechnen. Während allgemeine neuronale Netze für unterschiedliche Datentypen eingesetzt werden können, sind CNNs besonders für gitterförmig strukturierte Daten wie Bilder geeignet. Sie werden deshalb häufig in der Bildverarbeitung und Computer Vision eingesetzt. [52]

### *Grundlegende Verarbeitung*

Die wichtigste Operation eines CNNs ist die Faltung (Convolution). Dabei wird ein kleiner lernbarer Filter über das Bild bewegt. Der Filter untersucht jeweils einen lokalen Bildbereich und berechnet daraus einen neuen Wert. Auf diese Weise entsteht eine Merkmalskarte, die beispielsweise anzeigt, an welchen Stellen im Bild bestimmte Kanten oder Muster vorkommen. Solche Filter werden während des Trainings automatisch angepasst. [52]

Sogenannte Aktivierungsfunktionen entscheiden, welche berechneten Werte in welcher Form an die nächste Schicht weitergegeben werden. Dadurch ist das Netzwerk in der Lage, auch nichtlineare Zusammenhänge gut zu modellieren. Eine häufig verwendete Aktivierungsfunktion ist beispielsweise *ReLU*, bei dieser werden negative Werte auf null gesetzt und positive Werte beibehalten. [52]

Pooling-Schichten reduzieren die räumliche Größe von Merkmalskarten. Dabei werden Informationen aus benachbarten Bereichen zusammengefasst, beispielsweise indem jeweils der größte Wert ausgewählt wird. Dadurch sinkt der Rechenaufwand und das Netzwerk wird weniger empfindlich gegenüber kleinen Verschiebungen eines Merkmals im Bild. [52]

### *Tiefe neuronaler Netze*

Die Tiefe eines neuronalen Netzwerks bezeichnet die Anzahl seiner aufeinanderfolgenden Verarbeitungsschichten. In einem tiefen CNN werden zunächst einfache Strukturen wie Kanten erkannt. Nachfolgende Schichten kombinieren diese Informationen zu komplexeren Mustern und abstrakteren Merkmalen. Eine größere Tiefe kann dadurch leistungsfähigere Repräsentationen ermöglichen, erschwert jedoch häufig das Training des Netzwerks. [53]

Sollen besonders tiefe Netze trainiert werden, kommen sogenannte Skip-Connections zum Einsatz. Hierbei wird die Verarbeitung bestimmter Informationen in manchen Netzwerkschichten kurzgeschlossen bzw. übersprungen. Mehr dazu unter [54].

### *Räumliche und zeitliche Informationen*

Ein gewöhnliches 2D-CNN verarbeitet einzelne Bilder und erfasst daher hauptsächlich räumliche Informationen. Für Videodaten können spatiotemporale CNNs eingesetzt werden, die sowohl Positionen als auch zeitliche Veränderungen berücksichtigen. Dazu gehören beispielsweise 3D-CNNs, deren Filter zusätzlich über mehrere aufeinanderfolgende Frames bewegt werden und dadurch räumliche sowie zeitliche Muster direkt aus Videosequenzen lernen. [55]
Für die sehr rechenaufwändigen 3D-CNNs werden auch alternative Netzstrukturen wie ConvLSTM [56], das ist eine Erweiterung des LSTM (Long Short-Term Memory) Netzes für bildähnliche Dateneingaben, und TSM-Netze eingesetzt.

### *Temporal Shift Module*

Kurz TSM bietet eine rechen effiziente alternative zu einem 3D-CNN. Es verschiebt Informationen zwischen 2D-CNNs um ihnen einen Lerneffekt entlang der zeitlichen Achse zu geben, für was andernfalls das rechenintensivere 3D-CNN genutzt werden würde. [57]

## 2.14 Transformer

Ein Transformer ist eine neuronale Netzwerkarchitektur, die auf dem Attention-Mechanismus basiert. Im Gegensatz zu CNNs und rekurrenten Netzen verwendet er weder Faltungen noch Rekurrenz. Transformer berechnen mithilfe von Self-Attention, welche Elemente einer Eingabesequenz für die Verarbeitung besonders relevant sind. Dadurch können Abhängigkeiten zwischen weit voneinander entfernten Positionen direkt modelliert werden. Im Gegensatz dazu verschiebt ein CNN einen Filter über eine Bildstruktur und verarbeitet dabei zunächst lokale Bereiche. Transformer eignen sich daher insbesondere dafür, Beziehungen zwischen einzelnen Merkmalen einer Eingabesequenz zu modellieren und werden häufig in mehreren Schichten hintereinander eingesetzt. [58]

## 2.15 Menschliche Auditive Richtungswahrnehmung

### *Minimum Audible Angle*

Das Paper [59] untersucht den minimalen Audio-Winkel, in Englisch *minimum audible angle* oder kurz MAA. Genauer gesagt wird hier untersucht, wie klein die Richtungsänderung eines Geräusches sein kann, damit diese von einem Menschen noch wahrgenommen wird. Unter den kontrollierten Laborbedingungen der Studie lagen die Wahrnehmungsschwellen im Mittel bei 0,97° für Richtungsunterschiede im Azimut und bei 3,65° für Unterschiede in der Elevation.

### *MAA bei Ambisonics*

Etwas dichter an der Thematik der vorliegenden Arbeit, beschreibt das Paper [59] den MAA. Es beschreibt, wie die menschliche auditive Richtungswahrnehmung sich mit höherer Ambisonics Ordnung verhält.

Ambisonics ist ein gängiges Verfahren zur Aufnahme, Repräsentation und Wiedergabe dreidimensionaler Schallfelder. Im Gegensatz zu kanalbasierten Verfahren ist die Ambisonics-Repräsentation nicht an ein festes Lautsprecherlayout gebunden, sondern wird für die jeweilige Lautsprecher- oder Kopfhörerkonfiguration dekodiert. [60]

Der nachfolgenden Figure 16 kann entnommen werden, dass mit einer hochwertigen künstlichen Klangrekonstruktion durch eine hohe Ambisonics-Ordnung ein MAA von unter 5° Azimut möglich ist. Für die Wahrnehmung in Elevationsrichtung ist von einem höheren MAA auszugehen, da wie anfangs beschrieben die menschliche Richtungswahrnehmung in dieser Ebene weniger genau ist.

Zudem zeigt die Abbildung, dass die Richtungswahrnehmung mit zunehmender seitlicher und rückwärtiger Position der Schallquelle ungenauer wird. Darüber hinaus betont die Arbeit, dass die Art des Schallreizes und dessen Frequenzspektrum einen wesentlichen Einfluss auf die Genauigkeit der Richtungswahrnehmung haben.

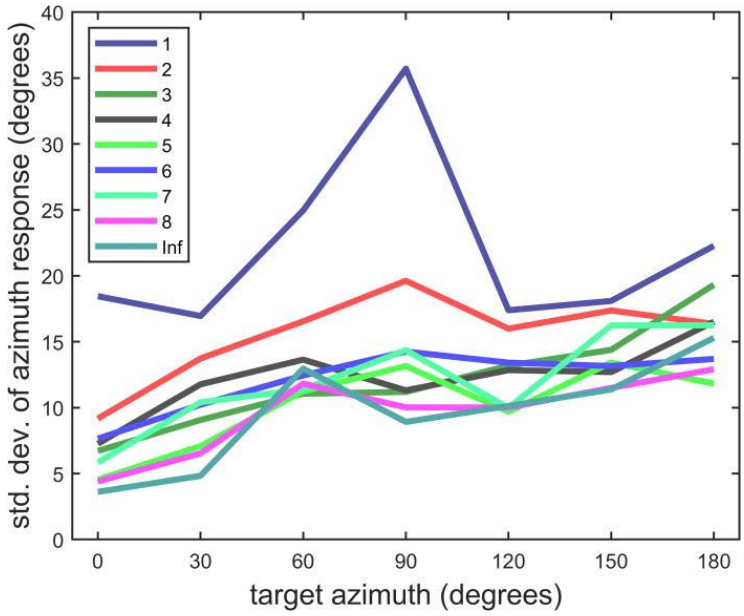



**Figure 16 - menschliche akustische Richtungsauflösung (MAA) in Abhängigkeit der Ambisonics-Ordnung [59]**

## 2.16 Sichtfeld Berechnungen

Ein rechteckiges Sichtfeld beispielsweise das Sichtfeld, welches sich durch Kameralinse und -sensor ergibt, kann in drei Richtungen gemessen werden: horizontal, vertikal und diagonal. In Englisch heißen diese Kamerasichtfelder: *horizontal field of view*, *vertical field of view* und *diagonal field of view*. Im Folgenden werden diese abgekürzt mit: *hFOV*, *vFOV* und *dFOV*.

Diese FOVs werden meist durch den Kameraöffnungswinkel in Grad angegeben. Hersteller von Kameras geben das Sichtfällt meist als vFOV + hFOV oder als dFOV + Seitenverhältnisse an. Da sich die Herstellerangabe nicht aussuchen lässt kann es sinnvoll sein, FOV-Werte ineinander umzurechnen.

Da es sich bei einem Kamerabild um ein Rechteck handelt und die diagonale, ein Rechteck in zwei rechtwinklige Dreiecke teilt, ergibt sich der Zusammenhang zwischen Bildhöhe, -breite und -diagonale über den Satz des Pythagoras [61]. Dieser zusammenhang lässt sich nur dann direkt auf ein Kamerasichtfält anwenden, wenn mit linearen Längenmaßen gerechnet wird.

Die Proportionalität zwischen Winkelmaß und linearem Längenmaß eines Sichtfeldes, bildet die Tangens-Funktion. Für abselute Längenwerte muss hierbei ein Abstand zur Kamera angegeben werden. Siehe Formel 6:

**Formel 6 - FOV Umrechnung von Winkel zu Länge [62]**

$$FOV_{len} = 2 \cdot \mathrm{t} \cdot \tan\left(\frac{FOV_{ang}}{2}\right)$$

$$mit$$

$$t\text{: } Abstand\ zur\ Kamera$$

Der Zusammenhang der Winkelgröße von vFOV, hFOV und dFOV ergibt sich daraus wie folgt:

**Formel 7 - dFOV Winkel aus vFOV und hFOV**

$$dFOV = 2 \cdot arctan\left(\sqrt{tan^2\left(\frac{vFOV}{2}\right) + tan^2\left(\frac{hFOV}{2}\right)}\right)$$

Der Zusammenhang zwischen den Seitenverhältnissen und Winkelangaben eines Sichtfeldes, ist in Formel 8 dargestellt:

**Formel 8 - vFOV, hFOV Winkel und Seitenverhältnisse**

$$vFOV = 2 \cdot \arctan\left(\frac{2 \cdot \tan\left(\frac{\mathrm{hFOV}}{2}\right) \cdot \frac{\mathrm{w}}{\mathrm{h}}}{2}\right)$$

$$mit$$

$$h : Bildhöhe$$

$$w : Bildbreite$$

## 2.17 Weitere Mathematische Grundlagen für diese Arbeit

*Umrechnung von Koordinatensystemen*

Für die Umrechnung eines zwei- oder dreidimensionalen Koordinatensystems in ein anderes, muss die Rotation und die Translation zwischen diesen Systemen bekannt sein. Auf der Basis der OpenCV-Dokumentation [63] entstand die Formel 9, diese beschreibt die Umrechnung eines 3D-Punktes von einem Kamera- in ein Raumkoordinatensystem.

**Formel 9 - Kamerakoordinaten zu Raumkoordinaten**

$$P_W = R \cdot P_K + T$$

$$mit$$

$$Punkt\ im\ Weldsystem : P_W = \begin{pmatrix} x_w \\ y_w \\ z_w \end{pmatrix}$$

$$Punkt\ im\ Kamerasystem : P_K = \begin{pmatrix} x_k \\ y_k \\ z_k \end{pmatrix}$$

$$\text{Rotationsmatrix} : R = \begin{pmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{pmatrix}$$

$$Translationsvektor : T = \begin{pmatrix} t_x \\ t_y \\ t_z \end{pmatrix}$$

Die Translation beschreibt einen Offset jeder Achse, genauer gesagt ist dies die Position des Koordinatenursprungs des Ausgangssystems (Kamera) innerhalb des Zielsystems

(Raum). Beispielsweise die Position der Kamera im Raum. Die Rotation beschreibt den Richtungsvektor für die X-, Y- und Z-Achsen eines Systems im jeweils anderen System. Sie wird durch eine 3 x 3 Matrix dargestellt. Spaltenweise gelesen beschreiben diese drei Vektoren der Matrix die X-, Y- und Z-Achsen des Ausgangssystems als Richtungsvektoren im Zielsystem, also ist $r_{31}$ der Z-Anteil des Raumsystems an der X-Achse des Kamerasystems. Zeilenweise gelesen ergibt sich der umgekehrte Zusammenhang.

### *Berechnung eines Winkels zwischen Punkten oder Vektoren*

Mit einer Arkustangens-2-Funktion lässt sich die Richtung eines zweidimensionalen Vektors berechnen, diese Funktion berücksichtigt Quadranten-Wechsel und Nulldivision [43]. In Python kann diese Funktion mit `math.atan2` [64] genutzt werden. Soll ein Winkel zwischen drei 2D-Punkten (z.B. Bildpunkten) berechnet werden, so führt Formel 10 zu einer Winkelangabe zwischen $\pm$ 180° bzw. $\pm\pi$.

**Formel 10 - Winkelberechnung mit Arcustangenz-2 [65]**

$$\phi_{bc} = \arctan2(y_c - y_b, x_c - x_b)$$
$$\phi_{ba} = \arctan2(y_a - y_b, x_a - x_b)$$
$$\phi = \phi_{bc} - \phi_{ba}$$
$$mit$$
$$a, b, c : 2D\ Punkte$$
$$\phi_{bc}, \phi_{ba} : Richtungswinkeln\ der\ Vektoren$$
$$\phi : Winkel\ zwischen\ den\ zwei\ Vektoren\ bzw. drei\ Punkten$$

Für eine Winkelberechnung zwischen zwei 2D- oder 3D-Vektoren kann Formel 11 genutzt werden. Diese berücksichtigt keine Quadrantenwechsel und enthält damit keine Informationen über die Öffnungsrichtung des Winkels. Sie gibt Winkelwerte zwischen 0° und 180° zurück. Die Vektoren $\vec{a}$ und $\vec{b}$ könnten auch, wie in Formel 10, als Resultat aus der Differenz zwischen drei Punkten dargestellt werden.

**Formel 11 - Winkelberechnung mit Arcuscosinus [66]**

$$\phi = \arccos\left(\frac{\vec{a} \cdot \vec{b}}{|\vec{a}| \cdot |\vec{b}|}\right)$$
$$mit$$
$$\vec{a}, \vec{b} : 2D\ oder\ 3D\ \ Vektoren$$

# 3 Stand der Technik

Dieses Kapitel gibt einen Überblick über ausgewählte wissenschaftliche Papers und dokumentierte Projekte aus den Bereichen Gestenerkennung und Gestensteuerung. Betrachtet werden verschiedene Verfahren zur Handgestenerkennung, Poseschätzung sowie zur Steuerung von Robotik- und Multimediasystemen. Insbesondere im Bereich der Robotik konnten sowohl zahlreiche aktuelle Arbeiten als auch Projekte mit einer hohen inhaltlichen Schnittmenge zur vorliegenden Arbeit identifiziert werden. Dazu zählen insbesondere Ansätze zur Robotersteuerung mittels Zeigegesten und zur räumlichen Abschätzung menschlicher Bewegungen. Im Mittelpunkt stehen die eingesetzten Methoden, Sensoren und technischen Umsetzungen. Abschließend werden die gewonnenen Erkenntnisse im Hinblick auf die vorliegende Arbeit bewertet.

## *3.1 Handgestenerkennung*

### *Verschiedene Verfahren*

Die Übersichtsarbeit [67] beschäftigt sich mit Verfahren zur Handgestenerkennung u.a. zur Erkennung von Gebärdensprache. Hierzu wurden 256 Studien genauer betrachtet. Die untersuchten Ansätze werden grundsätzlich in visuell basierte, sensorbasierte und hybride Verfahren unterteilt. Visuell basierte Verfahren basieren auf Kameradaten, beispielsweise RGB-Bilder, Tiefenbilder oder eine Kombination aus RGB- und Tiefeninformationen (RGB-D-Bilder). Sensorbasierte Verfahren erfassen die Hand- und Fingerbewegungen dagegen mit tragbaren Geräten, insbesondere Datenhandschuhen, die beispielsweise Flexionssensoren, Beschleunigungssensoren, Gyroskope oder elektromyografische Sensoren enthalten. Hybride Verfahren kombinieren Kameradaten mit solchen zusätzlichen Sensorsignalen, um sowohl die visuelle Erscheinung als auch die Bewegung, Orientierung oder Fingerbeugung der Hand zu berücksichtigen.

### *Visuellen Sensorik*

Hinsichtlich der visuellen Sensorik unterscheiden die Autoren von [67] insbesondere zwischen RGB und RGB-D-Daten. RGB-Kameras liefern Farb- und Intensitätsinformationen und sind kostengünstig sowie flexibel einsetzbar. Da sie jedoch keine direkte Tiefeninformation enthalten, müssen räumliche Lage und Handbewegung aus der Bildprojektion oder aus zeitlichen Bildfolgen geschätzt werden. Die zusätzliche Distanzinformation von RGB-D-Sensoren, macht die Erkennung stabiler. Es können Handposition, Fingeranordnung und dreidimensionale Bewegungen zuverlässiger erfasst werden. Dies ist insbesondere für Anwendungen relevant, bei denen nicht nur eine Geste erkannt, sondern auch eine räumliche Richtung oder eine 3D-Bewegung interpretiert werden soll.

### *Der Trend*

Bei kamerabasierten Verfahren werden zunächst häufig klassische Bildverarbeitungsschritte eingesetzt. Dazu gehören beispielsweise die Vorverarbeitung, die Segmentierung der Hand, die Extraktion von Konturen, Bewegungsmerkmalen oder geometrischen Handpunkten sowie die Reduktion störender Hintergrundinformationen. Die extrahierten Merkmale können anschließend entweder rein mit regelbasierter Geometrie oder auch mit klassischen maschinellen Lernverfahren (ML) ausgewertet werden, wie etwa:

- Support-Vector-Machines [68] zur Klassifikation von Merkmalsvektoren
- Hidden-Markov-Modelle [69] zur Analyse zeitlicher Sequenzen
- Entscheidungsbäumen
- statistischen Klassifikatoren

Moderne Ansätze verwenden dagegen Deep-Learning-Modelle, die relevante Merkmale direkt aus den Bild-, Tiefen- oder Bewegungsdaten lernen. Für statische Gesten kommen hierbei CNN-basierte Modelle zum Einsatz, während dynamische Gesten zusätzlich eine zeitliche Komponente betrachten, Modelle wie LSTM-Netze und 3D-CNNs werden genutzt. Siehe dazu auch Abschnitt 2.13 - *Convolutional Neural Networks*. Das Review [67] beschreibt insbesondere eine Entwicklung hin zu dynamischen Gesten und 3D-basierten Pose-Repräsentationen, die über die reine Klassifikation einzelner statischer Handformen hinausgehen.

### *Genauigkeit*

Das Review berichtet von Genauigkeiten aus unterschiedlichen Einzelstudien. Diese beruhen sich jeweils auf eigenen Datensätzen, Gestenklassen, Testpersonen und Evaluationsbedingungen. Viele dieser Studien berichten von Genauigkeiten über 90%. Die höchsten Genauigkeiten werden bei einer kleinen Anzahl vordefinierter Gesten, kontrollierten Aufnahmebedingungen und selbst erstellten Datensätzen erzielt. Bei personenunabhängigen Tests, bei denen die Testpersonen nicht bereits im Trainingsdatensatz enthalten sind, fallen die Ergebnisse typischerweise niedriger aus.

### *Einfluss auf die Genauigkeit*

Die Genauigkeit des Erkennungsverfahrens hängt, neben der verwendeten Erkennungstechnik auch von anderen Faktoren ab. Wesentlichen Einfluss haben die Anzahl und die Ähnlichkeit der Gesten, die Größe und Qualität des Datensatzes, die Anzahl und Variation der Testpersonen, der Abstand zur Kamera, ebenso wie Umgebungseinflüsse: die Beleuchtung, der Hintergrund, Verdeckungen. Zuletzt beeinflusst auch die Frage, ob Gesten statisch oder dynamisch erkannt werden sollen. Ein weiterer zu betrachtender Punkt bezüglich der Genauigkeit ist ihre Generalisierbarkeit: Ein Modell kann auf einem kontrollierten Datensatz eine sehr hohe Genauigkeit erreichen, aber bei unbekannten Personen oder veränderten Umgebungsbedingungen deutlich schlechter funktionieren. Das Review weist daher darauf hin, dass hohe Genauigkeitswerte allein keine ausreichende Aussage über die Robustheit eines Gestenerkennungssystems darstellen.

### *Gestenerkennung in der Gebärdensprache*

Das Paper [67] sieht Handgestenerkennung als zentrales Mittel zur Verbesserung der Kommunikation hörgeschädigter Menschen. Da Handgesten ein wesentlicher Bestandteil der Gebärdensprache sind, sollen Erkennungssysteme Gebärdensprache technisch erfassen und beispielsweise in Text oder Sprache übersetzen. Dadurch kann die Kommunikation zwischen hörgeschädigten und hörenden Personen erleichtert werden. Zugleich benennt das Paper die kontinuierliche Gestenerkennung und die begrenzte Verfügbarkeit großer Datensätze als zentrale Herausforderungen für den praktischen Einsatz.

### *3.2 Visuellen Handgestenerkennung*

Das Survey [70] untersucht 137 Arbeiten zur visuellen Handgestenerkennung und gibt einen Überblick über aktuelle Verfahren, Eingabedaten und Forschungsrichtungen. Allgemein überschneidet es sich zu großen Teilen mit der zuvor genannten Übersichtsarbeit [67] und schlussfolgert teils die gleichen Problemstellungen und Entwicklungstendenzen. Folgende Themenbereiche werden herausgearbeitet:

- Klassifikation von Handgesten
- Schätzung der Handpose und der räumlichen Gelenkpositionen
- Gebärdenspracherkennung
- Rekonstruktion von Hand oder Körper in 3D
- multimodale Fusion, beispielsweise aus RGB- und Tiefendaten
- echtzeitfähige Gestenerkennung

Das Paper unterscheidet klar zwischen Gesten Klassifizierung und Gesten Schätzung (in Englisch gesture estimation):

- Bei der Klassifikation wird eine Bild- oder Videosequenz einer vordefinierten Klasse zugeordnet, beispielsweise „Greifen“, „Winken“ oder „Daumen hoch“.
- Bei der Schätzung werden die räumlichen Eigenschaften der Geste bestimmt, beispielsweise 2D- oder 3D-Gelenkpositionen, Handorientierungen und Bewegungsbahnen.

In den Gestenerkennungsverfahren ist mittlerweile der Einsatz von Elementen aus dem KI-Bereich üblich. Hierbei unterscheiden die Autoren zwischen klassischen ML und neuronalen Deep-Learning Ansätzen. Klassische ML-Methoden sind. Die Entwicklung geht jedoch mehr in die Richtung der moderneren neuronalen Ansätze, hierbei sind folgende Methoden genannt:

- CNNs zur Merkmalsextraktion aus Bildern.
- LSTM-Netze und 3D-CNNs zur Modellierung zeitlicher Bewegungsabläufe.
- Graph Convolutional Networks, die Hand- oder Körper-Skelette als Graphen verarbeiten.
- Transformer-Modelle zur Verarbeitung einzelner räumlicher und zeitlicher Abhängigkeiten (siehe 2.14 - *Transformer*)
- Hybride Modelle, die beispielsweise RGB-Daten mit Tiefeninformationen und Skelettvektoren kombinieren.

Eine weitere Verfahrensunterteilung im Paper betrifft die Eingangs- bzw. Zwischendaten. Es wird zwischen box-/filterbasierten und skelettbasierten Ansätzen unterschieden. Bei box-/filterbasierten Verfahren wird die Hand zunächst mithilfe einer Bounding Box oder Region of Interest (ROI) lokalisiert. Anschließend werden aus diesem Bildausschnitt Merkmale wie Form, Textur oder Bewegungsmuster extrahiert, um die Geste zuzuordnen. Diese Verfahren sind direkter und meist mit weniger Rechenleistung umzusetzen und eignen sich insbesondere für 2D-Bilddaten. Sie reagieren jedoch empfindlicher auf Beleuchtungsänderungen, unruhige Hintergründe und Variationen im Erscheinungsbild der Hand.

Skelettbasierte Verfahren modellieren die Hand hingegen als Struktur miteinander verbundener Gelenkpunkte und Segmente. Aus diesen können Fingerbewegungen, Gelenkwinkel, Abstände und Bewegungsbahnen detaillierter analysiert werden. Sie sind besonders für Aufgaben geeignet, bei denen die räumliche Orientierung und Bewegung der Hand relevant sind, beispielsweise für die Schätzung eines Armwinkels. Dafür benötigen sie jedoch eine zuverlässige Gelenkpunkterkennung im Vorfeld und sind in der Regel rechenaufwendiger.

### *3.3 Deep-Learning-basierte Erkennung dynamischer Gesten*

Im Review [71] geben die Autoren einen Überblick über Verfahren zur Erkennung dynamischer Gesten aus Videosequenzen. Im Mittelpunkt steht die Frage, wie neben den räumlichen Merkmalen einzelner Bilder auch die zeitliche Entwicklung einer Bewegung berücksichtigt werden kann.

Die Autoren unterscheiden insbesondere zwischen Two-Stream-CNNs, 3D-CNNs und LSTM-basierten Verfahren. Two-Stream-CNNs verarbeiten räumliche Bildinformationen und Bewegungsinformationen in getrennten Netzwerken. Die Bewegungsinformationen können beispielsweise durch Optical Flow (bildliche Bewegungsvektoren) aus aufeinanderfolgenden Videoframes berechnet werden. Beide Merkmalsströme werden anschließend zur Gestenerkennung kombiniert.

3D-CNNs erweitern die Faltung um eine zeitliche Dimension. Siehe dazu auch Abschniutt 2.13 - *Convolutional Neural Networks*. 3D-CNNs sind hier am rechenintensivsten.

LSTM-basierte Verfahren verarbeiten dagegen eine zeitliche Folge von Merkmalen und modellieren Abhängigkeiten zwischen verschiedenen Zeitpunkten einer Bewegung. Ihr Rechenaufwand ist im Vergleich gering.

Allen drei Ansätzen ist gemeinsam, dass sie die spatiotemporale Struktur einer Geste berücksichtigen. Das Review zeigt damit die Entwicklung von Verfahren, die einzelne Bilder klassifizieren, hin zu Modellen, die Bewegungsabläufe aus Videosequenzen analysieren. Die zusätzliche zeitliche Modellierung erhöht jedoch meist den Rechenaufwand und die Modellkomplexität.

### *3.4 Gestensteuerung in der Robotik*

Die Gestenerkennung bildet ein fundamentales Element der visuellen Mensch-Roboter-Interaktion und ermöglicht es, Roboter über natürliche menschliche Bewegungen zu bedienen. Roboter erfassen menschliche Bewegungen über Videobilder sowie visuelle Sensordaten, teilen diese in Kategorien ein und werten sie als Signale zur Kommunikation aus. Diese Technik schafft eine direkt verständliche, berührungslose Verbindung. Menschen steuern Roboter darüber, deuten auf Objekte oder Richtungen und starten gezielte Handlungen.

Das Paper [72] ist eine Übersichtsarbeit und belegt diesen breiten Einsatz computerbasierter visueller Verfahren innerhalb der Mensch-Roboter-Interaktion und -Kollaboration. Die Autoren betrachteten 3.850 wissenschaftliche Veröffentlichungen und prüften dann 310 Arbeiten genauer. Sie machten dabei die folgenden Anwendungsfelder ausfindig:

- Gesten- und Aktionserkennung von Menschen
- Steuerung von Roboterbewegung im Umfeld von Menschen
- Objektübergabe und andere kollaborative Tätigkeiten
- soziale Mensch-Roboter-Kommunikation
- Lernen durch menschliche Demonstrationen

*Robotersteuerung per Zeigerichtung*

Die Übersichtsarbeit [72] verweist unter anderem auf das Projekt dokumentierende Paper [73], in diesem wird die Implementierung einer Roboter-Gestensteuerung anschaulich beschrieben. Ein selbst fahrender Roboter wird mit den Informationen aus einer Tiefenkamera gesteuert. Eine menschliche Zeigegeste und eine Raumkarte bilden die Grundlage für eine Bewegungsanweisung. Die daten eines Raumscanners werden mittels ROS (siehe 2.1 - *Robot Operating System*) in eine Umgebungskarte der möglichen Bewegungsziele überführt. Die Zeigerichtung wird, unter Berücksichtigung der Position von Menschen und Roboter bzw. seiner Kamera, verarbeitet. Es wird berechnet, wie dicht die Ziele an der Zeigelinie im Raum liegen. Auch wird über eine Rutenplanung (in Englisch Pathfinding) mittels ROS die Erreichbarkeit dieser Ziele bestimmt. Auf der Grundlage dieser zwei Informationen steuert der Roboter dann ein Ziel an. Das folgende Bild zeigt die Interaktion und die Raumkarte.

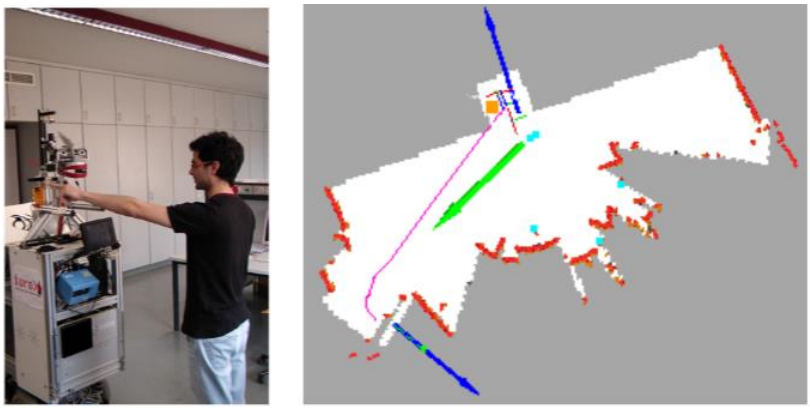


**Figure 17 - Robotersteuerung mit Raumkarte und Zeigerichtung (grün), Bewegungszielen (hellblau), Bewegungsroute (pink), Hindernisse (rot) [73]**

## 3.5 Pose-Schätzung

Das Paper [74] untersucht die Leistungsfähigkeit markerloser, monokularer (nur ein Kamerasensor) Verfahren zur menschlichen Posenschätzung. Ziel der Studie ist es, die Genauigkeit, Präzision und Verarbeitungsgeschwindigkeit verschiedener Open-Source-Modelle für die Analyse menschlicher Bewegungen zu vergleichen. Im Mittelpunkt steht die Frage, inwieweit Posenschätzung aus einzelnen RGB-Kamerabildern etablierte markerbasierte Motion-Capture-Systeme ersetzen oder ergänzen kann.

Die Autoren erstellen einen großen Datensatz annotierter RGB-Aufnahmen, deren Gelenkpositionen mithilfe eines markerbasierten Motion-Capture-Systems erfasst werden.

Verglichen werden insgesamt elf bzw. zwölf monokulare Posenschätzungsmodelle. Dazu gehören direkte 2D- und 3D-Verfahren. Die untersuchten Verfahren werden anhand der Genauigkeit, der Erkennungsrate und der Inferenzgeschwindigkeit bewertet. Untersucht werden:

- OpenPose – schätzt 2D-Körper-Landmarks mehrerer Personen
- AlphaPose – schätzt 2D-Körper-Landmarks einzelner Personen
- BlazePose Local – schätzt lokale 2D-Körper-Landmarks
- BlazePose World – schätzt 3D-Körper-Landmarks
- HRNet – schätzt 2D-Körper-Landmarks
- Detectron2 – schätzt 2D-Körper-Landmarks einzelner Personen
- RTMPose – schätzt 2D-Körper-Landmarks
- RTMW – schätzt 2D- Körper-Landmarks
- RTMO – schätzt 2D-Körper-Landmarks

- MotionBERT – rekonstruiert 3D-Landmarks aus 2D-Landmarks (Lifting)
- PoseFormerV2 – rekonstruiert 3D-Landmarks aus 2D-Landmarks (Lifting)
- MotionAGFormer – rekonstruiert 3D-Landmarks aus 2D-Landmarks (Lifting)

Zusammenfassend kommen die Autoren zu dem Ergebnis, dass monokulare Posenschätzungsverfahren eine kostengünstige und echtzeitfähige Alternative für die allgemeine Bewegungsanalyse darstellen. Für präzise klinische oder biomechanische 3D-Messungen sind sie jedoch nur eingeschränkt geeignet. BlazePose zeichnet sich insbesondere durch die Kombination aus hoher Verarbeitungsgeschwindigkeit und ausreichender 2D- beziehungsweise grober 3D-Posenschätzung aus und ist sozusagen der Testsieger.

## 3.6 Klassifikation einer Greifgeste

Das Paper [75] untersucht, ob Greifgesten und soziale Gesten anhand ihrer Bewegungsmerkmale unterschieden werden können. Dabei wird untersucht, welche Eigenschaften von Hand- und Armbewegungen für die Erkennung und Klassifikation von Gesten relevant sind.

Für die Untersuchung werden die Bewegungen mit einem sensorbasierten Motion-Capture-Handschuh erfasst. Dadurch können dreidimensionale Positionen von Punkten an Hand, Fingern und Arm bestimmt werden, siehe Figure 18. Aus diesen Skelettdaten werden verschiedene kinematische Merkmale abgeleitet, darunter Position, Geschwindigkeit, Beschleunigung, Bewegungsrichtung und Veränderungen der Fingerposition.

Die Autoren vergleichen die Einschätzungen menschlicher Versuchspersonen mit verschiedenen Verfahren des Machine Learnings. Die Ergebnisse zeigen, dass sich Greifgesten anhand der untersuchten Bewegungsmerkmale von Nicht-Greifgesten unterscheiden lassen. Dabei liefern insbesondere die Position und Bewegung der Finger relevante Informationen. Auch die untersuchten Verfahren des maschinellen Lernens können die von den Versuchspersonen vorgenommene Einteilung weitgehend nachvollziehen. Die Klassifikation sozialer Gesten erweist sich dagegen als schwieriger, da deren Bedeutung stärker vom jeweiligen Kontext abhängt.

Die Arbeit zeigt damit, dass und wie kinematische Merkmale von Hand und Fingern für die Erkennung von Greifgesten verwendet werden können. Ihre KI-Klassifikatoren erreichen eine Genauigkeit von über 90% bei der Erkennung von Greifgesten.

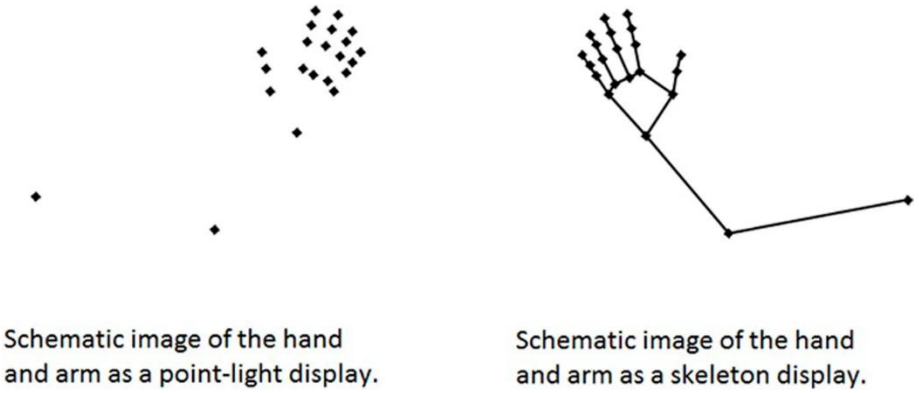



**Figure 18 - Landmarks des Motion-Capture-Systems [75]**

## 3.7 Gestensteuerung von Multimediasystemen

In Multimediasystemen können Gesten beispielsweise zur Steuerung einer Wiedergabe, zur Anpassung von Lautstärke oder zur Bedienung einer Benutzeroberfläche dienlich sein.

Der wissenschaftliche Artikel [76] untersucht eine gestenbasierte Steuerung für eine Media-Player-Anwendung. Das System erkennt Handgesten und ordnet diese den Steuerfunktionen des Players zu. Die Autoren vergleichen die Gestensteuerung mit Maus und Tastatur hinsichtlich Erkennungsgenauigkeit, physischer Ermüdung, Benutzerfreundlichkeit, pragmatischer Qualität und hedonischer Qualität. Ihre Studie ergibt, dass Gestensteuerung eine funktionsfähige Alternative zu klassischen Eingabegeräten für die Mediensteuerung darstellen kann.

Die erfassten Gesten werden in dem System über Handskelettdaten repräsentiert. Aus diesen Daten werden Merkmale der einzelnen Finger extrahiert. Jeder Finger wird dabei durch einen separaten Transformer (siehe 2.14 - *Transformer*) verarbeitet, um fingerbezogene Eigenschaften zu erfassen. Anschließend erfolgt eine Klassifikation der Gesten. Der Artikel zeigt damit ein konkretes Beispiel für eine landmarkenbasierte, KI-gestützte Gestenerkennung zur Mediensteuerung.

Das Verfahren erreicht in einer Nutzerstudie mit 30 Personen eine Erkennungsgenauigkeit von 91,67 % und einen mittleren Mean Opinion Score, also eine Nutzerbewertung von 4,4 aus 5 Punkten.

## 3.8 Gestensteuerungen mit MediaPipe

*"Vision Robotic Arm Gesture Recognition"*
Im Projekt [66] der University of Modena and Reggio Emilia, werden Ellenbogenwinkel, Schulterwinkel und Handöffnungsgrad rein bildlich erkannt und ausgewertet. Die Armwinkel und der Handöffnungsgerad werden dann auf einen simulierten Roboterarm übertragen, siehe Figure 19. Im Projekt werden unter anderem folgende Komponenten genutzt:

- Python 3 – als Programmiersprache

- MediaPipe – zur Erkennung von Körper- und Hand-Landmarks
- ROS – für die Kommunikation der Programmteile
- Gazebo [77] – zur Robotersimulation
- eine RGB-Webcam – Kamera ist waagerecht und stationär positioniert
- ein Computer mit Linux Ubuntu – als Recheneinheit

Die Gestensteuerung läuft in Echtzeit, erreicht allerdings nur 3-5 FPS auf einem nicht definierten Desktop Computer. Das Projekt ist ausführlich dokumentiert mit Python-Code auf GitHub [66] zu finden.

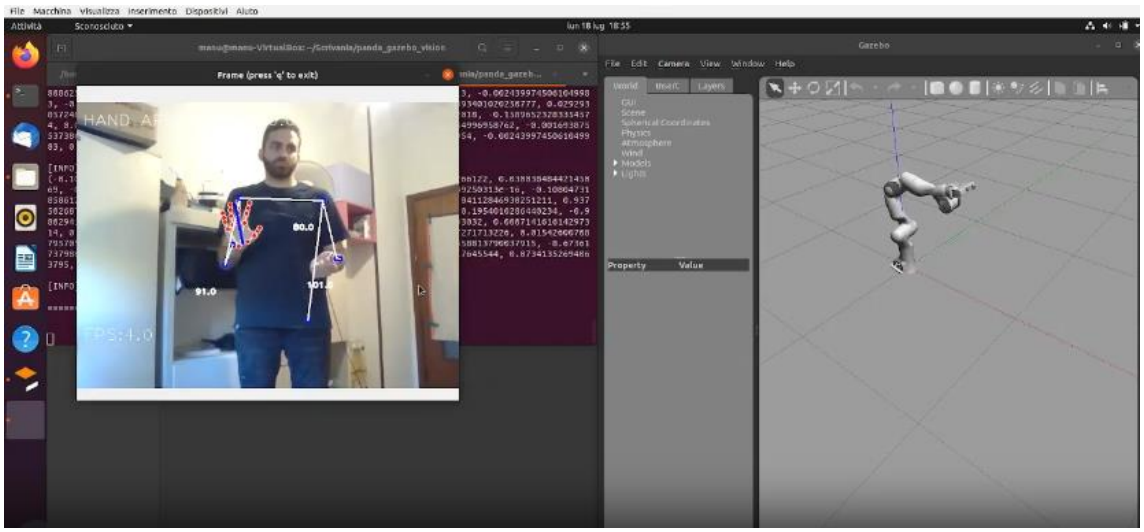


**Figure 19 -** *Vision Robotic Arm Gesture Recognition* **[66]**

*"GestureBot Vision System"*

[78] ist ein Projekt in dem ein Roboter menschliche Körperpositionen, Handgesten und Gegenstände im Raum erkennt, siehe Figure 20. Mittels Körper- und Handgesten lässt sich der Roboter auch steuern. Genutzt werden u.a. folgende Komponenten:

- Python 3 – als Programmiersprache
- MediaPipe – zur Erkennung von Körper- und Hand-Landmarks
- ROS 2 – zur Robotersteuerung, -navigation und Kommunikation
- Raspberry Pi Camera Module 3 (IMX708, 640×480 @ 30 FPS) – als mobile waagerechte RGB-Kamera
- Raspberry Pi 5 - 8 GB mit Ubuntu 24.04 LTS (Nobel) für Raspberry Pi – als Recheneinheit

In dem Projekt wurde die Hand- und Pose-Erkennung von MediaPipe mit 3-7 FPS auf einem Raspberry Pi 5 mit 8 GB RAM betreiben. Das Projekt ist samt Code auf GitHub [78].

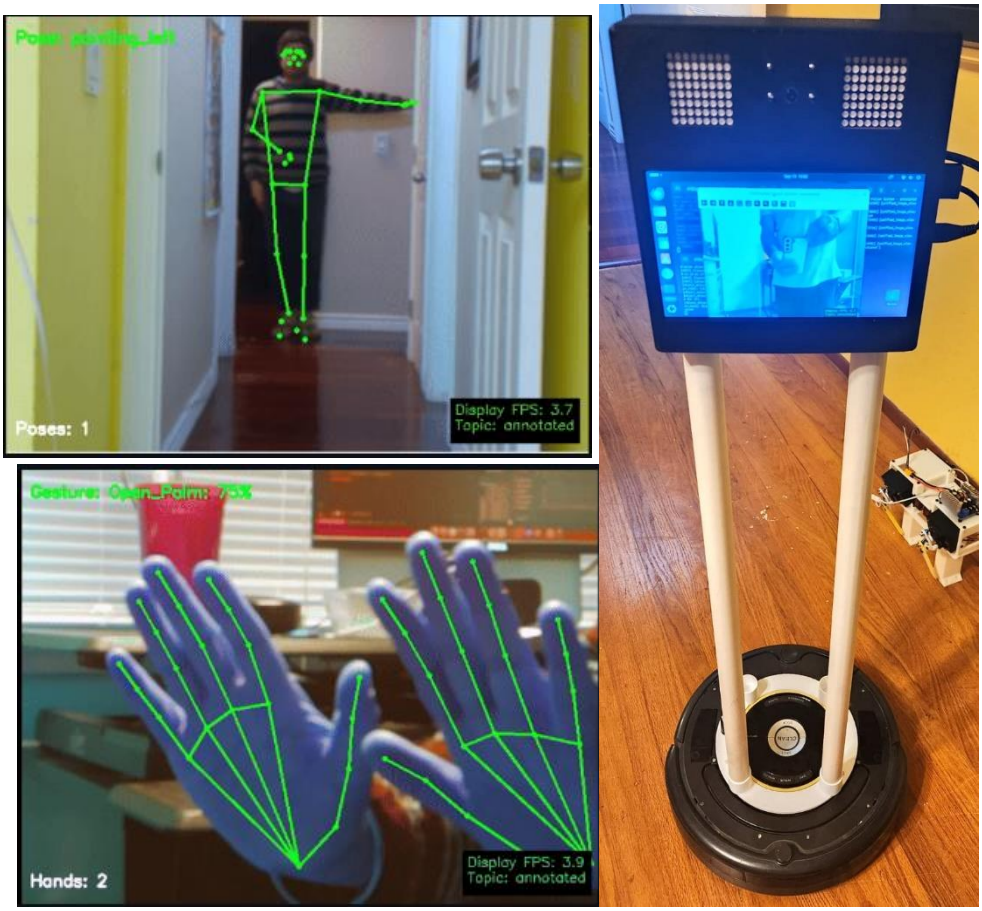



**Figure 20 -** *GestureBot Vision System* **[78]**

*"Control PIPER Arm with Hand Gestures"*

Das Projekt [79] ist von der chinesischen Firma AgileX Robotics [80]. Es dient zur Steuerung einer ihrer Roboterarme mittels Handbewegungen. Es wird eine Greifgeste und die Position sowie Ausrichtung der Hand im Raum ausgewertet, um den Arm zu steuern. Es werden u.a. folgende Komponenten genutzt:

- Python – als Programmiersprache
- MediaPipe – zur Erkennung von Hand-Landmarks
- ROS – zur Robotersteuerung und Kommunikation
- Intel RealSense D435 (640×480 @ 30fps with aligned depth/RGB) – Tiefenkamera ist waagerecht und stationär positioniert
- Orbbec Petrel [81] (640×400 @ 30fps with aligned depth/RGB) – gleichwertiges alternatives Kameramodul zum Einbauen im Endprodukt
- ein Computer mit Ubuntu – als Recheneinheit

Hier wurde eine der genannten Tiefenkameras in der Kombination mit MediaPipe verwendet. Dabei werden die Bildpositionen der Landmarks aus MediaPipe als Referenz genutzt, um die zugehörigen Tiefenwerte aus dem Tiefenbild zu beziehen. RGB- und Tiefenbild der Kamera müssen dafür synchronisiert (aligned) sein, siehe dazu Abschnitt 2.12 - *Intel RealSense Tiefenkamera Treiber*. Die Bildauflösung der Kameras ist in der Projektdokumentation geringer als auf den Herstellerseiten, denn diese kann über die Kameratreiber eingestellt werden. Das chinesische Projekt ist mit Code auf GitHub [82] zu finden. Eine englische Projektdokumentation gibt es auf hackster.io [79].

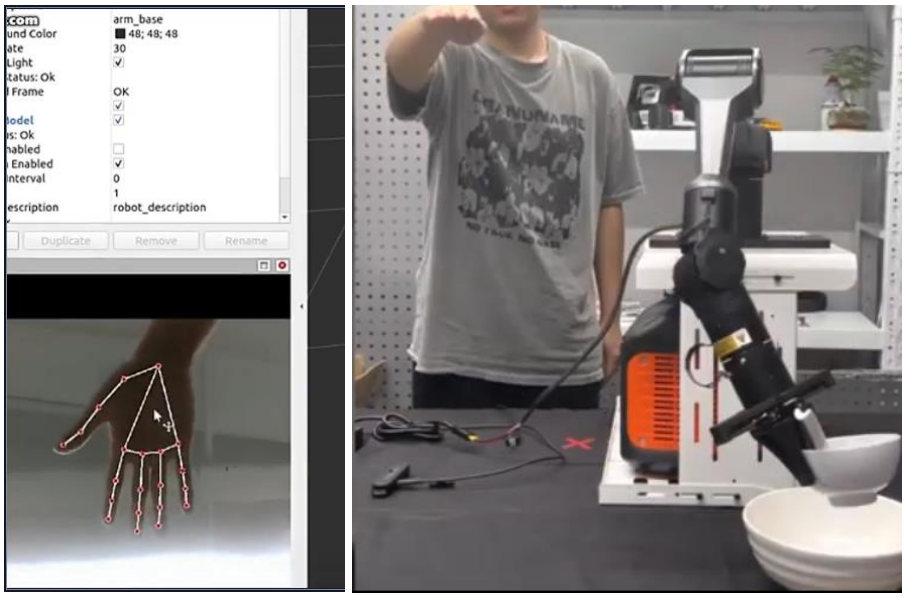



**Figure 21 -** *Control PIPER Arm with Hand Gestures* **[79]**

*"Robotic Arm Control"*

Auch in dem Projekt [65] wird ein Arm-Roboter gesteuert, er soll den menschlichen Arm nachahmen. Mehrere Armwinkel werden erkannt, berechnet und auf den Roboter übersetzt. Die Winkel werden nur im bildlichen 2D-Raum ausgewertet. Auch eine Greifgeste der menschlichen Hand wird auf den Roboter übertragen. Es wird u.a. folgendes genutzt:

- Python 3 – als Programmiersprache
- MediaPipe – zur Erkennung von Körper- und Hand-Landmarks
- eine Webcam – als RGB-Kamera, waagerecht und stationär positioniert
- ein Raspberry Pi – als Recheneinheit

Es gibt keine Angaben zur Performance. Aus dem Code geht hervor, dass es sich um eine Echtzeitanwendung handelt, in welcher das gesamte Webcam Bild zyklisch eingelesen wird. Das Projekt ist übersichtlich dokumentiert und quelloffen auf GitHub [65].

## 3.9 Gestenerkennung ohne KI

Eine Pose- oder Gestenerkennung ohne KI basiert ausschließlich auf klassischer Bildverarbeitung. Dabei werden relevante Merkmale wie Farbflächen, Konturen, Formen oder Bewegungsmuster direkt aus den Bilddaten extrahiert. Anschließend werden diese Merkmale mithilfe geometrischer Berechnungen und festgelegter Regeln ausgewertet, um eine Geste oder Pose zu erkennen.

*"Mimic Robot - ECE5725 Spring 2019 Final Project - Mark Li (mkl53) and Serena Krech (sk2282)"*
In [83] werden zweidimensionale Bewegungen von beiden Armen mittels klassischer Bildverarbeitung und geometrischen Berechnungen erkannt. Das Projekt nutzt keine KI-Anwendungen. Hände und Gesicht werden farblich über eine Hautsegmentierung erkannt. Der Oberkörper wird ebenfalls mittels Farbdifferenz bildlich segmentiert. Dazu wurde im Hintergrund der Person ein weißes Laken gespannt. Aus geometrisch berechneten Verbindungsgeraden dieser segmentierten Bereiche, werden zweidimensionale Armwinkel berechnet. Diese Winkel werden in Echtzeit auf vier Servomotoren eines kleinen Roboters übertragen. Genutzt wird unter anderem:

- Python 3 – Programmiersprache
- OpenCV – Bildverarbeitung
- multiprocessing [84] – Prozessparallelisierung (Rechenoptimierung)
- Cython [85] – Rechenoptimierung, genauer zur Umsetzung von C-Datentypen in Python
- Pi Cam – RGB-Kamera, stationär und waagerecht
- ein Raspberry Pi – Recheneinheit

Zu bemerken ist die hohe Performance von vollen 30 FPS (Grenze der Kamera) auf einem Raspberry Pi. Die Echtzeitperformanz in allen hier gefundenen KI-gestützten Projekten ist deutlich geringer. Um diese Performance zu erreichen, werden Berechnungen mit C-Datentypen gemacht, dafür wird die Python Library Cython genutzt.

Zudem wurde mit *multiprocessing* eine parallele Prozessverarbeitung der einzelnen Frames implementiert. Der rechenintensive Teil der Bildverarbeitung wird dabei auf alle vier Prozessorkerne des Raspberry Pis aufgeteilt. Mehr dazu im Abschnitt 4.6, Absatz *Multithreading (optional bei wenig Rechenleistung)*
.

Das Projekt ist ausführlich dokumentiert und samt Code auf courses.ece.cornell.edu [83] zu finden.

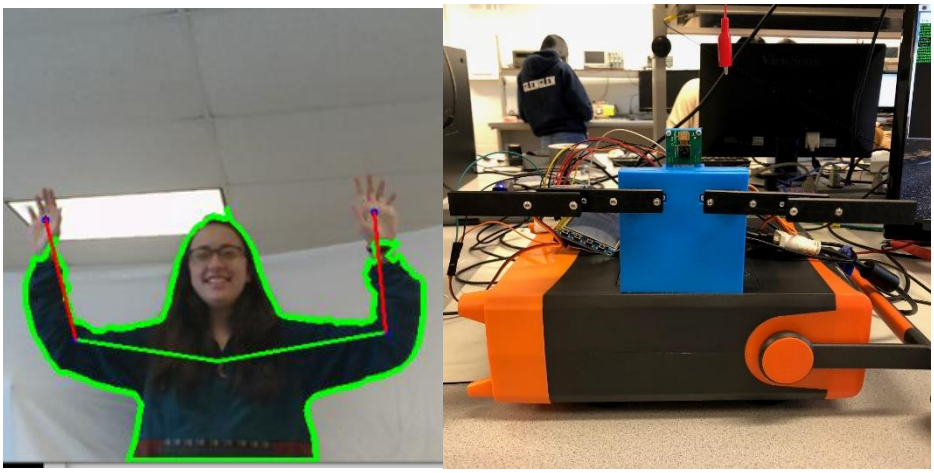


**Figure 22 -** *Mimic Robot* **[83]**

*"Vision-based arm gesture recognition for a long-range human–robot interaction"*
Das Paper [86] beschreibt ein Projekt in welchen klassische Bildverarbeitung zur Gestenerkennung genutzt wird. Erkannt werden vier verschiedene Gesten, teilweise sind diese dynamisch. Siehe Figure 23.

Für die Erkennung wird ein 320 x 240 Pixel großes Livebild einer undefinierten 2D-Kamera verarbeitet. Die Person befindet sich ca. in 5 Meter Entfernung. Die vier Gesten werden laut der Autoren zu über 97 % richtig erkannt. Dafür werden die folgenden Merkmale durch klassische Bildverarbeitung gewonnen:

- Position der Person – Mean-shift-Tracking [87]
- Gesichtslage nah dran – Gesichtserkennung über Merkmale also Muster
- Gesichtslage weit weg – Gesichtserkennung über Hautsegmentierung und Formerkennung
- Omega-Form – typisch menschliche Formen werden abgeglichen
- Bewegungsmuster der Arme – Background Subtraktion (siehe Abschnitt 2.8)
- Form der Armblöcke – Abgleich mit Formvorlagen

Für eine stabile Lokalisierung des Menschen im Bild, werden Gesichtserkennung, Mean-Shift-Farbtracking und die Erkennung der Omega-Körperform kombiniert. Das Gesicht liefert einen Referenzpunkt für die Bestimmung des Oberkörpers und der Armregion. Anschließend werden Position, Form und Bewegung des Arms analysiert, um die definierten Gesten zu klassifizieren.

Die Klassifikation der Gesten erfolgt regelbasiert anhand der erkannten Armregion. Nach der Segmentierung des Arms werden dessen Position, Form und Bewegung über mehrere Videoframes analysiert. Die Stoppgeste wird beispielsweise durch das Anheben und anschließende halten des Armes in einer bestimmten Position erkannt.

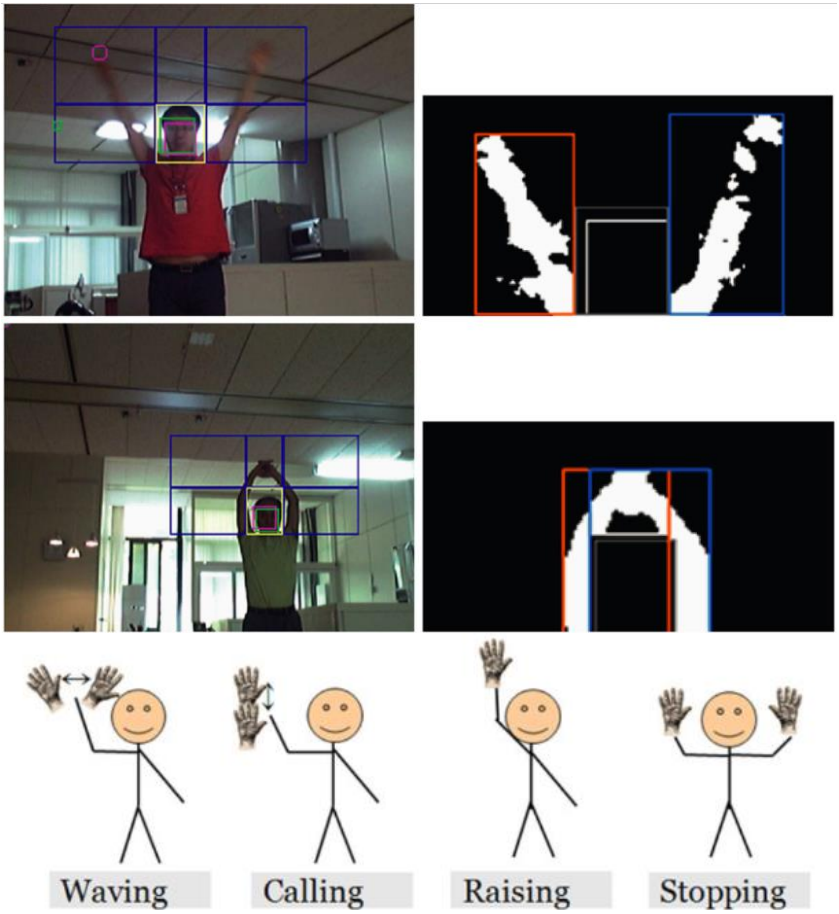



**Figure 23 –** *Vision-based arm gesture recognition* **[86]**

## 3.10 Gestenerkennung mit Tiefenkamera

*"Gesture Controlled Robotic Arm Using Kinect (Simulink Support For Kinect) and Arduino"*

Im Projekt [88] werden an verschiedenen Gelenken des menschlichen Arms, die Winkel erfasst, ausgewertet und simulativ dargestellt, siehe Figure 24. Es werden u.a. die folgenden Komponenten genutzt:

- MATLAB – als Programmiersprache
- MATLAB Simulink – für die Pose-Schätzung
- Microsoft Kinect V1 – als Tiefenkamera, waagerecht und stationär ausgerichtet
- ein Windows 7 Notebook – als Recheneinheit
- ein Arduino – für die Robotersteuerung

Im Projekt konnten, mit einer VGA-Farbbildauflösung (640 × 480p) und einer etwas geringeren Tiefenbildauflösung, die Arm- und Hand-Abwinkelungen, in alle drei

Dimensionen zuverlässig erkannt werden. Das Projekt lässt sich dokumentiert mit seiner Software-Implementation auf der MATLAB Community Seite [88] finden.

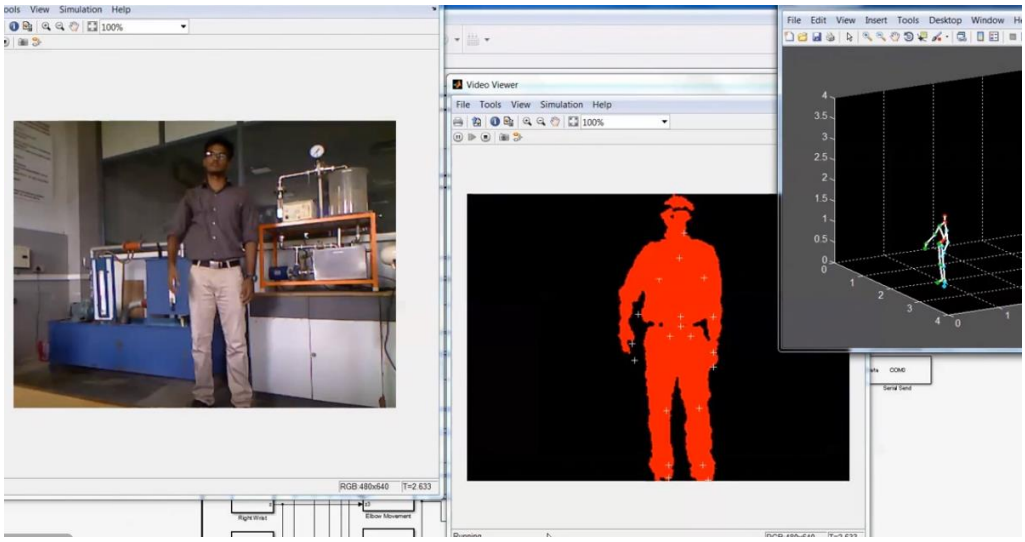


**Figure 24 - Gesture Controlled Robotic Arm [88]**

## 3.11 Haltungsanalyse mit OpenPose

*Sports2D* ist ein open source Programm, welches im Paper [89] dokumentiert ist. Es erkennt eine Vielzahl von Gelenkwinkeln von mehreren Menschen aus Videodaten oder Livestreams, siehe Figure 25. Es kommt ohne grafische Bedienoberfläche (GUI) daher und ist mit den drei gängigen Betriebssystemen: Windows, Linux und Mac OS kompatibel. Die Pese-Schätzung basiert auf OpenPose (siehe 2.5 - *OpenPose*) und ist in Python 3 geschrieben. Die Winkelerkennung ist rein zweidimensional, tiefe wird nicht berücksichtigt. Es bietet, neben der Winkelanalyse noch weitere Funktionen für Datenanalyse und Simulationen auf der Basis der Skeletterkennung und Pose-Schätzung.

Auch gibt es eine Anbindung für ein System, welches mehrere Kameras also Livestreams unterstützt, das soll das Problem mit der fehlenden Tiefenerkennung lösen und Verdeckungen Umgehen.

Das Projekt ist im Webbrowser nutzbar oder nach dem Download auf dem eigenen Computer. Angaben zur Echtzeit Performens gibt es keine. Das Programm ist ausführlich dokumentiert und samt Code auf GitHub [90] zu finden.




**Figure 25 -** *Sports2D* **[89]**

## 3.12 Gestensteuerungen mit Nvidia JetPack

Alle Mikrocomputer der *Jetson* Serie von NVIDIA, werden mit der NVIDIA eigenen Firmware JetPack [91] betrieben, diese basiert auf Linux. JetPack kommt mit eigenen Software Tools daher, welche die Performance verbessern und an das eigene Ökosystem anpassen. Diese beschleunigen insbesondere Berechnungen, die für KI-Anwendungen, Bildverarbeitung und Robotersteuerung genutzt werden.

*"Gestensteuerung eines Roboters auf dem Nvidia Jetson Nano"*
Im Projekt [92] werden verschiedene Handgesten bildlich erkannt. Es wird eine eindimensionale Roboterbewegung mit einigen der Gesten gesteuert, siehe Figure 26. Das Setup besteht u.a. aus:

- Python 3 – als Programmiersprache
- OpenCV – für die Bildverarbeitung
- TSM (siehe auch Abschnitt 2.13) – für neuronale Recheneffizienz und Gestenklassifizierung
- NVIDIA CUDA – zur Hardwarebeschleunigung
- Android Smartphone mit IP Webcam [93] – als Kamera, stationär und waagerecht
- NVIDIA Jetson Nano mit JetPack – als Recheneinheit

Es werden 13 FPS mit einer Echtzeit Gestensteuerung erreicht. Dokumentiert ist das Projekt auf blog.georgmill.de [92].

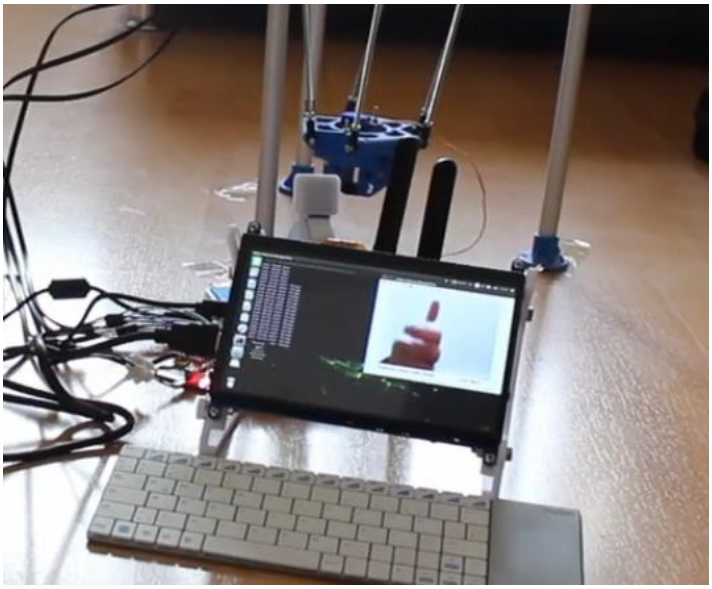


**Figure 26 -** *Gestensteuerung eines Roboters auf dem Nvidia Jetson Nano* **[92]**

*"Echtzeit Bewegungsklassifizierung mithilfe von Zeitfaltungsnetzen auf Basis von Gelenkerkennung mit Tf-Pose"*
Die Implementierung [94] erkennt Bewegungsmuster anhand von Gelenkpunkten und einer zusätzlichen KI-Klassifizierung. Hierzu werden sowohl die Positionen als auch deren zeitliche Änderungen ausgewertet. Für die Klassifizierung kommt ein 3D-CNN zum Einsatz, siehe auch 2.13 - *Convolutional Neural Networks*. Genutzt wird u.a. folgendes:

- Python 3 – als Programmiersprache
- TF-Pose (siehe dazu auch Abschnitt 2.5) – zur Erkennung der Gelenkspunkte
- TensorFlow und Keras [95] – zur Pose-Klassifizierung
- TensorRT – zur Inferenzbeschleunigung
- NVIDIA Jetson TX2 mit Jatpack 4.2 – als Recheneinheit
- eine Intel RealSense – als Tiefenkamera

Die Bewegungs- und Klassifizierungsprozesse werden mit einer Bildrate von etwa 7–8 FPS ausgeführt. Für die Anbindung der RealSense-Kamera an das Python-System war aufgrund der damaligen fehlenden ARM-Kompatibilität (siehe dazu 2.11 - *ABI-Kompatibilität*) ein selbst erstellter Wrapper erforderlich. Das Projekt ist dokumentiert auf GitHub [94].

## 3.13 Relevanz für diese Arbeit

In diesem Abschnitt werden Informationen aus den bisherigen Abschnitten dieses Kapitels (*Stand der Technik*) zusammengetragen. Die Informationen dienen als

Grundlage für die Konzeptionierung der Gestensteuerung der vorliegenden Arbeit. Es soll je ein Konzept mit und ohne KI-Anwendungen erstellt werden.

*Generelles*

Das Survey [70] ermöglicht eine genauere Zuordnung der zwei grundliegenden Gesten Informationen der vorliegenden Arbeit. Es unterscheidet zwischen Gestenklassifizierung und Gestenschätzung. Eine Greifgeste wird durch Klassifizierung erkannt, die Hand ist entweder offen oder geschlossen. Ein Armwinkel bzw. eine dadurch referenzierte Raumrichtung entstammt einer Gestenschätzung. Der Armwinkel ist ein kontinuierlicher Wert berechnet aus mehreren geschätzten Körperpositionen.

Die in der Übersichtsarbeit [67] genannten Genauigkeiten bei der Handgestenerkennung aus diversen anderen Arbeiten lassen auf Erfolg für die Greifgestenerkennung hoffen. Neben der Wahl der Erkennungsmethode wirft die Übersichtsarbeit auch die Frage auf, wie sich die Genauigkeit oder der Erfolg einer Methodik zur Gestenerkennung am besten bestimmen und ggf. messen und vergleichen ließe.

Grundsätzlich ist bei der Recherche nach Pose- und Gestenerkennung aufgefallen, dass der Trend schon seit mehreren Jahren zu KI-gestützten Systemen hin geht. Insbesondere die Verwendung von Deep-Learning-Architekturen wie CNNs und die Repräsentation über skelettbasierte Landmark-Modelle werden viel genutzt, wenn Hände oder der gesamte Körper erkannt werden sollen.

*MediaPipe*

Das Paper [74] beschreibt BlazePose als Testsieger bei der Erkennung von körper-Landmarks über ein RGB-Sensor. Genauigkeit und Performanz wurden betrachtet. BlazePose steht hinter der Pose-Erkennung von MediaPipe.

Ein großer Teil der Projekte, die Hand- oder Körpergesten oder deren Kombination erfolgreich erkennen, nutzt MediaPipe für die zugrunde liegenden Skeletterkennung. Zudem erleichtert die umfangreiche Verfügbarkeit von Informationen und Lösungsansätzen zu MediaPipe im Internet die Fehlerprävention, Fehlersuche und das Debugging.

Diese Aspekte sprechen für die Verwendung von MediaPipe im KI-basierten Konzept.

Das Projekt [65] nutzt MediaPipe, ist in einer klaren Struktur dokumentiert und besteht aus nur etwas über 300 Zeilen Python-Code. In diesem werden Armwinkel und eine Greifgeste erkannt. Das macht es attraktiv als Startpunkt für die Gestenerkennung der vorliegenden Arbeit. Einschränkend ist zu bemerken, dass in dem Projekt nur die zweidimensionalen Bildkoordinaten berücksichtigt werden.

Auch im Projekt [66] werden verschiedene Armwinkel und eine Greifgeste mit Hilfe von MediaPipe erkannt. Die Methode zur Greifgestenerkennung ist etwas komplexer und robuster gegenüber Positions- und Distanzänderungen als die Methode aus [65]. Sie gibt einen Wert zwischen 0 und 100 aus welcher den Öffnungsgrad der Hand beschreibt, siehe Figure 27. Auch dieser Python-Code ist überschaubar strukturiert und ließe sich für den Einstieg Übernehmen.

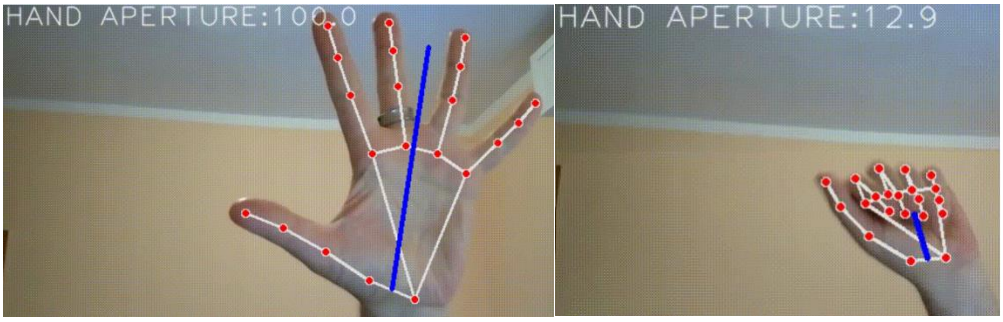



**Figure 27 – Handöffnungsgrad aus [66]**

### *Tiefeninformationen*

Das Projekt [79] kombiniert die Bildpositionen der von MediaPipe geschätzten Landmarks mit den Tiefeninformationen aus einem synchronisierten Tiefenbild. Es wird unteranderem eine Tiefenkamera von Intel RealSense genutzt. RealSense Kameramodelle sind zurzeit in Forschung und Robotik verbreitet.

Die beiden Papers [67] und [70] empfehlen bei räumlichen Informationen, die aus RGB-Bildern gewonnen werden sollen, die zusätzliche Verwendung eines Tiefenbildes, wenn das Ergebnis dreidimensional und genau werden soll. Die Schätzung der Zeigerichtung der geplanten Gestenerkennung unterliegt diesen Kriterien.

### *Rauminformation*

Die Zeigerichtung soll in der vorliegenden Arbeit in eine raum-referenzierende Information überführt werden. Das Konzept aus dem Paper [73] kann dafür aufgegriffen werden. In diesem wird eine erkannte menschliche Zeigegeste in eine räumliche Richtung überführt und eine Position im Raum geschlussfolgert. Hierfür wird aus der Zeigegeste ein Richtungsvektor geschätzt und unter der Kenntnis der Positionen von Kamera und der zeigenden Hand eine Raumrichtung berechnet. Diese wiederum wird mithilfe der räumlichen Dimensionen in eine Raumposition überführt.

### *Klassische Bildverarbeitung*

Das Projekt [83] beschreibt ausführlich wie mit Geometrie und Bildverarbeitungsmethoden aus OpenCV. Die Positionen von Kopf, Händen, Ellenbogen und Schultern bestimmt werden. Weitergehend werden diese für Armwinkelberechnungen genutzt. Das Programm weist eine gute Performanz auf.

Das Projekt aus dem Paper [86] kommt mit einer Bildauflösung von nur 320 x 240 Pixeln aus, um Ganzkörperbewegungen einer fünf Meter entfernten Person zu klassifizieren. Weiterhin beschreibt das Paper ein komplexeres mehrstufiges Vorgehen zur Gestenerkennung. Es kombiniert mehrere klassische Verfahren zielführend. Neben Bildformen werden hierfür auch Bewegungsmuster regelbasiert analysiert.

An [83] und [86] kann sich in der vorliegenden Arbeit orientiert werden, um ein Gestenerkennungskonzept mit rein klassischer Bildverarbeitung zu erstellen. Besonders [83] ist durch seinen übersichtlichen Code als Ausgangspunkt attraktiv.

# 4 Konzeption der Gestenerkennung

Zunächst wird der 3D-Audioraum simuliert, um räumliche Abhängigkeiten und deren Einfluss auf die Gestensteuerung, insbesondere auf das Kamerasichtfeld, zu untersuchen. Danach werden die Anforderungen an die Hard- und Softwarekomponenten des Projekts formuliert.

Es werden Kernaufgaben der Gestensteuerung benannt und ihre einzelnen funktionellen Abläufe konzipiert. Die zu nutzende Software muss definiert werden, um abschließend zwei feiner definierte Gesamtkonzepte aufzustellen: eines ohne und eines mit KI-gestützter Erkennung.

## 4.1 Raumsimulation, Kameraposition und -FOV

Im Folgenden werden die Anforderungen an das Kamerasichtfeld bestimmt. Es wird getestet, welches hFOV und vFOV unter bestimmten Bedingungen notwendig sind wodurch sie beeinflusst werden. hFOV und vFOV werden am Ende in ein dFOV mit dem gängigen Seitenverhältnis 16:9 umgerechnet. Die Größe des FOVs steht in Abhängigkeit zu folgenden Faktoren:

- Standzone: Bereich, am Boden, auf dem der Dirigent sich aufhalten darf, um mit dem System zu interagieren
- Armradius: Bereich, welchen die Arme des Dirigenten in Anspruch nehmen, wenn er „dirigiert“, also mit der Gestensteuerung interagiert.
- Entfernung der Kamera zum Dirigenten bzw. zur Standzone. Diese ist abhängig von der Positionierung der Kamera und der Standzone
- Neigungswinkel: aus welcher Höhe ist die Kamera auf den Dirigenten gerichtet. Das verändert bzw. verzerrt die Größe des Dirigenten im Bild.
- Größe des Dirigenten: die menschliche Körpergröße, der daraus resultierende Armradius sowie seine Armhöhe.

Ein 3D-Modell [96] des 3D-Audioraums soll helfen die Auswirkungen der genannten Faktoren zu sehen. Auswirkungen untereinander und auf das Kamera-FOV. Das Modell wurde mit der kostenfreien Version von onshape [97] erstellt, einer 3D-Modellierungsanwendung im Webbrowser.

Die Simulation zeigt nicht den gesamten Raum, in dem sich das Audiosystem befindet, sondern nur den Raum innerhalb des Systems, also den Gerüstaufbau mit dem Lautsprechersystem, in welchem sich der Dirigent aufhalten soll. Der Raum um das System herum ist nur schemenhaft dargestellt. Es existieren nur Boden und Fensterfront, um die Ausrichtung des Systems im Raum deutlich zu machen. Siehe Figure 28. Das Lautsprechergerüst hat in der Realität die Außenmaße 4 m x 4 m x 3 m (3m ist die Höhe)

mit Abweichungen innerhalb von +/- 2,5cm. Diese Toleranz von 2,5cm gilt auch für die Unterschiede zwischen Realität und dem simulierten Modell. Zudem sind kleine räumliche und strukturelle Merkmale des Modells ebenfalls anders als in der Realität. Grund dafür ist, dass Strukturen vereinfacht wurden, welche für die Simulationszwecke irrelevant sind. Kleinteile wie Lautsprecherhalterungen und elektrische Leitungen wurden weggelassen.

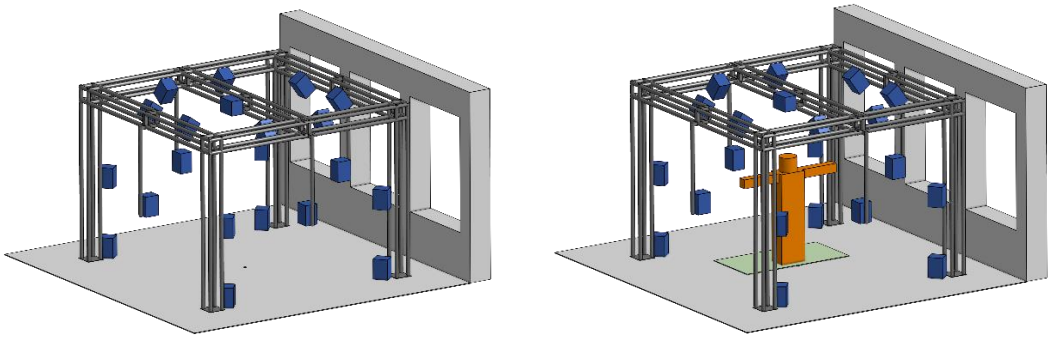


**Figure 28 – Modell Audiosystem leer und mir 1.80m Mensch und Standzone**

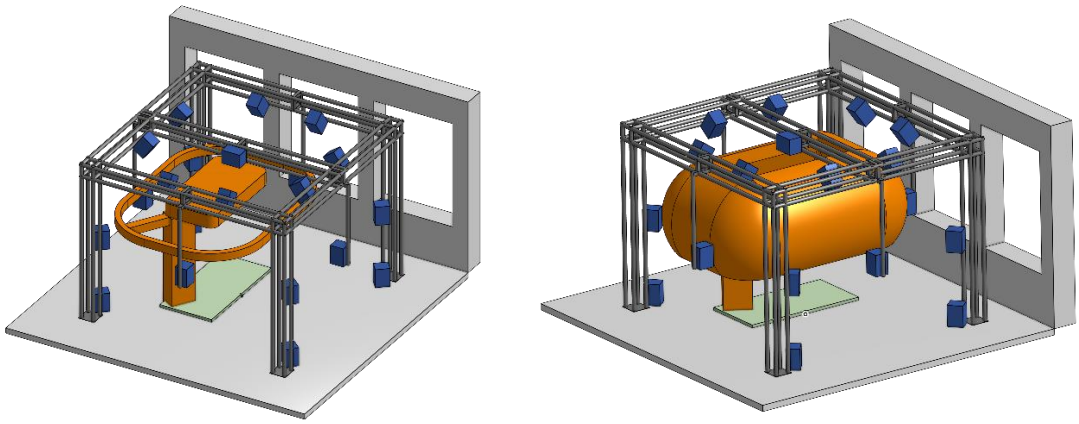


**Figure 29 - Modell Audiosystem mit Aktionsbereich**

Die Bilder in Figure 29 zeigen den Bereich im Raum in den Gesten erkannt werden sollen. Das Linke Bild zeigt neben dem Dirigenten auch zwei mit ihm verbundene quadratisch-rundliche Bereiche, in orange. Diese entstanden durch einen Ansatz, in dem Armradius und Kopfhöhe über die Fläche der Standzone ausgetragen wurden. Die Bereiche umschließen mögliche Positionen von Arm und Kopf. Der Ansatz im Bild rechts erweitert den Armbereich, um die Möglichkeit den Arm zu heben und zu senken. Dieser neue Armbereich umschließt alle möglichen Kopfpositionen und macht einen extra Kopfbereich redundant.

Die Simulation geht hier von einem 207 cm großen Menschen aus mit einer Armspanne von rund 223 cm, einer Streckhöhe (Arme nach oben) von 261 cm, einer Hals-Kopf-Partie von 34 cm Höhe über den Schultern und einer Schulterhöhe von 173 cm. Diese Maße sind aus den Abmessungen eines 190 cm großen Menschen linear hergeleitet. Für die hier angestrebte Genauigkeit soll diese lineare Skalierung reichen. Der 207 cm Mensch repräsentiert die maximale menschliche Größe, die in der Simulation berücksichtigt wird.

Diese Maximalmaße erlauben ca. 10 cm Freiraum zwischen einer gestreckten Haltung und dem Gerüstbalken oben in der Mitte. Dieser Mindestabstand ist der Grund für die Festlegung, der ansonsten willkürlichen Wahl der Maximalgröße.

Die untere Körpergrenze soll ein 140 cm großer Mensch repräsentieren. Es wird willkürlich davon ausgegangen, dass noch kleinere Personen nicht mit diesem System interagieren werden. Die entsprechenden Körpermaße werden auch hier einfach linear abgeleitet.

Die folgende Figure 30 zeigt zwei sich überschneidende Armbereiche im Sichtfeld der Kamera. In Orange, der Armbereich einer 207 cm großen Person und in Gelb, den einer 140 cm großen Person. Das Sichtfeld ist im Simulationsmodell durch eine hellblaue Pyramide dargestellt, deren Spitze die Position der Kameralinse repräsentiert. In Figure 30 lässt sich erkennen, dass das blaue Sichtfeld den orange-gelben Armbereich nicht vollständig abdeckt. Das rechte Bild zeigt das gleiche Szenario wie links, nur aus der Blickrichtung der Kamera. Das blaue Sichtfeld ist hierbei undurchsichtig, um Bereichsaustritte besser sichtbar zu machen.

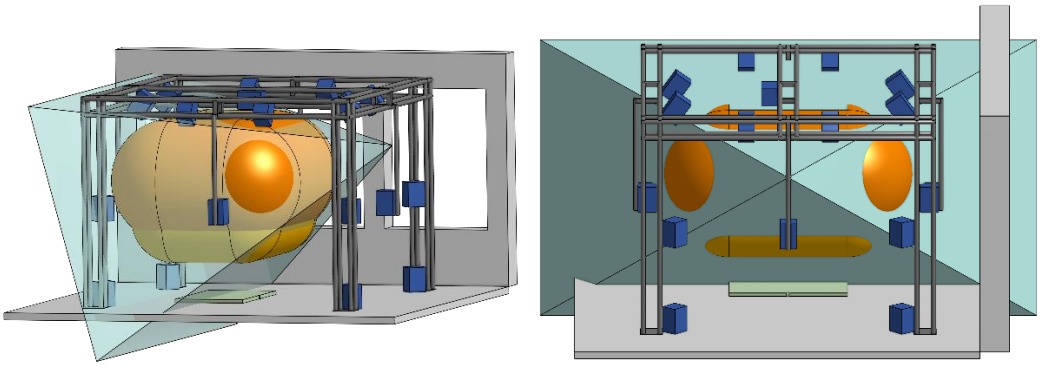


**Figure 30 - Modell Audiosystem Armbereiche 207 cm und 140 cm im Sichtfeld 80° x 50° und 30° Kameraneigung**

Der weitere Testablauf befindet sich im Anhang (*Ausführliche Testfortsetzung von Kameraposition und FOV*), dieser half bei der Entscheidungsfindung, ist allerdings aus dem Stand der aktuell vorliegenden Arbeit weniger relevant bis redundant.

Eine Kameraperspektive mit 0° Neigung (waagerecht) und in der Position mittig an einer Systemwand in 170 cm Höhe wurde gewählt. Diese wird hier *Frontalansicht* genannt, Figure 31 zeigt diese bildlich. Die Frontalansicht bietet im realen Raum einen gleichmäßigen Kamerabildhintergrund und eine angenehme Raumsymmetrie beim Interagieren mit dem System.

Ein Sichtfeld von 92° x 60° umfasst hierbei den gewünschten Aktionsbereich der Arme vollständig. Es wird im Folgenden als das *hinreichende FOV* bezeichnet und dient zur Orientierung bei der Kamerawahl. Für eine bessere Vergleichbarkeit mit den Herstellerangaben wird für das hinreichende FOV zusätzlich das dFOV berechnet. Dabei wird das gängige Seitenverhältnis 16:9 angesetzt. Aus Formel 2 ergibt sich, dass das hinreichende FOV bereits nahezu exakt dieses Seitenverhältnis aufweist, dies ist zufällig. Mit Formel 3 ergibt sich daraus ein dFOV von 99,8°≈100°.

Bei der Wahl einer Kamera mit kleinerem Sichtfeld liese sich die erlaubte Standzone verkleinern – beeinflusst besonders das hFOV, oder die berücksichtigte Personengröße reduzieren – beeinflusst besonders das vFOV.

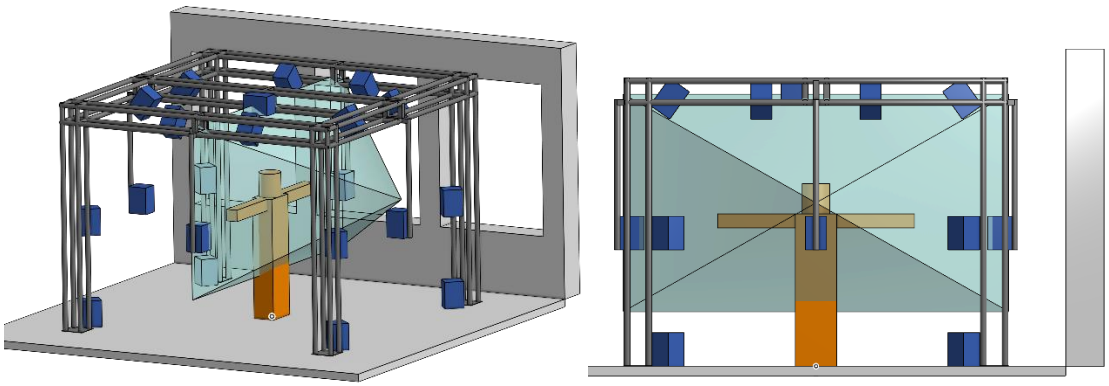


**Figure 31 - Modell Audiosystem Frontalansicht mit 170 cm, 0° Kamerazeigung und 180 cm Mensch**

## 4.2 Anforderungen an die Projektkomponenten

Die Wahl der Komponenten unterliegt Kriterien. Kriterien. Diese ergeben sich vorrangig aus dem Projekt und der physischen Projektumgebung, also dem 3D-Audioraum der BHT.

Weitere Komponentenkriterien ergeben sich während der Wahl der Komponenten selbst. Soll zum Beispiel eine Tiefenkamera zum Einsatz kommen, dann muss die Programmumgebung in der Lage sein, die notwendige Treiber Software zu integrieren. Soll eine bestimmte Software eingesetzt werden, dann muss die Recheneinheit und ihr Betriebssystem diese Software handhaben können. Weiterhin müsste ihre Rechenleistung ausreichend sein, um mit dieser Software eine Gestensteuerung in Echtzeit zu betreiben.

*Anforderungen an die Kamera*

Für die Umsetzung der Gestensteuerung wird eine 3D- bzw. Tiefenkamera vorausgesetzt. Versuche mit verschiedenen Methoden und einer 2D-Kamera haben gezeigt, dass die hier erreichbare Genauigkeit für die Bestimmung der Zeigerichtung unzuverlässig oder zu gering ist. Bei Interesse zu den genaueren Versuchen, siehe im Anhang unter *3D-Raum-Koordinaten mit MediaPipe world Landmarks* und *Fehlgeschlagene Konzepte zur Erkennung der Zeigerichtung*.

Die Kamera muss innerhalb des vorgesehenen Projektbudgets finanzierbar sein und eine ausreichende Bildqualität für die Gestenerkennung bereitstellen. Es sind eine ausreichende Auflösung und Bildwiederholungsrate relevant. Die Auflösung muss gewährleisten, dass auch kleinere Strukturen wie die Finger, in der vorgesehenen Entfernung erkannt werden können. Eine ausreichende Bildwiederholungsrate ist erforderlich, um Bewegungen ohne relevante Verzögerung zu erfassen und einem „flüssigen“ Benutzererlebnis nicht im Weg zu stehen. Eine HD-Auflösung mit 20 FPS soll hier als Mindestvoraussetzung angesetzt werden.

Die Befestigung der Kamera im 3D-Audio-Raum soll am vorhandenen Metallgerüst erfolgen können, an dem auch die Lautsprecher montiert sind.

Neben diesen Eigenschaften ist die Erfassung des vorgesehenen Arm- und Bewegungsbereichs relevant, damit die Gesten des Dirigenten vollständig im Bild sind. Das in Abschnitt 4.1 eingeschätzte FOV und die Kamerapositionierung im Raum dienen als Maßstab. Das FOV muss mit der Entsprechenden Kameraposition den Armbereich des Dirigenten abdecken.

*Anforderungen an die Recheneinheit*

Die Recheneinheit muss innerhalb des vorgegebenen Projektbudgets liegen und eine ausreichend hohe Rechenleistung bieten, um die Gestensteuerung in Echtzeit ausführen zu können. Sie sollte über mindestens eine leistungsstarke USB-3.x-Schnittstellen für den Anschluss der Tiefenkamera verfügen. Auch sollte an der Recheneinheit eine zeitgemäße Bildschnittstelle wie HDMI, DisplayPort (DP) oder DisplayPort Alt Mode (DP Alt Mode) über USB-C vorhanden sein. Netzwerkfähigkeit ist erforderlich, um den Datenaustausch mit dem Audiosystem zu ermöglichen. Da das System in einem Audioraum eingesetzt wird, muss die Recheneinheit im Betrieb leise sein. Lüftergeräusche sollen das Klangerlebnis nicht beeinträchtigen.

*Anforderungen an die Software*

Die Software soll innerhalb des Projektbudgets liegen, idealerweise kostenfrei verfügbar sein und Video- bzw. Bildverarbeitung in Echtzeit unterstützen. Wünschenswert ist eine breite Anbindungsmöglichkeit an Libraries und Programmbibliotheken im Bereich Bildverarbeitung und Gestenerkennung, insbesondere mit zeitgemäßen neuronalen ML-Verfahren. Die Umgebung sollte gut dokumentiert sein und in ähnlichen Gestenerkennungsprojekten erprobt worden sein. Zudem muss eine Kommunikationsschnittstelle für den Datenaustausch mit der MATLAB-Umgebung des Audiosystems verfügbar und kompatibel sein.

## 4.3 Kernaufgaben der Gestensteuerung

### 4.3.1 Benutzeroberfläche

Zur Unterstützung der Interaktion mit der Gestensteuerung wird eine Benutzeroberfläche (GUI) auf einem Bildschirm dargestellt. Die Benutzeroberfläche umfasst ein Menü zur Auswahl der Musikinstrumente, wobei die Auswahl über entsprechende Symbolfelder erfolgt, sowie weitere Schaltflächen und Interaktionsmöglichkeiten zur Steuerung des Systems. Als Grundlage der Darstellung dient das Kamerabild, das auf dem Bildschirm gespiegelt dargestellt wird. Der Benutzer sieht sich dadurch ähnlich wie in einem Spiegel und erhält visuelles Feedback über seine eigenen Bewegungen sowie deren Position innerhalb der GUI. Dies ermöglicht es dem Benutzer, seine Gesten gezielt auf die dargestellten Bedienelemente auszurichten und über die Gestensteuerung mit dem GUI zu interagieren. Der Bildschirm befindet sich dabei unmittelbar oberhalb der Kamera.

Die Erstellung und Darstellung von Bildelementen auf einem gespiegelten Kameraframe bildet somit die technische Grundlage für die Umsetzung der beschriebenen Benutzeroberfläche. Dadurch können grafische Elemente wie Schaltflächen,

Symbolfelder und weitere Anzeigeelemente direkt in das Kamerabild eingebettet und gemeinsam mit diesem dargestellt werden.

### 4.3.2 Positionierung eines Instruments

Um eine Soundquelle, also ein Instrument, im Raum zu positionieren oder zu verschieben muss dieser eine eindeutige räumliche Positionsinformation zugeteilt werden. Der Raum entspricht hier dem Raum innerhalb der Grenzen des Audiosystems, also des Metallgerüsts. Die Positionsinformation ist die *Raumrichtung*, diese besteht aus zwei Winkelangaben. Der Referenzpunkt jeder Winkelangabe ist das Raumzentrum, also die Position, an der jede der drei Raumdimensionen die Hälfte seines Maximalwertes hat. Weiterhin sind die Raumwinkel so definiert:

- Azimutwinkel: 0° ist eine Raumposition, welche horizontal zur Kamera zeigt, also vom Raumzentrum zur horizontalen Mitte der frontalen Systemwand. 90° ist dann die horizontale Mitte der rechten Systemwand

- Elevationswinkel: 0° ist hierbei eine Raumposition waagerecht zum Raumzentrum und 90° die Position direkt über dem Raumzentrum, also mittig an der Decke

Die Kameraperspektive ist hierbei die im Abschnitt 4.1 definierte *Frontalansicht* . Figure 32 zeigt die eben beschriebenen Winkelpositionen und die der Kamera.

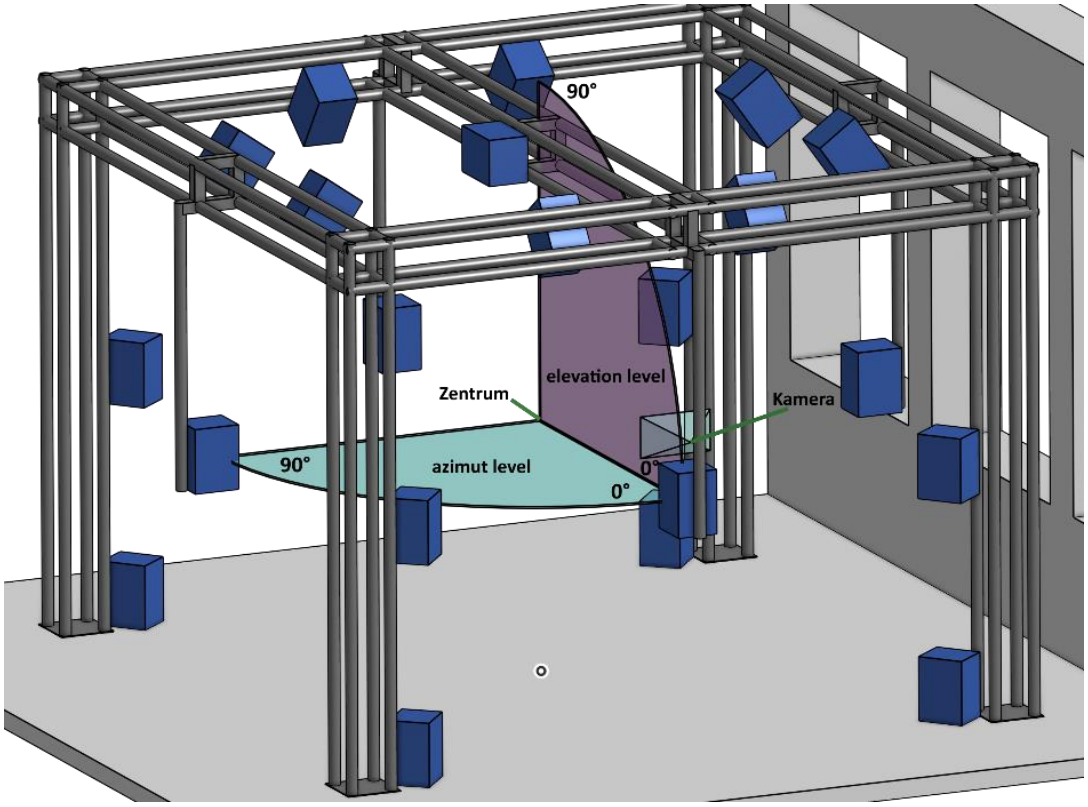



**Figure 32 - Azimut- und Elevations-Level im Raum**

Zur Bestimmung der Raumrichtung mittels Gestenerkennung werden die Position des Dirigenten sowie seine *Zeigerichtung* ermittelt. Die Definition dieser Zeigerichtung ergibt sich hier aus der Richtung einer Geraden zwischen zwei Körperlandmarks des Dirigenten im Raum. Die erste Landmark ist das Zentrum einer Hand, die Zweite ist die seitengleiche Schulter. Der Winkel dieser Armgeraden relativ zum Raum, soll in Azimut und Elevation aufgeteilt, die Zeigerichtung definieren. Mit einer Armwinkelschätzung soll die Zeigerichtung bestimmt werden.

Für die Raumrichtung muss nun der Punkt an der räumlichen Systemgrenze bestimmt werden auf den der Dirigent zeigt. Dafür werden seine Position und Zeigerichtung ausgewertet, ähnlich dem Vorgehen im Paper [73]. Damit sind Handzentrum und Schulter die zwei Punkte im Raum welche im Rahmen der Gestenerkennung erkannt und dreidimensional definiert werden müssen, um die Raumrichtung zu berechnen.

### 4.3.3 Auswählen eines Instruments

Um ein Instrument aus dem GUI auswählen zu können, wird die Bildposition (Pixelposition) der Handzentren mit der Bildposition der entsprechenden Symbolfläche auf dem Bildschirm verglichen. Um die Auswahl zu bestätigen, soll eine Greifgeste der selektierenden Hand erkannt werden. Solange die Hand dann geschlossen ist, wird die Zeigerichtung des entsprechenden Arms als Raumrichtung umgerechnet auf das Instrument übertragen. Wird anschließend eine Loslassengeste erkannt, also öffnet sich die Hand wieder, dann bleibt das Instrument an der Stelle im Bild zurück, an der sich die Hand zu der Zeit befindet. Es behält dann die zuletzt übertragene Raumrichtung bei.

Neben der Bestimmung des Handzentrums im Bildschirm, muss die Gestenerkennung das Schließen und Öffnen einer Hand erfassen. Figure 33 zeigt und definiert die Zustände dieser Greifgeste. Für diese wurden zwei Erkennungskonzepte erstellt:

- Statisch: Eine Greif- bzw. Loslassengeste kann durch einen Wechsel des Handstatus definiert werden. Wobei der Handzustand statisch auf Basis der Handgeometrie ausgewertet wird.

- Dynamisch: die Gesten werden direkt über eine definierte Größenveränderung in der Handgeometrie erkannt.

Auf die gleiche Weise, wie die Handinteraktion zur Auswahl eines Instruments verwendet wird, kann auch eine Interaktion mit den Schaltflächen des GUIs umgesetzt werden. Dadurch lassen sich neben der Instrumentenauswahl auch weitere Funktionen des Systems über die Handposition und Greifgeste bedienen.

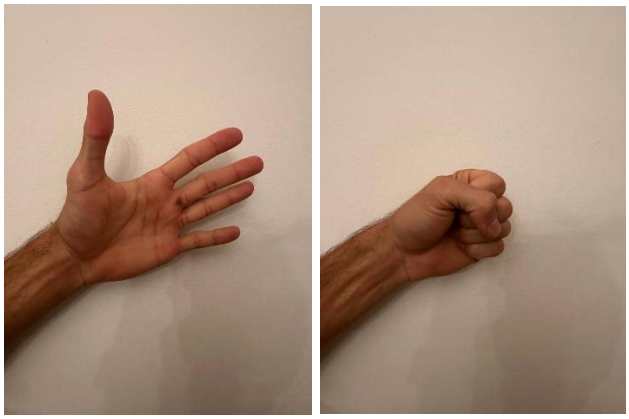


**Figure 33 - Handzustände offen und geschlossen**

### 4.3.4 Gesten für weitere Funktionalitäten

Um die Interaktion mit dem Menü zu erweitern und gleichzeitig die Anzahl der erforderlichen Schaltflächen im GUI zu reduzieren, sollen neben der Greifgeste und der Bestimmung der Armrichtung weitere fest definierte Gesten erkannt werden.

Eine Geste soll das Zurücksetzen des Systems ermöglichen. Dabei werden alle Instrumente wieder in ihren Ausgangszustand gesetzt und veränderte Einstellungen der Benutzeroberfläche auf ihren Initialzustand zurückgesetzt.

Eine weitere Geste soll den Anzeigemodus der Benutzeroberfläche ändern und zusätzliche Informationen über die im Hintergrund ausgeführten Verarbeitungsschritte ein- und ausblenden. Hierzu können beispielsweise erkannte Körper- und Handpositionen, berechnete Koordinaten oder weitere für die Gestenerkennung relevante Werte eingeblendet werden. Dies ermöglicht auf Wunsch einen Einblick in die zugrunde liegenden Verarbeitungsschritte.

Zusätzlich soll eine Geste zur Anzeige von Informationen zu allen verfügbaren Gesten und deren Funktion implementiert werden. Dadurch können dem Benutzer die möglichen Interaktionen und deren Verwendung innerhalb der Gestensteuerung erläutert werden.

Abschließend soll eine weitere definierte Geste das Beenden des Programms ermöglichen.

### 4.3.5 Systemkommunikation

Die von der Gestenerkennung ermittelten Steuerbefehle und Positionsinformationen, insbesondere die Raumrichtung der Instrumente, müssen an das Audiosystem übertragen werden. Umgekehrt soll das Audiosystem der Gestensteuerung mitteilen können, welche Instrumente gerade zur Verfügung stehen. Dadurch kann die Anzeige der Instrumente im GUI dynamisch angepasst werden.

Die Kommunikation erfolgt asynchron. In beiden Systemen läuft hierfür ein separater Empfangsthread, der eingehende Nachrichten entgegennimmt und in einem

Empfangspuffer ablegt. Der Hauptthread kann diese anschließend zeitlich unabhängig vom Empfangsvorgang verarbeiten.

## 4.4 Wahl der Software

Die Auswahl der hier betrachteten Software ist mit Linux, Windows und MAC OS kompatibel. Nur die NVIDIA Jetson Produkte kommen mit ihrer eigenen Linux-basierten Softwareumgebung. Basierend auf den Kapiteln *Stand der Technik* und

*Anforderungen an die* Projektkomponenten, kommt folgende Software als Basis für die Gestenerkennung in Frage:

- Googles MediaPipe in Python oder C++
- MATLAB mit Simulink
- OpenPose in Python oder C++
- OpenCV in Python oder C++ zur klassischen Bildverarbeitung
- NVIDIA Jetpack, beispielsweise mit MediaPipe Lite in Python

*Lieber Python als C++*
C++ steht nur zur Wahl, wenn sich für eine leistungsschwache Recheneinheit entschieden wird. Umsetzungen mit Python sind im Internet häufiger dokumentiert und die Sprache hat eine allgemein einfachere Syntax als C++.

*Stärken der Jetson-Umgebung*
Sollte sich für das JetPack-Softwarepaket entschieden werden, muss die Recheneinheit eine Jetson-Plattform sein, wie beispielsweise der Jetson Nano. Die Jetson-Umgebung ist für KI-Anwendungen optimiert. Ein Jetson kann dadurch bei KI-Anwendungen Vorteile gegenüber anderen Rechnern ähnlicher Größe haben. Dies wird auch durch die in verschiedenen Projekten erreichte Performance gestützt, die im Rahmen der Recherche betrachtet wurden. Einige davon sind im Kapitel *Stand der Technik* genannt. Die Jetson Umgebung ist also besonders dann interessant, wenn, viele KI-Berechnungen auf kleinem Raum mit ggf. wenig Stromverbrauch das Ziel sind. Die Co-Abhängigkeit von Software und Recheneinheit, macht diese Wahl weniger attraktiv, da sie dem Projekt Flexibilität nimmt. Sollte sich ein Problem in einer der Abhängigkeiten ergeben, schränkt diese Co-Abhängigkeit die Lösungsoptionen ein.

*MATLAB braucht Leistung*
MATLAB Simulink verfügt über eine einfache Syntax mit teils Grafischer Block-Programmierung. Die Verwendung eines Microcomputers ist hierbei unwahrscheinlich, das ergibt sich aus MATLABs Systemvoraussetzungen [98], es sind vollwertige Betriebssysteme und eine x86-64-bit-Prozessorstruktur nötig.

*MediaPipe wird viel genutzt*
Googles MediaPipe ist die mit Abstand meistgenutzte Software für Gestenerkennung in Kapitel *Stand der Technik*, das spiegeln auch die Erfarungen bei der generellen Recherche

wieder. Der einzige hier erfasste Nachteil ist der höhere Rechenaufwand im Vergleich zu rein klassischer Bildverarbeitung. Mildernd zu bemerken ist das MediaPipe Pose und MediaPipe Hands bereits einen starken Fokus auf gute Echtzeitperformance setzen, siehe Kapitel 2.4.2 und 2.4.3.

*OpenCV wird viel genutzt*
OpenCV bietet klassische Bildverarbeitungsmethoden. Es ist meistens die erste Software, welche im Internet genannt wird, wenn es um Computerbasierte Bildverarbeitung innerhalb einer Programmierumgebung geht. Keine andere Software wurde wären der Suche nach ähnlichen Projekten so oft genannt und dokumentiert genutzt. MidiaPipe nutzt ebenfalls OpenCV für die Vorverarbeitung der Bilddaten.

*OpenPose eher zur Mehrpersonenerkennung*
OpenPose ist ähnlich zu MediaPipe eine auf neuronalen KI-Anwendungen basierte Bildverarbeitung Software welche menschliche Körperlandmarks erkennt. Bei der Recherche nach ähnlichen Projekten ergab sich der Eindruck, dass der Schwerpunkt hier mehr auf der Mehrpersonenerkennung und modularen Softwareanbindung liegt und weniger auf Performanz, im Vergleich zu MediaPipe. Diese Aussage stützen auch der Absatz *Vergleich zu MediaPipe*
im Abschnitt 2.5 und das Projekt [89] in Abschnitt 3.11.

*Methodische Kompatibilität*
Für diese Masterarbeit werden zwei Konzepte zur Implementierung der Gestenerkennung entwickelt: eines mit und eines ohne KI-Anwendungen, diese sollen ggf. auch Implementiert werden. Es soll einen Vergleich von Verfahren mit und ohne KI ermöglichen. Für dieses Vorhaben sollten beide Methodiken mit der gleichen Software kompatibel sein. Dadurch muss bei der Implementierung lediglich die Methodik geändert werden, um sie an das restliche System anzubinden.

*Es wird MediaPipe mit OpenCV*
Ein naheliegender Ansatz ist die Kombination aus OpenCV (ohne KI) und MediaPipe (mit KI), da das MediaPipe-Framework ohnehin OpenCV nutzt. Diese Wahl begünstigt die hohe Verfügbarkeit ähnlicher Projekte, die OpenCV und MediaPipe verwenden. Die Lösung möglicherweise auftretender Probleme profitiert von einer hohen Informationsdichte. Relevante Gründe, die für eine andere Softwarewahl sprechen, liegen derzeit nicht vor. Es wird die Softwarekombination aus OpenCV und MediaPipe gewählt, vorzugsweise in einer Python-basierten Programmierumgebung.

## 4.5 Konzeptspezifikation mit KI

Der geplante Ablauf der Gestensteuerung basierend auf MediaPipe wird im Folgenden anhand der Verarbeitung eines einzelnen Frames innerhalb der Prozessschleife beschrieben.

*Einlesen und Vorverarbeitung*
Zunächst wird ein Frame der Tiefenkamera eingelesen und das Tiefenbild anhand der Kamerakalibrierung auf das RGB-Bild ausgerichtet (Alignment). Anschließend wird das Bild gespiegelt, um dem Benutzer über den Monitor ein natürliches visuelles Feedback zu ermöglichen. Der RGB-Frame wird danach dupliziert. Ein Frame dient der Analyse,

der andere wird, während der Verarbeitung um grafische Informationen ergänzt und dient am Ende als visuelles Feedback.

*Erkennung der Körperposition*
Im nächsten Schritt ermittelt MediaPipe Pose das Körperskelett aus relevanten Landmarks. Insbesondere die Positionen von Händen und Schultern sind wichtig.

*Erkennung des Handskelets*
Es werden sogenannte *Regions of Interest* (ROIs) definiert, also Bildausschnitte, in welchen eine Hand erwartet wird. Das Gesamtbild eignet sich nicht für die Verarbeitung mit MediaPipes Hand-Schätzung. Eigene Tests ergaben das die Hand eine Bestimmte Größe im zu analysierenden Bild haben sollte, siehe auch Figure 58 - *Hand Erkennung mit verschieden großen Bildausschnitten*. Bei der Positionierung des ROIs wird sich an den Handpunkten von MediaPipe Pose orientiert. MediaPipe Hands sucht dann in den ROIs nach den genauen Handskeletten.

Es wird die Bewegungsgeschwindigkeit der Hand im Bild bestimmt. Die Auswertung von Handgesten soll nur passieren, wenn die Hand stillsteht. Diese Maßnahme soll die Erkennung stabilisieren, Falscherkennungen reduzieren und verhindern das eine Greifgeste wären der Bewegung, bei der Positionierung eines Instruments, unterbrochen werden kann.

*Erkennung der Greifgeste*
Sind die Bedingungen erfüllt, wird das von MediaPipe Hands erkannte Handskelett zur Erkennung der Greif- oder Loslassgeste ausgewertet. Hierzu werden die Positionen ausgewählter Handlandmarks geometrisch zueinander in Beziehung gesetzt.

*Bestimmung der Raumrichtung*
Es werden die Positionen der Schulter und der zugehörigen Hand im Tiefenbild referenziert. Mittels der Tiefeninformationen werden dann dreidimensionale Raum Koordinaten errechnet. Der Richtungsvektor zwischen den 3D-Punkten (Hand und Schulter) wird dann unter Kenntnis der räumlichen Systemgröße in die Raumrichtung überführt. Die Raumrichtung wird in Azimut- und Elevationswinkel ausgegeben.

*Erkennung der Steuergesten*
Anschließend werden weitere fest definierte Steuergesten geprüft. Auch diese werden anhand geometrischer Beziehungen zwischen den erkannten Körper- oder Handlandmarken bestimmt.

*Verarbeitung der GUI*
Das GUI-Objekt erhält die Positionen der Hände innerhalb des Bildes sowie Informationen über erkannte Greif- und Steuergesten. Es wertet diese Informationen aus und reagiert darauf.

Für jedes Instrument existiert ein eigenes Instrumenten-Objekt im GUI. Dieses speichert den aktuellen Zustand des Instruments, beispielsweise die Lautstärke und die räumlichen Positionsinformationen.

Das GUI fungiert als logische Brücke zwischen den erkannten Gesten und dem Zustand der Instrumente. Es wertet die erkannten Gesten in Abhängigkeit von der aktuellen Situation aus und überführt ein Instrument gegebenenfalls in einen neuen Zustand.

*Kommunikation*
Änderungen eines Instrumentenzustands oder andere für das Audiosystem relevante Informationen werden an das Audiosystem übertragen. Das Audiosystem Wiederum kann der Gestensteuerung bzw. seiner GUI sagen welche Tonspuren, also Instrumente, zur Verfügung stehen und zur Auswahl angezeigt werden sollen.

*Ergebnisdarstellung am Loop-Ende*
Abhängig von einer gewählten Anzeigeeinstellung werden neben den GUI-Flächen auch relevante Prozessinformationen in den für die Darstellung vorgesehenen RGB-Frame eingezeichnet. Der resultierende Frame wird gegebenenfalls auf die erforderliche Anzeigegröße skaliert und anschließend auf dem Monitor ausgegeben. Damit ist die Verarbeitung eines Frames abgeschlossen und der nächste Kameraframe wird eingelesen.

## 4.6 Konzeptspezifikation ohne KI

Der Prozessablauf des Ansatzes ohne KI entspricht grundsätzlich dem zuvor beschriebenen Prozessablauf des Ansatzes mit KI. Die Verarbeitungsschritte zum Frame einlesen, Bestimmung der Raumrichtung aus dem Armvektor, Verarbeitung der GUI und Kommunikation mit dem Audiosystem bleiben dabei unverändert. Unterschiede bestehen insbesondere bei der Erkennung der Körperposition, der Hände sowie der Greif- und Steuergesten. Im Folgenden werden daher ausschließlich die abweichenden Verarbeitungsschritte beschrieben.

*Erkennung der Körperpositionen*
Die Körperpositionen werden direkt aus den im Bild erkannten Flächen bzw. ihrer Konturen abgeleitet. Ausgangspunkt ist die zuvor beschriebene Hautsegmentierung, siehe dazu Abschnitt 2.10. Es werden zunächst hautfarbene Bildbereiche bestimmt und anschließend durch morphologische Filter von kleineren Störungen und Flächenfehlern bereinigt. Siehe dazu auch Abschnitt 2.7 - *Morphologische Transformationen*. Aus der resultierenden Binärmaske werden die Konturen der erkannten Bereiche bestimmt, siehe dazu 2.9 - *Suzuki Algorithmus*. Bestimmte Flächen und Schwerpunktsberechnungen so wie form Erkennungen und weitere klassische Bildverarbeitungen, lassen sich gut auf der Basis der reinen Flächenkonturen durchführen. Dieses Vorgehen ist dem Code des Projektes [83] entnommen.

Um bereiche zu erkennen welche nicht Hautfabend sind, liese sich zudem eine Bewegungserkennung auf der basis von Hintergrundsubtraktion durchfüren, siehe dazu Abschnitt 2.8 - *Background Subtraction*. Das Paper [86] nutzt dieses Prinzip.

Körperbereiche, welche nicht direkt erkannt werden können, allerdings in einem festen Zusammenhang zu gut erkennbaren Bereichen stehen, ließen sich über diesen Zusammenhang bestimmen. Auch das wird bereits in [83] und [86] erfolgreich genutzt.

Für die genannten Verfahren der Filterung, Konturenerkennung und Hintergrundsubtraktion gibt es in OpenCV vorgefertigte Methoden. Eine

Hautsegmentierung kann aus Farbraum-Transformationen und Bildpixel-Schwellwert-Methoden zusammengesetzt werden, diese sind ebenfalls Bestand dieser Bildverarbeitungs-Library.

*Erkennung der Hände*
Die Hände werden über die Hautsegmentierung und die anschließende Konturerkennung lokalisiert. Dafür werden alle erkannten Konturen der Größe nach sortiert. Die größten drei Konturen werden als Hand- und Kopfbereiche interpretiert. Um aus einer erkannten Kontur eine konkrete Bildposition zu erhalten, kann ihr Mittelpunkt oder Schwerpunkt genommen werden. Die Zuordnung der Konturen zu Kopf, linker und rechter Hand, erfolgt anhand ihrer horizontalen und vertikalen Bildposition. Dieses Vorgehen stammt aus dem Projekt [83].

Dar die statische Bildposition eines Frames allein nicht ausreichend ist, um in dieser Gestensteuerung eindeutig zu sein, wird es einen zusätzlichen Kalibrierungsprozess geben. Dieser Prozess findet nach der initialen Seitenzuweisung kontinuierlich im Hintergrund statt. Er basiert auf der Verfolgung der Konturmittelpunkte. Der Kopf ändert seine vertikale Bildposition deutlich weniger und seltener als die Hände. Dadurch kann er identifiziert werden. Teil der Kalibrierung ist auch eine Langzeitauswertung der horizontalen Position der Hände, diese wird genutzt um die rechte und linke Hand zu labeln. Das soll erlauben das eine zeitliche kurze Positionsveränderung einer Hand zur jeweils anderen Bildseite, nicht die erkannten Handseiten vertauscht.

*Erkennung der Greifgeste*
Die geometrischen Informationen der Konturen werden ausgewertet, um den Handstatus zu unterscheiden. Eine bestimmte Veränderung der Flächengröße oder -form bildet hier die Erkennungsgrundlage, ggf. unter Berücksichtigung der Veränderungs-geschwindigkeit.

*Erkennung der Schulter*
Die Position der Schulter wird von der Position des Kopfes abgeleitet. Die Kopfhöhe und sein Abstand zur Kamera, dient dabei als Referenzmaß. Der Abstand zur Kamera wird hierbei aus dem kalibrierten Tiefenbild referenziert.

Sind die Positionen von Schultern und Händen bekannt, kann ein Armvektor und die Raumrichtung wie gewohnt berechnet werden.

Durch diese feste geometrische Zuordnung der Positionen kann eine Drehung der Schultern nicht erkannt werden, der Dirigent muss somit stets gerade zur Kamera stehen.

Gestaltet sich die Schultererkennung als problematisch, könnte alternativ die Kopfposition verwendet werden. Die Zeigerichtung wäre dann als Vektor zwischen Kopf und Hand definiert, sodass der Dirigent entlang dieser Linie zielen müsste.

*Erkennung der Steuergesten*
Die Steuergesten werden anhand von der Geometrie zwischen Kopf- und Handmittelpunkten definiert. Da diese drei Punkte wesentlich weniger Informationen enthalten als eine KI-basierte Skeleterkennung (Körper- und Hand-Landmarks), werden zusätzlich die Bewegungsmuster der Hände ausgewertet.

Dennoch ist zu erwarten, dass der Informationsgewinn geringer ausfällt als bei einer Skeleterkennung. Das erschwert die Implementierung zusätzlicher Steuergesten hier bei diesem klassischen Konzept.

*Multithreading (optional bei wenig Rechenleistung)*
MediaPipe verteilt seine einzelnen Prozessknoten auf die zur Verfügung stehenden Recheneinheiten und Kerne, siehe dazu 2.4.1 - *MediaPipe Framework*.

Um in diesem KI-losen Ansatz ebenfalls eine Aufteilung der Rechenlast zu realisieren. Kann ein Multithreading Konzept genutzt werden. Hierzu ist zu erwähnen, dass ein Konzept zur Aufteilung der Rechenlast, erwartungsgemäß nur dann einen sinnvollen Performanz-Gewinn schafft, wenn die Gestensteuerung auf einer leistungsschwache Recheneinheit mit mehreren Rechenkernen eingesetzt wird.

Ein Multithreading mit vier Prozessorkernen wird im Projekt [83] folgendermaßen implementiert:

> Für jeden der Prozessorkerne wird ein Thread erstellt und in einem Ringspeicher verwaltet. Mit jedem durchlauf der Berechnungsschleife (Loop) werden die Berechnungen der Bildverarbeitung eines Frames auf dem jeweils nächsten Thread bzw. Prozessorkern durchgeführt. Der Ablauf, Schritt für Schritt betrachtet, ist dabei wie folgt:
>
> - Initial bekommt jeder der vier Prozess-Threads eine Kopie des ersten Frames zur Verarbeitung.
> - Am Anfang des ersten Verarbeitungsloops werden die Resultate des ersten Threads ausgelesen.
> - Am Ende des Loops bekommt dieser Thread den nachfolgenden zweiten Frame von der Kamera zur Verarbeitung.
> - Am Start des zweiten Loops werden die Resultate des zweiten Threads ausgelesen
> - Dieser bekommt am Loopende den nachfolgenden dritten Frame zur Verarbeitung
> - Im fünften Loopdurchlauf wird wieder der erste Thread mit den Resultaten des zweiten Frames ausgewertet
> - Am Loopende bekommt der erste Thread den sechsten Frame. Und so weiter…

Figure 34 zeigt den gerade beschriebenen Prozess bildlich.
Das Multithreading Konzept garantiert eine Latenz von mindestens vier Frames zwischen dem Einlesen eines Frames und dem Erhalt seiner Resultate. Braucht die Frameverarbeitung weniger als die Dauer der Erfassung von vier Frames, erhöht dieses Konzept also die Latenz. Im Gegenzug sorgt es für eine „flüssigere“ Bildwiedergabe, sofern die Verarbeitung eines Frames länger dauert als seine Erfassung.

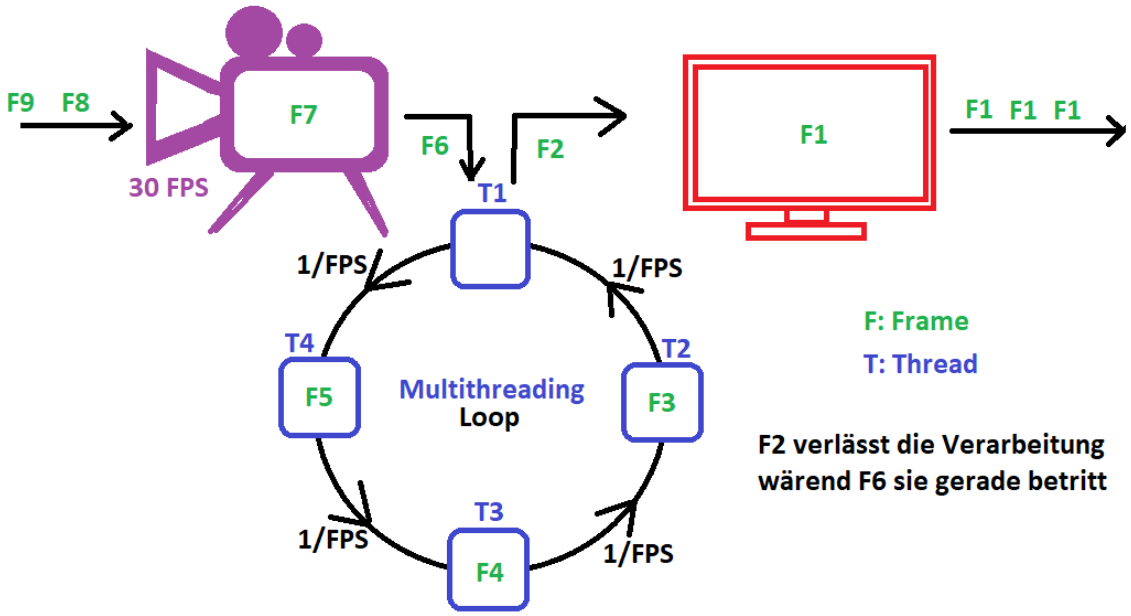



**Figure 34 – Schemata vom Multithreading im Projekt [83]**

# 5 Implementierung der Gestensteuerung

Zunächst wird die Projekthardware detailliert spezifiziert. Aus Gründen von Zeit und Aufwand wird nur die Konzeptspezifikation mit KI Implementiert. Die Gründe für diese Wahl werden diskutiert und der Implementierungseinstieg wird beschrieben. Die fertiggestellte Implementierung der Gestensteuerung zum Zeitpunkt der Abgabe dieser Masterarbeit wird *Stitchi 1.0* genannt. Stitchi 1.0 wird hier anhand ihrer Kernfunktionalitäten beschrieben. Der vollständige Programmablauf ist im Python-Code selbst nachzuvollziehen.

Zudem gibt es eine Bedienungsanleitung für die Nutzung der Gestensteuerung als User und für die Bedienung als Code-Entwickler im Projektordner und auch hier im Anhang:

*Bedienungsanleitung für die Gestensteuerung Version STITCHI 1.0*

Der gesamte Projektordner dieser Arbeit, einschließend dieses Dokuments und des Python-Quellcodes, ist auf GitHub veröffentlicht:

https://github.com/Eulelarve/Masterarbeit.git

## 5.1 Wahl der Hardware

Zunächst wird entschieden welche Kamera und Recheneinheit genutzt wird, um die Gestensteuerung zu realisieren. Auserdem wird noch ein Bildschirm für ein grafisches Benutzer Feedback bzw. das GUI benötigt.

### 5.1.1 Kamerawahl

Die Intel RealSense Tiefenkamera D455 [51] steht der BHT und diesem Projekt bereits zur Verfügung, ihr FOV von 86° x 57° ist etwas kleiner als das *hinreichende FOV* (92° x 60°). Mit der *Frontalansicht* und dieser Kamera unterliegt das Projekt somit kleinen Einschränkungen, bezüglich Personengröße und Standzone des Dirigenten. Die HD-Auflösung der Kamerasensoren bei 30 FPS ist ausreichend und der Arbeitsbereich des Tiefensensors, mit 0.6 bis 6 Metern ebenfalls. [99]

Weiterhin werden, nach eigener Recherche, RealSense-Sensoren aktuell häufig in Forschung und Robotik eingesetzt, was zuverlässige Genauigkeit und gute Kompatibilität erwarten lässt. Auch ist zu beachten das sich die von der Kamerakommende Bildauflösung über ihre Treibersoftware einstellen lässt. Auch ihre Kalibrierung (Tiefenbild-Alignment und Verzerrungskompensation) können die Treiber übernehmen, siehe dazu Abschnitt 2.12 - *Intel RealSense Tiefenkamera Treiber*. Die möglichen Einschränkungen des zuvor ermittelten FOVs werden als gering eingestuft und es wird sich für diese Tiefenkamera entschieden. Das folgende Bild die Kamera.



**Figure 35 - Intel RealSense Tiefenkamera D455 [100]**

### 5.1.2 Wahl der Recheneinheit

*Warum ein Mini-Desktop-PC*

Grundsätzlich wäre zu klären, ob ein Microcomputer oder ein Kleiner vollwertiger Desktop-PC (Mini-PC) zum Einsatz kommen soll. Eine größere Recheneinheit wie ein Stand-PC oder ein Notebook werden für dieses Projekt nicht in Betracht gezogen. Die Rechenleistung aktueller Mini-PCs sollte die obere Anforderungsgrenze des Projekts abdecken und ihr kleiner Formfaktor ermöglicht eine unauffällige räumliche Integration in das Audio-Video-System. Diese ist in diesem Projekt wünschenswert.

Vorteil eines Microcomputers ist seine leichte Integrierbarkeit an Orten, wo wenig Platz oder Traglast gegeben ist, sowie sein geringer Strom verbrauch. Vorteil eines Mini-desktop-PCs ist zum einen seine höhere Rechenleistung und zum anderen seine höhere Softwarekompatibilität, gegeben durch ein vollwertiges Betriebssystem und die x86-Prozessorarchitektur.

Stromverbrauch und Platz, in den hier diskotierten Dimensionen, ist kein limitierender Faktor, im 3D-Audio-Raum der BHT. Auch die ca. 500 - 1000g Gewicht eines Mini-Desktop-PCs stellen kein Problem dar. Höhere Rechenleistung und Kompatibilität sind dahingegen relevante Faktoren. Sie erhöhen die Flexibilität des Projekts und tragen zur Problemprävention bei. Daher soll für dieses Projekt ein Mini-PC als Recheneinheit zum Einsatz kommen.

*Welcher Mini-PC genau*

Es wird ein möglichst leiser und Performanter Mini-PC in der Preisklasse von etwa 500€ gesucht. In dieser Preisklasse ist keine performante dedizierte (eigenständige) grafische Prozessoreinheit (GPU / Grafikkarte) zu erwarten. Die GPU wird demzufolge im zentralen Prozessor integriert sein. Bei der PC-Wahl sind folgende Faktoren zu vergleichen:

- Speichergröße und -geschwindigkeit des Arbeitsspeichers (RAM)
- Speichergröße und -geschwindigkeit des Massenspeichers – also der Festplatte
- Performance der zentralen Prozessoreinheit (CPU)

- Lautstärkepegel der PC-Kühlung – also der Ventilatoren
- Schnittstellen für andere Geräte – also z.B.: USB, HDMI, LAN oder WLAN

Bezüglich der Schnittstellen sollen mindestens drei USB-A-Ports (inklusive einmal USB 3.0), USB-C, LAN und WLAN und für einem Bildschirm HDMI, DisplayPort (DP) oder USB-C mit DP Alt Mode zur Verfügung stehen.

Für den RAM wird ein Sollwert von mindestens 16GB DDR4 (Double Data Rate 4) angesetzt. Die Festplatte sollte 1TB SSD-Speicher aufweisen. Auf eine Genaue Spezifikation des SSD-Speichers wird verzichtet, es wird davon ausgegangen das die Handelsüblichen SSDs ausreichend performant sind und in diesem Projekt nicht den „Flaschenhals“ der Systemperformance darstellen werden.

Speicher und Schnittstellen der Mini-PCs in der angestrebten Preisklasse, sind bei den betrachteten Modellen ähnlich und erfüllen zumeist die genannten Anforderungen.

Der meiste Gewinn an Systemperformance wird sich von einer passenden Prozessorwahl erhofft.

MediaPipe kann von verschiedenen Recheneinheiten gleichzeitig gebrauch machen, siehe Abschnitt 2.4.1 Stichwort *Multithreading*
. Neben dem Prozessortakt und der Single-Core-Leistung ist somit auch die Gesamtleistung aller Prozessorkerne (Multi-Core-Leistung) zu berücksichtigen.

Die mindest Hardwarevoraussetzungen für MediaPipe sind für einen aktuellen Desktop-PC leicht zu erfüllen, siehe [101].

Weitere Angaben zu geeigneter Hardware für Bildverarbeitung oder KI-Anwendungen beziehen sich oft nur auf leistungsstarke Rechensysteme nicht auf die Dimensionen eines Mini-PCs. Beim Einsatz von MediaPipe Pose und Hands ist zu erwarten, dass die Hauptlast aus neuronalen KI-Anwendungen kommen wird, nicht aus der klassischen Bildverarbeitung. Für KI-Anwendungen wird betont, dass die gute GPU-Performance wichtig ist – aber das gibt wie gesagt das Budget nicht her. Weiterhin werden die Prozessoren Intel i7, i9 und AMD Ryzen 7, 9 empfohlen. [102], [103]

Diese CPU-Serien enthalten teilweise eine Recheneinheit, welche für die Anforderungen von neuronalen Netzen, wie CNNS, optimiert ist. Dieser sogenannte neuromorphe Prozessor [104] , kurz NPU, ist eine Art zusätzlicher Kern in der CPU. Bei Intel wird diese Technologie Core Ultra [105] genannt, bei AMD ist es Ryzen AI [106]. PCs mit NPU sind teilweise schon innerhalb des Budgets erhältlich, und werden für dieses Projekt priorisiert.

Neben dem Prozessor ist die Lautstärke des Mini-PCs ein relevanter Faktor. Der Minisforum UM760 Slim [107] ist ein sehr leiser MINI-PC, sagt heise online [108]. Sein AMD Ryzen 5 7640HS hat eine NPU und gute Benchmarks [109] (gemessene Rechenleistung bei verschiedenen Testanwendungen) in seiner Preisklasse. Er hat 16GB DDR4 RAM und 1TB SSD-Festplattenspeicher.

Der GMKtec NucBox M6 [110] hat die gleichen Komponenten nur nochmal 16 GB RAM mehr. Im Gegenzug wird seine Betriebslautstärke von CHIP.de [111] negativ bewertet.

In dem Preisbereich sind das die performantesten Modelle, welche hier gefunden wurden. Die Arbeitsspeichergröße wird besonders dann zum Leistungsfaktor, wenn eine Anwendung mehr Speicher benötigt, als verfügbar ist. Dann müssen Daten auf den „langsamen“ Festplattenspeicher ausgelagert werden. In diesem Projekt wird davon ausgegangen das 16GB RAM ausreichen. Die leisere Betriebslautstärke wird damit als relevanter betrachtet und die Wahl fällt auf den Minisforum UM760 Slim als Recheneinheit für dieses Projekt. Die folgende Figure zeigt diesen PC.



**Figure 36 - Minisforum UM760 Slim [107]**

### 5.1.3 Bildschirmwahl

Vorerst soll ein beliebiger PC-Monitor der BHT über dem frontalen mittleren Lautsprecherboxen befestigt werden, um überhaupt eine Bildausgabe und ein Benutzer Feedback zu haben. Es ist denkbar das dieser Standmonitor zu einem späteren Zeitpunkt durch eine dezente Lösung ersetzt wird. Bei einer späteren Auswahl wären volgende Punkte zu beachten:

- Preis
- Ist eine Bedingung über integrierten Touchscreen Möglich
- Kompatibilität mit den üblichen Bildschirmanschlüssen HDMI oder DisplayPort
- Möglicher Verzicht auf ein extra Netzteil durch Stromversorgung per USB-C DP Alt Mode Anschluss
- Gerätegröße und Gewicht
- Bildgröße und -auflösung
- Befestigungsmöglichkeiten für die Befestigung am Gerüst im 3D-Audioraum

## 5.2 Konzeptwahl und Einstieg

Aus Gründen von Zeit und Aufwand wird in dieser Arbeit nur eines der beiden entwickelten Konzepte implementiert. Die Wahl fällt auf das KI-basierte Konzept mit MediaPipe, da von diesem Ansatz ein besseres Endergebnis erwartet wird. Dafür sprechen folgende Gründe:

- Gute Informationsgrundlage: Die Skeletterkennung liefert eine umfangreiche Datengrundlage für die Spezifikation und Umsetzung der Gesten.

- Umfangreiche Dokumentation und Referenzprojekte: Zahlreiche Projekte im Internet nutzen MediaPipe erfolgreich zur Gestenerkennung. Die verfügbaren Informationen und Lösungsansätze erleichtern die Entwicklung sowie die spätere Fehlersuche.

- Ausreichende Rechenleistung: Durch die Auswahl eines geeigneten Mini-PCs werden keine wesentlichen Leistungsprobleme erwartet.

- Geringerer Implementierungsaufwand: MediaPipe übernimmt bereits die Erkennung relevanter Körperbereiche und die Bereitstellung der zugehörigen Landmarks.

*Kurzer erster Test von MediaPipe*

Als Startpunkt wird der Code aus dem Projekt [66] übernommen da dieser bereits MediaPipe Pose und MediaPipe Hands erfolgreich nutzt. Er wir zunächst unverändert getestet. Als Testmaterial dienen RGB-Videoaufnahmen, die mit der Intel RealSense D455 im 3D-Audioraum erstellt wurden.

Die Körpererkennung mit MediaPipe Pose liefert in etwa 98–99 % der Frames, in denen eine Person sichtbar ist, Ergebnisse für die 33 geschätzten Körperlandmarks. Die Positionen der Landmarks erscheinen bei visueller Überprüfung ausreichend präzise für die geplante Anwendung.

Beim ersten Testlauf funktioniert die Erkennung des Handskeletts hingegen noch nicht zufriedenstellend. Die Ursache und Lösung dafür werden in Abschnitt 5.4 näher erläutert.

## 5.3 Erkennung der Raumrichtung

### 5.3.1 Hand- und Schulterposition

Für die Erkennung der Raumrichtung muss grundsätzlich erstmal der zeigende Arm erkannt werden. Bzw. die Position von Schulter und dem Handzentrum. MediaPipe Pose beinhaltet in seinen Landmark-Resultaten vier Landmarks für jede Hand und auch je eine Schulter-Landmarke. Das Handzentrum wird für dieses Projekt durch die Mitte der äußeren beiden Hand-Landmarks (kleinen Fingers und Zeigefingeransatz) definiert. Sollte MediaPipe Hands im späteren Verarbeitungsverlauf eine valide Hand in dieser Nähe erkennen, wird das Handzentrum überschrieben. Die erwartungsgemäß präziseren

Positionsdaten aus der separaten Handdetektion werden dann für das Handzentrum verwändet (Landmarke 5 und 17, siehe Figure 5). Zunächst werden jedoch nur die Bildlandmarks und World-Landmarks von MediaPipe Pose gespeichert.

### 5.3.2 3D-Raum-Koordinaten

*Nur aus dem RGB-Bild*
Für die Berechnung der Zeige- und dann der Raumrichtung ist eine möglichst exakte 3D-Position der Schulter und Hand (Handzentrum) im Raum nötig. Für die Ermittlung der dreidimensionalen Raumkoordinaten wurden zunächst die World-Landmarks (siehe, *Landmarks* in 2.4.1) genutzt. Mit diesen wurden Tests durchgeführt, eine zuverlässige Winkelgenauigkeit konnte nur in etwa 45°-Schritten erzielt werden. Mehr dazu im Anhang unter *3D-Raum-Koordinaten mit MediaPipe world Landmarks*.

*Mit Tiefenkamera*
Für die Berechnung der Raumkoordinaten werden die relativen Bild-Koordinaten der MediaPipe Bild-Landmarks (siehe, *Landmarks* in 2.4.1) zusammen mit der Tiefeninformation der RealSense-Kamera genutzt. Der Ansatz mittels World-Landmarks wurde dennoch in der Gestensteuerung implementiert gelassen, dieser wird automatisch verwändet sollte eine 2D-Kamera ohne Tiefensensoren angeschlossen werden.

Das Vorgehen zur Raumkoordinatenermittlung mittels Tiefensensor ist wie folgt:

- mit der RealSense-Treibersoftware wird ein Kameraframe eingelesen, dieser beinhaltet ein Objekt für den RGB-Frame und eins für den Tiefenframe.

- MediaPipe verarbeitet den RGB-Frame und gibt die relativen Bild-Koordinaten in seinen Bild-Landmarks aus

- diese relativen Koordinaten werden mit den Frame-Dimensionen multipliziert, um Pixel-Koordinaten zu erhalten.

die Pixelpositionen des Tiefenbilds werden mit denen des RGB-Bilds synchronisiert. Siehe dazu auch *Alignment für 3D-Koordinaten* im Abschnitt 2.11. Dazu wird über die Treibersoftware ein `pyrealsense2.align` Objekt erstellt.

- die RGB-Pixelkoordinaten aus MediaPipe können jetzt auf die entsprechende Stelle im Tiefenbild zeigen. Das Tiefenbild Objekt der Treibersoftware hat eine `get_distance()` Methode, mit dieser wird die entsprechende Tiefeninformation eines Bildpunktes ermittelt.

- die X-Y-Pixelkoordinaten und die Z-Tiefeninformation werden unter Kenntnis der synchronisierten intrinsischen Kameraparameter in 3D-Kamerakoordinaten umgerechnet. Dazu dient die Funktion:
  `pyrealsense2.rs2_deproject_pixel_to_point()`.

*Koordinatensystem*
Die Kamerakoordinaten sind jetzt bereits in der Einheit Meter. Sie werden schlussendlich noch über die Kameraposition und ggf. -neigung zu Raumkoordinaten verrechnet, siehe dazu auch Formel 9 - *Kamerakoordinaten zu Raumkoordinaten*. Die Kamera ist nicht rotiert und gerade ausgerichtet, somit ist nur die Position der Kamera relativ zur Raummitte zu addieren. Eine mögliche Kameraneigung nach unten in Gerad, kann in den Einstellungen gesetzt werden, diese wird dann automatisch verrechnet.

### 5.3.3 Zeigerichtung des Arms

Die Zeigerichtung des Arms wird durch zwei Winkelkomponenten beschrieben: den Azimut- und den Elevationswinkel. Als Grundlage dient der Armvektor, der zwischen dem dreidimensionalen Schulterpunkt und dem dreidimensionalen Mittelpunkt der Hand aufgespannt wird. Der Armvektor beschreibt somit die räumliche Richtung vom Schulterpunkt zum seitengleichen Handzentrum.

*Azimutwinkel*
Für die Berechnung des Azimutwinkels werden ausschließlich die X- und Z-Komponenten des Armvektors berücksichtigt. Die Y-Komponente (höhe) wird dabei nicht betrachtet, da sie für die horizontale Richtungsbestimmung nicht relevant ist. Als Referenz dient ein Raumvektor, der ausgehend vom Schulterpunkt in negativer Z-Richtung (zur Kamera hin) aufgespannt wird. Der Azimutwinkel entspricht dann dem Winkel zwischen diesem Referenzvektor und dem Armvektor, welcher nur auf der X-Z-Ebene abgebildet wird. Dadurch wird die horizontale Zeigerichtung unabhängig von der Höhe des Arms bestimmt. Der Azimutwinkel wird mithilfe der Arcustangens-2-Funktion (Formel 10) im 2D-X-Z-Raum bestimmt. Das Ergebnis sind Vollkreiswerte zwischen $\pm 180°$, Pluss ist hierbei Arm nach rechts.

*Elevationswinkel*
Für die Berechnung des Elevationswinkels werden hingegen alle drei Komponenten des Armvektors berücksichtigt. Als Referenz wird ein Raumvektor verwendet, der ausgehend vom Schulterpunkt in positiver Y-Richtung verläuft, also gerade nach unten. Der Winkel zwischen dem Armvektor und diesem Referenzvektor wird mithilfe der Arcus-Kosinus-Funktion (Formel 11) bestimmt. Anschließend wird vom berechneten Winkel ein Wert von 90° subtrahiert. Dadurch wird der Elevationswinkel so definiert, dass ein nach unten gerichteter Arm einen Winkel von −90° und ein nach oben gerichteter Arm einen Winkel von +90° entspricht.

### 5.3.4 Raumrichtung

Über ein Raytracing-Verfahren wird ein Punkt an der Raumgrenze des Systems referenziert. Nahe der Systemraum-Wände und -Decke befinden sich auch alle 21 Lautsprecher des Audiosystems. Der Vektor zwischen diesem Punkt und dem Zentrum (Hälfte jeder räumlichen Ausdehnung) des Systems ist die Raumrichtung. Die Raumrichtung wird in Azimut und Elevationswinkel aufgeteilt angegeben. Seine Aufteilung gleicht der Berechnung, beschrieben im vorherigen Absatz *Zeigerichtung des Arms*.

### 5.3.5 Raytracing zur Raumgrenze

Zur Bestimmung des durch eine Zeigerichtung referenzierten Punktes an der Raumgrenze wird ein vereinfachtes Prinzip des Raytracings, zu Deutsch Strahlenverfolgung verwendet. [112]
Ausgehend von einem Ursprungspunkt und der ermittelten räumlichen Richtung wird ein Strahl verfolgt, bis dieser auf eine der Begrenzungen des definierten Raumes trifft. Der Schnittpunkt bzw. Kollisionspunkt zwischen dem Strahl und einer raumbegrenzenden Fläche beschreibt dabei die durch die Zeigerichtung bestimmte Zielposition. Für die Implementierung in Python werden folgende Parameter definiert:

- Startpunkt des Strahls: der Schulterpunkt des Dirigenten
- Raumgrenzen: Position der Systemwände, -decke und -boden
- Sinus- und Cosinus-Anteile der Azimut- und Elevations-Komponenten der Zeigerichtung für die schrittweise Verfolgung des Strahls durch seine Anteile in die drei Raumrichtungen.
- Schrittweite mit welcher der Strahl verfolgt wird. Diese bestimmt die Genauigkeit für das Ergebnis.

Die Schrittweite wird über die Sinus- und Cosinus-Anteile gewichtet in drei räumliche Komponenten aufgeteilt, siehe Formel 12.

**Formel 12 - Richtungsanteile von Azimut und Elevation im Raumkoordinatensystem**

$$x = step \cdot \cos(\alpha_E) \cdot \sin(\alpha_A)$$
$$y = step \cdot \sin(\alpha_E)$$
$$z = step \cdot \cos(\alpha_E) \cdot \cos(\alpha_A)$$
$$mit$$
$$x : Breite\text{nschritt}$$
$$y : Höhe\text{schritt}$$
$$z : Tiefe\text{schritt}$$
$$\alpha_A : Azimut\ Winkel$$
$$\alpha_E : Elevations\ Winkel$$

In einer Loop wird der 3D-Startpunkt schrittweise mit den räumlichen Komponenten aufaddiert, bis seine Position eine der sechs Raumgrenzen erreicht. Dann bricht die Loop ab und gibt diese aufaddierte 3D-Position aus.

### 5.3.6 Sonderfall

Zeigt ein gestreckter Arm direkt in die Kamera, liegen Hand und Schulter im Bild direkt übereinander, sodass die Hand die Schulter verdeckt. In diesem Fall kann die korrekte Tiefeninformation der Schulter nicht aus dem Tiefenbild ermittelt werden, da stattdessen die Tiefe der Hand erfasst wird. Eine sinnvolle Berechnung der Zeigerichtung ist dadurch nicht möglich.

In diesem Sonderfall wird der Vektor zur Berechnung der Raumrichtung zwischen der Kameraposition und dem Raumzentrum aufgespannt. Mit anderen Worten: Der durch das Raytracing ermittelte Punkt an der Raumgrenze wird durch die Kameraposition ersetzt und nicht anhand der Zeigerichtung referenziert.

Zur Erkennung dieses Sonderfalls wird der Abstand zwischen der Hand- und der Schulter-Landmarke betrachtet. Ist dieser Abstand zu gering, wird davon ausgegangen, dass die Hand die Schulter verdeckt.

Für alle Armpositionen, bei denen die Hand bildlich vom Körper verdeckt wird, gibt es hier keine alternative Bestimmungsmöglichkeit. Ein sinnvolles Ergebnis ist in diesen Fällen nicht zu erwarten.

## 5.4 Erkennung der Hand

Für jede gefundene Handposition aus MediaPipe Pose wird ein sogenanntes Region of Intrest (ROI) definiert, jedes ROI dient als eingangs Bild für eine Instanziierung einer separaten MediaPipe Handerkennung. Alle Erkennungen von Handgesten (so auch die Greifgeste) basieren auf Geometrischen Berechnungen der Bildkoordinaten aus den 21 Hand-Landmarks (siehe Figure 5) von MediaPipe Hands.

*Region of Intrest*

Ein ROI ist notwendig damit MediaPipe Hands, die Hand nicht im Gesamtbild suchen muss. Die Erkennung eines Handskeletts funktionierte hier am besten, wenn das zu analysierende Bild, etwa 2 - 5 mal den Durchmesser der Hand im Bild hat. Siehe dazu auch Abschnitt *ROI-Test* im Anhang.

Erkennt MediaPipe Pose eine Hand innerhalb des Bildbereichs und liegt deren Visibility-Wert über einem festgelegten Schwellenwert, wird für diese Hand ein ROI erstellt. Liegt der Visibility-Wert unterhalb des einstellbaren Schwellenwerts, wird davon ausgegangen, dass die Hand verdeckt ist, beispielsweise weil sie sich hinter dem Rücken oder in einer Hosentasche befindet. In diesem Fall ist keine zuverlässige Erkennung der Hand durch MediaPipe Hands zu erwarten.

Das ROI wird als Quadrat definiert. Dessen Mittelpunkt entspricht der Position des von MediaPipe Pose bestimmten Handzentrums. Dieses wird aus dem Mittelwert zweier Pose-Landmarks berechnet. Dabei handelt es sich um die von MediaPipe Pose geschätzten Ansatzpunkte des kleinen Fingers und des Zeigefingers.

Damit sich die Größe des ROI an den Abstand der Person zur Kamera und die damit verbundene bildliche Handgröße anpassen kann, wird die erkannte Person als Maßstab verwendet. Die Seitenlänge des quadratischen ROI entspricht dabei der Länge des Oberkörpers der erkannten Person. Diese wird als Abstand zwischen den Landmarks der Schultern zu den Hüften berechnet. Figure 37 zeigt unter anderem beide Hände und ihre ROIs.

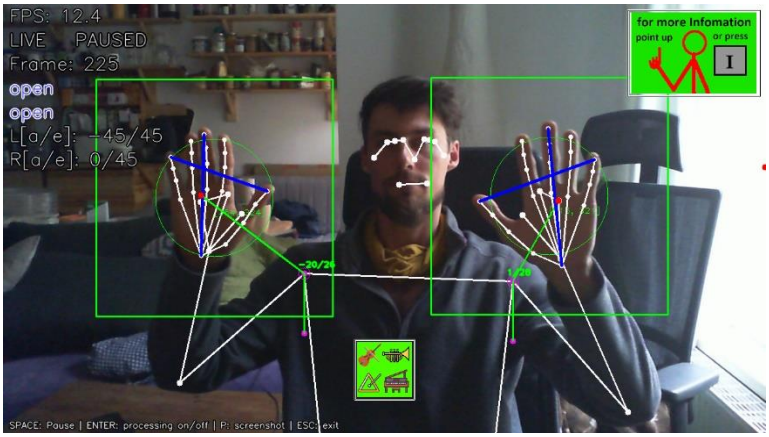



**Figure 37 - MediaPipe Pose und Hand Erkennung mit ROIs als grünes Quadrate**

### *Validierung der Hand*

Um Falscherkennungen entgegenzuwirken, wird die Position der Hand aus der Hand-Erkennung mit der Handposition aus der Pose-Erkennung abgeglichen. Der Mittelpunkt eines ROIs ist ein Handzentrum aus der Pose-Erkennung. Der euklidische Abstand von diesem zum Zentrum aus der Handerkennung wird geprüft. Es wird ein Maximalabstand definiert, dieser ist durch die dünnen grünen Kreise in Figure 37 dargestellt. Liegt das Zentrum der erkannten Hand aus der Hand-Erkennung außerhalb dieses Kreises, wird von einer Falscherkennung ausgegangen und die Hand wird nicht weiter ausgewertet.

Zudem soll eine Hand nicht analysiert werden, wenn sie sich während der Aufnahme über das Bild bewegt. Hierzu wird die Pixelposition des Handzentrums von Frame zu Frame verfolgt, um daraus die Bewegungsgeschwindigkeit der Hand zu bestimmen. Dafür werden zwei Parameter verwendet: eine maximal zulässige Bewegungsgeschwindigkeit in Pixeln pro Sekunde sowie ein Zeitintervall, über das die Positionsänderung betrachtet wird. Das Zeitintervall legt dabei fest, über wie viele Frames der Ringspeicher die Handposition verfolgt.

Die Wahl des Zeitintervalls beeinflusst dabei die Empfindlichkeit und Reaktionsgeschwindigkeit der Erkennung. Bei einem zu kurzen Zeitintervall können bereits geringe, unbeabsichtigte Bewegungen oder Schwankungen des ermittelten Handzentrums als relevante Bewegung erkannt werden. Ein zu langes Zeitintervall führt hingegen zu einer verzögerten Erkennung von Bewegungen und kann dazu führen, dass Bewegungen mit einer eigentlich zu großen Amplitude noch innerhalb des zulässigen Bereichs liegen.

## 5.5 Greifgestenerkennung

Es werden zwei Methoden zur Erkennung einer Greifgeste implementiert und getestet, eine statische und eine dynamische.

### *Statische-Methode*

Basierend auf der Bestimmung des Hand-Aperture-Werts (Handöffnungsgrads) aus dem Projekt [66] wird die statische Methode zur Erkennung der Greifgeste konzipiert.

Der Hand-Aperture-Wert liegt in einem Wertebereich von 0 bis 100 und wird anhand zweier Schwellwerte ausgewertet. Bei einem Wert über 70 (einstellbare Schwelle) wird die Hand als offen erkannt, bei einem Wert unter 60 (einstellbare Schwelle) als geschlossen. Liegt der Wert zwischen diesen beiden Schwellwerten, bleibt der bisherige Handstatus unverändert. Dadurch wird ein Wechsel des Handstatus verhindert, wenn sich die Hand in einem teilweise geöffneten Zustand zwischen 60 und 70 befindet.

Zusätzlich wird der Handstatus über mehrere Frames erfasst. Die Anzahl der zu berücksichtigenen Frames ist einstellbar. Werte zwischen $x = 3$ und $x = 6$ Frames haben sich als geeignet erwiesen. Nur wenn bei mindestens $n$ der insgesamt $m$ betrachteten Frames derselbe Handstatus erkannt wird, wird dieser als neuer Handstatus übernommen. Als geeigneter Wert für $n$ hat sich $n = m$ oder $n = m - 1$ erwiesen. Dadurch können einzelne fehlerhafte Erkennungen ausgeglichen und ein unbeabsichtigter Statuswechsel verhindert werden. Eine Erhöhung der Anzahl der betrachteten Frames verbessert zwar die Robustheit gegenüber kurzzeitigen Fehldetektionen, führt jedoch gleichzeitig zu einer höheren Latenz bei der Erkennung der Greifgeste.

Wechselt der Handstatus von offen zu geschlossen, wird eine Greifgeste erkannt und der Greifstatus aktiviert. Wechselt der Handstatus anschließend wieder von geschlossen zu offen, wird eine Loslassgeste erkannt und der Greifstatus aufgehoben.

Der in [66] verwendete Hand-Aperture-Wert wird ausschließlich anhand der Handlänge bestimmt. Zeigt die Handfläche direkt in Richtung der Kamera, kann aufgrund der perspektivischen Projektion keine zuverlässige Handlänge bestimmt werden. Die Handbreite ist aus dieser Perspektive hingegen gut erkennbar. Zeigt dagegen die Handkante in Richtung der Kamera, ist die Handbreite nur eingeschränkt bestimmbar.

Um diese Einschränkungen zu berücksichtigen, werden zunächst sowohl die Länge als auch die Breite der Handfläche in Pixeln bestimmt. Ist der ermittelte Längenwert größer als der Breitenwert, wird der Hand-Aperture-Wert auf Grundlage der Handlänge nach dem Verfahren des ursprünglichen Projekts bestimmt. Überwiegt hingegen der Breitenwert, werden die Handbreite sowie der Abstand zwischen Daumen und kleinem Finger als Grundlage für die Berechnung des Hand-Aperture-Werts verwendet. Die hierfür verwendeten Strecken sind in Figure 38 als blaue Geraden dargestellt.

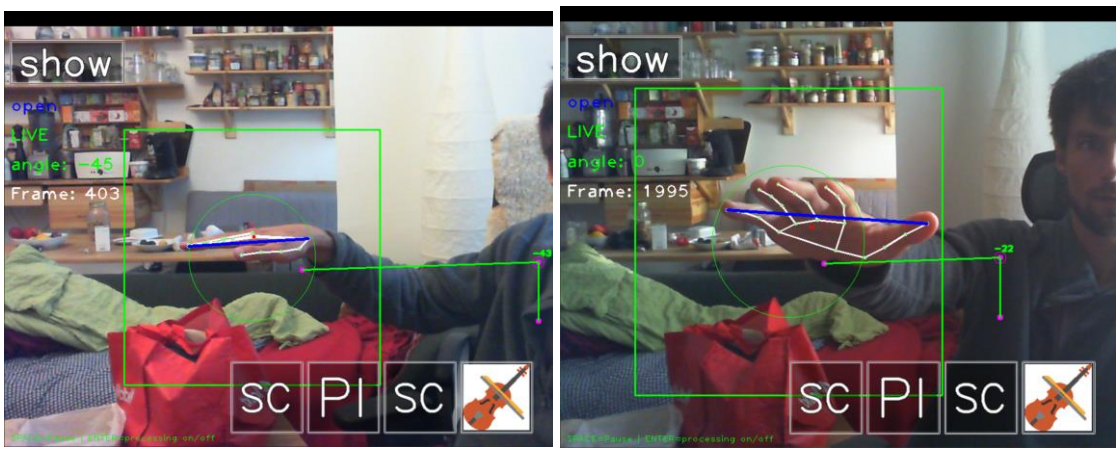



**Figure 38 - Handstatus mit statischer Methode**

*Dynamische-Methode*

Für die Erkennung des Handstatus werden zwei Distanzen zwischen den Hand-Landmarks bestimmt:

- Handlänge: Abstand zwischen Handgelenk und Mittelfingerspitze (Hand-Landmark 0 zu 12, siehe Figure 5)

- Handbreite: Abstand zwischen Daumenspitze und Spitze des kleinen Fingers (Hand-Landmark 4 zu 20)

Figure 39 zeigt diese Distanzen als blaue Linien. Bei einer geöffneten Hand sind beide Distanzen räumlich annähernd gleich groß. Für die weitere Auswertung wird jeweils die perspektivisch größere Distanz verwendet. Die ausgewählte Distanz wird anschließend in einem Ringspeicher abgelegt, der die Messwerte der letzten 0,8 Sekunden (einstellbarer Wert) enthält.

Für die Auswertung werden der kleinste (Minimum) und der größte (Maximum) Distanzwert innerhalb des Ringspeichers bestimmt. Übersteigt der maximale Wert den minimalen Wert um den eingestellten Differenzfaktor, wird von einer Veränderung des Handstatus ausgegangen. Folgt der minimale Wert zeitlich dem maximalen Wert, wird der Handstatus auf *geschlossen* gesetzt. Tritt die umgekehrte Reihenfolge auf, wird der Handstatus auf *offen* gesetzt.

Mit dieser Methode wird ein Statuswechsel nur dann erkannt, wenn die Veränderung der Handstellung innerhalb eines bestimmten Zeitraums ausreichend schnell erfolgt. Die hierfür erforderliche Geschwindigkeit kann über den Differenzfaktor, also das Verhältnis zwischen Minimal- und Maximalwert, sowie über die Länge des Ringspeichers eingestellt werden. Dadurch können langsame und beiläufige Veränderungen der Handstellung von einer gezielten Greifgeste unterschieden werden.

Allerdings können auch andere Bewegungen der Hand zu einer Veränderung der gemessenen Distanz führen. Insbesondere eine Drehung der Hand kann die perspektivisch gemessene Distanz verändern und unter Umständen eine ausreichende Differenz zwischen Minimal- und Maximalwert erzeugen, sodass fälschlicherweise ein Wechsel des Handstatus erkannt wird.




**Figure 39 - Handstatus mit dynamischer Methode**

Zusätzlich zu einer dieser Methoden kommt eine Regel zum Einsatz:

> Wird ca. zwei Sekunden keine Hand von MediaPipe Hands erkannt, so gilt die Hand als abwesend im Bild oder perspektivisch verdeckt und der Handstatus wird auf *no hand* gesetzt.

Ein Test mit Annotiertem Videomaterial soll eine bessere Einschätzung für die Güte der Methoden ermöglichen. Auch soll der Einfluss der Schwellwerte und Ringspeichergrößen eingeschätzt werden. Eine genauere Testbeschreibung ist im Anhang unter *Greifgestenerkennungen im Test* zu finden. Die statische Methode mit den genannten Schwellwerten hatte die höchste erkennungsrate, diese erkannte in 87.0% der Frames den richtigen Handstatus. Ein Probandentest mit einem Prototyp der Gestensteuerung ist für eine realitätsnähere Einschätzung der Güte sinnvoll, siehe dazu Kapitel 6 - *Test der Gestensteuerung*.

## 5.6 Steuergesten

Es sind vier Gesten für die Steuerung des Programms definiert:

- Zeigefinger nach oben: ein Info-Fenster wird ein oder Ausgebländet
- Daumen nach unten: das GUI wird zurückgesetzt
- Hände vor die Augen: die Menge der angezeigten Live-Informationen ändert sich
- Arme kreuzen: das Programm wird beendet

Die ersten beiden Gesten sind Handgesten, diese Funktionieren nur wenn Hand-Landmarks gefunden wurden. Alle Gesten Werten die Bildliche Position von Landmarks geometrisch aus.

*Infogeste*

Für die Erkennung der Infogeste wird zuerst geprüft, ob der Zeigefinger nach oben zeigt. Es wird die Differenz der X- und Y-Achse zwischen Fingeransatz und -spitze verglichen. Ist $dy > 2 \cdot dx$, gilt dies als erfüllt. Danach wird geschaut, ob die restlichen Finger eine Faust bilden, hier für wird der euklidische Abstand von jeder der drei anderen Fingerspitzen zur Zeigefingerspitze und zur Daumenspitze gemessen. Sind alle drei Fingerspitzen dichter am Daumen als an der Zeigefingerspitze, gilt auch dies als erfüllt.

Als letztes kommt eine zeitliche Komponente „ins Spiel“ diese soll verhindern, dass eine beiläufige Handbewegung durch Zufall die Bedingungen für diese Geste erfüllt. Die zuvor beschriebene Handhaltung muss lückenlos über einen Zeitraum von 0.7 Sekunden in jedem Frame erkannt werden, um die Infogeste auszulösen.

Die Geste wird anschließend als Trigger verwendet. Sie löst nur beim Übergang von einem nicht erkannten zu einem erkannten Gestenzustand aus und wird nicht bei jedem weiteren Frame erneut ausgelöst, solange die Handhaltung beibehalten wird.

Das folgende Figure zeigt diese Geste als Schaubild und in echt.

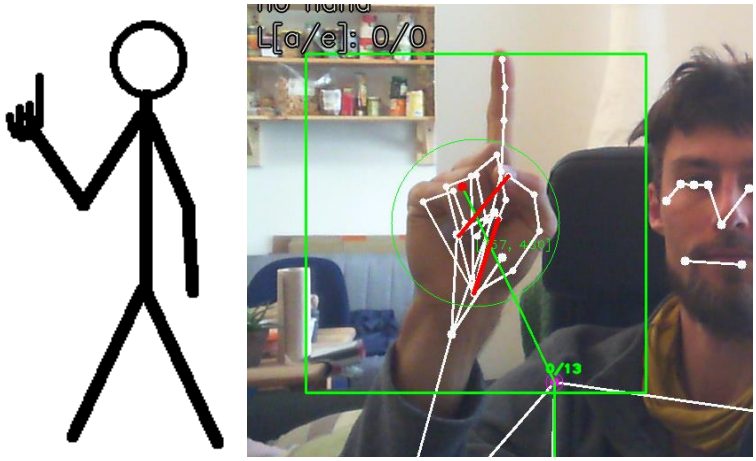



**Figure 40 – Infogeste als Schaubild und in echt**

*Reset-Geste*

Für die Erkennung der Reset-Geste wird zunächst geprüft, ob sich die Hand zwischen den beiden Schultern befindet. Die Geste soll nur erkannt werden, wenn diese vor dem Körper, also zwischen den vertikalen Schulterlinien ausgeführt wird.

Anschließend wird anhand der Position der Handfläche und des Daumens bestimmt, ob der Daumen nach unten zeigt. Hierfür wird zunächst die Orientierung der Handfläche ausgewertet. Die Handfläche muss dabei annähernd vertikal ausgerichtet sein. Zusätzlich muss die Ausdehnung des Daumens auf der Y-Achse größer sein als auf der X-Achse, zudem muss diese nach unten zeigen. Zeigen also Handfläche und Daumen in einer Linie nach unten, wird eine nach unten gerichtete Daumenstellung erkannt.

Anschließend wird geprüft, ob die übrigen vier Finger eine Faust bilden. Hierzu werden für jeden Finger die Abstände zwischen Handgelenk und Fingerspitze sowie zwischen Handgelenk und dem jeweiligen Fingeransatz bestimmt. Die Summe der Abstände zu den Fingerspitzen muss kleiner sein als die Summe der Abstände zu den Fingeransätzen. Sind diese Bedingungen erfüllt, wird die Geste als gültig erkannt.

Wie auch bei der Infogeste, wird schlussendlich eine zeitliche Komponente verwendet, um eine zufällige oder nur kurzzeitig eingenommene Handstellung nicht als Reset-Geste zu interpretieren. Die beschriebene Handhaltung muss mindestens 1.0 Sekunden lang durchgehend erkannt werden. Erst anschließend wird die Reset-Geste ausgelöst.

Auch diese Geste wird, wie die Infogeste als Trigger verwendet. Das folgende Figure zeigt diese Geste als Schaubild und in echt.

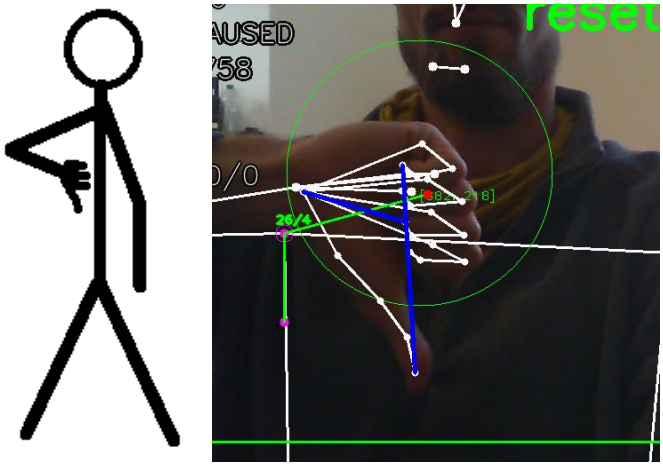


**Figure 41 - Reset-Geste als Schaubild und in echt**

*Ansicht-ändern-Geste*

Für die Erkennung der Ansicht-ändern-Geste wird geprüft, ob sich beide Hände in der Nähe der Augen befinden. Hierzu werden die von MediaPipe Pose erkannten Positionen der Hände und Augen verwendet. Zunächst wird der Abstand zwischen den beiden Augen als Referenzgröße bestimmt. Anschließend wird für jede Hand der Abstand zum seitengleichen Auge berechnet. Liegen beide Hände innerhalb eines festgelegten Abstands (eineinhalbmal der Augenabstand) zu den Augen, wird davon ausgegangen, dass die Hände die Augen bedecken.

Zusätzlich wird überprüft, ob beide Handpositionen durch MediaPipe Pose ausreichend sichtbar erkannt werden. Hände, welche hinter dem Kopf verschränkt sind, sollen diese Geste nicht auslösen. Sind die genannten Bedingungen erfüllt, beginnt die zeitliche Erfassung der Geste. Die Hände müssen mindestens 0,5 Sekunden lang in der beschriebenen Position verbleiben, bevor die Ansicht-ändern-Geste als erkannt gilt. Dadurch werden kurzzeitige Bewegungen der Hände in den Gesichtsbereich nicht unmittelbar als Geste interpretiert.

Auch diese Geste wird, wie die zwei Gesten zuvor als Trigger verwendet. Die folgende Figure zeigt diese Geste als Schaubild und in echt.



**Figure 42 - Ansicht-ändern-Geste als Schaubild und in echt**

*Beenden-Geste*

Für die Erkennung der Beenden-Geste werden die Positionen beider Hände und Ellbogen sowie die Position des Augenzentrums ausgewertet. Zunächst wird geprüft, ob sich beide Hände oberhalb des Kopfes befinden. Hierzu muss die vertikale Position beider Hände oberhalb der Augen liegen. Das verhindert ein Auslösen durch vor der Brust verschränkte Arme.

Anschließend wird überprüft, ob beide Hände jeweils oberhalb des gegenüberliegenden Ellbogens liegen. Als Maß für die notwendige vertikale Differenz wird ein Drittel der Länge eines Unterarms verwendet. Abschließend wird anhand der horizontalen Positionen von Händen und Ellbogen festgestellt, ob die Arme gekreuzt sind. Dazu wird in X-Richtung verglichen, ob die Reihenfolge der Hände (linke Hand, rechte Hand) von der Reihenfolge der Ellbogen abweicht.

Sind die Arme oberhalb des Kopfes gekreuzt, beginnt die zeitliche Erfassung der Geste. Die Position muss mindestens 2.0 Sekunden lang durchgehend erkannt werden. Erst danach wird die Beenden-Geste ausgelöst. Dadurch werden kurzzeitig gekreuzte oder zufällig über den Kopf bewegte Arme nicht unmittelbar als Beenden-Geste interpretiert. Die folgende Figure zeigt diese Geste als Schaubild und in echt.

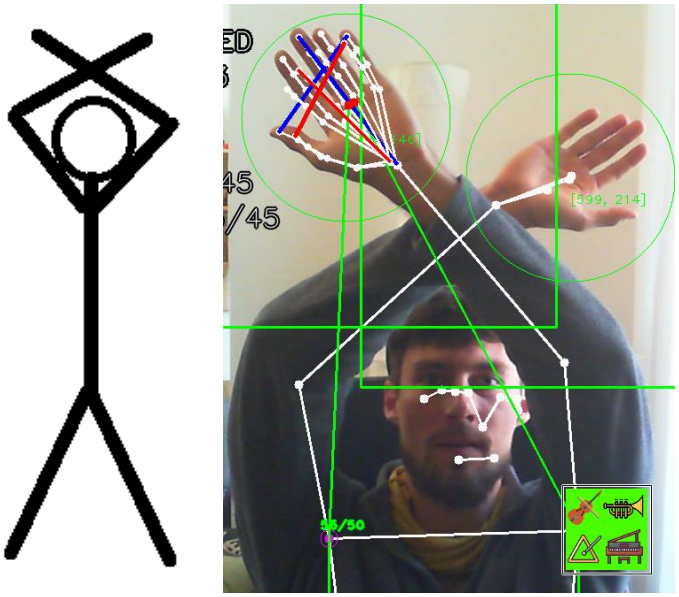



**Figure 43 - Beenden-Geste als Schaubild und in echt**

## 5.7 Grafische Benutzeroberfläche

Die grafische Benutzeroberfläche (GUI) dient zur Auswahl und Steuerung der im virtuellen Raum dargestellten Instrumente. Sie wird als Overlay auf dem von der Kamera aufgenommenen Bild dargestellt und verarbeitet die Positionen sowie die Greif- und Loslassgesten der erkannten Hände. Zusätzlich kann die Maus als weiterer Eingabekanal verwendet werden. Das Drücken der linken Maustaste ist hierbei das Äquivalent zur Greifgeste. Für die drei möglichen Eingabekanäle (linke/rechte Hand und Maus) werden jeweils die Position, der Greifstatus und der Loslassstatus an die GUI übergeben.

### *Auswahl*

Die GUI besteht aus verschiedenen grafischen Elementen, die als einzelne Kacheln

implementiert sind. Ein Instrument wird zunächst als Kachel in einer Auswahlleiste dargestellt. Durch das Positionieren eines Zeigers (linke/rechte Hand oder Maus) über einer Kachel wird das entsprechende Element ausgewählt. Eine 2D-Punkt-zu-Rechteck Kollisionsberechnung wird einmal pro Frame für jede sichtbare Kachel und jeden aktiven Zeiger durchgeführt. Dabei wird für jeden Eingabekanal separat gespeichert, welches GUI-Element durch die Positionsüberschneidung ausgewählt wurde. Befindet sich der Zeiger nicht mehr über dem entsprechenden Element, wird die Auswahl aufgehoben.

Wird ein ausgewähltes Instrument mit einer Greifgeste aktiviert, wird es aus der Auswahlleiste in den virtuellen Raum übernommen. Gleichzeitig wird das aktivierte Instrument grafisch hervorgehoben und sein Einschaltzustand auf `True` gesetzt. Die räumliche Positionierung ist jetzt aktiv.

### *Positionierung*

Während ein Instrument mit einer Hand gegriffen wird, bestimmt die Position der Hand dessen Position innerhalb des Kamerabildes. Zusätzlich wird bei einer Handsteuerung der zuvor berechnete Azimut- und Elevationswinkel an das Instrument übertragen. Dadurch wird neben der zweidimensionalen Position des Instruments auch die räumliche Zeigerichtung des Arms berücksichtigt. Die Winkelwerte werden dabei nur für die beiden Hand-Eingabekanäle verwendet. Wird ein Instrument mit der Maus gegriffen, wird es nur auf der grafischen Oberfläche verschoben. Das folgende Bild zeigt das Verschieben eines Instruments.

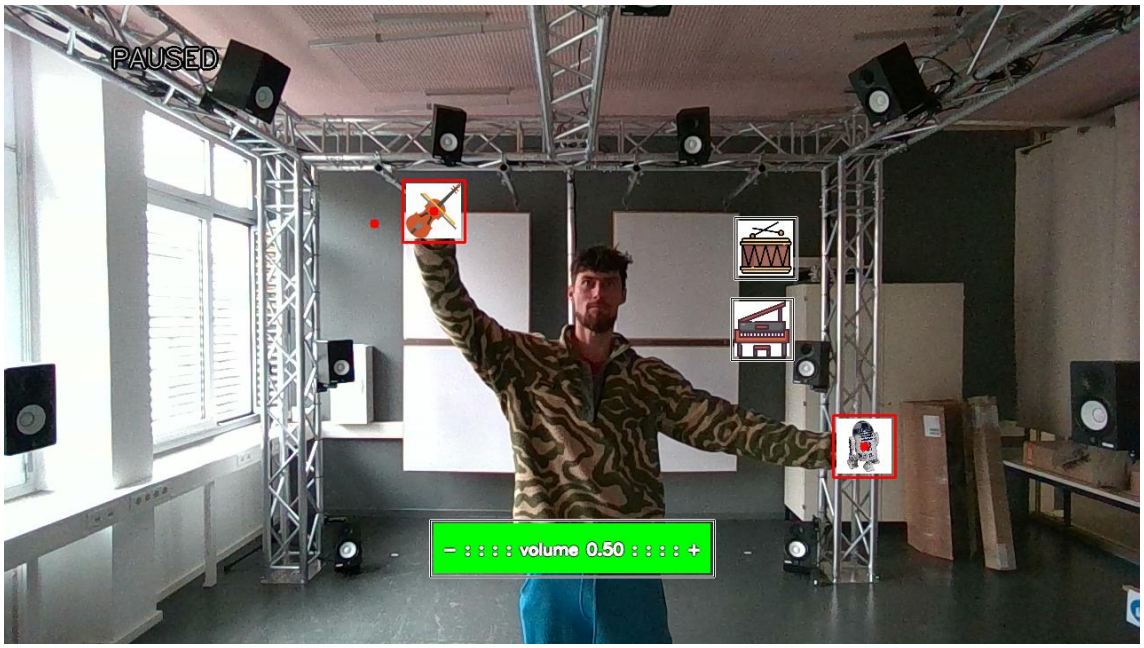



**Figure 44 – GUI: Verschieben eines Instruments mit jeder Hand, im 3D-Audioraum**

### *Reset-Zone*

Durch das Loslassen der Greifgeste wird die Steuerung des Instruments beendet. Befindet sich das Instrument beim Loslassen innerhalb einer definierten Reset-Zone, wird es aus dem virtuellen Raum entfernt, deaktiviert und zurück in die Auswahlleiste verschoben. Die Reset-Zone umfasst die Auswahlleiste und den Kachelbereich des Lautstärkereglers. Auch eine Positionierung im Bereich außerhalb des Kamerabildes setzt das Instrument wieder zurück.

*Lautstärke*

Für ein bereits im virtuellen Raum befindliches Instrument kann zusätzlich die Lautstärke verändert werden, sobald es sich unter der Auswahl eines Zeigers befindet. Hierzu wird eine Lautstärkekachel eingeblendet. Die Lautstärke wird anhand der horizontalen Position eines beliebigen Zeigers innerhalb dieser Kachel bestimmt. Dabei entspricht die linke Seite dem kleinsten und die rechte Seite dem größten Lautstärkewert. Der ermittelte Wert wird auf den Bereich von 0 bis 1 gemappt und anschließend im Kachelobjekt des Instruments gespeichert. Das folgende Bild zeigt das Ändern der Lautstärke eines Instruments.

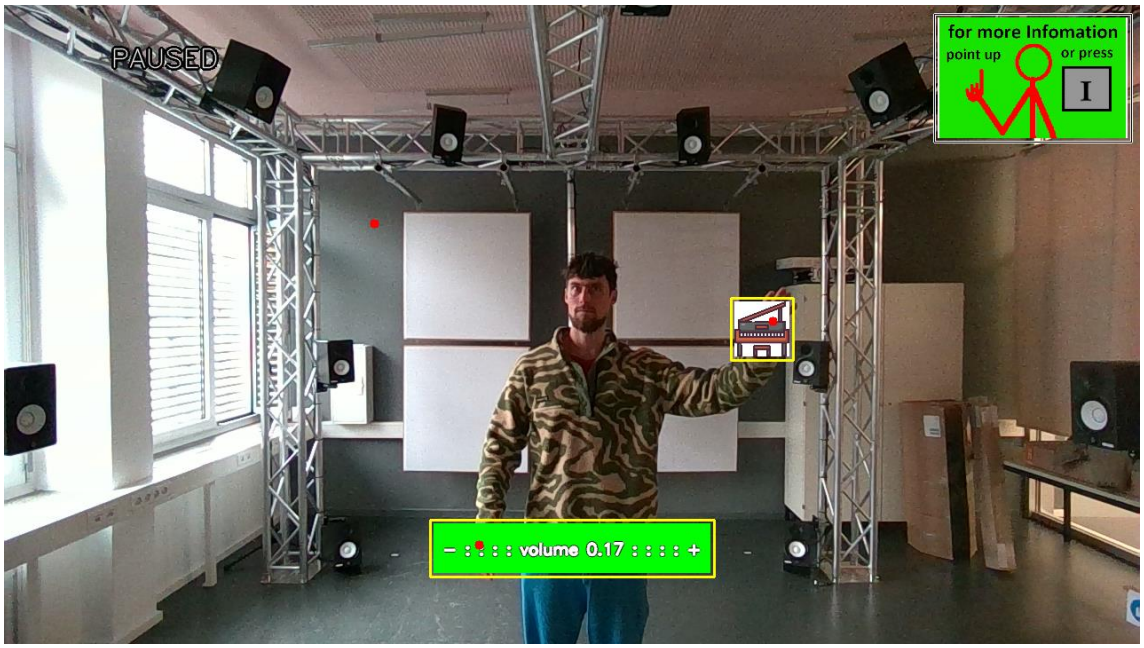



**Figure 45 – GUI: Lautstärke ändern, im 3D-Audioraum**

*Instrumentenmenü*

Die Auswahloberfläche für die Instrumente, wird dynamisch aus den Kacheln der aktuell verfügbaren Instrumente aufgebaut. Ihre Anordnung wird abhängig von der Anzahl der vorhandenen Instrumente bestimmt. Wird die Kachel für das Instrumentenmenü, durch eine Positionsüberschneidung mit einem Zeiger, aktiviert, werden die darin enthaltenen Instrumente eingeblendet und können anschließend ausgewählt werden. Das folgende Bild zeigt dieses Instrumentenmenü.

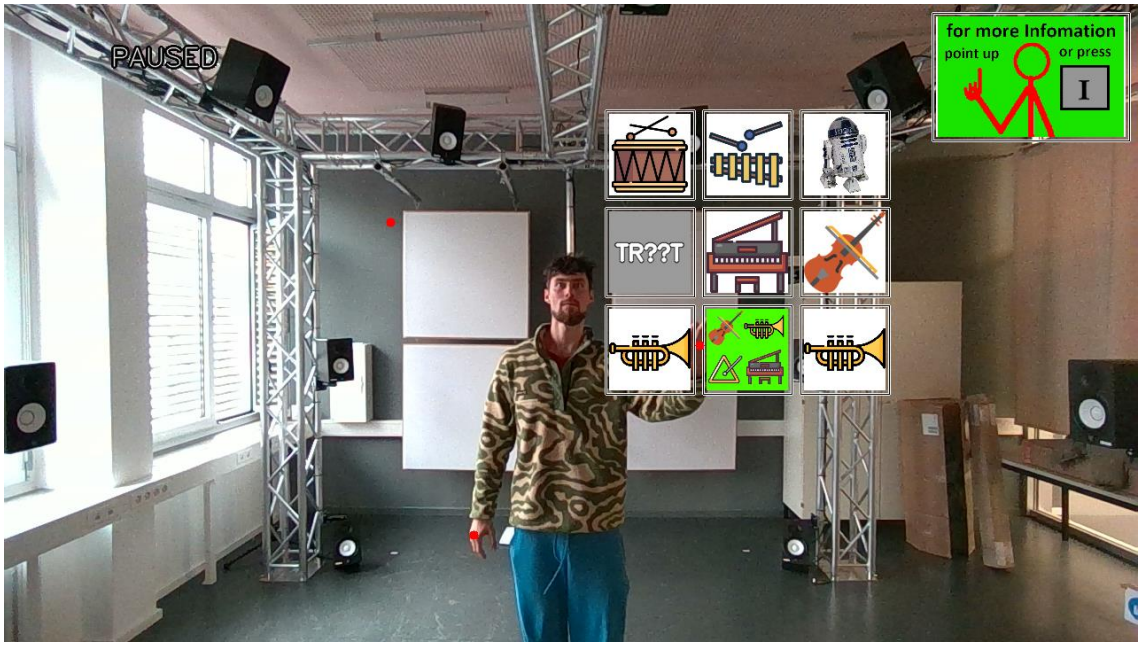



**Figure 46 – GUI: Instrumentenmenü, im 3D-Audioraum**

*Steuergesten*

Zusätzlich stehen drei Gesten zur Interaktion mit der GUI zur Verfügung. Die Infogeste schaltet die Anzeige des Informationsfensters (siehe Figure 47) ein oder aus. Die Ansicht-ändern-Geste wechselt zwischen den verfügbaren Darstellungsmodi der GUI. In der aktuellen Implementation hat die GUI jedoch nur einen Anzeigemodus, in der *settings.py* kann dies geändert werden. Die Reset-Geste setzt die GUI, alle Instrumente sowie den Darstellungsmodus auf den definierten Ausgangszustand zurück. Diese drei Gesten werden unabhängig von der Instrumentensteuerung verarbeitet. Nur ihre funktionellen Auswirkungen werden direkt in der Main-Loop auf die GUI angewendet.

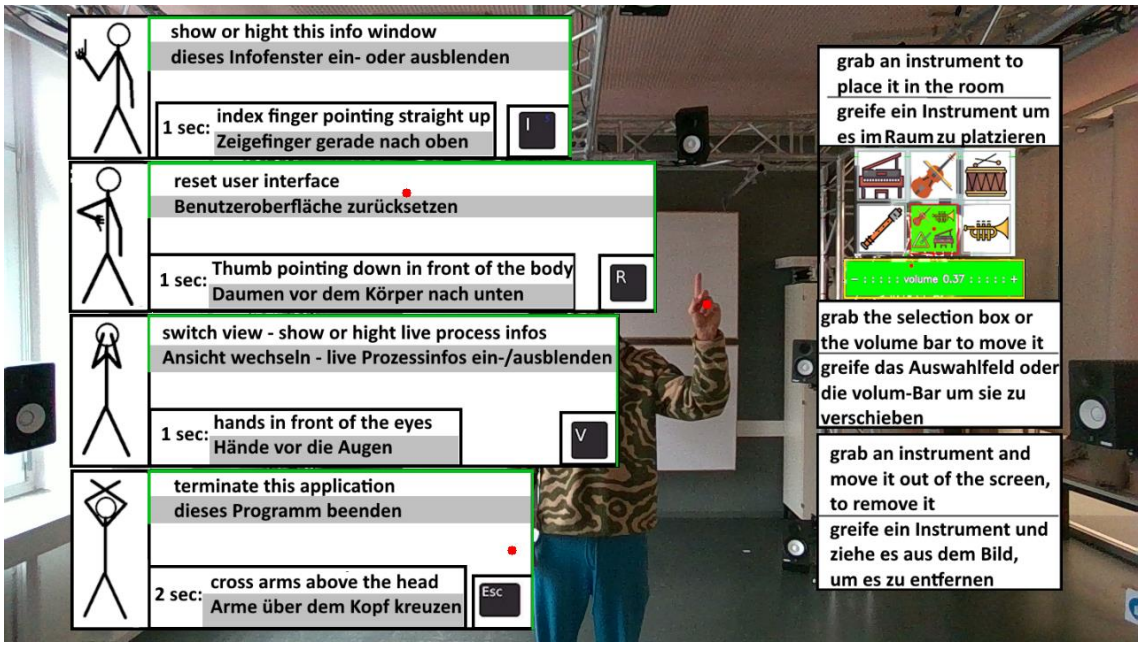



**Figure 47 – GUI: Informationsfenster, im 3D-Audioraum**

*Info-Button*
Neben den Instrumenten enthält die GUI auch eine Info-Schaltfläche, siehe oben links in Figure 46. Über diese kann alternativ zur Infogeste das Informationsfenster eingeblendet werden. Während das Infofenster angezeigt wird, werden die übrigen GUI-Elemente nicht gezeigt, da dieses den gesamten Bildschirm einnimmt.

*Instrumentenstatus*
Die GUI erzeugt bei Änderungen des Instrumentenzustands Informationsdaten, die von der Hauptschleife ausgelesen werden können. Für ein Instrument werden dabei unter anderem dessen Name, Typ, Einschaltzustand, Lautstärke sowie Azimut- und Elevationswinkel in einem Python-Dictionary bereitgestellt. Jedes dieser Dictionaries wird zu einer Liste hinzugefügt. Der Listeninhalt wird dann elementweise von einem Kommunikationsobjekt an das Audiosystem übertragen.

*GUI in der Main-Loop*
Im Hauptprogramm wird die GUI innerhalb jeder Verarbeitungsschleife zunächst mit den aktuellen Eingabedaten versorgt. Anschließend werden Auswahl, Greifstatus, Zeigerbewegung und Loslassstatus sowie die drei zusätzlichen Gesten verarbeitet. Danach zeichnet die GUI ihren aktuellen Zustand auf das Kamerabild. Abschließend werden eventuell vorliegende Zustandsänderungen eines Instruments aus der GUI ausgelesen und an die Kommunikationsschnittstelle zum Audiosystem übergeben.

Die GUI ist damit die Schnittstelle zwischen der Gestenerkennung und der Steuerung des virtuellen Audiosystems.

## 5.8 Systemkommunikation

Die Kommunikation zwischen Video- und Audiosystem erfolgt über UDP. Siehe dazu Abschnitt 2.2 - *User Datagram Protocol*. UDP ist einfach in Python- und MATLAB-Code zu integrieren. Die sonstigen Anforderungen an die Kommunikationsschnittstelle sind für dieses Programm gering. Daher wurde ohne weitere Begründung das „nächstbeste“ Kommunikationsprotokoll integriert. Die auszutauschenden Daten werden als JSON-Nachrichten übertragen. Siehe dazu auch 2.3 - *JavaScript Object Notation*.

*Übertragung vom Videosystem zum Audiosystem*
Ändert sich der Zustand eines Instruments, wird das Dictionary mit den aktuellen Instrumentendaten an das Kommunikationsobjekt `SendOnChange` übergeben. Dieses vergleicht die Daten mit dem zuletzt übertragenen Zustand und erzeugt nur bei einer Änderung eine neue Nachricht. Dadurch werden redundante Übertragungen vermieden. Anschließend wird das Dictionary in einen JSON-String und danach in eine UTF-8-codierte Bytefolge umgewandelt, die als UDP-Datagramm an die hinterlegte IPv4-Adresse und den Port des Audiosystems gesendet wird. In MATLAB kann die UDP-Empfangs-Loop mit `parfeval` als Hintergrundprozess gestartet werden.

Beispielnachricht:

```
{"type": "instrument", "name": "violin", "azimuth": -90,
"elevation": 0, "volume": 0.5, ...}
```

### *Übertragung vom Audiosystem zum Videosystem*

Das Videosystem besitzt einen separaten UDP-Empfänger, der in einem eigenen parallelen Thread betrieben wird. Dieser wartet auf eingehende UDP-Datagramme, dekodiert deren Nutzdaten und wandelt den enthaltenen JSON-String mit `json.loads()` wieder in ein Python-Dictionary um. Eingegangene Nachrichten werden in einer Liste zwischengespeichert. Die Hauptschleife ruft diese Nachrichten über `get_msg()` einzeln ab. Dabei wird die jeweils erste Nachricht aus der Liste entfernt und zur weiteren Verarbeitung zurückgegeben. Somit können mehrere eingegangene Nachrichten nacheinander verarbeitet werden. Beispielcode für das Senden in MATLAB befindet sich im Projektordner.

### *Instrumente hinzufügen*

Ein Nachricht-Dictionary vom Audiosystem an die Gestensteuerung kann den Schlüssel "`add_instruments`" enthalten. In diesem Fall erwartet die Gestensteuerung entweder einen String mit dem Namen eines hinzuzufügenden Orchesterinstruments oder eine Liste mit mehreren Instrumentennamen. Die empfangenen Instrumentennamen werden aus dem Dictionary ausgelesen und der GUI zur Verfügung gestellt, die daraus die entsprechenden Instrumentenkacheln in der Auswahlleiste erzeugt.

Beispielnachricht:

```
{"add_instruments": ["piano", "violin", "flute"]}
```

# 6 Test der Gestensteuerung

Es gab vier Termine an denen insgesamt 14 Probanden teilnahmen, zwei der Probanden waren Kinder (10 und 12 Jahre alt). Die Probanden testeten die Gestensteuerung und gaben Feedback. Gleichzeitig wurde Ihre Interaktion mit der Steuerung von mir beobachtet und ausgewertet. Zwischen den Terminen wurden die Gestensteuerung an die Testergebnisse angepasst, im Rahmen zur Verfügung stehenden Zeit. Die Aufteilung der Probanden auf die Termine war wie folgt:

- T1: 1 Proband (der Betreuer dieser Arbeit) am 01.09.2026 von 11 - 13 Uhr
- T2: 9 Probanden (Freunde und Bekannte) am 11.09.2026 von 18 - 20 Uhr
- T3: 3 Probanden (zwei Kinder mit Vater) am 15.09.2026 von 17 - 19 Uhr
- T4: 1 Proband (der zuständige des Audioraums) am 18.09.2026 von 10 - 12 Uhr

Der erste Test (T1) war formlos und unterlag keiner vordefinierten Testprozedur. Die anderen Testdurchläufe orientierten sich an einem, dafür konzipierten einseitigen Testbogen. Dieser wurde von T2 zu T3 einmalig überarbeitet.

## 6.1 Testprozedur

*Raumvorbereitung*

Der Test wurde im 3D-Audioraum (B217a) in der BHT durchgeführt. Kameraposition und -ausrichtung wurden kalibriert und gleichen der Vorsehung für das Endprodukt, siehe Figure 48. Der Systemraum wurde vermessen und es wurden Markierungen für die Raummitte und die Azimutwinkel von -90° bis +90° in 15° Schritten am Boden erstellt, siehe Figure 49. Die Fensterjalousien waren geschlossen und das Deckenlicht eingeschaltet (das ist auf den Bildern nicht der Fall).



**Figure 48 - 3D-Audioraum Kameraposition und -kalibrierung**



**Figure 49 - 3D-Audioraum Bodenmarkierungen für Systemmitte (schwarz) und 15° Schritte**

### *Metadaten*

Alter und Größe des Probanden wurden im Testbogen notiert. Weiterhin gab es ein Feld für einen Probanden-Alias und optische Merkmale. Optische Merkmale wurden insbesondere dann erfasst, wenn sie den Bereich der Hände betrafen. Einige der Merkmale waren beispielsweise: Armbanduhren, lange oder kurze Ärmel, eine von Weiß abweichende Hautfarbe und besonders bunte Kleidung.

### *Einführung*

Der Proband wurde instruiert, dass er ein virtueller Dirigent sei und mit einer Greifgeste Instrumente im Raum positionieren könne. Die Greifgeste wurde „trocken“ vorgeführt, das heißt ohne Systemfeedback. Ein Instrument würde später aus der Richtung erklingen, in die sein Arm im dreidimensionalen Raum zeigt, wenn er es positioniert. Das Audiosystem ist noch nicht mit der Gestensteuerung verbunden, sodass keine Soundwiedergabe erfolgt.

### *Freier Test*

Im ersten Testschritt sollte ein erster Eindruck hinsichtlich der intuitiven Bedienbarkeit und der Funktionsweise des Systems gewonnen werden. Dabei standen insbesondere die Verständlichkeit der Bedienung sowie die wahrgenommene Qualität und Benutzerfreundlichkeit der Funktionen im Fokus. Der Proband wurde dazu aufgefordert, die Gestensteuerung eigenständig auszuprobieren und seine persönliche Wahrnehmung der Bedienung zu beurteilen. Dabei sollte er nach Möglichkeit sämtliche verfügbare Funktionen erkunden. Auf dem Testbogen wurde vermerkt, welche Funktionen selbstständig gefunden und ob diese anschließend genutzt werden konnten. Während des Tests wurden mündliches Feedback und eigene Beobachtungen meinerseits auf dem Testbogen notiert. Nach dem ersten Test erhielt der Proband die Möglichkeit, schriftliches Feedback zu geben. Der Testbogen enthielt die Fragen, ob die Bedienung einfach war und ob der Proband Änderungsvorschläge hatte.

### *Winkelgenauigkeit*

Im zweiten Testabschnitt wurden zwei konkrete Tests durchgeführt. Der erste Test diente der Überprüfung der Winkelgenauigkeit. Hierzu sollte der Proband ein Instrument an fünf verschiedenen Positionen im Raum positionieren. Die ermittelten Azimut- und Elevationswinkel wurden anschließend mit den erwarteten Werten abgeglichen.

### *Greifgestenerkennung*

Im zweiten Test wurden verschiedene Verfahren zur Greifgestenerkennung untersucht. Der Proband wurde aufgefordert, ein Instrument mehrfach zu greifen, zu bewegen und zu verstellen. Anschließend wurde das Erkennungsverfahren für die Greifgeste geändert und der Test wiederholt. Nach jeder Änderung wurde der Proband gefragt, ob sich die Bedienung mittels Greifgeste für ihn verbessert oder verschlechtert hat.

*Mitschriften*
Während der Tests wurden die Aussagen des Probanden sowie meine eigenen Beobachtungen auf dem Testbogen dokumentiert. Um die Niederschriften der Probanden klar von meinen eigenen Notizen zu trennen, wurden die Notizen mit einem Bleistift eckig umrahmt. Alle Testbögen befinden sich im Anhang unter *Testböden der Probanden*.

## 6.2 Resultate aus dem Test

*Probandenverteilung*
Die Termine und die Anzahl der Probanden waren an deren Verfügbarkeit sowie den begrenzten zeitlichen Rahmen dieser Masterarbeit gebunden. Für eine effiziente Optimierung des Programms und seiner Gestensteuerung war diese Probandenverteilung jedoch nicht optimal. Sinnvoller wäre es gewesen, zunächst mit einer kleineren Anzahl an Probanden zu beginnen und diese nach den ersten Programmoptimierungen schrittweise zu erhöhen.

Die Fehler und Verbesserungsvorschläge aus dem ersten dokumentierten Testdurchlauf (T2) konnten meist schnell identifiziert werden und wiederholten sich größtenteils bei den Probanden. Für die Bewertung der finalen Version wäre ein größerer Kreis an Probanden sinnvoll, um unterschiedliche Optimierungsansätze zu beurteilen, die teilweise gegensätzliche Auswirkungen auf die Bedienung haben können. Darüber hinaus wären längere Zeiträume zwischen den Testdurchläufen vorteilhaft gewesen, um eine ausführlichere Optimierung des Programms zu ermöglichen.

Aus zeitlichen Gründen konnten nicht alle Probanden sämtliche Testschritte der Testprozedur absolvieren.

*Instrument im Raum*
Besonders auffällig waren die folgenden Probleme. Mehrere Probanden gingen zunächst davon aus, dass sich die Zielrichtung auf die Position des Instruments in der GUI bezieht. Erst nach wiederholten Hinweisen wurde deutlich, dass für die Steuerung die tatsächliche Ausrichtung des Arms im dreidimensionalen Raum maßgeblich ist. Dieses Verständnisproblem wird sich voraussichtlich durch die Ergänzung eines akustischen Feedbacks, also durch die Verbindung mit dem Audiosystem, reduzieren. Dann können die Probanden anhand der Veränderung des Klangsignals unmittelbar erkennen, ob sich die Ausrichtung verändert und durch welche Bewegung die gewünschte Richtung erreicht wurde.

*Mit dem Arm zielen*
Zudem zeigten sich unterschiedliche Zielstrategien bei der Ausrichtung auf bestimmte Raumpositionen. Während einige Probanden entlang der vorgesehenen Linie zwischen Schulter und Hand zielten, orientierte sich ein Großteil an der Verbindungslinie zwischen

Augen und Hand. Durch das auditive Feedback des Endsystems ist eine automatische Korrektur der Zielstrategie zu erwarten. Wie gut die erfasste Armstellung mit der wahrgenommenen Klangrichtung übereinstimmt und ob sich ein stimmiges Gefühl bei der Bewegung eines Instruments ergibt, kann erst mit dem vollständigen System, mit seiner vorgesehenen Anbindung an das Audiosystem beurteilt werden.

*Winkelgenauigkeit*
Durch die von der Vorgabe abweichenden Zielstrategien entstanden teilweise deutliche Abweichungen zwischen den erwarteten und den tatsächlich erkannten Azimut- und Elevationswinkeln. Da diese Abweichungen überwiegend auf eine nicht unterstützte Zielstrategie und damit auf einen systematischen Versatz zurückzuführen sind, eignen sie sich nicht als aussagekräftige Repräsentation der Winkelgenauigkeit der implementierten Methode.

Für eine belastbare Bewertung der methodischen Genauigkeit wäre es sinnvoll gewesen, wenn alle Probanden die unterstützte Zielstrategie verwendet hätten. Eine entsprechende Instruktion wäre daher hilfreich gewesen. Abgesehen davon wären die Testergebnisse auch nutzbarer, wenn die jeweils verwendete Ziellinie der Probanden dokumentiert worden wäre.

*Dynamische Gesten*
Darüber hinaus erwiesen sich die für das Zurücksetzen der GUI verwendeten dynamischen Gesten als anfällig für unbeabsichtigte Auslösungen. Gleichzeitig waren sie schwieriger bildlich und textlich zu beschreiben, wodurch ihre korrekte Anwendung erschwert wurde. Obwohl dynamische Gesten als angenehm und intuitiv wahrgenommen werden können, erwiesen sich statische Gesten im Test als zuverlässiger, da sie sich hier eindeutiger beschreiben und durch klar definierte Kriterien besser gegen unbeabsichtigte Auslösungen abgrenzen ließen.

*Zu lange Kleidung*
Bei der Erkennung der Greifgesten zeigte sich, dass sehr lange Ärmel, die teilweise die Hände bedecken, die Erkennung erschweren. Dies trat insbesondere beim Bilden einer Faust auf, vermutlich da hierbei ein noch größerer Teil der Hand durch den Ärmel verdeckt werden kann.

*Die Interagierende Hand*
Ein weiteres Problem ergab sich durch die Festlegung auf eine aktive Hand. Diese wurde anhand der jeweils schnellsten Bewegung bestimmt und wechselte entsprechend zwischen der linken und rechten Hand. Dadurch kam es wiederholt vor, dass ein Proband mit einer Hand interagieren wollte, während die andere Hand als aktive Hand erkannt wurde. Dies erschwerte die Bedienung und warf die Frage auf, ob sich die Auswahl der relevanten Hand durch andere Kriterien, beispielsweise die Position oder die Bewegungsgeschwindigkeit, zuverlässig bestimmen lässt. Da sich jedoch keine dieser Strategien als uneingeschränkt geeignet erwies, stellte sich grundlegend die Frage, warum überhaupt nur eine Hand als aktiv erkannt werden muss und ob nicht stattdessen beide Hände gleichzeitig erkannt und für die Interaktion mit dem System verwendet werden können.

*Greifgestenerkennung*

Beim Vergleich der verschiedenen Verfahren zur Greifgestenerkennung konnte keine eindeutig überlegene Methode festgestellt werden. Lediglich einzelne Kombinationen, wie eine statische Erkennung mit sehr langer Pufferzeit oder eine dynamische Erkennung mit sehr kurzer Auslösezeit, erwiesen sich als weniger geeignet. Bei moderaten Parametereinstellungen zeigten sich hingegen keine eindeutigen Vorteile der statischen oder dynamischen Erkennung. Zudem erwies sich die Wahrnehmung einer intuitiven Greifgeste als subjektiv und personenabhängig. Über alle Verfahren hinweg zeigte sich jedoch, dass sich die Bedienung nach einer gewissen Eingewöhnungszeit deutlich verbesserte.

*Winkelgenauigkeit*
Im Probandentest zur Winkelgenauigkeit waren die Abweichungen zwischen den erwarteten und den tatsächlich angezeigten Azimut- und Elevationswinkeln deutlich größer als in den eigenen Tests. Dies ist hauptsächlich darauf zurückzuführen, dass die Probanden nicht mit der vorgesehenen Ziellinie vertraut waren und sich überwiegend an der Verbindungslinie zwischen Augen und Hand orientierten, wie bereits beschrieben. Probanden, welche das Zielsystem kannten, erreichten hingegen deutlich höhere Zielgenauigkeiten mit Abweichungen von maximal 15°. Bei den Probanden ohne dieses Wissen lagen die Abweichungen teilweise bei 30°.

*Positionsdifferenz*
In einzelnen Fällen traten deutlich größere Abweichungen von etwa 45° bis 90° auf. Diese sind auf Fehler bei der Erfassung der Tiefenposition der Hand zurückzuführen. Da die Hand, insbesondere bei Kindern, nur einen kleinen Bildbereich einnimmt, kann bereits eine geringe Abweichung bei der Bestimmung der Bildposition dazu führen, dass die Tiefeninformation an einer falschen Stelle ausgelesen wird. Dadurch kann eine deutlich fehlerhafte räumliche Position der Hand und infolgedessen ein stark abweichender Winkel berechnet werden. Derartige Fehler traten jedoch nur selten auf und erklären insbesondere alle Abweichungen ab 45°.

*Menüposition*
Bezüglich der Bedienbarkeit zeigte sich insbesondere bei kleineren Personen, dass die Position des Menüs teilweise ungünstig war. Nach dem ersten Testdurchlauf wurde das Menü daher überarbeitet und verschiebbar gestaltet, sodass die Position an die jeweilige Körpergröße und räumliche Position angepasst werden kann.

## 6.3 Programmoptimierungen

Auf Grundlage der Ergebnisse aus den Testdurchläufen wurde die Gestensteuerung schrittweise angepasst.

*Die Interagierende Hand*
Eine wesentliche Änderung betraf die Auswahl der aktiven Hand. Diese wurde zunächst anhand der höchsten im Bild erkannten Hand bestimmt und anschließend durch die Hand mit der höchsten Bewegungsgeschwindigkeit ersetzt. Im weiteren Verlauf wurde dieses Konzept vollständig entfernt und auf die gleichzeitige Erkennung beider Hände umgestellt. Dadurch können nun beide Hände unabhängig voneinander für die Systeminteraktion verwendet werden.

Eine Hand kann nun zum Beispiel das Instrumentenmenü öffnen und die andere ein Instrument daraus Greifen. Oder beide Hände können gleichzeitig je ein Instrument Greifen und Verschieben. Auch können jetzt die Steuergesten jederzeit mit jeder Hand ausgelöst werden.

### *Verdekte Hände*

Die Erkennung einer Hand wird nun nur dann für die Bedienung verwendet, wenn der von MediaPipe bereitgestellte Sichtbarkeitswert einen definierten Schwellenwert erreicht. Dadurch sollen Hände, die beispielsweise hinter dem Rücken oder in einer Hosentasche gehalten werden, keine Gesten mehr auslösen oder GUI-Flächen bedienen können.

### *Instrumentenmenü*

Das zunächst fest positionierte Instrumentenmenü wurde durch ein verschiebbares und ausklappbares Menü ersetzt. Das Menü wird eingeblendet, sobald eine Hand über den entsprechenden Bereich gehalten wird und kann per Greifgeste umpositioniert werden. Dadurch konnte ein, ursprünglich für die Instrumentenpositionierung gesperrter Bereich im unteren Teil des Bildes entfernt werden. Dieser war zuvor durch das dauerhaft eingeblendete Menü blockiert. Die Änderung ermöglicht den gesamten sichtbaren Bereich zur Positionierung von Instrumenten zu verwenden und insbesondere kleinere Personen können nun Positionen mit einer nach unten gerichteten Elevation leichter ansteuern.

### *Instrumentenwahl*

Das ausklappbare Menü wird nun bereits beim Greifen eines Instruments automatisch geschlossen. Zuvor wurde es erst geschlossen, nachdem das gegriffene Instrument aus dem Bereich des Menüs herausbewegt worden war. Das war jedoch weniger intuitiv.

### *Lautstärkeregelung*

Die Lautstärkeregelung erfolgt weiterhin über eine Volume-Bar. Deren Position kann nun jedoch ebenfalls per Greifgeste verändert werden. Auch muss für die Bedienung das Instrument nicht mehr zwingend direkt über die Volume-Bar bewegt werden. Stattdessen reicht es aus, das gewünschte Instrument mit einer Hand zu berühren, die Volume-Bar erscheint dann und kann mit der anderen Hand bedient werden. Das vorherige Verfahren erforderte das Greifen des Instruments und veränderte dadurch auch seine räumliche Position.

### *Instrument entfernen*

Das Entfernen einzelner Instrumente wurde ebenfalls angepasst. Ein Instrument kann nun aus dem Raum entfernt werden, indem es über die Bildschirmgrenze hinausgeschoben, in das Instrumentenmenü zurückgezogen oder über der Volume-Bar losgelassen wird.

### *Reset-Geste*

Die dynamische Geste zum Zurücksetzen des Raums wurde zunächst durch eine andere dynamische Geste ersetzt. Da dadurch keine ausreichende Verbesserung erzielt wurde, wurde sie anschließend durch eine statische Geste ersetzt. Die Raum-Reset Geste ist jetzt Daumen nach unten. Zusätzlich wurde die Funktion des Raum-Resets erweitert, sodass nicht mehr nur die Instrumente, sondern die gesamte GUI in ihren Ausgangszustand zurückgesetzt wird. Nennen wir sie nun GUI-Reset.

*Beenden-Geste*
Die Geste zum Beenden des Programms erfordert nun das Kreuzen der Arme über dem Kopf. Das verhindert versehentliches Auslösen, wenn die arme vor dem Körper verschränkt werden.

*Infogeste*
Die Infogeste mit dem nach oben gerichteten Zeigefinger wurde von einer halte Funktion zu einem Umschalt-Trigger für das Infofenster geändert. Durch die Geste wird das Infofenster nun ein- beziehungsweise ausgeblendet. Der Finger musste zuvor die ganze Zeit in der Gestenposition gehalten werden, um das Infofenster zu sehen. Beim Zurücksetzen des GUIs wird das Infofenster ebenfalls ausgeblendet.

*Feinjustierung*
Die Schwellwerte für die Erkennung der Handöffnung wurden sowohl für die statische als auch für die dynamische Greifgestenerkennungsmethode geringfügig angepasst. Zudem wurden die geometrischen Erkennungsverfahren für die Steuergesten überarbeitet. Dabei wurden unter anderem Toleranzen angepasst, sodass beispielsweise die Infogeste nicht mehr eine nahezu vollständig senkrechte Ausrichtung des Zeigefingers voraussetzt.

*Dynamische Gesten*
Im Verlauf der Optimierung wurden alle dynamischen Gesten durch statische Gesten ersetzt. Dadurch konnten die verfügbaren Gesten eindeutiger definiert und die für ihre Ausführung erforderlichen Bedingungen eindeutiger im Infofenster dargestellt werden.

*Gesten-Feedback*
Für erkannte Gesten und ausgelöste Kommandos wurde ein kurzes visuelles Feedback ergänzt. Nach der Erkennung wird für einen kurzen Zeitraum eine entsprechende Meldung im Bild eingeblendet. Dadurch kann während der Entwicklung und im Einsatz unmittelbar nachvollzogen werden, ob eine Geste erkannt und das zugehörige Kommando ausgelöst wurde.

*Infofenster*
Das Infofenster wurde optisch überarbeitet. Die einzelnen Bereiche wurden übersichtlicher, das heißt geometrisch einheitlicher, angeordnet und die Beschreibungen durch Rahmen eindeutiger den jeweiligen Funktionen und den zugehörigen Gesten und Tasten zugeordnet. Zusätzlich werden im Infofenster nun die erforderlichen Haltezeiten der einzelnen Gesten angezeigt.

*Das Kind braucht einen Namen*
Als abschließende Änderung erhielt die Gestensteuerung einen Versionsnamen. Auf Anordnung der jüngsten Probandin (10 Jahre) wird die erste Abgabeversion des Programms, als *Stitchi 1.0* bezeichnet. Stitchi 1.0 beschreibt damit den Zustand der Gestensteuerung zum Zeitpunkt der Abgabe der Vorliegenden Arbeit.

# 7 Ergebnisse

Die Ergebnisse dieser Arbeit und ihrer Tests werden im Folgenden noch einmal zusammengefasst und eingeordnet.

*MediaPipe und Performanz*

Für die Skeletterkennung mit MediaPipe Pose lieferten die Tests in etwa 98–99 % der Frames, in denen eine Person sichtbar war, Ergebnisse für seine Körperlandmarks. Diese waren optisch bewertet ausreichend präzise. Die Handerkennung erwies sich zunächst als weniger zuverlässig, als hier ebenfalls das Gesamtbild für die Analyse genutzt wurde. Durch die Verwendung eines an die geschätzte Handposition angepassten Bildausschnitts (ROI) stieg der Anteil der Frames mit erkannter Hand in den untersuchten Videos von etwa 30-70 % auf rund 80-90 %. Diese Erkennungsrate ist ausreichend, denn in einigen Frames mit schneller Armbewegung ist das Bild verschwommen und eine Handskeletterkennung erwartungsgemäß generell nicht möglich. Die Gestensteuerung mit einem MediaPipe Pose und einem MediaPipe-Hands-Objekt (Klassenimplementierung des Erkennungsverfahrens in Python) lief auf der Projekthardware mit ca. 24 FPS. Die aktuelle Version Stitchi 1.0 hat zwei separate Objekte zur Handerkennung und läuft mit ca. 19 FPS. Auch das ist noch deutlich schneller als Kennzahlen aus ähnlichen Projekten, welche hier betrachtet wurden. Die bessere Performanz ist wahrscheinlich der Projekthardware, mit DDR4 RAM und der performanten CPU mit NPU, zu verdanken.

*Performanzgewinn*

Durch die Verwendung von zwei separaten MediaPipe-Handerkennungsobjekten wird die benötigte Rechenleistung erhöht. Alternativ könnte stattdessen ein einziges ROI um beide Hände aufgespannt werden, sodass MediaPipe Hands beide Hände innerhalb dieses gemeinsamen Bereichs erkennt. Diese Variante würde den Rechenaufwand voraussichtlich reduzieren und damit die Performanz verbessern. Diese sind im Anhang zu finden.

*Greifgestenerkennung*

Bei der untersuchten Greifgestenerkennung auf Videobasis lagen die True-Positive-Raten der sechs getesteten Varianten zwischen 81,7 % und 89,2 %. Diese Werte sind als orientierende Ergebnisse innerhalb des Entwicklungsprozesses zu verstehen. Insbesondere, da die Greifgesten nur „trocken“, also ohne Systeminteraktion abgefilmt wurden. Sie dienten primär der anfänglichen Optimierung, dem Vergleich und der Aussortierung von Greiferkennungsmethoden. Im Stitchi 1.0 sind die zwei erfolgreichsten Methoden implementiert, genannt: statische Methode und dynamische Methode. Im Probandentest wurden im Schnitt beide Methoden ähnlich gut gewertet, die Vorlieben waren hier subjektiv. Die Bedienung erwies sich dabei nach einer kurzen Eingewöhnungsphase, bei der sich an das System gewöhnt wurde, als kontrollierter und „flüssiger“.

*3D-Daten*

Eine ausreichende Genauigkeit in der Tiefeninformation war nur unter Verwendung einer Tiefenkamera möglich. Zwar gibt MediaPipe selbst auch geschätzte 3D-Koordinaten zurück, diese sind jedoch in ihrer Tiefe zu ungenau, um die hier gewollte Zeigerichtung genau genug daraus zu berechnen. Halbwegs zuverlässig war hierdurch nur eine Schätzung von etwa 45°-Schritten möglich. Auch alternative Ansätze zur Bestimmung

der Zeigerichtung allein aus dem RGB-Bild blieben erfolglos. Die Tests dazu sind im Anhang zu finden.

### *Winkelgenauigkeit*

Die Tests meiner Person zur Berechnung der Zeigerichtung bzw. der daraus resultierenden Raumrichtung ergaben einen Azimut-Winkelfehler von unter ±7,5° mit einer Zuverlässigkeit von etwa 75 %. Das bedeutet, dass in etwa 75 % der Fälle eine maximale Abweichung von unter ±7,5° auftrat. In etwa 95-98 % dieser Selbstversuche lag der Azimut-Winkelfehler unter 22,5°. In der Elevation sind die Abweichungen etwas höher. Da Stitchi 1.0 die Winkel auf 15°-Schritte rundet, sind die 7,5° und die 22,5° die entsprechenden Rundungsschwellen. Eine Abweichung von $7{,}5° < \varepsilon < 22{,}5°$ wird dadurch als Winkelwert ausgegeben, der den erwarteten Wert um 15° verfehlt. Diese eigenen Tests meiner Person wurden in ihrer Methodik und Durchführung nicht dokumentiert und sollen hier primär der Relativierung des dokumentierten Probandentests dienen.

Im Probandentest traten größere Abweichungen auf. Personen, die sich an der Verbindungslinie zwischen Augen und Hand statt an der vorgesehenen Linie zwischen Schulter und Hand orientierten, wichen teilweise um etwa 30° vom Erwartungswert ab. Im Elevationswinkel waren es teilweise sogar 45°. In Einzelfällen führten fehlerhafte Tiefenzuordnungen zu Abweichungen von etwa 45° bis 90°. Zu bemerken ist, dass letzteres eher bei kleinen Personen mit kleinen Händen zustande kam.

Da die Probandentests zu großen Teilen mit einer nicht unterstützten Zielstrategie durchgeführt wurden und sich zudem alle Erwartungswerte der Testverfahren am Augenmaß orientieren, wurde der Auswertung dieser Tests keine besonders große Bedeutung beigemessen.

Da das Audiosystem zum Zeitpunkt dieser Arbeit noch nicht angebunden ist, konnte die Qualität der Steuerung nur visuell und ohne akustisches Feedback bewertet werden. Aussagen über die tatsächliche Nutzererfahrung und die wahrgenommene Genauigkeit der Klangpositionierung sind daher hier nicht möglich. Für das Endprodukt und seine Bewertung wird das auditive Feedback jedoch als zentral angesehen: Erst durch die unmittelbare Wahrnehmung der Klangquelle können Nutzende ihre Zielbewegung richtig einschätzen und ein Gefühl für die räumlich-auditive Bewegungsverbindung bewerten.

Hierzu sei noch gesagt: unter guten Voraussetzungen kann das menschliche Gehör bei künstlichen Klangquellen Richtungsunterschiede von etwa 5° wahrnehmen. Siehe dazu Abschnitt 2.15 - *Menschliche Auditive Richtungswahrnehmung*.

### *Dynamische Gesten*

Die Nutzertests zeigten, dass dynamische Gesten häufiger unbeabsichtigt ausgelöst wurden als statisch definierte Gesten.

### *Lange Ärmel*

Lange Ärmel begünstigten die Handerkennung, vermutlich da sie den Arm farblich von der Hand abgrenzten und dadurch die Segmentierung der Hand für MediaPipe erleichterten. Zu lange Ärmel wiederum erschwerten die Erkennung der Hand hingegen, wenn diese Teile der Hand verdeckten.

*Programmentwicklung*

Die automatische Auswahl einer einzelnen aktiven Hand führte zu Bedienproblemen. In der überarbeiteten Version Stitchi 1.0 können beide Hände unabhängig voneinander verwendet werden. Zusätzlich wurde das Instrumentenmenü verschiebbar gestaltet, das visuelle Gestenfeedback ergänzt, die separaten Steuergesten überarbeitet und weitere Bedienabläufe angepasst und vereinfacht.

# 8 Resümee

Ziel dieser Arbeit war die Konzeption und Umsetzung einer kamerabasierten Gestensteuerung, mit der ein Dirigent virtuelle Instrumente im dreidimensionalen Raum positionieren kann. Dazu wurden zunächst die räumlichen Anforderungen im 3D-Audioraum der BHT untersucht, Anforderungen an die Hardware und Software formuliert und Konzepte mit sowie ohne KI-Anwendungen entworfen. Ein KI-basiertes Konzept auf der Grundlage von MediaPipe wurde implementiert.

*Die Gestensteuerung*

Das System erfasst Körper- und Handlandmarks mit MediaPipe, leitet daraus die Zeigerichtung des Arms und den Handzustand ab und stellt die Interaktion über eine grafische Benutzeroberfläche dar. Steuerinformationen werden als JSON-Nachrichten über UDP übertragen. Der Kommunikationspartner ist das 3D-Audiosystem dieses ist, zum Stand der Abgabe noch nicht angebunden. Für die Evaluation wurde daher die räumliche Positionierung rein visuell und ohne die eigentliche akustische Rückmeldung der virtuellen Instrumente bewertet. Diese Bewertung ist für das Endprodukt nur bedingt aussagekräftig.

Die Umsetzung zeigt, dass eine kamerabasierte Gestensteuerung für virtuelle Klangquellen grundsätzlich realisierbar ist. Für eine präzise räumliche Erkennung war hierbei eine Kamera mit Tiefensensor erforderlich. Es entstand ein funktionierendes System, das Körper- und Handlandmarks erfasst, daraus Zeigerichtung und Handzustand ableitet und virtuelle Klangquellen über eine grafische Benutzeroberfläche steuert. Die beschriebenen Tests deuten zudem darauf hin, dass die verwendeten Erkennungs- und Berechnungsverfahren unter geeigneten Bedingungen ausreichend zuverlässig arbeiten. Trotz ausreichender Funktionalität besteht bei der Winkelbestimmung und der Greifgestenerkennung weiterhin methodisches Verbesserungspotenzial. Denkbar wäre beispielsweise der Einsatz einer KI-Klassifikation statt einer rein geometrischen Auswertung der Landmarks, um Greifgesten zuverlässiger zu erkennen. Ebenso wäre eine Methode von Interesse, mit der sich die dreidimensionale Zeigerichtung mit hinreichender Winkelgenauigkeit ohne Tiefensensor aus einem einzelnen RGB-Bild oder -Stream bestimmen ließe.

*Ausblick*

Allgemein entwickelt sich Gestensteuerung von der Erkennung einzelner vordefinierter Haltungen hin zu Systemen, die Körperpose, Handbewegung, zeitliche Abläufe und räumliche Kontextinformationen gemeinsam auswerten. Fortschritte bei Effizienz und Leistung von KI-Modellen, Genauigkeit der Sensorik und der Verarbeitung direkt auf Endgeräten können solche Anwendungen reaktionsschneller, präziser und unabhängiger von externer Recheninfrastruktur machen. Zugleich bleiben Robustheit, Barrierefreiheit, eine intuitive Bedienung und eine verständliche Rückmeldung zentrale Gestaltungsaufgaben.

Aus Sicht des Autors liegt das Potenzial zukünftiger Gestensteuerung in der Bedienung von Multimediasystemen und der Mensch-Roboter-Interaktion. Automatisierte Systeme könnten zunehmend auf menschliche Gesten und Bewegungen reagieren, um Interaktion direkter und intuitiver zu gestalten. Bei bewegten Systemen kann eine automatische und vorausschauende Analyse des menschlichen Verhaltens relevant für die Sicherheit sein.

Zu denken ist an kraftvolle Roboterbewegungen in der Gegenwart von Menschen oder an autonomes Fahren.
Auch in der Medizin bzw. Sportmedizin bietet die automatisierte Analyse von Bewegungsmustern und Körperhaltungen Potenzial, beispielsweise für gesundheitsförderndes Feedback.

Für das vorliegende Projekt liegt das Potenzial insbesondere in der Kombination aus natürlicher Armbewegung und räumlicher Klangwahrnehmung mit direktem Audiofeedback. Damit könnte eine solche Gestensteuerung über das virtuelle Orchester hinaus auch für immersive Medien, interaktive Installationen und berührungslose Bedienoberflächen relevant werden.

## Literatur und Quellen


[1] „ROS“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://docs.ros.org/

[2] M. Quigley *u. a.*, „ROS: an open-source Robot Operating System“.

[3] „Interfaces (topics, services, actions)“, ROS 2 Documentation. Zugegriffen: 5. Oktober 2026. [Online]. Verfügbar unter: https://docs.ros.org/en/humble/Concepts/Basic/Interfaces-Topics-Services-Actions.html

[4] „ROS on DDS“. Zugegriffen: 5. Oktober 2026. [Online]. Verfügbar unter: https://design.ros2.org/articles/ros_on_dds.html

[5] J. Postel, „User Datagram Protocol“, Internet Engineering Task Force, Request for Comments RFC 768, Aug. 1980. doi: 10.17487/RFC768.

[6] L. Eggert, G. Fairhurst, und G. Shepherd, „UDP Usage Guidelines“, Internet Engineering Task Force, Request for Comments RFC 8085, März 2017. doi: 10.17487/RFC8085.

[7] T. Bray, „The JavaScript Object Notation (JSON) Data Interchange Format“, Internet Engineering Task Force, Request for Comments RFC 8259, Dez. 2017. doi: 10.17487/RFC8259.

[8] C. Lugaresi *u. a.*, „MediaPipe: A Framework for Building Perception Pipelines“, 14. Juni 2019, *arXiv*: arXiv:1906.08172. doi: 10.48550/arXiv.1906.08172.

[9] „layout: forward target: https://developers.google.com/mediapipe/framework/framework_concepts/calculators title: Calculators parent: Framework Concepts nav_order: 1 — MediaPipe v0.7.5 documentation“. Zugegriffen: 21. September 2026. [Online]. Verfügbar unter: https://mediapipe.readthedocs.io/en/latest/framework_concepts/calculators.html

[10] google-ai-edge, „mediapipe/docs/index.md at master · google-ai-edge/mediapipe“, GitHub. Zugegriffen: 21. September 2026. [Online]. Verfügbar unter: https://github.com/google-ai-edge/mediapipe/blob/master/docs/index.md

[11] „mp.tasks.components.containers.NormalizedLandmark“, Google AI for Developers. Zugegriffen: 22. September 2026. [Online]. Verfügbar unter: https://ai.google.dev/edge/api/mediapipe/python/mp/tasks/components/containers/NormalizedLandmark

[12] „mediapipe/docs/solutions/pose.md at master · google-ai-edge/mediapipe“, GitHub. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: https://github.com/google-ai-edge/mediapipe/blob/master/docs/solutions/pose.md

[13] „Landmark | Google AI Edge“, Google for Developers. Zugegriffen: 10. Juli 2026. [Online]. Verfügbar unter: https://developers.google.com/edge/api/mediapipe/java/com/google/mediapipe/tasks/components/containers/Landmark

[14] V. Bazarevsky, I. Grishchenko, K. Raveendran, T. Zhu, F. Zhang, und M. Grundmann, „BlazePose: On-device Real-time Body Pose tracking“, 17. Juni 2020, *arXiv*: arXiv:2006.10204. doi: 10.48550/arXiv.2006.10204.

[15] „On-device, Real-time Body Pose Tracking with MediaPipe BlazePose“. Zugegriffen: 11. August 2026. [Online]. Verfügbar unter: https://research.google/blog/on-device-real-time-body-pose-tracking-with-mediapipe-blazepose/

[16] F. Zhang *u. a.*, „MediaPipe Hands: On-device Real-time Hand Tracking“, 18. Juni 2020, *arXiv*: arXiv:2006.10214. doi: 10.48550/arXiv.2006.10214.

[17] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, und Y. Sheikh, „OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields“, *IEEE Trans. Pattern Anal. Mach. Intell.*, Bd. 43, Nr. 1, S. 172–186, Jan. 2021, doi: 10.1109/TPAMI.2019.2929257.

[18] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, und Y. Sheikh, „OpenPose: Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields“, 30. Mai 2019, *arXiv*: arXiv:1812.08008. doi: 10.48550/arXiv.1812.08008.

[19] D. Osokin, „Real-time 2D Multi-Person Pose Estimation on CPU: Lightweight OpenPose“, 29. November 2018, *arXiv*: arXiv:1811.12004. doi: 10.48550/arXiv.1811.12004.

[20] „OpenPose: Main Page“. Zugegriffen: 3. Oktober 2026. [Online]. Verfügbar unter: https://cmu-perceptual-computing-lab.github.io/openpose/web/html/doc/

[21] „Human keypoint detection for close proximity human-robot interaction“. Zugegriffen: 3. Oktober 2026. [Online]. Verfügbar unter: https://arxiv.org/html/2207.07742v2

[22] „[1909.13423] Single-Network Whole-Body Pose Estimation“. Zugegriffen: 3. Oktober 2026. [Online]. Verfügbar unter: https://ar5iv.labs.arxiv.org/html/1909.13423

[23] Z. Cao, *ZheC/tf-pose-estimation*. (31. Juli 2026). PureBasic. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://github.com/ZheC/tf-pose-estimation

[24] „tensorflow“, PyPI. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://pypi.org/project/tensorflow/

[25] S. K. Thakkar Nisarg, *Mastering OpenCV Android Application Programming*. Zugegriffen: 23. September 2026. [Online]. Verfügbar unter: https://www.oreilly.com/library/view/mastering-opencv-android/9781783988204/ch01s02.html

[26] „Platform Support Overview“, OpenCV. Zugegriffen: 23. September 2026. [Online]. Verfügbar unter: https://mintlify.wiki/opencv/opencv/platforms/overview

[27] „OpenCV: Video I/O with OpenCV Overview“. Zugegriffen: 12. Oktober 2026. [Online]. Verfügbar unter: https://docs.opencv.org/4.13.0/d0/da7/videoio_overview.html

[28] „OpenCV: Introduction“. Zugegriffen: 23. September 2026. [Online]. Verfügbar unter: https://docs.opencv.org/4.12.0/d1/dfb/intro.html

[29] „OpenCV: opencv2/imgproc.hpp File Reference“. Zugegriffen: 23. September 2026. [Online]. Verfügbar unter: https://docs.opencv.org/4.12.0/d1/d4f/imgproc_2include_2opencv2_2imgproc_8hpp.html

[30] „What is MediaPipe? A Guide for Beginners“, Roboflow Blog. Zugegriffen: 23. September 2026. [Online]. Verfügbar unter: https://blog.roboflow.com/what-is-mediapipe/

[31] „Mathematische Morphologie“, *Wikipedia*. 28. November 2024. Zugegriffen: 2. Juni 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Mathematische_Morphologie&oldid=250766250

[32] M. Werner, „Morphologische Transformationen“, in *Digitale Bildverarbeitung: Grundkurs mit neuronalen Netzen und MATLAB®-Praktikum*, M. Werner, Hrsg., Wiesbaden: Springer Fachmedien, 2021, S. 197–240. doi: 10.1007/978-3-658-22185-0_8.

[33] „OpenCV: Morphological Transformations“. Zugegriffen: 2. Juni 2026. [Online]. Verfügbar unter: https://docs.opencv.org/4.x/d9/d61/tutorial_py_morphological_ops.html
[34] P.-L. St-Charles, G.-A. Bilodeau, und R. Bergevin, „Flexible Background Subtraction with Self-Balanced Local Sensitivity“, in *2014 IEEE Conference on Computer Vision and Pattern Recognition Workshops*, Columbus, OH, USA: IEEE, Juni 2014, S. 414–419. doi: 10.1109/CVPRW.2014.67.
[35] R. Wang, F. Bunyak, G. Seetharaman, und K. Palaniappan, „Static and Moving Object Detection Using Flux Tensor with Split Gaussian Models“, in *2014 IEEE Conference on Computer Vision and Pattern Recognition Workshops*, Columbus, OH, USA: IEEE, Juni 2014, S. 420–424. doi: 10.1109/CVPRW.2014.68.
[36] x-jeff, „AI_Papers/Topological Structural Analysis of Digitized Binary Images by Border Following.pdf at master · x-jeff/AI_Papers“, GitHub. Zugegriffen: 10. Juni 2026. [Online]. Verfügbar unter: https://github.com/x-jeff/AI_Papers/blob/master/Topological%20Structural%20Analysis%20of%20Digitized%20Binary%20Images%20by%20Border%20Following.pdf
[37] Kang & Atul, „suzuki contour algorithm opencv“, TheAILearner. Zugegriffen: 5. Juni 2026. [Online]. Verfügbar unter: https://theailearner.com/tag/suzuki-contour-algorithm-opencv/
[38] „YUV-Farbmodell“, *Wikipedia*. 4. September 2025. Zugegriffen: 29. Mai 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=YUV-Farbmodell&oldid=259454114
[39] „RGB-Farbraum“, *Wikipedia*. 23. Juli 2026. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=RGB-Farbraum&oldid=269049392
[40] „Skin Segmentation Using YUV and RGB Color Spaces“, *JIPSJournal Inf. Process. Syst.*, Bd. 10, Nr. 2, S. 283–299, Juni 2014, doi: 10.3745/JIPS.02.0002.
[41] B. C. Ennehar, O. Brahim, und T. Hicham, „An Appropriate Color Space to Improve Human Skin Detection“, *INFOCOMP J. Comput. Sci.*, Bd. 9, Nr. 4, S. 1–10, Dez. 2010.
[42] „YCbCr-Farbmodell“, *Wikipedia*. 5. August 2026. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=YCbCr-Farbmodell&oldid=269396758
[43] „Hunter Lab“, *Wikipedia*. 7. Dezember 2025. Zugegriffen: 7. Oktober 2026. [Online]. Verfügbar unter: https://en.wikipedia.org/w/index.php?title=Hunter_Lab&oldid=1326184020
[44] K. B. Shaik, P. Ganesan, V. Kalist, B. S. Sathish, und J. M. M. Jenitha, „Comparative Study of Skin Color Detection and Segmentation in HSV and YCbCr Color Space“, *Procedia Comput. Sci.*, Bd. 57, S. 41–48, Jan. 2015, doi: 10.1016/j.procs.2015.07.362.
[45] „Unity: Develop, Deploy, and Grow | The World’s Leading Game Engine“, Unity. Zugegriffen: 12. Oktober 2026. [Online]. Verfügbar unter: https://unity.com
[46] „librealsense2 — librealsense2 2.57.7 documentation“. Zugegriffen: 21. Mai 2026. [Online]. Verfügbar unter: https://docs.ros.org/en/jazzy/p/librealsense2/
[47] „librealsense/wrappers/python/readme.md at master · realsenseai/librealsense“, GitHub. Zugegriffen: 21. Mai 2026. [Online]. Verfügbar unter: https://github.com/realsenseai/librealsense/blob/master/wrappers/python/readme.md
[48] realsenseai, „librealsense/doc/record-and-playback-legacy-ros1.md at master · realsenseai/librealsense“, GitHub. Zugegriffen: 24. September 2026. [Online].

Verfügbar unter: https://github.com/realsenseai/librealsense/blob/master/doc/record-and-playback-legacy-ros1.md
[49] realsenseai, „librealsense/doc/record-and-playback.md at master · realsenseai/librealsense“, GitHub. Zugegriffen: 24. September 2026. [Online]. Verfügbar unter: https://github.com/realsenseai/librealsense/blob/master/doc/record-and-playback.md
[50] realsenseai, „Failing to run rtabmap-ros against streams recorded in rosbag · Issue #3272 · realsenseai/realsense-ros“, GitHub. Zugegriffen: 24. September 2026. [Online]. Verfügbar unter: https://github.com/realsenseai/realsense-ros/issues/3272
[51] „D455“, RealSense. Zugegriffen: 24. September 2026. [Online]. Verfügbar unter: https://realsenseai.com/products/real-sense-depth-camera-d455f/
[52] A. Krizhevsky, I. Sutskever, und G. E. Hinton, „ImageNet Classification with Deep Convolutional Neural Networks“, in *Advances in Neural Information Processing Systems*, Curran Associates, Inc., 2012. Zugegriffen: 5. Oktober 2026. [Online]. Verfügbar unter: https://papers.nips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html
[53] K. Simonyan und A. Zisserman, „Very Deep Convolutional Networks for Large-Scale Image Recognition“, 10. April 2015, *arXiv*: arXiv:1409.1556. doi: 10.48550/arXiv.1409.1556.
[54] K. He, X. Zhang, S. Ren, und J. Sun, „Deep Residual Learning for Image Recognition“, gehalten auf der Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, S. 770–778. Zugegriffen: 5. Oktober 2026. [Online]. Verfügbar unter: https://openaccess.thecvf.com/content_cvpr_2016/html/He_Deep_Residual_Learning_CVPR_2016_paper.html
[55] D. Tran, L. Bourdev, R. Fergus, L. Torresani, und M. Paluri, „Learning Spatiotemporal Features With 3D Convolutional Networks“, gehalten auf der Proceedings of the IEEE International Conference on Computer Vision, 2015, S. 4489–4497. Zugegriffen: 5. Oktober 2026. [Online]. Verfügbar unter: https://openaccess.thecvf.com/content_iccv_2015/html/Tran_Learning_Spatiotemporal_Features_ICCV_2015_paper.html
[56] X. Shi, Z. Chen, H. Wang, D.-Y. Yeung, W. Wong, und W. Woo, „Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting“, 19. September 2015, *arXiv*: arXiv:1506.04214. doi: 10.48550/arXiv.1506.04214.
[57] J. Lin, C. Gan, und S. Han, „TSM: Temporal Shift Module for Efficient Video Understanding“, 22. August 2019, *arXiv*: arXiv:1811.08383. doi: 10.48550/arXiv.1811.08383.
[58] A. Vaswani *u. a.*, „Attention Is All You Need“, 2. August 2023, *arXiv*: arXiv:1706.03762. doi: 10.48550/arXiv.1706.03762.
[59] D. R. Perrott und K. Saberi, „Minimum audible angle thresholds for sources varying in both elevation and azimuth“, *J. Acoust. Soc. Am.*, Bd. 87, Nr. 4, S. 1728–1731, Apr. 1990, doi: 10.1121/1.399421.
[60] „Ambisonics“, *Wikipedia*. 22. August 2026. Zugegriffen: 28. August 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Ambisonics&oldid=269872883
[61] „Satz des Pythagoras“, *Wikipedia*. 13. September 2026. Zugegriffen: 25. September 2026. [Online]. Verfügbar unter:

https://de.wikipedia.org/w/index.php?title=Satz_des_Pythagoras&oldid=270455092

[62] P. Kumar, „Sichtfeld vs. Brennweite – alles, was Sie wissen müssen“, e-con Systems. Zugegriffen: 14. August 2026. [Online]. Verfügbar unter: https://www.e-consystems.com/blog/camera/de/technologie/field-of-view-focal-length-everything-you-need-to-know/

[63] „OpenCV: Camera Calibration“. Zugegriffen: 30. September 2026. [Online]. Verfügbar unter: https://docs.opencv.org/doc/doxygen/html/d4/d93/group__calib.html?utm_source=chatgpt.com

[64] „math — Mathematical functions“, Python documentation. Zugegriffen: 30. September 2026. [Online]. Verfügbar unter: https://docs.python.org/3/library/math.html

[65] R. Yeung, *keyframesfound/robotic-arm*. (18. Februar 2026). Python. Zugegriffen: 20. Mai 2026. [Online]. Verfügbar unter: https://github.com/keyframesfound/robotic-arm

[66] E. Candeloro, *e-candeloro/Vision-Robotic-Arm-Gesture-Recognition*. (14. Mai 2026). Python. Zugegriffen: 20. Mai 2026. [Online]. Verfügbar unter: https://github.com/e-candeloro/Vision-Robotic-Arm-Gesture-Recognition

[67] A. Osman Hashi, S. Zaiton Mohd Hashim, und A. Bte Asamah, „A Systematic Review of Hand Gesture Recognition: An Update From 2018 to 2024“, *IEEE Access*, Bd. 12, S. 143599–143626, 2024, doi: 10.1109/ACCESS.2024.3421992.

[68] „Support Vector Machine“, *Wikipedia*. 19. Februar 2026. Zugegriffen: 12. Oktober 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Support_Vector_Machine&oldid=264471992

[69] „Hidden Markov Model“, *Wikipedia*. 28. Juni 2026. Zugegriffen: 12. Oktober 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Hidden_Markov_Model&oldid=268311991

[70] M. Linardakis, I. Varlamis, und G. Papadopoulos, „Survey on Hand Gesture Recognition From Visual Input“, *IEEE Access*, Bd. 13, S. 135373–135406, Jan. 2025, doi: 10.1109/ACCESS.2025.3593428.

[71] Y. Shi, Y. Li, X. Fu, M. Kaibin, und M. Qiguang, „Review of dynamic gesture recognition“, *Virtual Real. Intell. Hardw.*, Bd. 3, Nr. 3, S. 183–206, Juni 2021, doi: 10.1016/j.vrih.2021.05.001.

[72] N. Robinson, B. Tidd, D. Campbell, D. Kulić, und P. Corke, „Robotic Vision for Human-Robot Interaction and Collaboration: A Survey and Systematic Review“, *ACM Trans. Hum.-Robot Interact.*, Bd. 12, Nr. 1, S. 1–66, März 2023, doi: 10.1145/3570731.

[73] M. Van den Bergh *u. a.*, *Real-time 3D hand gesture interaction with a robot for understanding directions from humans*. 2011, S. 362. doi: 10.1109/ROMAN.2011.6005195.

[74] D. Rode, A. Dunkel, R. Willi, P. Wolf, M. Xiloyannis, und R. Riener, „Assessment of monocular human pose estimation models for clinical movement analysis“, *Sci. Rep.*, Bd. 15, Nr. 1, S. 38767, Nov. 2025, doi: 10.1038/s41598-025-22626-7.

[75] P. Hemeren, P. Veto, S. Thill, C. Li, und J. Sun, „Kinematic-Based Classification of Social Gestures and Grasping by Humans and Machine Learning Techniques“, *Front. Robot. AI*, Bd. 8, Okt. 2021, doi: 10.3389/frobt.2021.699505.

[76] A. Floris *u. a.*, „Controlling Media Player with Hands: A Transformer Approach and a Quality of Experience Assessment“, *ACM Trans. Multimed. Comput. Commun. Appl.*, Bd. 20, Nr. 5, S. 1–22, Jan. 2024, doi: 10.1145/3638560.
[77] „Gazebo“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://gazebosim.org/home
[78] V. Mehta, *mvipin/gesturebot*. (3. Juli 2026). Python. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://github.com/mvipin/gesturebot
[79] „Control PIPER Arm with Hand Gestures“, Hackster.io. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://www.hackster.io/agilexrobotics/control-piper-arm-with-hand-gestures-7c2779
[80] „Mobile Robots, Robotic Arms, Embodied AI Solutions | Mobile the World“, Agilex Robotics. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://global.agilex.ai/
[81] „Petrel-3D结构光模组-奥比中光官网“. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://www.orbbec.com.cn/index/Product/info.html?cate=38&id=28
[82] agilexrobotics, „Agilex-College/README.md at master · agilexrobotics/Agilex-College“, GitHub. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://github.com/agilexrobotics/Agilex-College/blob/master/README.md
[83] „Mimic Robot“. Zugegriffen: 27. Mai 2026. [Online]. Verfügbar unter: https://courses.ece.cornell.edu/ece5990/ECE5725_Spring2019_Projects/mkl53_sk2282/index.html
[84] „multiprocessing — Process-based parallelism“, Python documentation. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://docs.python.org/3/library/multiprocessing.html
[85] „Basic Tutorial — Cython 3.3.0b1 documentation“. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://cython.readthedocs.io/en/latest/src/tutorial/cython_tutorial.html
[86] D. Kim, J. Lee, H.-S. Yoon, J. Kim, und J. Sohn, „Vision-based arm gesture recognition for a long-range human–robot interaction“, *J. Supercomput. - TJS*, Bd. 65, S. 1–17, Jan. 2010, doi: 10.1007/s11227-010-0541-9.
[87] „Mean-Shift Tracker“. [Online]. Verfügbar unter: https://www.cs.cmu.edu/~16385/s17/Slides/15.2_Tracking__Mean_Shift.pdf
[88] „Gesture Controlled Robotic Arm Using Kinect (Simulink Support For Kinect) and Arduino“. Zugegriffen: 12. August 2026. [Online]. Verfügbar unter: https://de.mathworks.com/matlabcentral/fileexchange/58189-gesture-controlled-robotic-arm-using-kinect-simulink-support-for-kinect-and-arduino
[89] D. Pagnon und H. Kim, *Sports2D: Compute 2D human pose and angles from a video or a webcam*. (September 2024). Python. doi: 10.21105/joss.06849.
[90] *Soprts2D Programmcode*. [Online]. Verfügbar unter: https://github.com/davidpagnon/Sports2D/tree/main
[91] „NVIDIA JetPack Software Stack“, NVIDIA Developer. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://developer.nvidia.com/embedded/jetpack
[92] Georg, „Gestensteuerung eines Roboters auf dem Nvidia Jetson Nano | Künstliche Intelligenz, 3D-Druck, Open Source und Linux“. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://blog.georgmill.de/2019/10/17/gestensteuerung-eines-roboters-auf-dem-nvidia-jetson-nano/
[93] U. T. SL, „IP Webcam (Android)“, Uptodown. Zugegriffen: 6. Oktober 2026. [Online]. Verfügbar unter: https://ip-webcam.de.uptodown.com/android

[94] P. Zeimet, *Pol-Zeimet/Gestenerkennung_auf_Skelettdaten_mit_CNN*. (3. Februar 2021). Python. Zugegriffen: 13. August 2026. [Online]. Verfügbar unter: https://github.com/Pol-Zeimet/Gestenerkennung_auf_Skelettdaten_mit_CNN

[95] „keras“, PyPI. Zugegriffen: 13. Oktober 2026. [Online]. Verfügbar unter: https://pypi.org/project/keras/

[96] jonas wegener, *Symulation des 3D-Audioraums der BHT*. (25. September 2026). onshape. [Online]. Verfügbar unter: https://cad.onshape.com/documents/00b008a3f533eca98fac5bba/w/b6eb2c8f02007af9c686e203/e/60674b425d3e898ddf7b70c7

[97] O. Business a PTC, „Onshape | Product Development Platform“. Zugegriffen: 10. Oktober 2026. [Online]. Verfügbar unter: https://www.onshape.com/en/

[98] „Linux System Requirements“. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://de.mathworks.com/support/requirements/matlab-linux.html

[99] „Intel RealSense D455 für Unitree G1 Anleitung“, OpenELAB Technology Ltd. Zugegriffen: 16. August 2026. [Online]. Verfügbar unter: https://openelab.io/de/blogs/learn/intel-realsense-d455-depth-camera-for-unitree-g1-edu-complete-guide

[100] „Intel Realsense D455 Tiefenkamera für Unitree G1 Edu Humanoiden-Robote“, OpenELAB Technology Ltd. Zugegriffen: 17. August 2026. [Online]. Verfügbar unter: https://openelab.io/de/products/intel-realsense-d455-for-unitree-robot

[101] „MediaPipe – Build Smart Media Apps Fast“. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://mediapipe.org/

[102] „Hardware Requirements for Machine Learning“, GeeksforGeeks. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://www.geeksforgeeks.org/machine-learning/hardware-requirements-for-machine-learning/

[103] „Hardware Recommendations“, Puget Systems. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://www.pugetsystems.com/solutions/ai/develop/hardware-recommendations/

[104] „Neuromorpher Prozessor“, *Wikipedia*. 5. Juli 2026. Zugegriffen: 15. Oktober 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Neuromorpher_Prozessor&oldid=268522480

[105] „Quick overview of Intel’s Neural Processing Unit (NPU) — Intel® NPU Acceleration Library documentation“. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://intel.github.io/intel-npu-acceleration-library/npu.html?utm_source=chatgpt.com

[106] „AMD Ryzen™ AI – Windows-PCs mit integrierter KI“, AMD. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://www.amd.com/de/products/processors/consumer/ryzen-ai.html

[107] „MINISFORUM UM760 Slim“, Minisforum EU. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://minisforumpc.eu/products/minisforum-um760-slim-mini-pc

[108] „Mini-PC Minisforum UM760 Slim mit Ryzen 5 im Test: Weniger Kerne, mehr Leistung“, heise online bestenlisten. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://www.heise.de/bestenlisten/testbericht/mini-pc-minisforum-um760-slim-mit-ryzen-5-im-test-weniger-kerne-mehr-leistung/tn00rrx

[109] „AMD Ryzen 5 7640HS - Benchmarks & technische Daten“. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://www.cpu-monkey.com/de/cpu-amd_ryzen_5_7640hs

[110] „GMKtec NucBox M6 Ultra AMD Ryzen™ 5 7640HS Mini PC“, GMKtec EU. Zugegriffen: 9. Oktober 2026. [Online]. Verfügbar unter: https://de.gmktec.com/products/gmktec-nucbox-m6-ultra-amd-ryzen-5-7640hs-mini-pc

[111] T. Nowag, „Neuer Preis-Leistungs-Sieger: GMKtec NucBox M6 Ultra im Test“, CHIP. Zugegriffen: 15. Mai 2026. [Online]. Verfügbar unter: https://www.chip.de/test/GMKtec-NucBox-M6-Ultra-im-Test_186385852.html

[112] „Raytracing“, *Wikipedia*. 20. Februar 2026. Zugegriffen: 2. September 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Raytracing&oldid=264478529

[113] B. Hangün und Ö. Eyecioğlu, „Performance Comparison Between OpenCV Built in CPU and GPU Functions on Image Processing Operations“, 2017.

[114] E. Yan, L. Luo, und L. Ceze, „Characterizing and Taming Resolution in Convolutional Neural Networks“, 28. Oktober 2021, *arXiv*: arXiv:2110.14819. doi: 10.48550/arXiv.2110.14819.

[115] A. Rudzitis und M. A. Zaeva, „Alternative Inverse Perspective Mapping Homography Matrix Computation for ADAS Systems Using Camera Intrinsic and Extrinsic Calibration Parameters“, *Procedia Comput. Sci.*, Bd. 190, S. 695–700, Jan. 2021, doi: 10.1016/j.procs.2021.06.081.

[116] L. Reiher, B. Lampe, und L. Eckstein, „A Sim2Real Deep Learning Approach for the Transformation of Images from Multiple Vehicle-Mounted Cameras to a Semantically Segmented Image in Bird’s Eye View“, arXiv.org. Zugegriffen: 28. Juli 2026. [Online]. Verfügbar unter: https://arxiv.org/abs/2005.04078v1

[117] „Erde Globus Länder“, Getty Images. Zugegriffen: 3. Juli 2026. [Online]. Verfügbar unter: https://www.gettyimages.de/detail/illustration/spinning-earth-globe-icon-set-latitude-15-n-lizenfreie-illustration/154417058

[118] „Weitwinkelobjektiv“, *Wikipedia*. 20. August 2024. Zugegriffen: 14. August 2026. [Online]. Verfügbar unter: https://de.wikipedia.org/w/index.php?title=Weitwinkelobjektiv&oldid=247906697

## Glossar

| Begriff | Definition |
|---|---|
| 3D-Audiosystem | Räumliches Lautsprechersystem im Raum B217a, mit dem Schallquellen dreidimensional positioniert und bewegt werden können. |
| Attention-Mechanismus | Verfahren, mit dem ein Modell unterschiedliche Teile einer Eingabe je nach ihrer Relevanz unterschiedlich stark gewichtet. |
| Azimut | Horizontaler Winkel, der die Richtung einer Schallquelle oder Zeigegeste relativ zu einer festgelegten Bezugsrichtung beschreibt. |
| Background Subtraction | Bildverarbeitungsverfahren, bei dem ein Hintergrundmodell genutzt wird, um sich davon unterscheidende Vordergrundbereiche zu erkennen. |
| CNN | Neuronale Netzarchitektur, die insbesondere bei Bilddaten Faltungsoperationen zur Erkennung räumlicher Merkmale verwendet. |
| Debugging | Systematisches Auffinden, Analysieren und Beheben von Fehlern in Software. |
| Dynamische Geste | Geste, die anhand einer Bewegung oder einer zeitlichen Abfolge von Haltungen erkannt wird. |
| Elevation | Vertikaler Winkel einer Richtung relativ zur Horizontalebene. |
| Greifgeste | In dieser Arbeit die Geste, mit der ein Instrument ausgewählt, gehalten oder wieder losgelassen wird. |
| Handöffnungsgrad | Numerischer Wert, der beschreibt, wie weit die Hand geöffnet ist und zur Unterscheidung zwischen offener Hand und Faust verwendet werden kann. |

| Begriff | Definition |
|---|---|
| hinreichendes FOV | In der Simulation ermitteltes Kamerasichtfeld, das den vorgesehenen Aktionsbereich unter den festgelegten Bedingungen vollständig abdeckt. |
| Inferenz | Anwendung eines trainierten KI-Modells auf neue Daten, um Vorhersagen oder Ergebnisse zu erzeugen. |
| JSON | Textformat zur strukturierten Darstellung von Daten, etwa in Form von Objekten und Arrays. |
| Landmark | Markanter, vom Erkennungsverfahren geschätzter Punkt, beispielsweise an einem Körpergelenk oder an der Hand. |
| MediaPipe | Framework zur Erstellung von Verarbeitungspipelines für Wahrnehmungsaufgaben, unter anderem zur Schätzung von Körper- und Handlandmarks. |
| Motion Capture | Erfassung und Aufzeichnung von Bewegungen, häufig durch optische oder sensorbasierte Messsysteme. |
| Pose-Estimation | Schätzung der Position und gegebenenfalls Orientierung eines Körpers oder seiner Gelenkpunkte anhand von Sensordaten. |
| Raumreferenzpunkt | Zusätzlicher Punkt, der bei der geometrischen Berechnung als Referenz zur Bestimmung eines Winkels dient. |
| Raumreferenzvektor | Vektor, der eine räumliche Bezugsrichtung für die Berechnung der Raumrichtung festlegt. |
| Raumrichtung | In dieser Arbeit die horizontale und vertikale Richtung von der Systemmitte zu einer räumlichen Position am Rand des 3D-Audiosystems. |

| **Begriff** | **Definition** |
|---|---|
| Region of Interest (ROI) | Ausgewählter Bildbereich, der für eine bestimmte Erkennungsaufgabe verarbeitet wird. |
| Skip-Connection | Verbindung, die Informationen an Schichten eines neuronalen Netzes vorbeiführt und dadurch das Training tiefer Netze erleichtern kann. |
| Spatiotemporal | Bezeichnet Zusammenhänge, die sowohl vom Raum als auch von der Zeit abhängen. |
| Statische Geste | Geste, die anhand einer Körper- oder Handhaltung erkannt wird, ohne dass ein zeitliches Bewegungsmuster erforderlich ist. |
| Stitchi 1.0 | Versionsname der in dieser Arbeit entwickelten Gestensteuerung zum Zeitpunkt der Abgabe. |
| Transformer | Neuronale Netzwerkarchitektur, die mithilfe von Attention relevante Beziehungen zwischen Elementen einer Eingabe gewichtet. |
| True-Positive-Rate | Anteil der tatsächlich positiven Fälle, die korrekt als positiv erkannt werden; hier der Anteil korrekt erkannter markierter Handzustände. |
| UDP-Datagramm | Einzelne Dateneinheit, die über UDP ohne Verbindungsaufbau übertragen wird. |
| Winkelpunkt | Punkt, an dem die beiden Vektoren zusammentreffen, deren Winkel für die Berechnung bestimmt wird. |
| Zeigerichtung | In dieser Arbeit die horizontale und vertikale Richtung des Vektors zwischen der Hand und der seitengleichen Schulter des Dirigenten. |

# Anhang

**Die Elemente im Anhang unterlagen KEINER finalen Prüfung auf Rechtschreibung, Grammatik und inhaltliche Kausalität. Ich bitte dies zu entschuldigen.**

**Bedienungsanleitung für die Gestensteuerung Version STITCHI 1.0**

**User Manual**

Die folgende Nutzeranleitung befindet sich auch im Projektordner unter: *User_Guide_Stitchi1.0.docx*

# Stitchi 1.0 — User Guide

*Operating the camera-based gesture-control interface*

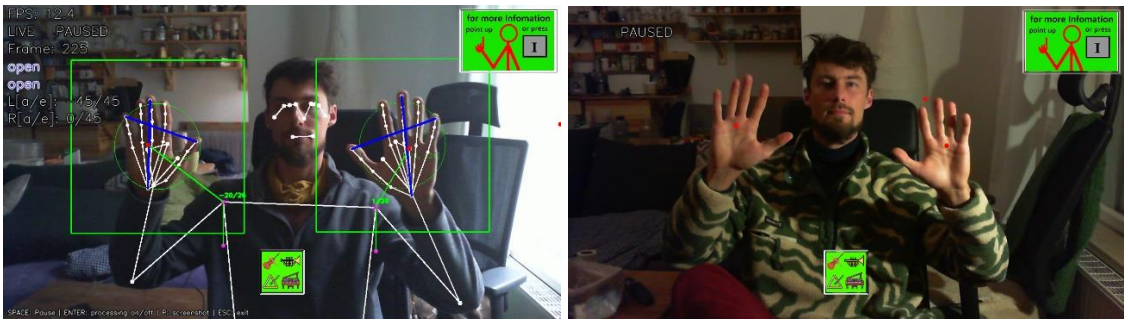
This guide is for a person standing in front of the camera and using the gesture interface. It assumes that the application has been installed and configured by a developer.

## 1. Before you start

1. Place the camera in its calibrated position and connect it to the computer. For depth-based operation, connect the configured Intel RealSense camera.
2. Stand far enough from the camera for the upper body and both arms to remain visible.
3. Ask the operator to start `main.py`. Wait until the camera window and instrument interface are visible.

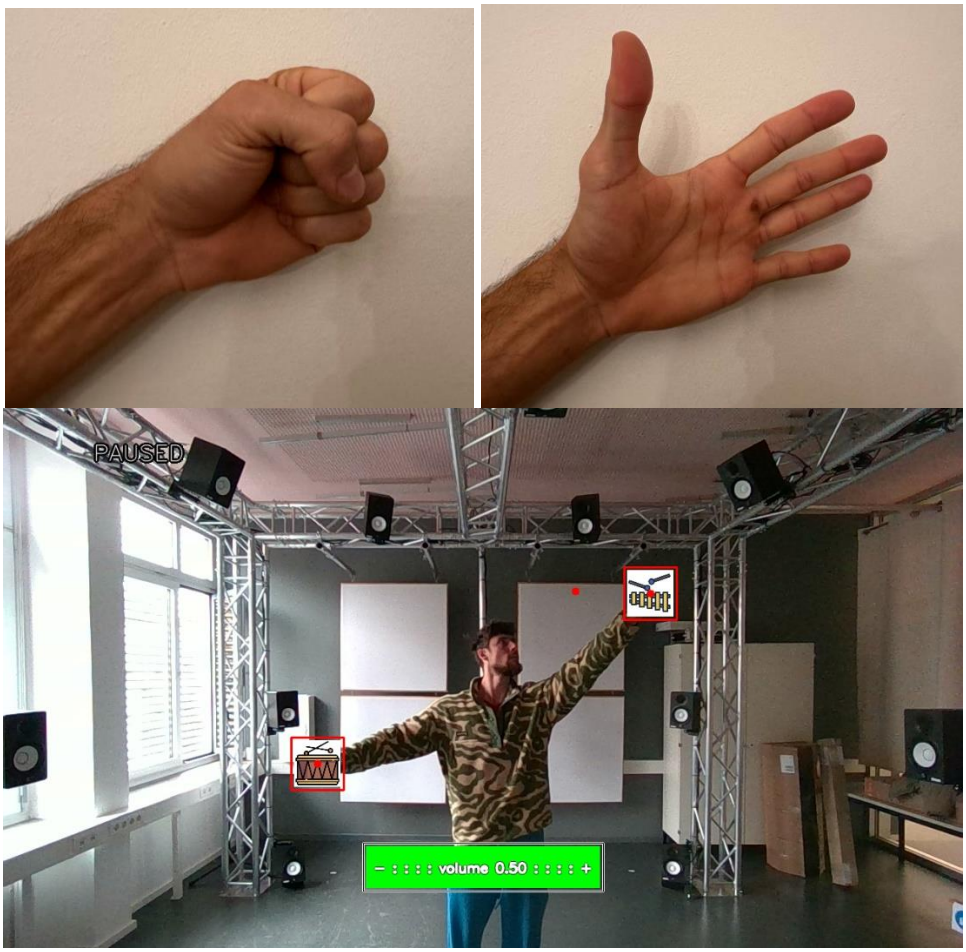
## 2. Understanding the interface

The camera image is the background for the GUI overlay. Instrument tiles can be selected and moved with hand gestures. Depending on the display mode, the window may also show body/hand landmarks, a hand region of interest (ROI), aperture information, pointing angles, frame number, and FPS. These diagnostic markings are for feedback and may be hidden in the GUI-only mode.



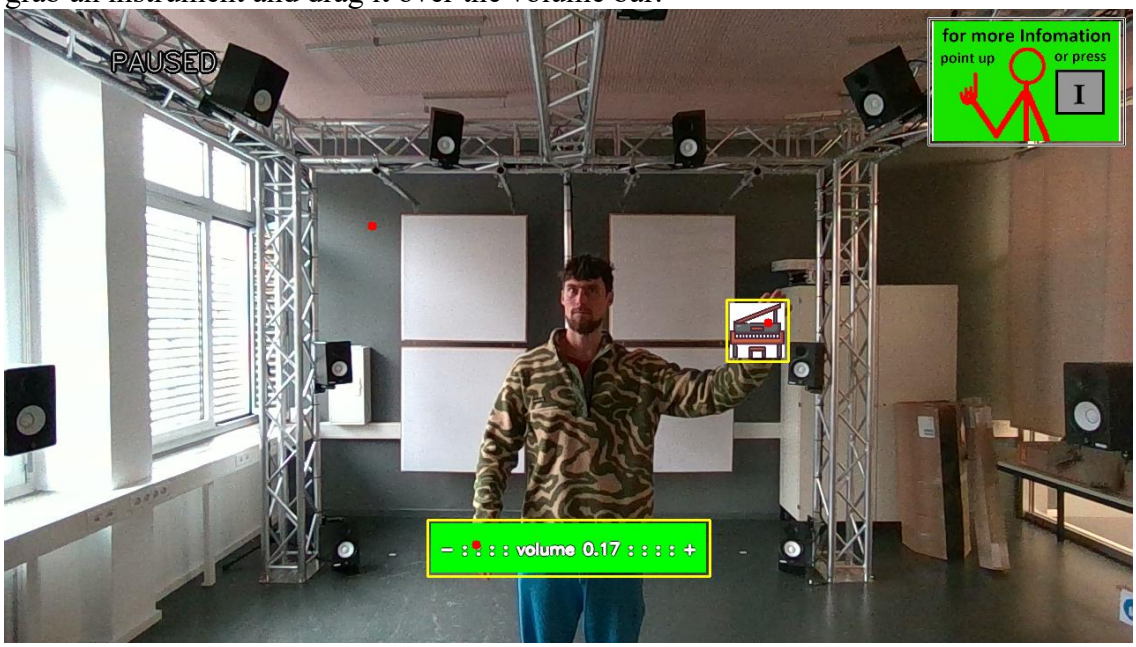
### 3. Select, move, and release an instrument

4. Locate the green instrumentmenu.
5. Move your hand so its displayed hand centre (a red dot) is over the tile.
6. Are selection of instruments will show up.
7. Locate the instrument tile you want to control.
8. Keep the hand clearly visible to the camera.
9. Close your hand to grab the tile. The application detects the transition from an open hand to a closed hand.
10. While holding the closed hand, move your arm to point in the desired direction. The application calculates the arm direction from the shoulder and hand landmarks and updates the instrument's direction.
11. The arm–shoulder vector determines the direction from which the instrument is heard in the physical space, while its position on the GUI serves solely as visual feedback.
12. Open your hand to release the tile. The instrument remains at the position/direction reached when released.
13. If both-hand mode is enabled, the application can track both hands
14. You can also use the mouse if mouse_control is enabled. Then left klick is grabbing. The mouse can not change the sound direction.




## 4. Change Volume

To adjust the volume, hold one hand over an instrument to display the volume bar. Then, hover your other hand over the desired position on the bar to set the volume. Alternatively, grab an instrument and drag it over the volume bar.



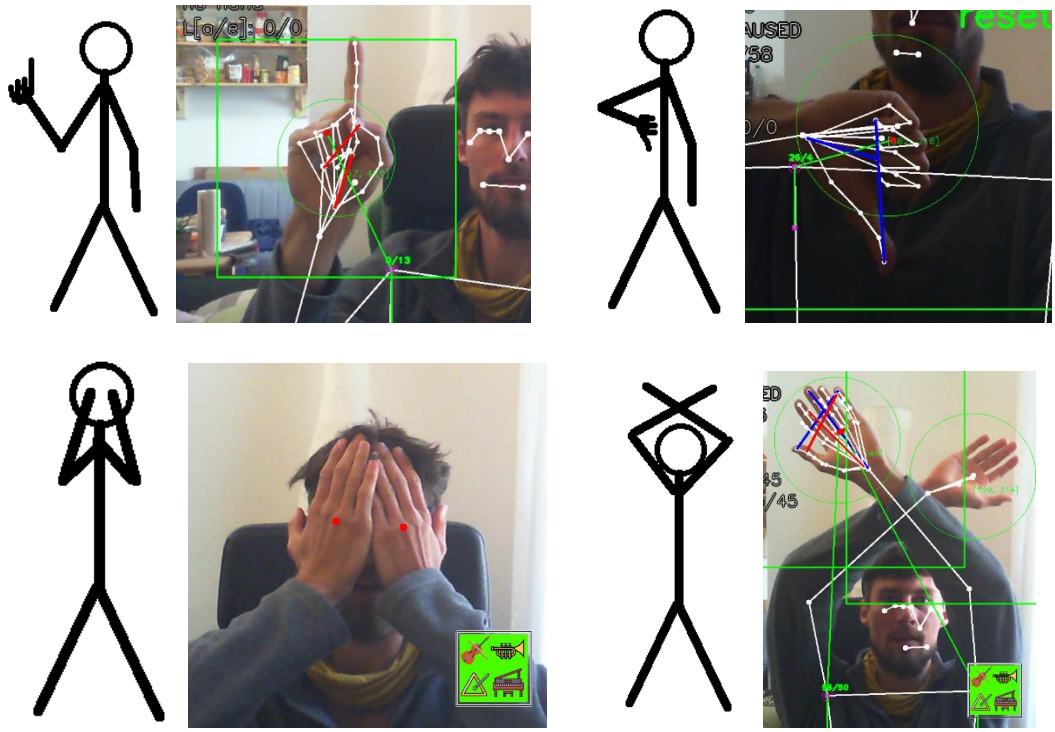

**5. Adjust Menu**

Just like the instruments, you can move the instrument menu and the volume bar around the screen. This only affects their visual position, allowing you to place them wherever they are least distracting.

**6**. Additional controls

| Action | How to use it / key |
|---|---|
| **Show or hide the information menu** | Hold the index finger pointing upward for about 1 second (gesture). Press `I` for the keyboard shortcut. Hover over the Info button in the top-right corner with your hand or mouse pointer. |
| **Clear/reset the GUI** | Use the configured clear gesture: Thumb held pointing downward in front of the body for 1 second, or press `R`. This resets the instrument GUI state. |
| **Change display/feedback mode** | Use the visibility-mode gesture: cover your eyes for 1 second, or press `V`. The available modes are configured by the developer. |
| **Pause/resume video processing** | Press `Space`. This is mainly useful during video playback/testing. |
| **Toggle processing overlay** | Press `Enter` to toggle on and off the processing of the entire gesture control |
| **Save screenshot** | Press `P`, saves the shown Frame in "...\Stitchi1.0\screenshots". |
| **Exit** | Use the ending gesture by crossing your hands above your head or press `Esc`. This terminates the entire gesture control application. |

| | |
|---|---|
| **Restart application** | Press `Q`. |
| **Navigate a normal video file** | `W` / `S` step by one frame; `A` / `D` jump by more frames. These controls apply to ordinary video playback, not live camera input or bag-file. |



### 7. Good operating practice

- Keep hand landmarks visible during pointing.
- Move deliberately when grabbing, pointing, and releasing.
- Wait for the visible feedback after each control action before starting the next one.
- Do not change camera position after calibration unless a developer updates the room/camera settings.
- Treat the current software as a research prototype; recognition can be affected by lighting, clothing, body position, and camera geometry.

**Developer Manual**

Die folgende Code-Anleitung befindet sich auch im Projektordner unter: *Developer_Guide_Stitchi1.0.docx*

# Stitchi 1.0 — Developer Guide

*Architecture, setup, configuration, and MATLAB/UDP integration*

## 1. Requirements and environment

The supplied project was developed/tested in the following environment. These are the reported installed versions, not a guarantee that every version is required or that the same combination will work unchanged on every machine.

| Component | Reported version / setting |
|---|---|
| **Operating environment** | Windows PowerShell; virtual environment shown as `(venv)` |
| **Python** | 3.11.4 |
| **MediaPipe** | 0.10.9 |
| **OpenCV** | opencv-python 4.13.0.92; opencv-contrib-python 4.13.0.92 |
| **NumPy** | 2.4.2 |
| **Intel RealSense Python API** | pyrealsense2 2.55.1.6486 |
| **Other notable packages** | protobuf 3.20.3; absl-py 2.4.0; attrs 25.4.0; Pillow 12.1.1; requests 2.34.2; jsonschema 4.26.0 |
| **MATLAB** | Must support `udpport`, `jsonencode`, `jsondecode`, and the string APIs used by the scripts. The exact MATLAB release was not provided. |
| **Hardware** | A camera compatible with the chosen input mode; for depth-based operation, an Intel RealSense camera and compatible RealSense SDK/driver. A network connection between the video-system PC and audio-system PC is required for remote UDP communication. |

You can just run `python -m pip install -r requirements.txt` to install all packages in python or its virtual environment. The `requirements.txt` contains all library names and versions.

All of the other packages listed by pip are indirect dependencies that should automatically installed alongside the main libraries and are required for their functionality. The full pip list supplied with the project also included: asttokens 3.0.1, backcall 0.2.0, beautifulsoup4 4.15.0, bleach 6.4.0, certifi 2026.6.17, cffi 2.0.0, charset-normalizer 3.4.7, colorama 0.4.6, contourpy 1.3.3, cycler 0.12.1, decorator 5.3.1, defusedxml 0.7.1, docopt 0.6.2, executing 2.2.1, fastjsonschema 2.21.2, flatbuffers 25.12.19, fonttools 4.61.1, idna 3.18, IPython 8.12.3, jedi 0.20.0, Jinja2 3.1.6, jsonschema-specifications 2025.9.1, kiwisolver 1.4.9, MarkupSafe 3.0.3, Matplotlib 3.10.8, mistune 3.3.2, nbclient 0.11.0, nbconvert 7.17.1, nbformat 5.10.0, packaging 26.0, pandocfilters 1.5.1, parso 0.8.7, pickleshare 0.2.2, pip 26.1.2, pipreqs 0.5.0, platformdirs 4.10.0, prompt-toolkit 3.0.52, pure-eval 0.2.3, Pygments 2.20.0, pycparser 3.0, pyparsing 3.3.2, python-dateutil 2.9.0.post0, pyzmq 27.1.0, referencing 0.37.0, rpds-py 2026.6.3, setuptools 65.5.0, six 1.17.0, SoundDevice 0.5.5, soupsieve 2.8.4, tornado 6.5.7, traitlets 5.15.1, typing_extensions 4.16.0, urllib3 2.7.0, wcwidth 0.8.2, webencodings 0.5.1, yarg 0.1.9.

# 2. Project structure

| File | Responsibility |
|---|---|
| **main.py** | Application orchestration and frame loop; connects all subsystems. |
| **settings.py** | Central configuration read by the other modules. |
| **frame_handler.py** | Selects, initializes, reads, displays, and closes camera/video/depth sources. |
| **MediaPipe_handler.py** | Pose and hand landmark inference, visibility/movement data, hand ROI handling. |
| **coordinates_handler.py** | Pixel/depth-to-3D conversion and coordinate compensation. |
| **angle_handler.py** | Arm vector, azimuth/elevation, room ray tracing, angle smoothing and special camera-pointing case. |
| **gesture_handler.py** | Interprets landmarks as grab/release transitions and control gestures. |
| **GUI.py** | Instrument tiles and controls; applies selection, grab, movement, release and GUI state. |
| **communication.py** | UDP/JSON sender, receiver thread, message queue, send-on-change logic. |

| | |
|---|---|
| **own_functions.py** | Shared buffers, movement/aperture helpers, drawing, mouse, file and geometry utilities. |
| **analyse.py** | Test-data capture, file discovery, status annotation, JSON/list persistence, and offline comparison. |
| **matlab-code/send_udp.m** | Send-only instrument-list smoke test. |
| **matlab-code/receiv_udp.m** | Receive-only UDP diagnostic. |
| **matlab-code/UDP_send_instruments_then_receiv_orders.m** | Send a fixed instrument list, then print incoming messages. |
| **matlab-code/UDP_send_instruments_per_comandline_and_reseiv.m** | Interactive instrument-add test and receive loop. |
| **matlab-code/README.md** | Short original German instructions for starting Python and MATLAB communication. |

# 3. Runtime call flow

15. Running `python main.py` enters the `if __name__ == '__main__'` block and calls `main()` using values from `settings.py`.
16. `main()` creates `GuiOverlay`, `FrameSource`, two `HandDetector` instances, `PoseDetector`, `RoomAngleDetector`, `GestureDetector`, `SendOnChange`, and `SaveUdpToDict`. The receiver starts in a background thread.
17. `FrameSource` reads each frame. In RealSense mode it can provide aligned colour and depth frames; in ordinary webcam/video mode only the image is available.
18. `PoseDetector` estimates body landmarks and exposes shoulder/hand reference points. `HandDetector` estimates hand landmarks. The results are passed to `GestureDetector` and the angle module.
19. `RoomAngleDetector` estimates pointing azimuth/elevation from hand and shoulder coordinates. With RealSense depth it uses depth/intrinsics; other paths use available MediaPipe world coordinates or fallback approximations.
20. `GestureDetector` detects hand open/closed states and transitions. A transition from open to closed starts a grab; reopening releases the currently grabbed GUI item. Additional gestures trigger GUI actions.
21. `GuiOverlay` uses the hand positions and grab/release events to select and move instrument tiles. The selected instrument state is exposed by `get_info()`.
22. `SendOnChange` sends a JSON/UDP message when the GUI instrument state changes. Separately, `SaveUdpToDict` receives messages such as `add_instruments` from MATLAB and places them in a queue for `main()` to process.
23. On exit, `main()` closes the frame source and prints/saves available analysis statistics.

# 4. Module interaction summary

The live dependency chain is approximately:

```
main.py
  ├─ frame_handler.FrameSource  → camera / video / RealSense frames
  ├─ MediaPipe_handler.PoseDetector + HandDetector → landmarks
  │    └─ coordinates_handler → 3D conversion / camera-room compensation
  ├─ gesture_handler.GestureDetector → grab/release and control triggers
  ├─ angle_handler.RoomAngleDetector → azimuth/elevation in room coordinates
  ├─ GUI.GuiOverlay → instrument selection and movement
  └─ communication.SendOnChange / SaveUdpToDict ⇔ MATLAB audio-system process

Shared helpers: own_functions.py
Test and analysis support: analyse.py
Shared configuration: settings.py
```

# 5. Starting the application

24. Install Python 3.11.4 and the compatible Intel RealSense SDK/driver if using a RealSense camera. Connect the camera before launching the program.
25. Open PowerShell in the project root (`Stitchi1.0`) and activate the project virtual environment.
26. Install the project dependencies in that environment. Prefer a pinned requirements file for reproducibility; the archive did not contain one, so use the package versions listed in Section 1 as the recorded environment.
27. Check `settings.py`: input source, room/camera geometry, network IP addresses, UDP port, and GUI options.
28. Run `python main.py`. The configured camera/video window should open. Use `ESC` to exit.

The current `video_source = 'depth'` setting selects the RealSense depth camera path. For an ordinary webcam, set it to an integer camera index such as `0`. For a video file, set it to the filename/path. RealSense recordings use `.bag`; the configured `rs_save_type` is `.bag`.

# 6. settings.py — parameter reference

The following table explains the practical effect of each setting present in the supplied file. Values in parentheses are the values in the archive. Changes can interact; tune one group at a time and validate with repeatable videos.

| Setting | Current value | Effect |
|---|---|---|
| **name / version** | `Gesture control` / `Stitchi 1.0` | Window/application label only. |
| **fps** | 30 | Processing frame-rate cap and requested camera frame rate. Higher values increase responsiveness only if the camera and computer can sustain them; lower values reduce load but add latency. |
| **start_frame / end_frame** | 1 / 0 | Video playback start and optional ending frame. `end_frame = 0` disables an explicit end limit. |
| **live_stream_resulutuin** | (1280, 720) | Requested live image dimensions. Higher resolution can improve landmark detail but increases processing load. Must be supported by the selected camera. |
| **window_size** | (1920, 1080) | GUI/mouse coordinate reference and display size; changing it can affect mouse mapping and layout. |
| **hand_width_factor / hand_opening_factor** | 1.2 / 1.3 | Scaling factors used by hand/gesture sizing logic; may change hand-region/aperture interpretation and should be tuned with recorded examples. |
| **skip_frames** | 100 | Frame skip/analysis-related parameter; verify its call sites before relying on it for live recognition because it is not a general frame-rate setting. |
| **icon_folder / screenshots_folder / gui_info_window_image_path** | relative paths | Locations of GUI assets and screenshots. Paths are relative to the working directory; missing assets can affect icons/info visuals. |
| **video_folder** | relative test-video folder | Location searched for offline test videos and RealSense recordings. |

| | | |
|---|---|---|
| **port** | 5005 | UDP destination/listener port used by the Python communication classes. Must match MATLAB target/local ports. |
| **IPv4_audiosystem / IPv4_videosystem** | 192.168.0.10 / 192.168.0.11 | IP addresses of audio-system PC and video-system PC. Replace with the actual addresses on the current network. |
| **bind_ip** | 0.0.0.0 | Python UDP receiver binds to all local IPv4 interfaces. Use `127.0.0.1` only for same-machine tests. |
| **printout_udp** | True | Enables console output for UDP messages when the call requests printing. |
| **add_inst_key / start_instruments** | `add_instruments` / empty list | JSON key used to add instruments; optional initial list inserted at startup. |
| **video_source** | `depth` | Selects RealSense live depth camera (`depth`), a standard camera index (`0`, `1`, …), an ordinary video file, or a RealSense `.bag` recording. |
| **default_buffer_mode** | `median` | Default buffer aggregation mode. Median filtering can reduce outlier jitter; smoothing can add lag. |
| **cam_inclination_angle** | 0 degrees | Camera inclination compensation used in 3D coordinate conversion. Set to the calibrated physical tilt; an incorrect value biases pointing angles. |
| **visibility_threshold** | 0.92 | Minimum landmark visibility used by pose/point checks. Higher values reject uncertain points more often; lower values may increase detections but admit noisy landmarks. |
| **align_depth** | True | Aligns RealSense depth to the colour stream, so landmark pixels can be sampled from corresponding depth coordinates. Usually needed for this workflow. |
| **active_hand** | `both` | Hand selection mode: `right`, `left`, `moving`, `top`, or `both`. `moving` chooses based on motion; `top` chooses the highest image hand; `both` processes both hands. |
| **no_hand_frame_count** | 50 frames | Frames without a detected hand before treating it as absent. At 30 FPS, 50 frames are about 1.67 s; increasing it delays loss/reset recognition. |
| **moving_min_speed** | 150 px/s | Minimum image-space speed used to classify a hand as moving in the moving-hand mode. |
| **moving_check_time** | 0.3 s | Time interval over which hand motion is assessed. Longer intervals smooth speed estimates but respond more slowly. |
| **hand_change_min_speed** | 200 px/s | Speed threshold for switching the selected hand in `moving` mode. Higher values reduce accidental switching but may miss slower hand changes. |
| **use_roi** | True | Restricts hand detection to a region around the expected hand. Can improve speed/detection in the intended region, but may miss a hand outside the ROI. |
| **roi_min_size** | 50 px | Minimum ROI dimension. Larger minimum regions can help retain enough context but increase image area processed. |
| **hand_shoulder_intersect_dist** | 0.10 m | Depth-camera threshold for treating hand and shoulder as too close in image/depth geometry, used for the special case where the arm points toward the camera. |
| **hand_shoulder_intersect_pixel** | 20 px | Image-space threshold for the same special case when depth-based distance is |

| | | |
|---|---|---|
| | | unavailable. Despite the source comment, the unit is pixels, not metres. |
| **hand_center_hands** | (5, 17) | MediaPipe Hands landmark indices used to estimate hand centre. Changing indices changes the point used for cursor/pointing calculations. |
| **left_hand_center_pose / right_hand_center_pose** | (17,19) / (18,20) | MediaPipe Pose landmark pairs used as hand-centre approximations when using pose landmarks. |
| **grab_detection_methode** | `dynamic` | Selects hand aperture method (`static` or `dynamic`). The chosen method affects how open/closed states are inferred and therefore GUI grab/release reliability. |
| **dynamic_grab_distance_difference** | 1 | Minimum aperture/distance change for dynamic open/closed detection. Larger values require a stronger change; smaller values are more sensitive and may trigger false changes. A value of 1 means that the opening distance must double or halve to trigger a change. A value of 0.5 means that it must increase by a factor of 1.5 or decrease to 0.75 of its original value. |
| **dynamic_grab_time_difference** | 1 second | Time interval for dynamic aperture comparison. Longer intervals are less sensitive to fast changes; shorter intervals react faster but may be noisier. |
| **hand_status_buffer_size / hand_status_buffer_atleast** | 3 / 3 frames | Number of recent hand-state samples and minimum agreeing samples for buffered/static classification. More smoothing reduces flicker but can delay transitions. For static hand detection method. |
| **hand_open_thrashold / hand_close_thrashold** | 65 / 75 | for the static hand detection method, values between zero and 100 determine the degree of hand opening at which a hand transitions to a state of being "too open" or "too closed." |
| **angle_buffer_size** | 5 frames | Number of recent angle samples used for smoothing. Larger buffers stabilize pointing but add lag. |
| **volume_decimal_place** | 2 | Number of decimal places used when quantizing volume values sent or displayed. |
| **real_depth_angle_resulutuin** | 15 degrees | Quantization step/resolution for room pointing angles when depth is used. Coarser steps reduce jitter but reduce directional precision. |
| **ray_tracing_step_size** | 0.1 m | Step distance for ray tracing to the room boundary. Smaller steps can improve boundary resolution but require more iterations. |
| **room_size** | (3.4, 2.7, 3.4) m | Room dimensions in x/y/z used for ray tracing. Must match the coordinate convention and modeled room. |
| **room_center** | (1.7, 1.5, 1.7) m | Reference centre of the room used for room-relative direction calculations. |
| **cam_pos_in_room** | (1.7, 1.7, -0.1) m | Camera position in room coordinates. Errors here bias the reconstructed pointing direction. |
| **estimated_hip_hight** | 0.95 m | Estimated hip height used to offset MediaPipe world landmarks into the room coordinate system. |
| **show_gui** | True | Enables/disables the instrument GUI overlay and related GUI-state sending. |

| | | |
|---|---|---|
| **gui_tile_max_size** | 100 px | Maximum instrument tile size used by GUI layout. |
| **gui_hight** | 1/4 of frame height | GUI panel height proportion relative to the video frame. |
| **controll_trigger_text_time** | 1.0 s | Duration of temporary on-screen feedback after a control gesture/key. |
| **instrument_start_volume** | 0.5 | Initial volume for new instrument tiles, on a 0–1 scale. |
| **no_mouse_control** | False | When True, disables mouse selection/grab/release input; gesture control remains available. |
| **mirrow_view** | True | Mirrors the displayed image/coordinate handling. Must stay consistent with landmark/depth coordinate interpretation. |
| **draw_aperture / draw_hand_skeleton / draw_body_skeleton / draw_roi / draw_hand_vlidation_dist** | True | Debug overlays for aperture, hand/body landmarks, ROI, and hand-validation distance. Useful for tuning; overlays may add display/rendering overhead. |
| **show_depth_frame** | False | Controls display of the depth image for debugging. |
| **type_instrument** | `instrument` | Type field used to identify outgoing instrument-control messages. |
| **rs_save_type** | `.bag` | File extension used to identify RealSense recordings. |
| **display_feedback_mode** | two modes: processing+GUI; GUI only | Cycle of visual feedback modes. Add/remove valid mode strings only if the GUI code supports them. |
| **red / blue / green / white / black / yellow** | BGR colour tuples | Colours for overlays and GUI feedback; OpenCV uses BGR ordering. |

# 7. Adjust hand grabbing detection

Use ‘grab_detection_methode’ to set the static or dynamic hand-detection method. If you wish to modify the dynamic hand-detection method, focus on parameters ‘dynamic_grab_distance_difference’ and ‘dynamic_grab_time_difference’.
If you wish to modify the static hand-detection method, focus on parameters ‘hand_status_buffer_size’ , ‘hand_status_buffer_atleast’, ‘hand_open_thrashold’ and ‘hand_close_thrashold’. These are described in greater detail in section 6.

# 8. MATLAB audio-system communication (UDP/JSON)

The video system performs gesture recognition and GUI interaction; the MATLAB/audio-system process receives instrument state and can send instrument names back to populate the GUI. UDP is connectionless: the sender does not establish a session or receive a delivery guarantee. JSON is the payload format.

## 8.1 Addressing and ports

- In `settings.py`, `IPv4_audiosystem = 192.168.0.10` is the Python sender's target, `IPv4_videosystem = 192.168.0.11` is the video PC address used by MATLAB scripts, and `port = 5005` is the shared destination/listener port.
- The Python receive thread binds to `bind_ip:port`; with `bind_ip = 0.0.0.0`, it listens on all local IPv4 interfaces. MATLAB scripts bind their local `udpport` to port 5005 and send to `192.168.0.11:5005`.

- Using port 5005 on both computers is valid: the local IP addresses differ. On one computer, two independent sockets cannot normally bind exclusively to the same local IP/port. If both apps run on one PC, configure a single listener or use different local ports.
- Ensure both PCs are on a reachable network, use the correct IPv4 addresses, and allow inbound UDP port 5005 in the Windows firewall. A successful `write()` call does not prove that the other application received the packet.

## 8.2 Message flow

29. Start `python main.py` on the video-system PC. It creates `SaveUdpToDict`, which starts a background receive loop, and `SendOnChange`, which sends changed GUI instrument state to the audio-system IP.
30. Run a MATLAB sender/receiver script on the audio-system PC. It sends JSON such as `{"add_instruments":["piano","violin"]}` to the video PC.
31. The Python receive loop decodes the JSON and queues it. `main.py` reads the queued `add_instruments` field and calls `gui.add_instrument(...)` for each name.
32. When an instrument is selected and moved/released, the GUI exposes its current state. `SendOnChange` serializes and sends the changed state as JSON to the audio-system PC.
33. MATLAB reads incoming datagrams, converts UTF-8 bytes to text, applies `jsondecode`, and prints the decoded structure. Replace the print statement with the audio engine's parameter-update function when integrating the real audio system.

## 8.3 Recommended communication test

34. Check and record the actual IPv4 addresses on both PCs. Update `IPv4_audiosystem`, `IPv4_videosystem`, `targetIp`, `targetPort`, and `localPort` as needed.
35. Temporarily allow UDP port 5005 through the firewall on both systems.
36. Start Python `main.py` first. Confirm that the video window opens and no port-binding exception appears.
37. Run `UDP_send_instruments_then_receiv_orders.m` in MATLAB. The instrument names should appear in the GUI; MATLAB should print any UDP messages sent back by Python when instrument state changes.
38. Select an instrument using the hand gesture, move the pointing direction, and release it. Confirm that the received JSON fields and angle values change as expected.
39. If testing on a single PC, use `127.0.0.1` (special IPv4 loopback address) and ensure only one application owns each local listener port. For separate PCs, use the actual LAN IPs.
40. If nothing arrives, verify IP/port, firewall, that the Python receiver thread is running, and that no other process already occupies local UDP port 5005. On Windows, `netstat -ano | findstr :5005` helps identify the PID holding the port.

## 8.4 Which MATLAB script should be used?

| Script | Purpose |
|---|---|
| **send_udp.m** | One-shot send-only smoke test; it does not bind local port 5005 or receive replies. |
| **receiv_udp.m** | Receive-only diagnostic on local port 5005. |
| **UDP_send_instruments_then_receiv_orders.m** | Recommended basic round-trip test: sends a fixed instrument list and continuously receives responses. |

| | |
|---|---|
| **UDP_send_instruments_per_comandline_and_reseiv.m** | Interactive test: adds one instrument name per prompt and receives responses. Since input() blocks, received messages are only processed after the user returns to the loop. |

# 9. Adding instruments and audio-system connection

The available instrument tiles can be provided by the MATLAB/audio-system application over UDP. When the communication is configured, the audio system sends a JSON message using the key `add_instruments`, for example an array containing instrument names. For example: ["trumpet", "trumpet2", "piano", "violin","drum","r2d2"]. Each name can appear only once. The names are then added to the GUI. If no instruments appear, check network addresses, port 5005, firewall rules, and the MATLAB sender must be configured first.

Instruments can also be pre-configured by the state in the settings.py file under `start_instruments`.

The 'icons' folder contains the image files for all GUI tiles. When a new instrument is added, the system automatically checks whether a suitable image for that instrument exists in this folder. It selects the first image that includes the full instrument name in its title. You can add images there. If you do so, make sure they are square and measure approximately 100x100 to 300x300 pixels.

# 10. Adjrust GUI menu tiles and information

The image ‘info_window.png’ contains the information displayed in the info window, which is opened either by the info gesture, info button or by pressing the I key. The icons for the instrument menu and the info button are located in the icons folder. If you want to adjust any of them, in particular the info window, it is recommended to use the raw files in the ‘appendix’ folder to create new PNG files. I used the free image editing software ‘panit.net’.
The volume bar is generated dynamically from text within the ‘VolumeBar’ class in the ‘GUI.py’ file.

# 10. Operational caveats

- The supplied Python source starts a receive thread and calls a receive function that binds to port 5005. Do not start multiple application instances or multiple receive loops on the same local port.
- UDP can drop or reorder packets. If the audio system needs guaranteed delivery or state recovery, add sequence numbers/acknowledgements or periodic full-state updates.
- The current MATLAB scripts are diagnostics: they print JSON but do not implement actual audio synthesis/control.

**Kamerawahl (ursprüngliche anforderungen)**

Grundsätzlich ist zunächst zu entscheiden, ob für das Projekt eine Kamera mit reiner 2D-Bildaufnahme oder eine Kamera mit zusätzlicher Tiefenmessung eingesetzt werden soll. Auf Grundlage, der in den vorherigen Kapiteln zu Hardware und Software ermittelten Anforderungen lassen sich für beide Varianten folgende Kriterien gegenüberstellen.

**Kamera mit Tiefensensor:**

- Das Sichtfeld wird wahrscheinlich kleiner als das hinreichende FOV sein, welches im Kapitel *Raumsimulation, Kameraposition* bestimmt wurde. Dadurch können Einschränkungen hinsichtlich der Kameraposition, der Standzone oder der zu erfassenden Personengröße entstehen.

- Höhere Anschaffungskosten.

- Je nach Kameramodell ist eine spezielle Treibersoftware erforderlich. Deren Kompatibilität mit der verwendeten Bildverarbeitungssoftware und der Prozessorarchitektur der Recheneinheit muss gewährleistet sein.

- Die Tiefeninformation wird direkt durch den Sensor erfasst und muss nicht aus den 2D-Bilddaten geschätzt werden. Dadurch wird eine einfachere und potenziell genauere Bestimmung der räumlichen Armwinkel erwartet.

**Kamera ohne Tiefensensor:**

- Die Bestimmung des Armwinkels muss ausschließlich auf Grundlage der 2D-Bildinformationen erfolgen. Hierfür ist gegebenenfalls eine Methodik zur Gewinnung von Tiefeninformationen aus einem 2D-Bild nötig.

- Reine 2D-Kameras sind weit verbreitet und daher in einer großen Auswahl an Sichtfeldern erhältlich. Dadurch besteht eine hohe Flexibilität bei der Wahl des gewünschten FOVs, beispielsweise auch bei der Auswahl eines Ultraweitwinkelsensors.

- Geringere Anschaffungskosten.

- Hohe Kompatibilität mit den übrigen Projektkomponenten, da die Treibersoftware herkömmlicher 2D-Kameras in der Regel schon von Bildverarbeitungsprogrammen unterstützt wird.

Soll eine Kamera ohne Tiefensensor verwendet werden, muss zusätzlich zwischen einem Objektiv mit Weitwinkel und einem Objektiv mit kleinerem Sichtfeld gewählt werden. Ein Weitwinkelobjektiv bildet einen größeren Bereich der dreidimensionalen Umgebung auf derselben zweidimensionalen Bildfläche ab. Dadurch steht für einen bestimmten zentralen Ausschnitt des Sichtfelds eine geringere Pixeldichte zur Verfügung. Gleichzeitig nehmen die perspektivischen und objektivbedingten Verzerrungen in den Randbereichen zu. Dieser Zusammenhang ist vergleichbar mit der Abbildung einer gekrümmten Erdoberfläche auf eine ebene Weltkarte.

Bei gleicher Bildauflösung kann der zentraler Bildbereich, der bei einer Kamera mit kleinerem Sichtfeld einen großen Teil der verfügbaren Pixel einnimmt als bei einer Weitwinkelkamera. Dadurch sinkt die Informationsdichte in diesem Bereich. Mildernd hierfür sind die Erkenntnisse aus dem Kapitel ***Fehler! Verweisquelle konnte nicht g efunden werden.***, in verschiedenen Arbeiten mit teils ähnlichen Distanzen zur Kamera war bereits eine geringe Bildauflösungen ausreichend, für eine erfolgreiche Gestenerkennung. Gering Auflösung, im Vergleich zu einer gängigen Full HD Auflösung. Daher wir hier davon ausgegangen, dass die verbleibende Informationsdichte in der Bildmitte auch bei einer Weitwinkelkamera ausreichen ist. Bei einer HD oder Full HD Auflösung auch um ggf. kleinere Strukturen wie Finger aus 2-4 m Entfernung zu erkennen. Ein größeres FOV vergrößern den räumlich möglichen Aktionsbereich des Videosystems.

Zur Bildauflösung ist zu bemerken, dass die Verarbeitung eines Bildes rechenaufwendiger wird, wenn das Bild mehr Pixel enthält. In der Klassischen Bildverarbeitung ist der Zusammenhang zwischen Rechenaufwand und Pixeln meist relative linear [113]. Bei neuronalen Netzen kann die Abhängigkeit je nach Architektur und Verarbeitungsschritten abweichen und auch einen überproportionalen Anstieg aufweisen [114]. Dennoch sollte eine HD oder Full HD Auflösung keine signifikanten Probleme darstellen, da sich Bilder vor der Weiterverarbeitung runter skalieren lassen.

Das Runterskalieren an sich benötigt auch Rechenleistung, diese skaliert linear mit der Bildgröße. Dieser Skalierungsaufwand ist bei einem HD oder Full HD Bild allerdings eher gering, im Vergleich zum erwarteten Rechenaufwand der restlichen Gestenerkennung. Besonders wenn diese auf neuronalen Netzen basiert. Wird jedoch ein rechenschwacher Microcomputer wie ein Raspberry Pi als Recheneinheit gewählt, ist unter Umständen auch geringer zusätzliche Aufwand nicht zu vernachlässigen, um eine „flüssige“ Echtzeitverarbeitung zu gewährleisten.

Der Vorteil einer höheren Bildauflösung hingegen ist, dass eine Erkennung in größerer Entfernung möglich sein. Bei kleineren Strukturen wie Fingern könnte das vorteilhaft sein. Für diesen Informationsgehalt ist, wie schon beschrieben nicht nur die reale räumliche Entfernung entscheidend, sondern auch die optisch erzeigte Entfernung durch das FOV der Kamera. Es lässt sich damit sagen das eine höhere Auflösung ebenfalls den räumlich möglichen Aktionsbereich des Videosystems vergrößern.

### 3D-Raum-Koordinaten mit MediaPipe world Landmarks

Auf der Basis der 3D-Koordinaten soll der Armwinkel für die Zeigerichtung berechnet werden. Hierfür werden zunächst die MediaPipe-World-Koordinaten in eigene Raum-Koordinaten umgerechnet. Dies dient hauptsächlich dazu die Kameraneigung der Schrägansicht rauszurechnen, so dass die Koordinatenachsen gerade zum Raum stehen. Die X-Koordinate bleibt hier dabei unverändert, y und z werden mit Winkelfunktionen gewichtet und neu aufaddiert. Es gilt:

**Formel 13 - Raum Koordinaten, Kameraneigung kompensieren**

$$x_{Raum} = x$$
$$y_{Raum} = y \cdot \cos(\alpha) + z \cdot \sin(\alpha)$$

$$z_{Raum} = z \cdot \cos(\alpha) - y \cdot sin(\alpha)$$
$$mit$$
$$x : Breite$$
$$y : Höhe$$
$$z : Tiefe$$
$$\alpha : Kameraneigung$$

Es werden vier, leicht unterschietliche Implementierungen getestet. Im den Testvideos wird ein Arm in 45° schritten von -90° (Zeigerichtung links) nach 90° (rechts) bewegt, das wird je einmal aus der Frontalansicht und aus der Schrägansicht gefilmt. Zur Berechnung der Zeigerichtung werden drei Punkte genommen, welche sich zu zwei Vektoren verbinden lassen, siehe **Fehler! Verweisquelle konnte nicht gefunden w erden.**. Die vier Implementierungen unterscheiden sich in der Wahl der drei Punkte:

- X-Blau: Handzentrum, Schultermitte, Schulter mit gespiegelter X-Koordinate
- Z-Blau: Handzentrum, Schultermitte, Schulter mit addierter Z-Komponente
- X-Grün: Handzentrum, Schulter, Schulter mit gespiegelter X-Koordinate
- Z-Grün: Handzentrum, Schulter, Schulter mit addierter Z-Komponente

Es werden nur die Raum Koordinaten x und z der Punkte bzw. Vektoren betrachtet, da an dieser Stelle lediglich der Azimut-Anteil der Zeigerichtung betrachtet wird. Der dritte Punkt (Raumreferenzpunkt) ist jeweils so gewählt, dass der Vektor zwischen diesem und dem Schulterpunkt bzw. Schultermittelpunkt (Winkelpunkt) nur eine Richtungskomponente hat – also sich nur in X- oder Z-Richtung erschreckt (Raumvektor). Ein zusätzlicher Vergleich der X-Koordinaten, von Hand- und Schulterpunkte, bestimmt das Vorzeichen des Winkels in diesen Testimplementierungen.

Die Ergebnisse der vier Implementationen werden in der folgenden Tabelle verglichen. Bilder aus dem Testvorgang sind unter der Tabelle zu sehen.

**Tabelle 1: MediaPipe World Koordinaten, ermittelte Zeigerichtungen**

| Kameraneigung | Winkelpunkt | Referenzachse | 90° links | 90° rechts | 45° links | 45° rechts | 0° links | 0° rechts | Max. Diff. | Mittl. Diff. |
|---|---|---|---|---|---|---|---|---|---|---|
| 0° | Schulter | Z Grün | 92 | 81 | 49 | 63 | 6 | 14 | 18 | 8,2 |
| 0° | Schulter | X Grün | 88 | 81 | 49 | 63 | −6 | −14 | 18 | 8,8 |
| 0° | Schultermitte | Z Blau | 81 | 75 | 50 | 64 | 5 | 3 | 19 | 9,3 |
| 0° | Schultermitte | X Blau | 90 | 84 | 59 | 73 | 15 | 12 | 28 | 12,5 |
| 32,5° | Schulter | Z Grün | 86 | 80 | 59 | 60 | 13 | 10 | 15 | 11 |
| 32,5° | Schulter | X Grün | 86 | 80 | 59 | 60 | −13 | −10 | 15 | 11 |
| 32,5° | Schultermitte | Z Blau | 91 | 91 | 76 | 73 | 29 | 20 | 31 | 18,3 |

| | | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|
| **32,5°** | **Schultermitte** | **X Blau** | **89** | **89** | **76** | **73** | **29** | **20** | **31** | **18,3** |

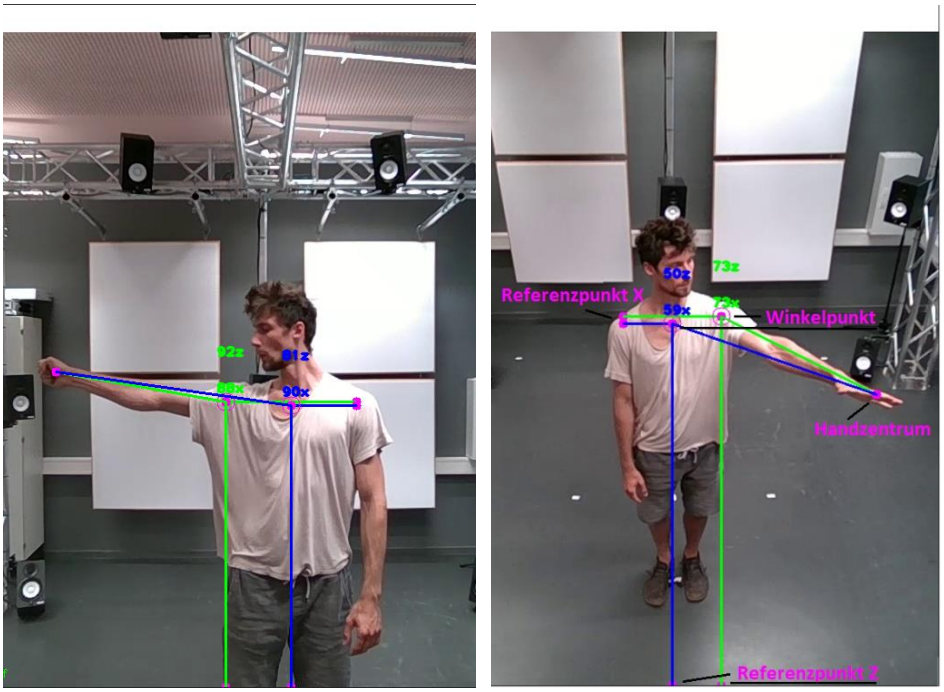



**Figure 50 - drei Punkte für die Zeigerichtung, vier Implementierungen, 0° und 32,5° Kameraneigung, (Werte im Bild sind fehlerhaft)**

Die Wahl der Referenzachse – also ob der Raumreferenzvektor nach vorne (z) oder zur Seite (x) weg geht, macht mathematisch nur einen Unterschied von genau 90°, welche in der Implementierung berücksichtigt werden, sie sollten daher in der Theorie die zu den gleichen Ergebnissen führen. In der Tabelle ist das nicht so. Die teils unterschiedlichen Ergebnissen der, mit X und Z bezeichneten Implementationen, entstehen durch Rundungsfehler bei kleinzahligen Zwischenergebnissen der Winkelberechnung. Bei der Erstellung des Raumreferenzpunktes durch Spiegelung der X-Koordinate des Winkelpunkt – also der Referenzpunkt gleicht dem Winkelpunkt bis auf das Vorzeichenwechsel seiner X-Komponente, können sich kleinere Werte für den Raumreferenzvektor ergeben. Die Aufaddierung einer X- oder Y-Komponente ist daher die Zuverlässigere Methode, um den Referenzpunkt und damit einen Raumreferenzvektor zu erstellen.

Die kleinste mittlere Abweichung von 8,2° liefert Z-Grün mit der Frontalansicht (0°). Die kleinste maximale Abweichung von 15° haben X- und Z-Grün mit der Schrägansicht (32,5°). Mit einer maximalen Abweichung von 15° lässt sich Schwellwert so platzieren, dass eine Winkelauflösung von > 30° möglich wäre. Die Datenmenge dieses Testes ist jedoch zu gering für präzise Aussagen. Eine Winkelzuordnung in 45° Schritten (wie beim Test) wird als: mit geringer Fehlerquote möglich, eingeschätzt. Diese Auflösung soll vorerst implementiert werden, um diese dann weiter zu testeten und ggf. zu verringern.

## Fehlgeschlagene Konzepte zur Erkennung der Zeigerichtung

### Inverse Perspective Mapping

Mit dem Inverse Prospective Mapping (IPM) lässt sich aus einem Bild welches schräg von oben aufgenommen wurde eine perspektive berechnen, als wäre das Bild gerade von oben Aufgenommen worden. Die perspektive Gerade von oben nennt sich auch Vogelperspektive oder in Englisch *Bird's Eye View*, unterdiesen Namen lässt sich diese Bildtransformation ebendfalls finden. [115]

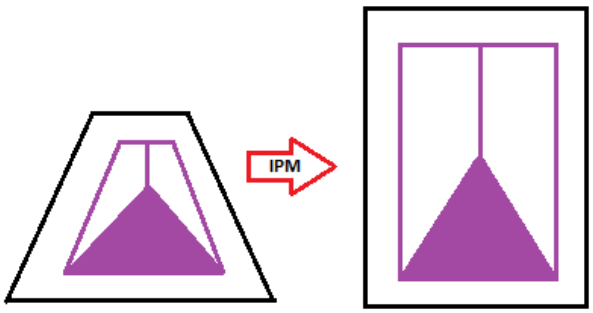



**Figure 51 - perspektivische Entzerrung durch Inverse Prospective Mapping**

Die Überlegung war es, dass Videobild aus der Schrägansicht perspektivisch zu transformieren, damit es wie von oben erscheint. Dann ließe sich die Zeigerichtung einfach abläsen, aber das geht nicht ohne weiteres.

Diese Transformation basiert auf der Annahme, dass sich alle betrachteten Bildpunkte auf einer gemeinsamen Ebene mit bekannter Höhe befinden, beispielsweise einer Fahrbahn oder einer ebenen Arbeitsfläche. [116]
Diese Voraussetzung ist bei der Bestimmung der Zeigerichtung des menschlichen Arms nicht erfüllt. Seine Höhe variiert in Abhängigkeit von der Körpergröße der Person sowie von der jeweiligen Armhaltung. Die Höhe der einzelnen Bildpunkte ist unbekannt. Das IPM projiziert daher alle Bildpunkte fälschlicherweise auf die angenommene Referenzebene, wodurch Punkte abseits dieser Ebene an einer falschen Position dargestellt werden.
Folglich kann die durch IPM erzeugte Bird's-Eye-View die tatsächliche horizontale Zeigerichtung des Arms nicht wiedergeben. Eine korrekte Transformation wäre nur möglich, wenn zusätzlich die dreidimensionale Position beziehungsweise die genaue Höhe des Arms bekannt wäre.

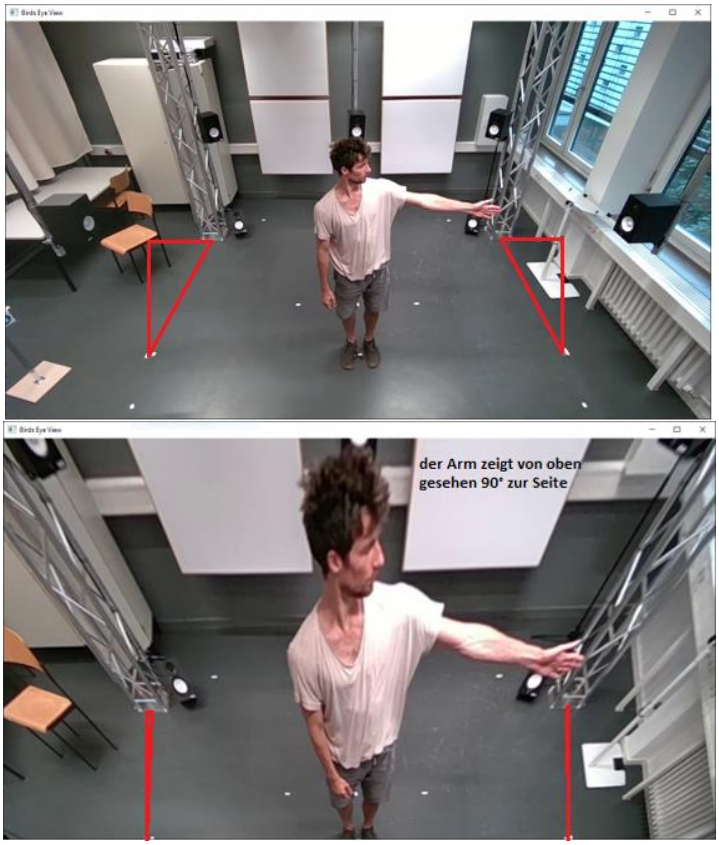



**Figure 52 - Inverse Prospective Mapping der Bodenebene**

Dieser Ansatz wir Hiermit als ungeeignet eingeschätzt und nicht länger verfolgt.

### Der Zeitzonenansatz

Die horizontale Zeigerichtung (Azimut) einer Person im Raum lässt sich durch den Steigungswinkel einer geraden zwischen Schulter und Hand exakt bestimmen, wenn die Person senkrecht von oben zu sehen ist. Siehe Figure 53.

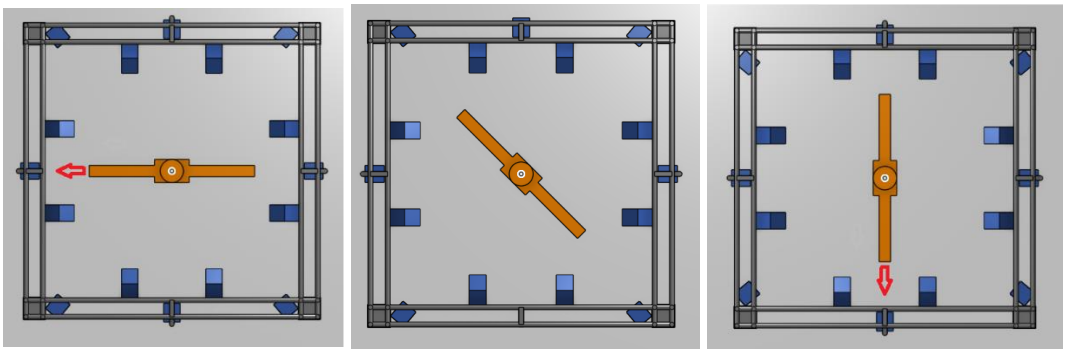


**Figure 53 - horizontale Zeigerichtung von oben**

Soll die Person von vorne betrachtet werden, funktioniert diese Azimut-Ermittlung nicht so direkt. Siehe Figure 54.

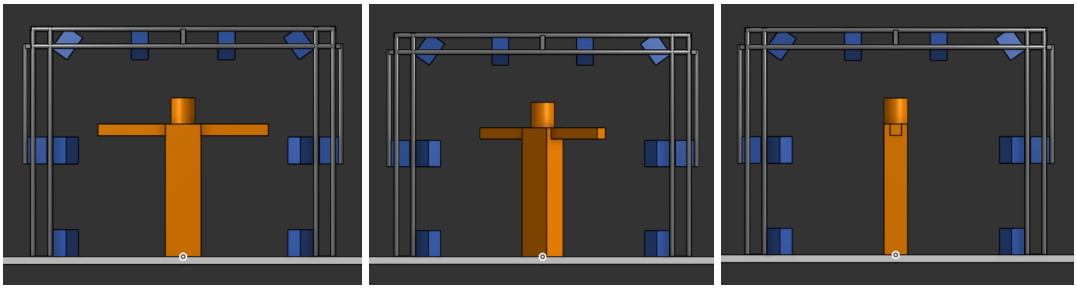


**Figure 54 - horizontale Zeigerichtung von vorne**

Aus der Betrachtung der Zeitzonen auf einem Globus ergibt sich ein Ansatz zur Ermittlung der horizontalen Zeigerichtung auch aus einer Frontalansicht.

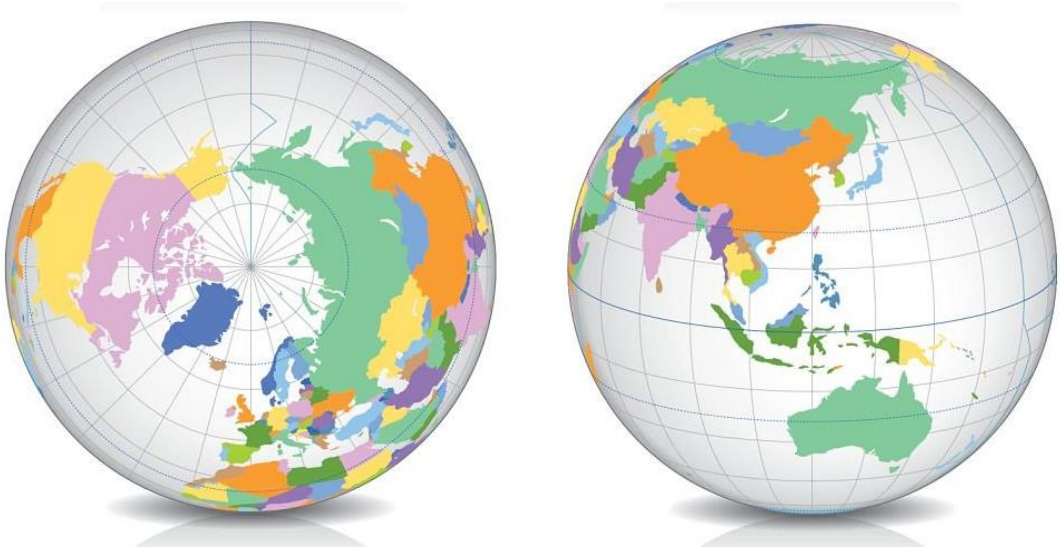


**Figure 55 – Zeitzonen von oben (Nordpol) und vorne (Äquator) [117]**

Jede Zeitzone ist eine Kreisbahn welche 15° rotiert zur nächsten Kreisbahn auf einer Kugeloberfläche angeordnet ist. Vom Nordpol betrachtet lässt sich diese 15° Verschiebung einfach zwischen den hier optisch geraden Linien der Zeitzonen messen. Siehe das linke Bild in Figure 13. Diese Betrachtung dient als Äquivalent zur Zeigerichtung in Figure 53. Von Äquator aus gesehen sich die Zeitzonen gekrümmte Linien und die 15° Verschiebung lassen sich nicht direkt messen, rechtes Biln in Figure 55. Es dient hier als Äquivalent zu Figure 54.
Unter der Annahme das die Armlänge des Dirigenten beim Zeigen konstant ist, lässt sich hier eine Analogie feststellen. Die Armlänge entspricht dem Erdradios und die Verschiebung zwischen den Zeitzonen der Zeigerichtung.

Die Verschiebung der Zeitzonen lässt sich auch aus der Frontansicht (Äquatoransicht) mathematisch ermitteln. Hierzu brauch es zwei Zusammenhänge. Zum Ersten, die Funktion hinter einer Kreisbahn, genauer gesagt einer halben Kreisbahn bildlich von oben nach unten. Diese lautet:

**Formel 14 - Kreisbahn (Halbkreis)**

$$x = \sqrt{r^2 - y^2}$$
$$mit$$
$$-r \leq y \leq r$$

Eine Kugel wird aus einer Perspektive 90° zu einer Drehachse betrachtet, wie es bei der Frontalansicht der Fall ist. Die optische Stauchung einer Strecke zwischen einem Punkt auf einer Kugeloberfläche und der Drehachse, ergibt sich dann über die Sinus- oder Cosinus-Funktion, je nach Wahl der Null-Grad-Ausrichtung des Drehwinkels. Damit ist der zweite Zusammenhang, welcher benötigt wird, folgender:

**Formel 15 – optische Distanz eines Punktes auf einer Kugeloberflächen 90° zur Drehachse**

$$d_o = \sin(\omega) \cdot d_r$$
$$mit$$
$$d_r : distanz\ zur\ Dreachse$$
$$d_o : optische\ Distanz\ zur\ Drehachse$$
$$\omega : Drehwinkel$$
$$\omega = 0° : zum\ Betrachter\ gedreht$$

$\omega = 0°$ ist dort wo die Kreisbahn der Zeitzone optisch wie ein senkrechte Gerade aussieht. Es ergibt sich folgende Funktion für die Bahnen der Zeitzonen eines von vorne betrachtetem Globus:

**Formel 16 - Kreisbahn aus einer Perspektive 90° zur Drehachse**

$$\boldsymbol{x = \sin(\omega) \cdot \sqrt{r^2 - y^2}}$$
$$mit$$
$$x : bildliche\ Breite$$
$$y : bildliche\ Höhe$$
$$r : Radius$$
$$-r \leq x \leq r$$
$$-90° \leq \omega \leq 90°$$

Ist der Kugel- bzw. Kreismittelpunkt und der Radius bekannt, dann lässt sich mit der Formel 16 nach $\omega$ umgestellt, bestimmen auf welcher Kreisbahn ein Koordinatenpunkt liegt. Der Verschiebungs- bzw. Drehwinkel $\omega$ der Kreisbahn, entspricht dann der Zeigerichtung. Somit ließe sich eine Zeigerichtung im vorderen, horizontalen Halbkreis bestimmen, also im Bereich $-90° \leq \omega \leq 90°$. Dieser Azimut Winkel wäre damit auch unabhängig von der Höhe des Arms – also von der Elevation. Das folgende Bild zeigt die Implementierung dieses Zeitzonenansatzes.

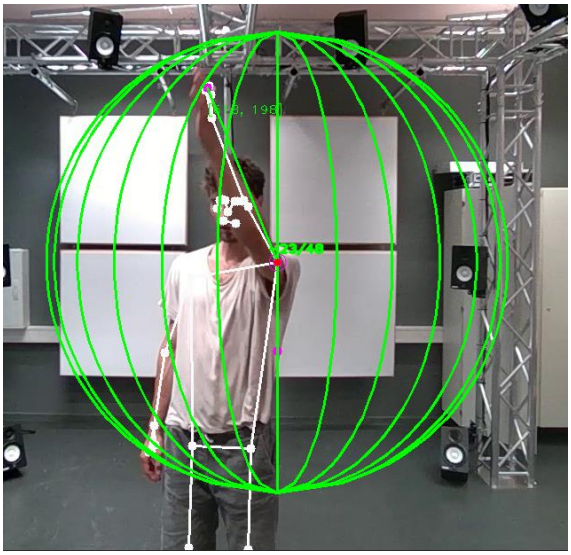


**Figure 56 – Zeitzonenansatz zur Ermittlung der horizontalen Zeigerichtung**

In der Praxis ist dieser Zusammenhang zwischen $\omega$ und bildlichen Position nicht eins zu eins nutzbar. Denn die Betrachtung durch einer Kameralinse sorg für zusätzlich Änderungen der Armlänge. Ist der Arm bzw. die Hand dichter an der Kamera, so ist diese Optisch größer. Das Sorgt für folgenden Effekt:

> Die optische Armlänge bei 70° Zeigerichtung ist länger als bei 90°, bei 60° ist sie fast gleich und bei 45° ist diese dann wieder kürzer. Abgesehen von der zusätzlichen optischen Verzerrung, sorgt dieser Effekt für Zweideutigkeiten im Bereich $-90° \leq \omega \leq 90°$. Zweideutig ist hier der Zusammenhang zwischen der optischen Armlänge und der Zeigerichtung. Figure 57 zeigt Das beschriebene Problem.




**Figure 57 - Zeitzonenansatz, Problem mit der Kameralinsenverzerrung**

Ein weiteres Problem bei diesem Ansatz ist das die Armlänge bekannt seien muss, genauer gesagt die optische Armlänge an und in einer bestimmten Position im Raum. Das System müsste demnach eine Kalibrierung durchführen, wenn ein neuer Dirigent den Systembereich betritt. Und ebenfalls jedes Mal, wenn dieser seine Position im Raum ändert. Das wäre mit zusätzlichen und unintuitiven Verhalten des Dirigenten verbunden. Für eine Kalibrierung könnte der Dirigent beispielsweise seine Arme für zwei Sekunden gerade zu den Seiten ausstrecken und damit eine bildliche Messung initialisieren.

Dieser Ansatz wir Hiermit als ungeeignet eingeschätzt und nicht länger verfolgt.

**ROI-Test**

Die Hand Erkennung vom MediaPipe Hand, findet je nach Video nur in 30-70% der Frames eine Hand wenn das gesamte Bild verarbeitet wird (siehe in Figure 58 die 0-Position von „HOI Size“). Im manschen Frames wurde ein beispielsweise Teil des Arms als Hand erkannt, obwohl die Pose-Erkennung das ROI auf die richtige Hand zentriert hat. Solche Falscherkennungen lassen sich einfach reduzieren. Zum Beispiel in dem ein kleineres ROI gewählt wird. Ein Test mit dem eigenem Videomaterial wird durchgeführt, um die Auswirkungen von der ROI-Größe auf die Handerkennungsrate zu untersuchen. ROI-Testergebnisse sind in Figure 58, Figure 59 und Figure 60 zu sehen. Die „ROI size“ von 0 Pixels, bedeutet das kein ROI verwendet und das gesamte Bild analysiert wurde.

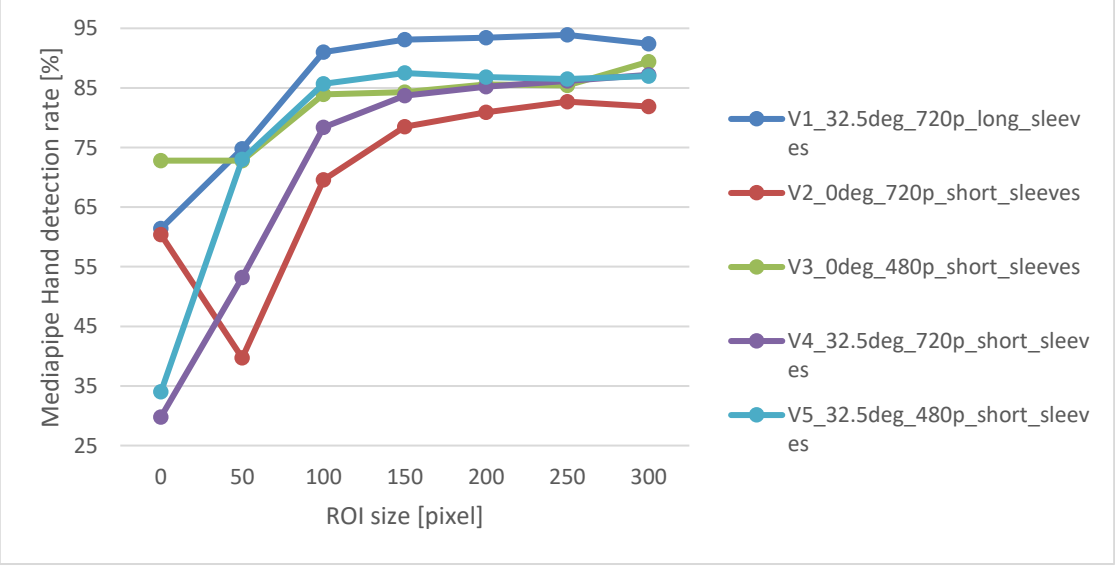



**Figure 58 - Hand Erkennung mit verschieden großen Bildausschnitten**

Der Test ergibt, dass ein kleineres ROI zu weniger Frames mit erkannter Hand führt. Das hat folgende Gründe:

- Die Hand ist zu groß im verarbeiteten Bildausschnitt – nahm die Hand das gesamte Bild ein sank die Erkennungsrate im Test.

- Die Hand wird vom ROI abgeschnitten – Die Hand verlässt teilweise das ROI, wenn dieses sehr klein ist. Das liegt an sich ändernden Handgrößen durch unterschietliche Distanzen zur Kamera und einer wackelnde Handposition aus MediaPipe Pose.

- Falscherkennung – in Frames, in welchen die Hand von MediaPipe Hands nicht erkannt wurde, wird etwas anderes im ROI als Hand erkannt. Ist das ROI größer gibt es mehr Strukturen welche potenziell als Hand erkannt werden können.

Wird nur der erste Grund betrachtet, so funktioniert die MediaPipe Hand-Erkennung augenscheinlich am besten, wenn das verarbeitete Bild etwa einen Durchmesser vom zwei- bis fünffachen der Hand hat. Bestätigend hierzu zeigt der ROI-Test einen Knickpunkt in den Kurven zwischen 100 und 150 Pixel. Das ist in etwa dann, wenn das ROI durchschnittlich den doppelten Durchmesser der Hand aufweist. Damit sich das ROI der Größe der zu erkennende Hand etwas anpassen kann, wird die erkannte Person als Maßstab verwendet.

Durch diese Maßnahmen steigern sich die Frames mit erfolgreicher Hand Erkennung in etwa von 30-70% auf 80-90%. Bzw. auf 94% in dem Video, in welchem ein langärmliges Oberteil getragen wurde.

Interessant zu bemerken war, dass sich die Performance dieses Codes zur Landmark-Erkennung nicht geändert hat, als das ROI eingeführt wurde. Die Handerkennung bekam dabei zeitweise eingangs Bilder welche 20-mal weniger Pixel beinhalteten als das Gesamtbild. Die Performance lag im Mittel je nach Testvideo zwischen ca. 12,5 und 14,5 FPS. Lediglich das Format des gesamten Videos beeinflusste die Performance. Dabei hatte HD (1280x720p@16:9) durchschnittlich ca. ein FPS mehr als VGA (640x480p@4:3). Dass die Verarbeitung von mehr Pixeln genauso viel oder sogar weniger Zeit kostet ist nicht intuitiv. Die Gründe liege wahrscheinlich in der Art und Weise wie MediaPipe die Frames vorverarbeitet und auswertet. Im Kapitel MediaPipe Hands wird beispielsweise von einer Palm-Detektion gesprochen, welcher selbst ein ROI für die Handerkennung erstellt.

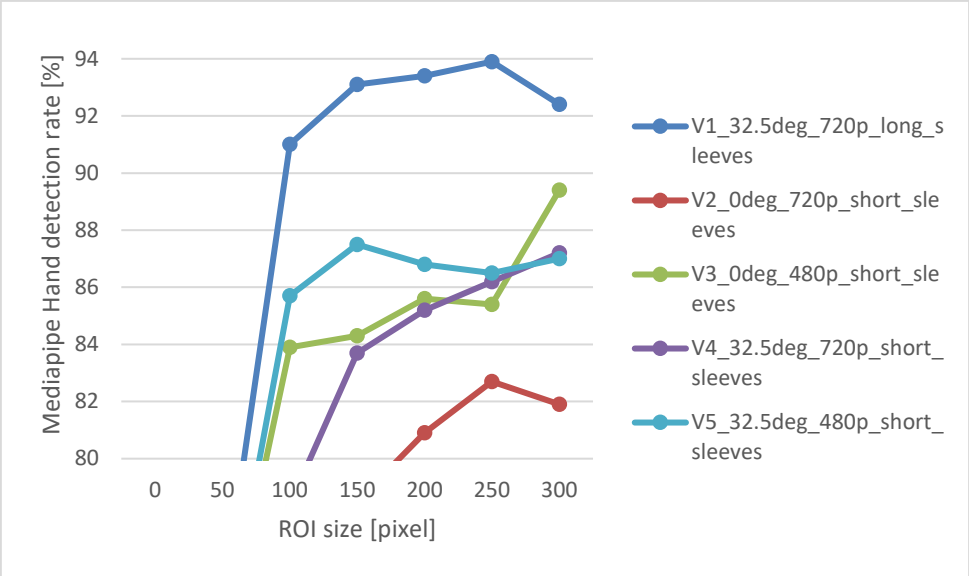



**Figure 59 - Hand Erkennung mit verschieden großen Bildausschnitten (gezoomt)**

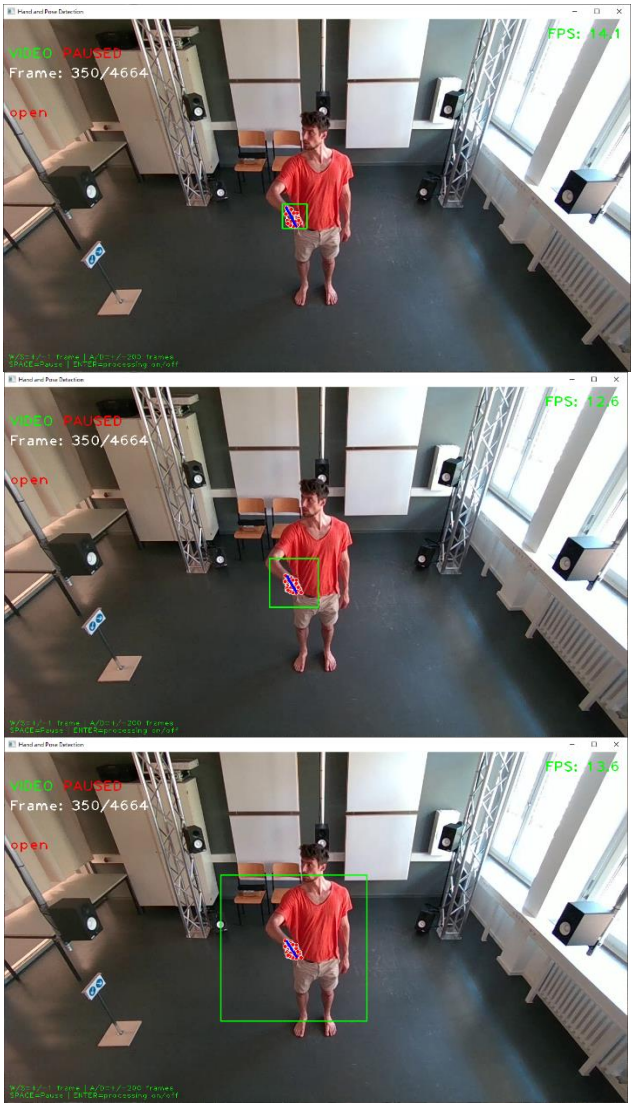



**Figure 60 - Hand Erkennung mit verschieden großen Bildausschnitten (V4 mit 50, 100, 300 Pixel)**

**Greifgestenerkennungen im Test**

Die Greifgestenerkennung wird mit fünf Videos getestet, zwei im VGA-Format und drei im HD-Format von je zwei bis drei Minuten Länge. Hierbei ist anzumerken, dass die Videos entstanden, bevor die Funktionsweise der Gestenerkennung entwickelt wurde. Für den Test wurden zunächst die Frames händisch markiert, in welchen eine Hand eine Greifgeste ausführt. Diese Frames bekamen je einen Status zugewiesen: *offen* oder *geschlossen*, je nach dem in welchem Zustand sich die Hand während der Greifgeste befand. Frames, in welchen keine gewollte Greifgeste ausgeführt wird, wurden nicht ausgewertet. Ausgewertet wurde nur die *True-Positiv-Rate*. Sie besagt der vorgegebene Handstatus *offen* oder *geschlossen* wurde richtig erkannt. *True-Positiv-Wert* soll hier stellvertretend für die Wahrscheinlichkeit steht eine Greifgeste zu erkennen.

Die in jedem der fünf Videos wurden die zwei Handstatus-Erkennungsmethoden in je drei Variationen getestet, die Varianten unterscheiden sich in dem Parameter, von dem sich der größte gewinn in der Feinjustierung versprochen wurde:

- V1: Aperture-Methode mit den Schwellwerten 50 für *geschlossen* und 70 für *offen*
- V2: Aperture-Methode mit den Schwellwerten 55 für *geschlossen* und 70 für *offen*
- V3: Aperture-Methode mit den Schwellwerten 60 für *geschlossen* und 70 für *offen*

- V4: Difference-Methode mit den Differenzfaktor 0.8 für einen Statuswechsel
- V5: Difference-Methode mit den Differenzfaktor 1.0 für einen Statuswechsel
- V6: Difference-Methode mit den Differenzfaktor 1.2 für einen Statuswechsel

Je Video und Methoden-Variation wurde ein *True-Positiv-Wert* erstellt. Die spanne beträgt hierbei 81,7% bis 89,2%. Am besten abgeschnitten hat V3. In vier von fünf Videos hatte V3 die höchste Rate. Die über die Videos gemittelt ergab sich hier ein *True-Positiv-Wert* von 87,0%.

Es wurden zudem Unterschiede zwischen den Videoformaten HD und VGA erkannt. In den drei HD-Videos lag der *True-Positiv-Wert* im Schnitt ca. 4% höher.

Durch das Weglassen der eigenen, zusätzlichen Positionsglättung der MediaPipe Landmarkes wurde die durchschnittlicher *True-Positiv-Wert* um 0,9% erhöhe, mit einem Maximalwert 92,4% bei V3 in einem HD-Video.

Dieser Testergebnisse dient nur als Richtwerte zur groben Bewertung der Methoden zu verstehen. Sie haben nur bedingt eine Aussage darüber, inwiefern eine Greifgeste wehrend der tatsächlichen Gestensteuerung erkannt wird. Steuert ein Nutzer das System mit seinen Gesten so erhält dieser währenddessen optisches Feedback vom Videosystem dieses wird seine Gesteninteraktion beeinflussen. Entscheidend ist am Schluss, wie zuverlässig sich das Anwenden einer Greifgeste für die Nutzer anfühlt.

**Ausführliche Testfortsetzung von Kameraposition und FOV**

Dieser Abschnitt ist ein Anschluss an Abschnitt 4.1 - *Raumsimulation, Kameraposition*.

Durch die Bereichsaustritte vom Armbereich aus dem Sichtfeld, wird im Folgenden festgestellt, welche Parameter notwendig sind, um Armgesten bildlich voll einzufangen. Zudem werden zwei Kamera Ausrichtungen getestet, hier 45° Ausrichtung und 90° Ausrichtung genannt. Figure 61 zeigt Armbereich und Kamera Sichtfeld in diesen beiden Ausrichtungen, linkes Bild 45° und rechtes Bild 90°.

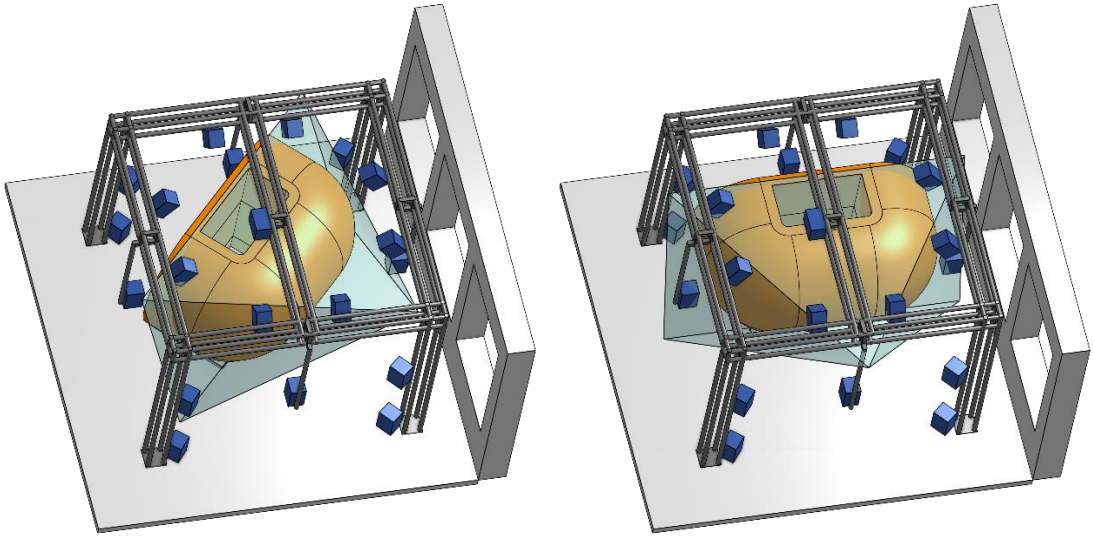


**Figure 61 -Modell Audiosystem 45° und 90° Ausrichtung**

Die Wahl der Standzone fiel vorerst auf 160 x 100 cm. Der Armbereich eines 207 cm Großen Menschen passt mit diesen Maßen gut in beide Ausrichtungen und hat dabei noch einen Freiraum von ca. 10 cm zu den Gerüstobjekten. Figure 62 zeigt die türkise Standzone von oben in beiden Ausrichtungen. Teilweise ist diese verdeckt vom orangenen Armbereich. Die rot umrahmten Bereiche sind Stellen, mit denen eine Kollision am ehesten möglich wäre, diese begrenzen den Armbereich und folglich die Standzone.

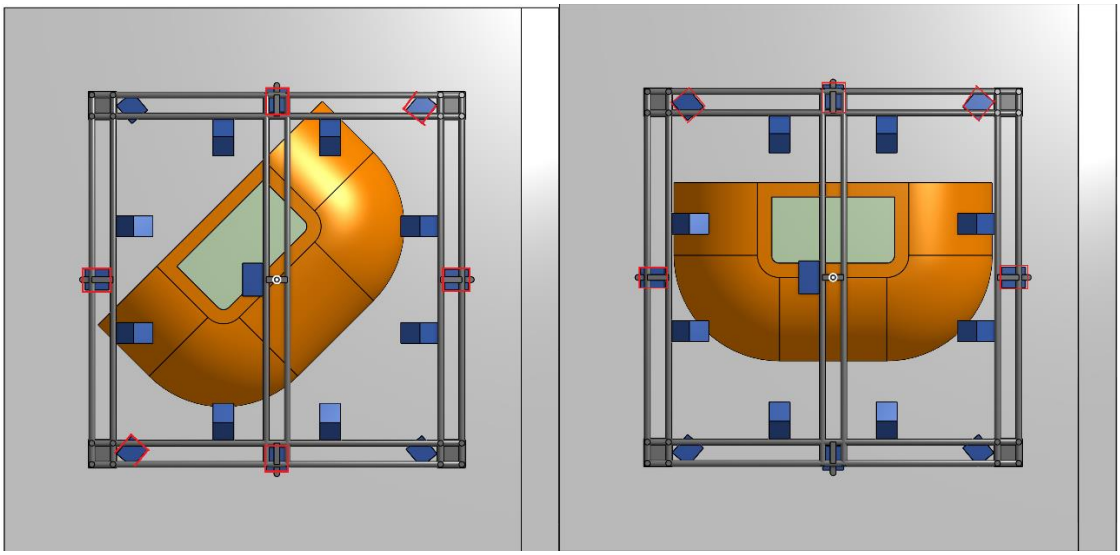


**Figure 62 - Modell Audiosystem Kollisionsbereiche der Ausrichtungen**

Innerhalb der zwei Ausrichtungen werden je fünf verschiedene Kamerapositionen simulativ getestet. Bezugspunkt der Positionierung ist die jeweilige obere äußere Gerüstecke. Von da aus werden zwei 50 cm Schritte gemacht, je einmal nach unten und einmal nach innen. Die folgenden zwei Bilder zeigen die getesteten Positionierungen beider Kameraausrichtungen.

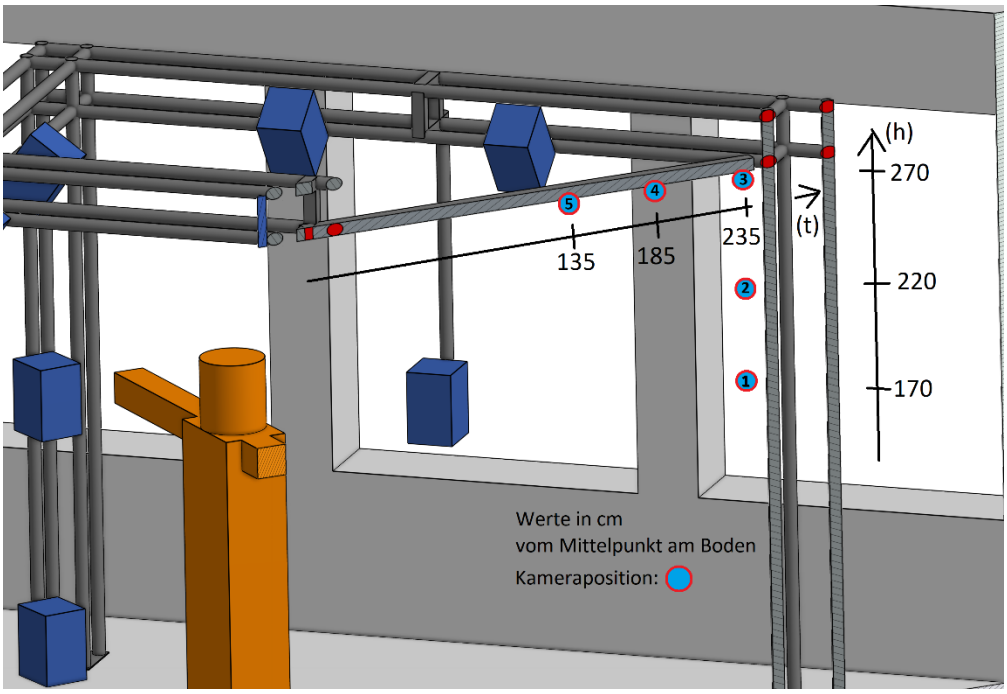



**Figure 63 - Modell Audiosystem Schnittansicht der 5 Kamerapositionen in Ausrichtung 45°**

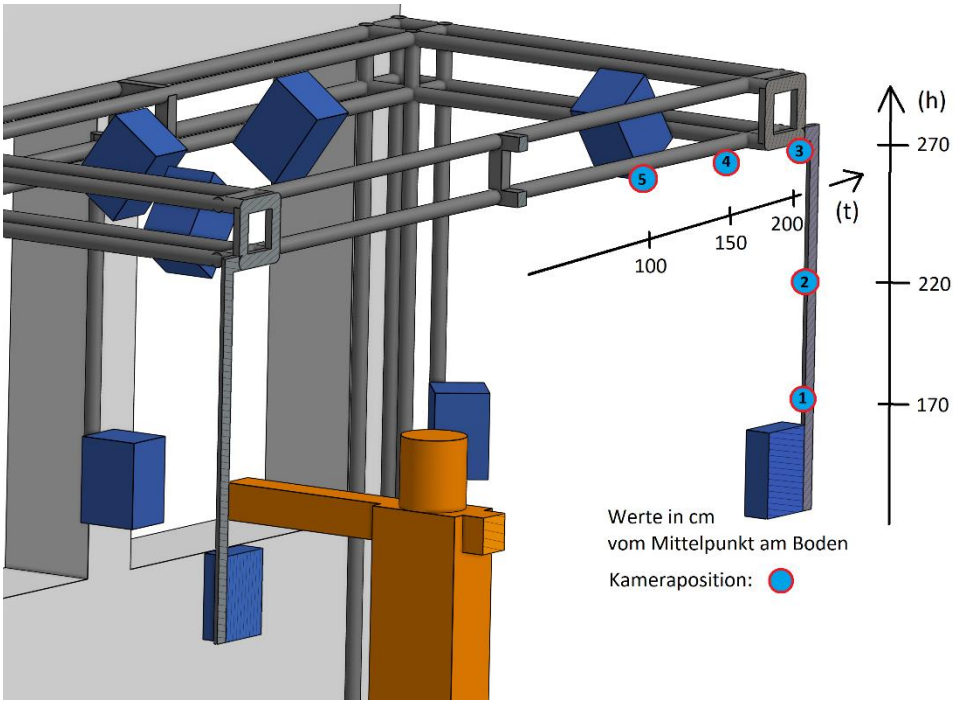



**Figure 64 - Modell Audiosystem Schnittansicht der 5 Kamerapositionen in Ausrichtung 90°**

Für die simulativen Tests wurden die folgenden fünf Parameter variiert:

- h: Höhe der Kameraposition in cm, 0 ist die Bodenbender
- t: Tiefe der Kameraposition in cm, 0 ist die Gerüstmitte
- n: Neigungswinkel der Kamera, 0° Horizontalen, 90° Blick zum Boden
- hF: horizontaler Kamera Öffnungswinkel – hFOV
- vF: vertikaler Kamera Öffnungswinkel – vFOV

Das Testziel ist es hF und vF immer so klein wie möglich zu halten, ohne dass der Armbereich die Sichtgrenzen verlässt. Die Folgenden Tabellen zeigen die Simulationsresultate:

**Tabelle 2 - FOV bei verschiedenen Kamerapositionen, 45° Ausrichtung, Standzone 160 x 100 cm**

| Position | h [cm] | t [cm] | n [deg] | hF [deg] | vF [deg] |
|---|---|---|---|---|---|
| P1 | 170 | 235 | 2 | 75 | 51 |
| P2 | 220 | 235 | 13 | 74 | 49 |
| P3 | 270 | 235 | 23 | 70 | 45 |
| P4 | 270 | 185 | 28 | 84 | 53 |
| P5 | 270 | 135 | 35 | 103 | 67 |
| Änderung der Standzonen Breite, Standzone ist hier 250 x 100 cm | | | | | |
| P3 | 270 | 235 | 23 | 85 | 45 |

**Tabelle 3 - FOV bei verschiedenen Kamerapositionen, 90° Ausrichtung, Standzone 160 x 100 cm**

| Position | h [cm] | t [cm] | n [deg] | hF [deg] | vF [deg] |
|---|---|---|---|---|---|
| P1 | 170 | 200 | 2 | 92 | 60 |
| P2 | 220 | 200 | 15 | 90 | 56 |
| P3 | 270 | 200 | 27 | 84 | 51 |
| P4 | 270 | 150 | 32 | 102 | 61 |

| | | | | | |
|---|---|---|---|---|---|
| P5 | 270 | 100 | 45 | 127 | 82 |

Die Positionen P4 und P5 werden, aufgrund ihrer steilen Näge zum Dirigenten und des dem entsprechent großen FOVs, als ungeeignet eingestuft. Für die relevanten Positionen, das sind P1 bis P3, ist ein Sichtfeld von 92° x 60° ausreicht. Dieses Sichtfeld wird im Folgenden als *hinreichendes FOV* bezeichnet. Bei der Wahl einer Kamera mit kleinerem Sichtfeld, sollten die wegfallenden Positionierungsmöglichkeiten berücksichtigt werden. Alternative könnte ließe sich die Standzone verkleinern (beeinflusst besonders das hFOV) oder die berücksichtigte Personengröße reduzieren (beeinflusst besonders das vFOV).

Für eine bessere Verhleichbarkeit mit den Verschiedenen Herstellerangeben wird für das hinreichende FOV noch das dFOV berechnet. Die gänigen Seitenverhältnisse 16:9 werden gierbei angesete. Formel 8 ergibt, dass das hinreichende FOV bereits fast exakt diese Seitenverhältnisse hat. Das ist zufall. Mit Formel 7 ergibt sich dann ein dFOV von $99{,}8° \approx 100°$.

Ein dFOV von 100° fällt bereits unter dem Begriff Ultraweitwinkel. An den Rändern des Bilds ist eine deutliche Bildverzerrung zu erwarten, der sogenannte Fischaugeneffekt [118]. Kernaufgabe der Gestenerkennung soll es sein den Armwinkel zu interpretieren, eine starke Bildverzerrung würde die wahrgenommene Armlänge unterschiedlich stark beeinflussen, je nach Verzerrungsgerad der Bildbereiche, über welchen sich der Arm erstreckt. Eine solche Verzerrung müsste mathematisch mithilfe der Intrinsischen Parameter der Kamera korrigiert werden, um reale Winkel in verschiedenen Bildbereichen auszulesen.

*Wahl der Kameraausrichtung*

Es wurde entschieden das die 90° Ausrichtung (siehe Figure 64) verwendet werden soll, die Gründe sind:

- ein gleichmäßigerer Kamerabildhintergrund im Raum
- und vor allem, die angenehmere Raumsymmetrie beim Interagieren mit dem System.

In der 90°-Ausrichtung ist zum jetzigen Stand der Arbeit die Position P3 bevorzugt, da sie perspektivisch gute Informationen für die Berechnung des Azimut-Armwinkel enthält und die weiteste Distanz zum Dirigenten hat. Das ist in dem Maß vorteilhaft, weil das benötigte FOV und damit auch die Bildverzerrung abnimmt.
Position 1 wäre dann passender, wenn eine Tiefenkamera zum Einsatz kommen soll. Hierbei wäre die perspektivische Tiefeninformation im RGB-Bild nicht relevant, da diese aus dem Tiefenbild genommen werden kann. Eine Horizontale Kameraneigung wäre geeignet, um die Positions- und Winkelberechnung einfach zu halten.
Wird eine Interaktion mit dem Videosystem mithilfe von visuellem Feedback über einen Bildschirm integriert, hätte die waagerechte Kameraposition P1 folgenden Vorteil. Angenommen eine Person sieht sich selbst auf einem Bildschirm und soll ihre Hände zu bestimmten Bildbereichen bewegen, um mit einem GUI zu interagieren. Dann wäre diese Steuerung am intuitivsten, wenn die Perspektive im Bildschirm, der eines Spiegels

gleicht. Mit anderen Worten, Kamera und Bildschirm sollten möglichst die gleiche Position zum Dirigenten haben, damit dieser Intuitiv mit seinen Gesten die GUI bedienen kann. Bei der Position P1 könnte der Dirigent bequem nachvorne auf einen Bildschirm auf etwa Schulterhöhe sauen.
Das hinreichende FOV beschreibt genau das FOV welches, unter den in diesem Kapitel beschriebenen Bedinnungen, für P1 aus der 90°-Ausrichtung (hier Frontalperspektive genannt) nötig wäre. P3 hat etwas geringere Anforderungen an das minimale FOV.
Das hinreichende FOV beschreibt keine harte Grenze, sondern ist mehr als Richtwert zu verstehen.

**GUI Bildquellen**



https://www.flaticon.com/free-icon/violin_199396



https://www.magnific.com/de/icons/piano



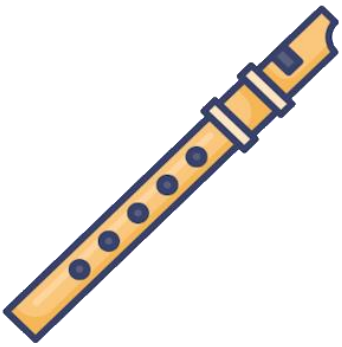
https://www.magnific.com/de/icon/trompete_2690140#fromView=search&page=1&position=4&uuid=582791b4-36fa-4996-8f84-f0dc9d3a2ae2



https://www.magnific.com/de/icon/klassische-gitarre_4488318#fromView=search&page=2&position=53&uuid=1540a2ed-f0ee-4d5f-89d7-699145aa06fc



https://www.magnific.com/de/icon/elektrische-gitarre_18694723#fromView=search&page=1&position=88&uuid=cc79cedd-620c-4e4e-ae4c-a07f283184cb



https://www.magnific.com/de/icon/floete_3816649#fromView=search&page=1&position=29&uuid=416a17bd-bd3d-4b55-9173-a7ddfb73ceb8



https://www.magnific.com/de/icon/musik_11983530#fromView=search&page=1&position=2&uuid=3b8cdb48-d287-4201-8edb-6f202b3446ee



https://www.magnific.com/de/icon/shehnai_15371634



https://www.magnific.com/de/icon/fagott_1913278#fromView=search&page=1&position=18&uuid=e1e8272b-6557-4010-be96-e0f84d6503b8



https://www.magnific.com/de/icon/trompete_2318260#fromView=search&page=1&position=13&uuid=0e4dfb07-d4c0-47f9-9e0b-a3fd959b6a01



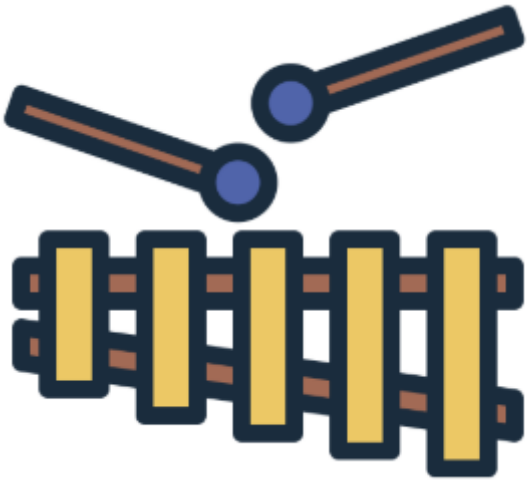
https://www.magnific.com/de/icon/tuba_1118006#fromView=search&page=1&position=26&uuid=e7658a4f-f27e-493a-87b8-d691c60f119a



https://www.magnific.com/de/icon/trommel_5201576#fromView=search&page=1&position=33&uuid=e7426326-0e32-47d4-bf3a-021ea4006141



https://www.magnific.com/de/icon/dreieck_199414#fromView=search&page=1&position=18&uuid=49890dfd-f028-4d09-a1b8-ddf9c8cd126f



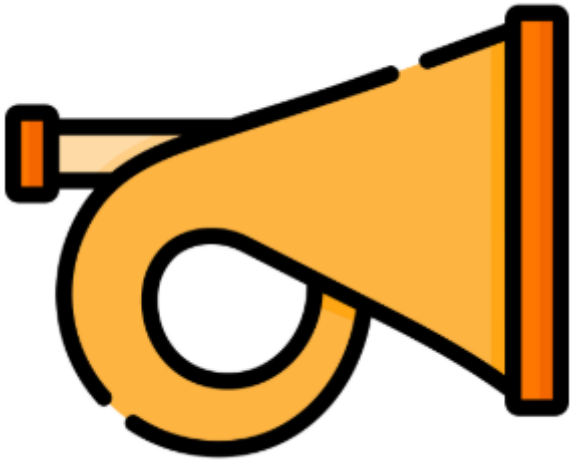
https://www.magnific.com/de/icon/xylophon_8939192#fromView=search&page=1&position=26&uuid=c407dbb9-5508-4c55-b0fa-c8a8e97081ae



https://www.magnific.com/de/icon/harfe_9810449#fromView=search&page=1&position=16&uuid=bba449ed-6ec0-459e-9f5e-4a1cee3b4ea2



https://www.magnific.com/de/icon/floete_934105#fromView=search&page=1&position=85&uuid=a31e8da0-181f-4175-ba36-a70dfe72d828



https://www.magnific.com/de/icon/tuba_2591476#fromView=search&page=1&position=72&uuid=13f856b4-4a8c-4e71-a9cd-56807a814508



https://www.magnific.com/de/icon/instrument_15484757#fromView=search&page=1&position=3&uuid=3c95bc42-9eeb-4906-9ebf-09827c224fce



https://www.pngmart.com/image/170255




https://hdwrglobal.com/de/blog/22580/beschreibung-der-grundtasten-und-nuetzlichen-tastenkombinationen-auf-einer-computertastatur/

**Weitere Bildquellen**



https://www.magnific.com/de/vektoren-premium/musiknoten-mit-kurven-und-wirbeln-schlagzeug-treble-schluesselnoten-musiker-flacher-abstrakter-cartoon-vektor_138914654.htm#fromView=keyword&page=1&position=22&uuid=fd03f063-ae97-469c-ba39-ad215bc78971&track=ais_hybrid&query=Noten+png

**Testböden der Probanden**

1-2

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener 2 Personen

Ort: BHT-Berlin, Raum B217a | Datum: 11.09.2026 | Proband: Vielfältig | Alter: 29 | Größe: 180 cm

Unterarme sind durch Kleidung bedeckt: [ja] / [nein] | Sonstige optische Merkmale des Probanden:

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☑ ☑ | ☑ ☑ |
| Lautstärke ändern | ☐ ☑ | ☐ ☑ |
| Informations-Fenster erhalten | ☑ ☑ | ☑ ☑ |
| Prozess Live-Infos ein und aus blenden | ☑ ☑ | ☑ ☑ aus Versehen ausgelöst |
| alle Instrumente zurücksetzen | ☑ ☑ | ☑ ☑ |
| das Programm beenden | ☑ ☑ | ☑ ☑ |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?

Die Gesten-Steuerung hat soweit gut funktioniert, nur hat es teilweise etwas gedauert bis die „aktive" Hand erkannt wurde. Wie man die Handfläche verstellt hatte ich leider nicht herausgefunden.

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| unit | Azimut soll | Azimut erkannt | | Elevation soll | Elevation erkannt | |
|---|---|---|---|---|---|---|
| 1 | 0° | -15 | ~~-75~~ -75 | 0° | 15 | ~~30~~ 15 |
| 2 | 15° | 30 | 0 | 0° | 30 | 0 |
| 3 | -45° | -60 | -60 | 45° | ~~45~~ 45° | 45° |
| 4 | 90° | 105 | 105 | -45° geht nicht | | |
| 5 | Egal | | | 90° | 60 | 25 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschietlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | | |
| Statisch 5 von 6 | | |
| Statisch 9 von 10 | | |
| Dynamisch 1s | | |
| Dynamisch 0.5s | | |

Sonstige Anmerkungen oder Feedback: _____

1-2 ~~Prozess in Gas aus~~

greifgeste und Infoangest werden gleichzeitig ausgelöst

info-geste geht nun mit der Aktiven Hand

2 Hand zur Bedienung zu instigrieren ist Gest für Volume

Die Handerkennung war manchmal verzögert

Hand zum Kamera wird manchmal nicht beim öffnen erkannt

3

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 17.09.2026 | Proband: Sissi | Alter: 69 | Größe: 171 cm

Unterarme sind durch Kleidung bedeckt: (ja) /nein | Sonstige optische Merkmale des Probanden: dunkle Sachen

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☑ | ☑ |
| Lautstärke ändern | ☐ | ☐ |
| Informations-Fenster erhalten | ☐ | ☐ |
| Prozess Live-Infos ein und aus blenden | ☐ | ☐ |
| alle Instrumente zurücksetzen | ☐ | ☐ |
| das Programm beenden | ☐ | ☐ |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

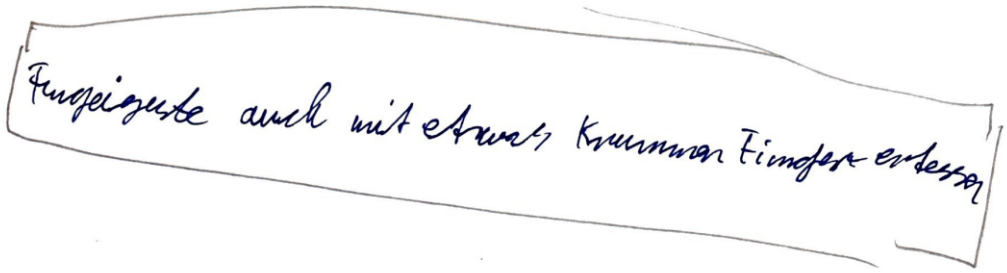
| unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | | 0° | |
| 2 | 15° | | 0° | |
| 3 | -45° | | 45° | |
| 4 | 90° | | -45° | |
| 5 | Egal | | 90° | |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschietlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | | |
| Statisch 5 von 6 | | |
| Statisch 9 von 10 | | |
| Dynamisch 1s | | |
| Dynamisch 0.5s | | |

Sonstige Anmerkungen oder Feedback:

3



4

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 11.09.2026 | Proband: Kenny | Alter: 40 | Größe: 171 cm

Unterarme sind durch Kleidung bedeckt: (ja)/nein | Sonstige optische Merkmale des Probanden: Schwarze Haut

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☐ | ☐ |
| Informations-Fenster erhalten | ☐ | ☐ |
| Prozess Live-Infos ein und aus blenden | ☐ | ☐ |
| alle Instrumente zurücksetzen | ☐ | ☐ |
| das Programm beenden | ☐ | ☐ |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?

## Konkrete Tests

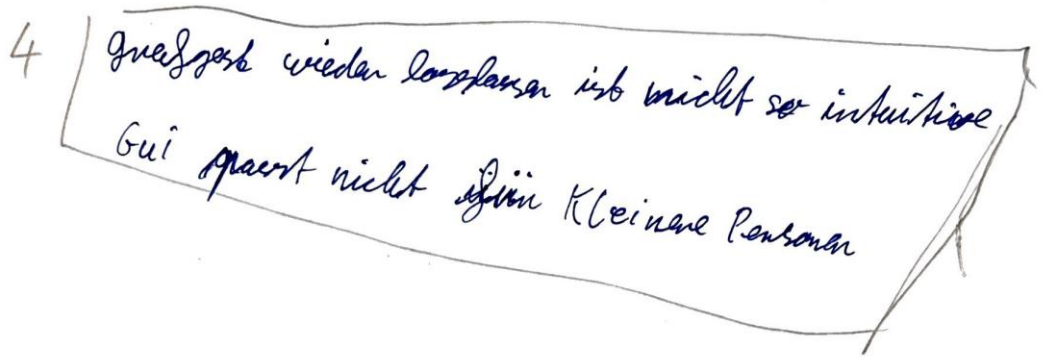
Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | | 0° | |
| 2 | 15° | | 0° | |
| 3 | -45° | | 45° | |
| 4 | 90° | | -45° | |
| 5 | Egal | | 90° | |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschietlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | Schneller ist angenehmer aber es fällt öfter aus der Hand | |
| Statisch 5 von 6 | | |
| Statisch 9 von 10 | | |
| Dynamisch 1s | | |
| Dynamisch 0.5s | | |

Sonstige Anmerkungen oder Feedback:


4
Grundgerüst wieder komplizierter ist nicht so intuitive
Gui spaart nicht Dinn Kleinere Personen

5-6

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener 2 Personen

Ort: BHT-Berlin, Raum B217a | Datum: 11.09.2026 | Proband: J.V | Alter: 18 | Größe: 1.70 cm

Unterarme sind durch Kleidung bedeckt: ~~ja~~/nein | Sonstige optische Merkmale des Probanden:

Uhr und Armbanden

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☒ | ☒ |
| Informations-Fenster erhalten | ☒ | ☒ |
| Prozess Live-Infos ein und aus blenden | ☒ | ☒ |
| alle Instrumente zurücksetzen | ☒ | ☒ |
| das Programm beenden | ☒ | ☒ |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?

9/10

Bewegungen werden schnell erkannt

Bilder ohne Hintergrund, Greifen

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | | 0° | |
| 2 | 15° | | 0° | |
| 3 | -45° | | 45° | |
| 4 | 90° | | -45° | |
| 5 | Egal | | 90° | |

Hand geschlossen halten und beim öffnen Instrument loslassen

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschiedlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | | |
| Statisch 5 von 6 | | |
| Statisch 9 von 10 | | |
| Dynamisch 1s | | |
| Dynamisch 0.5s | | |

Sonstige Anmerkungen oder Feedback: _____

5-6

Info Bild länger zeigen
anne X nicht lang genug

7

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 11.09.2026 | Proband: 151 | Alter: 29 | Größe: 158 cm

Unterarme sind durch Kleidung bedeckt: (ja)/nein | Sonstige optische Merkmale des Probanden: dunkle Klamotten

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☐ | ☐ |
| Informations-Fenster erhalten | ☒ | ☒ |
| Prozess Live-Infos ein und aus blenden | ☒ | ☒ |
| alle Instrumente zurücksetzen | ☒ | ☐ |
| das Programm beenden | ☒ | ☒ |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?
wegen der entfernung der Bildschirmer ein paar sachen spät gefunden. war ganz okay zu bedienen. an sich alles sehr cooles programm. änderungen: vielleicht am anfang das informations fenster zeigen. tutorial mäßig

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

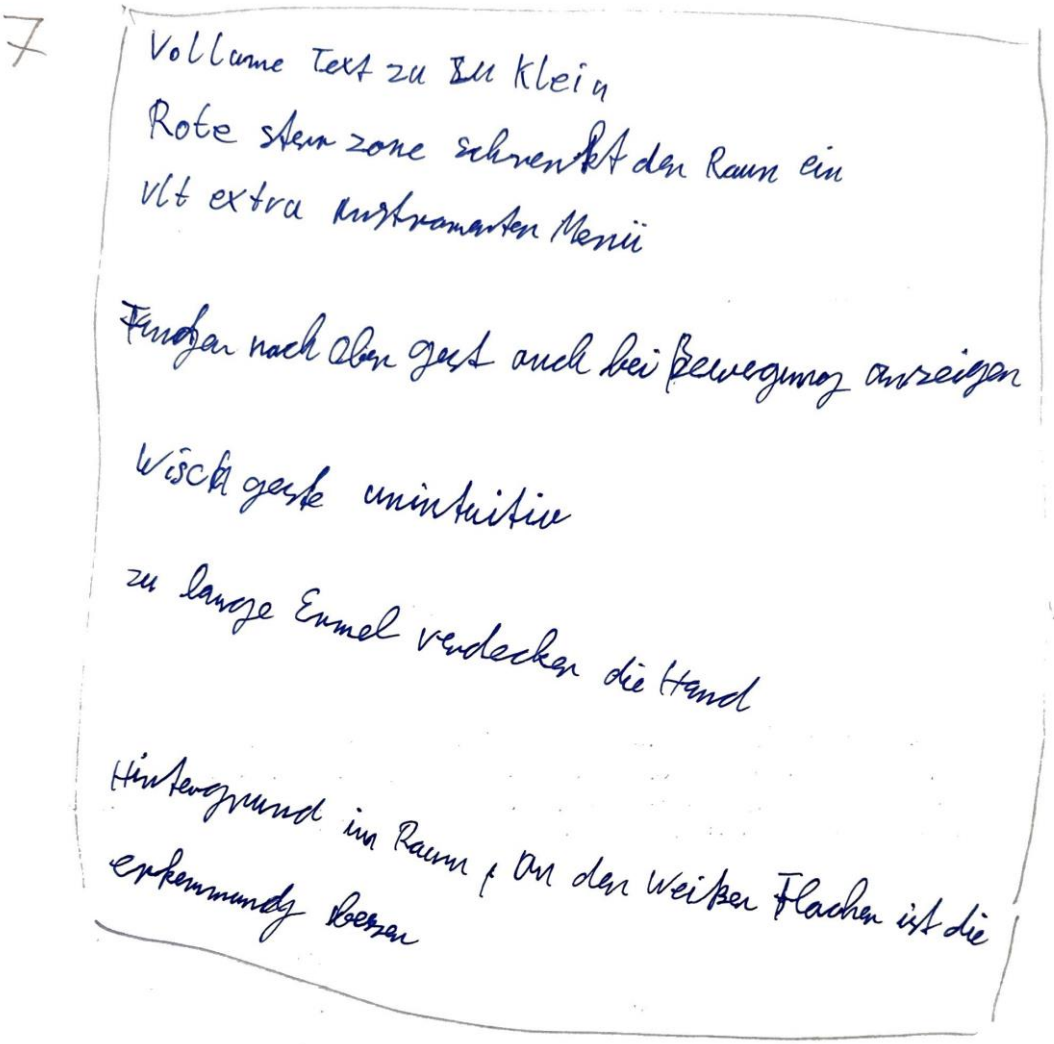
| unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | -15 | 0° | 45 |
| 2 | 15° | 15 | 0° | 30 |
| 3 | -45° | -43 | 45° | 60 |
| 4 | 90° | 90 | ~~-45°~~ | |
| 5 | Egal | | 90° | 75 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschietlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | schneller ist besser | |
| Statisch 5 von 6 | | |
| Statisch 9 von 10 | | |
| Dynamisch 1s | übersichtlich aber schön schnell | |
| Dynamisch 0.5s | lässt manchmal nicht los | |

Sonstige Anmerkungen oder Feedback:

7


Volume Text zu klein
Rote Sperrzone schränkt den Raum ein
Vlt extra Kurzwahlen Menü
Finger nach oben geht auch bei Bewegung anzeigen
Wisch geste unintuitiv
zu lange Enmel verdecken die Hand
Hintergrund im Raum, an den weißen Flächen ist die
erkennung besser

8

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 11.09.2026 | Proband: Joshi | Alter: 30 | Größe: 175 cm

Unterarme sind durch Kleidung bedeckt: ja / (nein) | Sonstige optische Merkmale des Probanden: (
Uhr, Armbelle, Ringe

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☑ | ☑ |
| Lautstärke ändern | ☑ | ☒ Nein |
| Informations-Fenster erhalten | ☑ | ☑ |
| Prozess Live-Infos ein und aus blenden | ☑ | ☑ |
| alle Instrumente zurücksetzen | ☑ | ☒ |
| das Programm beenden | ☑ | ☒ |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?
Ja es war sehr intuitiv und es hat auch intuitive Handgesten erkannt zum greifen.
Die Infotafel war super, hat die Lautstärkeregelung jedoch nicht erkannt.
Verbesserung: Lautstärke erklären.

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | 15 | 0° | 15 |
| 2 | 15° | 15 | 0° | 15 |
| 3 | -45° | -60 | 45° | 45° |
| 4 | 90° | 90 | ~~45°~~ | |
| 5 | Egal | | 90° | 75 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschiedlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| | Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|---|
| 1 | Statisch 2 von 3 | instabieler | |
| 2 | Statisch 5 von 6 | 4 war Besser | |
| 3 | Statisch 9 von 10 | | |
| 4 | Dynamisch 1s | besser als 5 | |
| 5 | Dynamisch 0.5s | besser als 2 | aber lässt manchmal nicht los |

Sonstige Anmerkungen oder Feedback:

8

Der Greifer einer Hand sollte einen Hand wechsel Initialisieren

Wischgeste wird nicht groß genug gemacht

Greifgeste wird oft anders ausgeführt und dann nicht erkannt

Menschen wackeln mit der Hand um auf diese aufmerksam zu machen, kleine Amplituden werden aber nicht als Bewegung erkannt

9

# Testprotokoll für eine Gestensteuerung

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 11.09.2026 | Proband: Sex | Alter: 24 | Größe: 190cm

Unterarme sind durch Kleidung bedeckt: ja / ~~nein~~ | Sonstige optische Merkmale des Probanden: Kleidung schwarz

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt | |
|---|---|---|---|
| Greifen und Verschieben | ☒ | ☒ | |
| Lautstärke ändern | ☒ | ☒ | |
| Informations-Fenster erhalten | ☒ | ☒ | |
| Prozess Live-Infos ein und aus blenden | ☒ | ☒ | |
| alle Instrumente zurücksetzen | ☒ | ☒ | manchmal aus Versehen |
| das Programm beenden | ☒ | ☒ | |

War es einfach und intuitiv zu bedienen? Was hat gefallen? Was sollte anders sein?

#1 Es war definitiv intuitiv, ulkt wenn jmd. aber nicht weiß das man greifen muss weiß es nicht wie man es bedient, hinweis das es ein drag and drop System ist. Swipen fürs reseten passiert unabsichtlich Lautstärkeregelung würde ich anders gestalten evtl. bzw verschieben ulkt nach oben.

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | -15 | 0° | 0 |
| 2 | 15° | 15 | 0° | 0 |
| 3 | -45° | -45 | 45° | 30 |
| 4 | 90° | nicht möglich | -45° | |
| 5 | Egal | | 90° | 750 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 5 mal mit unterschietlichen Greifer-Kennungsmethoden (ich stelle das zwischendurch um)

| | Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|---|
| 1 | Statisch 2 von 3 | Bevorzugt 2&3 | |
| 2 | Statisch 5 von 6 | | |
| 3 | Statisch 9 von 10 | | |
| 4 | Dynamisch 1s | nicht anders gleich wie 1 | |
| 4 | Dynamisch 0.5s | ~~Schlechter~~ aber die anderen | |

Sonstige Anmerkungen oder Feedback:

Beide Hände gleichzeitig nutzen können

9

Grabpunkt zu weit hinten im Arm
unbeabsichtigter Raum resetet
Verdeckte Hand wird trotzdem noch erkannt

*1 Lautstärke der einzelnen Module an dem platzort wo das Modul ist einstellen können.

Idee: Piano ist oben rechts, man hält mit einer Geste das Piano und es öffnet sich für das Piano die Lautstärke regelung. ~~Für~~

Position der Hst. im Bild kann als Ausrichtung interpretiert werden.

Hand von der Seite wird schlecht erkannt

10

# Testprotokoll für eine Gestensteuerung v2

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 15.09.2026 | Proband: Artur | Alter: 12 | Größe: 149 cm

Unterarme sind durch Kleidung bedeckt: ja (nein) | Sonstige optische Merkmale des Probanden: Uhr

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Für die Raumrichtung orientiere dich am echten Raum, nicht am Bildschirm. Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☒ | ☒ |
| ein Instrument entfernen | ☐ | ☐ |
| Informations-Fenster erhalten | ☒ | ☒ |
| Prozess Live-Infos ein und aus blenden | ☐ | ☐ |
| alle Instrumente zurücksetzen | ☒ | ☐ |
| das Programm beenden | ☒ | ☑ (aus versehen) |

War es einfach und intuitiv zu bedienen? Wie kann man die Benutzererfahrung verbessern?

Info Program ein bisschen unübersichtlich
Vielleicht eine Einführung am Anfang (damit es auch ohne erklärung verständlich wird)

## Konkrete Tests

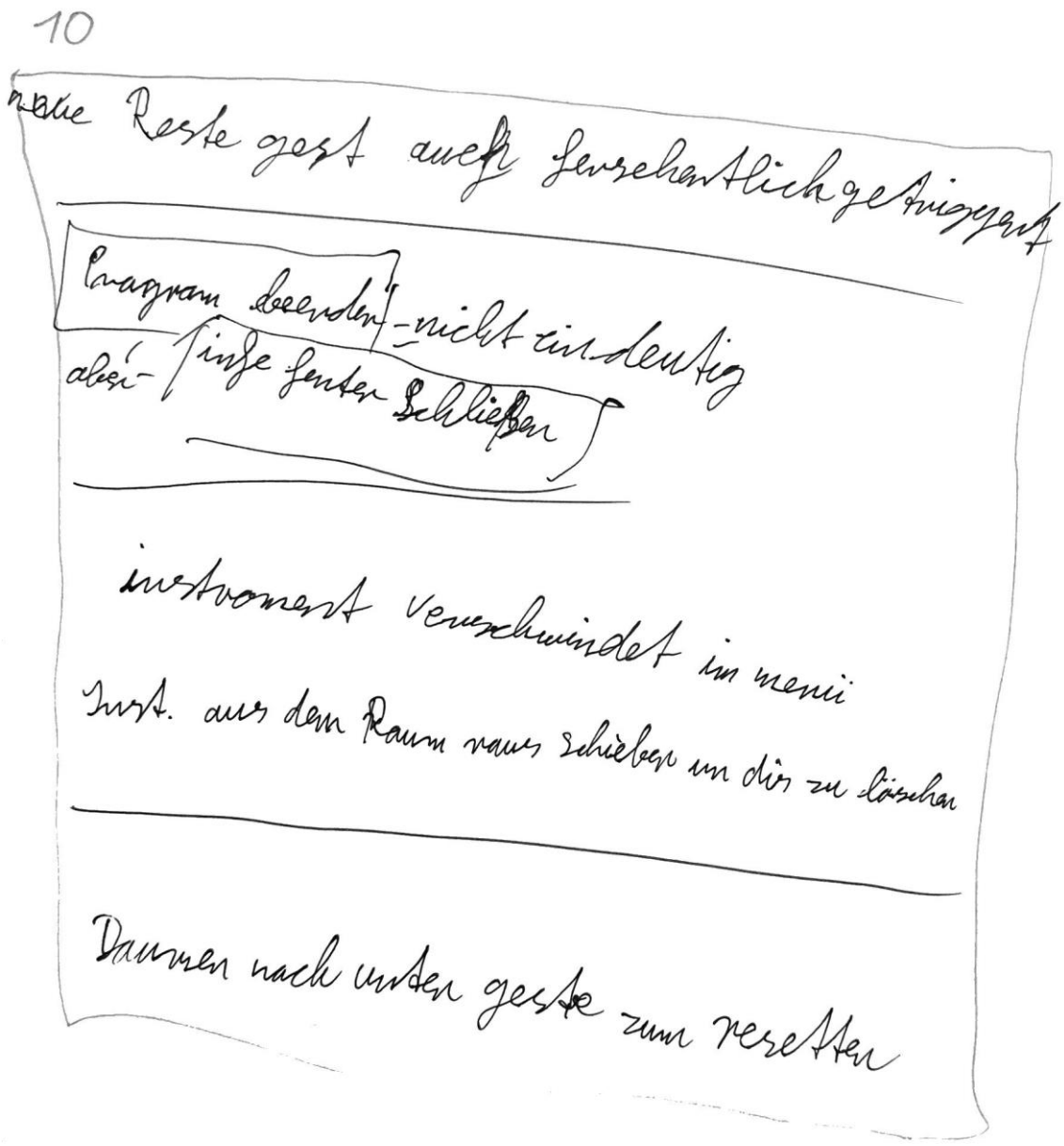
Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| Unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | 0 | 0° | 15 |
| 2 | 15° | 15 | 0° | 30 |
| 3 | -45° | -105/-60 | 45° | 60/60 |
| 4 | 90° | +105 | -45° | -15 |
| 5 | Egal | | 90° | 60 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 3 mal mit unterschietlichen Greifer-Erkennungsmethoden (ich stelle das zwischendurch um).

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | Keinen Unterschied gemacht | |
| Statisch 5 von 6 | | |
| Dynamisch 1s | wenigen Male Triggert nicht richtig | |

Sonstige Anmerkungen oder Feedback:


10
Programm beenden - nicht eindeutig
aber - linke Seiten Schließen
Instrument verschwindet im menü
Inst. aus dem Raum raus schieben um dies zu löschen
Daumen nach unten geste zum verletzen

11

# Testprotokoll für eine Gestensteuerung v2

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 15.09.2026 | Proband: Livia | Alter: 10 | Größe: 140 cm

Unterarme sind durch Kleidung bedeckt: ja / nein | Sonstige optische Merkmale des Probanden:

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Für die Raumrichtung orientiere dich am echten Raum, nicht am Bildschirm. Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☐ | ☐ |
| ein Instrument entfernen | ☒ | ☒ |
| Informations-Fenster erhalten | ☒ | ☒ |
| Prozess Live-Infos ein und aus blenden | ☒ | ☒ |
| alle Instrumente zurücksetzen | ☒ | ☒ |
| das Programm beenden | ☐ | ☐ |

War es einfach und intuitiv zu bedienen? Wie kann man die Benutzererfahrung verbessern?

Es wahr einfach zu bedienen aber man könte ein ~~gerä~~ geräusch machen das Unfair das macht: ~~Pfff~~ Pffff und wenn man es alles ~~löscht~~ hat macht es Bing.

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

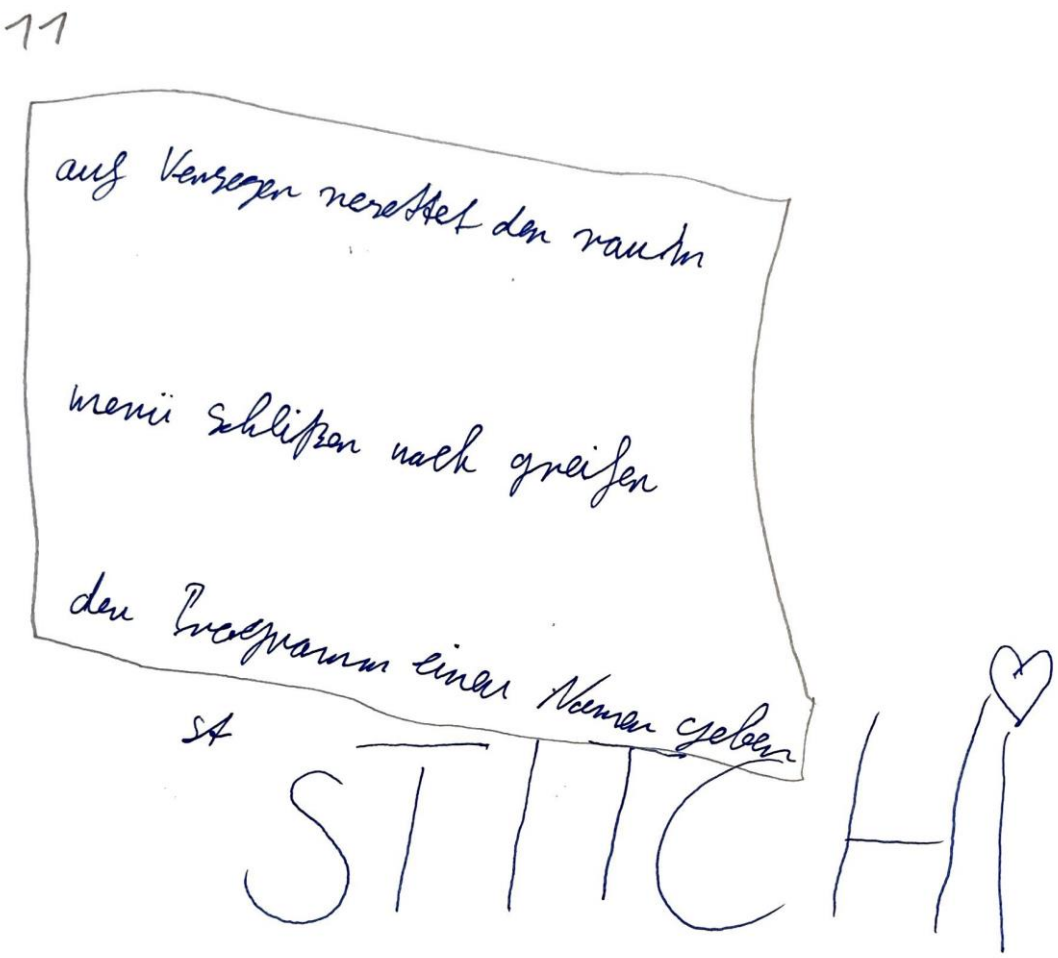
| Unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | 15 | 0° | 45 |
| 2 | 15° | 30 | 0° | 45 |
| 3 | -45° | -75 | 45° | 45 |
| 4 | 90° | 90 | -45° | 0 |
| 5 | Egal | | 90° | 75 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 3 mal mit unterschietlichen Greifer-Erkennungsmethoden (ich stelle das zwischendurch um).

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | Mittelmäßig | Lässt manchmal los |
| Statisch 5 von 6 | | |
| Dynamisch 1s | | |

Sonstige Anmerkungen oder Feedback:

11


auf Versagen reagiert der rauchen
mensch schlafen nach greifen
den Programm einen Namen geben
STICHI

12

# Testprotokoll für eine Gestensteuerung v2

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 15 .09.2026 | Proband: HB | Alter: 57 | Größe: 179,5 cm

Unterarme sind durch Kleidung bedeckt: (ja)/nein | Sonstige optische Merkmale des Probanden: Glatze & Bauch LOL

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Für die Raumrichtung orientiere dich am echten Raum, nicht am Bildschirm. Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☒ | ☒ |
| ein Instrument entfernen | ☒ | ☒ |
| Informations-Fenster erhalten | ☒ | ☒ |
| Prozess Live-Infos ein und aus blenden | ☒ | ☒ |
| alle Instrumente zurücksetzen | ☒ | ☐ |
| das Programm beenden | ☒ | ☐ |

War es einfach und intuitiv zu bedienen? Wie kann man die Benutzererfahrung verbessern?

Soundeffekte zur Unterstützung der Bedienerleichterung wären super z.B. Signal bei Aktivierung & Visuelle Positionierung der Elemente. ~~Es war~~ unübersichtlich, kleinere Symbole wären gut

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

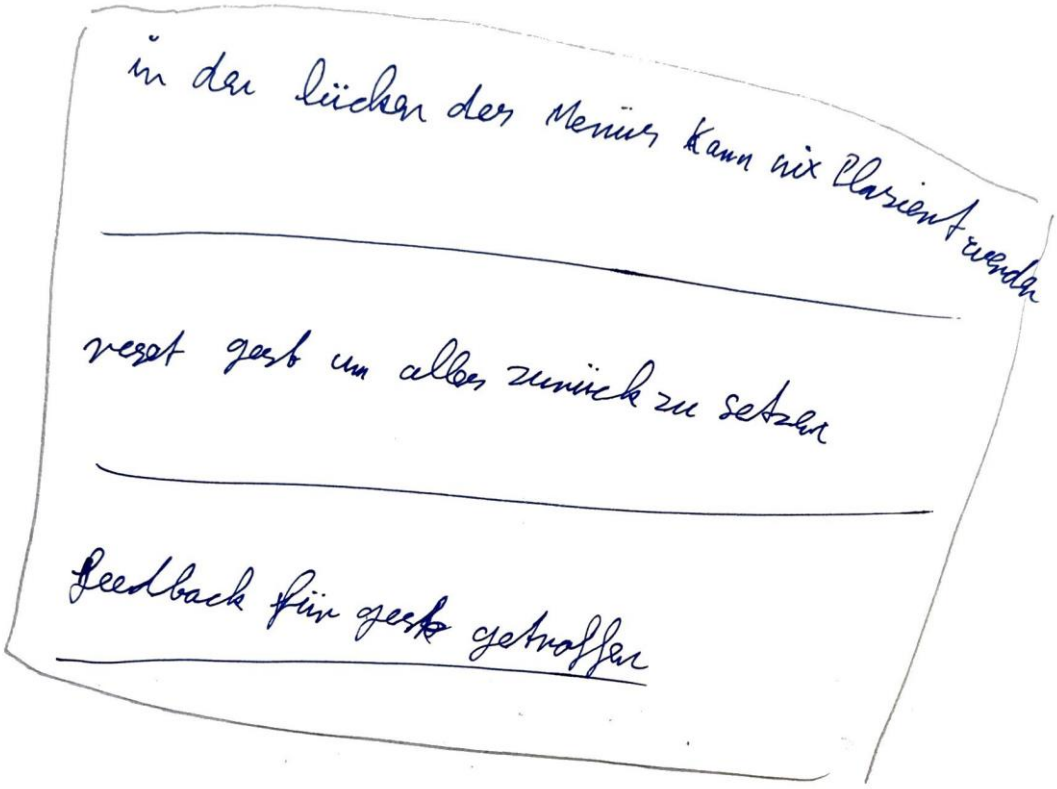
| Unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | | 0° | |
| 2 | 15° | | 0° | |
| 3 | -45° | | 45° | |
| 4 | 90° | | -45° | |
| 5 | Egal | | 90° | |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 3 mal mit unterschietlichen Greifer-Erkennungsmethoden (ich stelle das zwischendurch um).

| Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|
| Statisch 2 von 3 | | |
| Statisch 5 von 6 | | |
| Dynamisch 1s | | |

Sonstige Anmerkungen oder Feedback:

12


in den Lücken der Menüs kann nix Platziert werden
reset geht um alles zurück zu setzen
Feedback für gesetzt getroffen

13

# Testprotokoll für eine Gestensteuerung v2

Im Rahmen der Masterarbeit von Jonas Wegener

Ort: BHT-Berlin, Raum B217a | Datum: 18.09.2026 | Proband: AS | Alter: 58 | Größe: 165 cm

Unterarme sind durch Kleidung bedeckt: (ja) nein | Sonstige optische Merkmale des Probanden: Brille
Ärmel ziemlich lang

## Erster Eindruck

Du spielst den Dirigenten und kannst deine Musikinstrumente im Raum platzieren (die Soundwiedergabe ist noch nicht verbunden). Für die Raumrichtung orientiere dich am echten Raum, nicht am Bildschirm. Probiere die Steuerung aus und entdecke alle Funktionen!

Welche Funktionen und Steuergesten wurden gefunden und konnten genutzt werden:

| Funktion | gefunden | erfolgreich genutzt |
|---|---|---|
| Greifen und Verschieben | ☒ | ☒ |
| Lautstärke ändern | ☐ | ☒ |
| ein Instrument entfernen | ☐ | ☒ |
| Informations-Fenster erhalten | ☒ | ☒ |
| Prozess Live-Infos ein und aus blenden | ☒ | ☒ |
| alle Instrumente zurücksetzen | ☒ | ☒ |
| das Programm beenden | ☒ | ☒ |

War es einfach und intuitiv zu bedienen? Wie kann man die Benutzererfahrung verbessern?

Beenden durch andere Geste (verschränkte Arme beenden ansonsten auch, normale, häufige Geste)

## Konkrete Tests

Test 1: setze fünf Instrumente in bestimmte Raumrichtungen

| Unit | Azimut soll | Azimut erkannt | Elevation soll | Elevation erkannt |
|---|---|---|---|---|
| 1 | 0° | -15 | 0° | 15 |
| 2 | 15° | 0 | 0° | 0 |
| 3 | -45° | -60 | 45° | 45 |
| 4 | 90° | 90 | -45° | -15 |
| 5 | Egal | | 90° | 75 |

Test 2: setze fünf Instrumente in den Raum, ändere die Lautstärke von zweien davon, entferne eins wieder aus dem Raum. Mache das 3 mal mit unterschietlichen Greifer-Erkennungsmethoden (ich stelle das zwischendurch um).

| | Methode | Wertung im vergleich | Anmerkungen |
|---|---|---|---|
| 1 | Statisch 2 von 3 | Kein unter Schied zu 2 | |
| 2 | Statisch 5 von 6 | | |
| 3 | Dynamisch 1s | neu Ticken Besser als 2 | |

Sonstige Anmerkungen oder Feedback:

13

(Zweite) Hand weg wenn ~~nicht~~ Verdeckt

Menü wird die ganze Zeit getriggert

Finger der inlegerte muss sehr gerade sein

in der Selektion das Menü Presivsiener

Menü Klarer Trennen

Beenden geht Versehentlich getriggert



[a/e] → [a2/el]

15° in die Settungt

**Sonstiges**

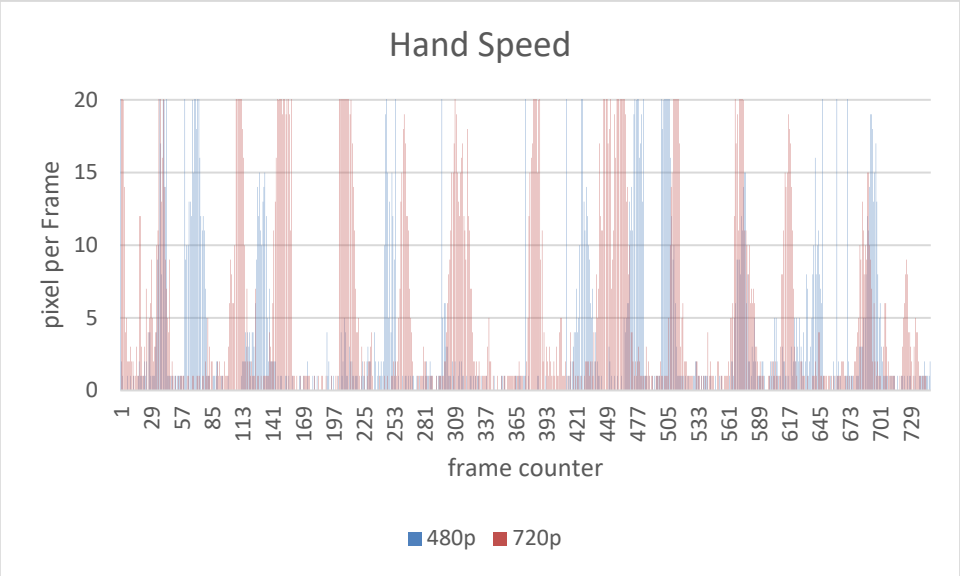



**Figure 65 - Hand Geschwindigkeit in zwei Testvideos**

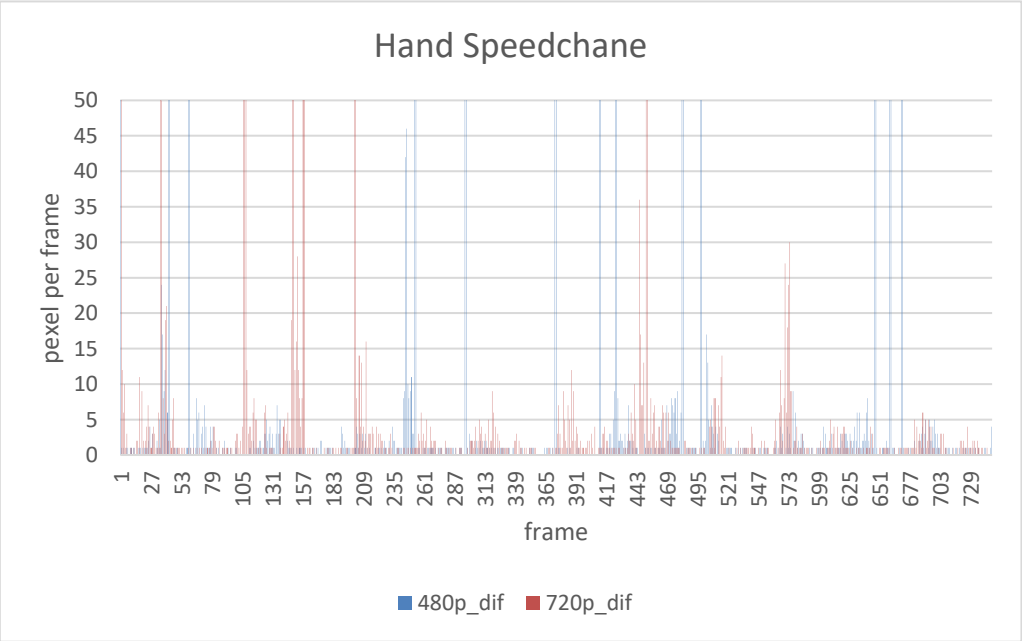
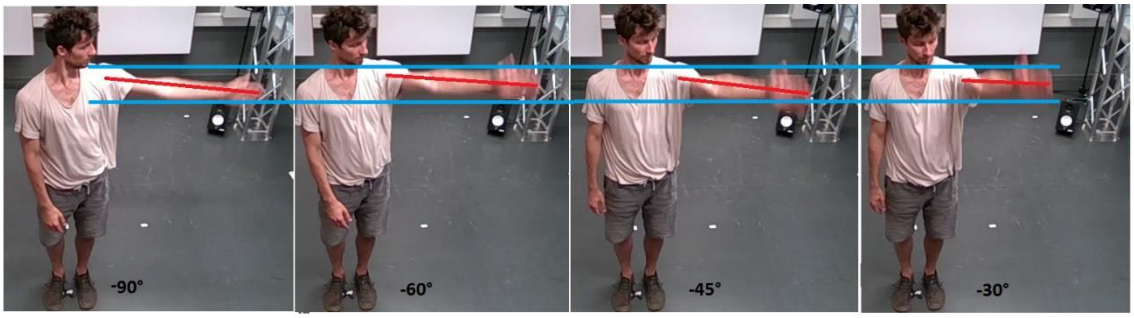
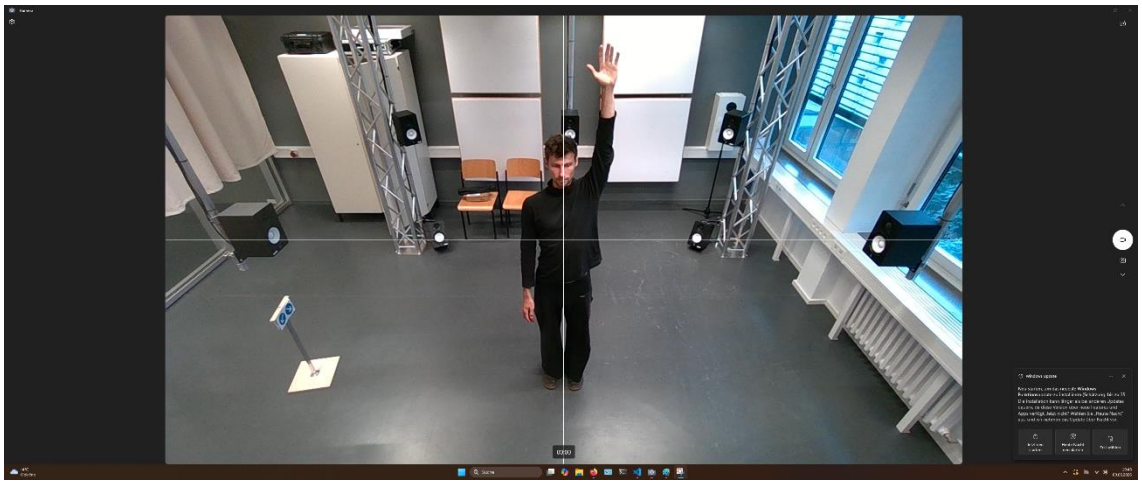
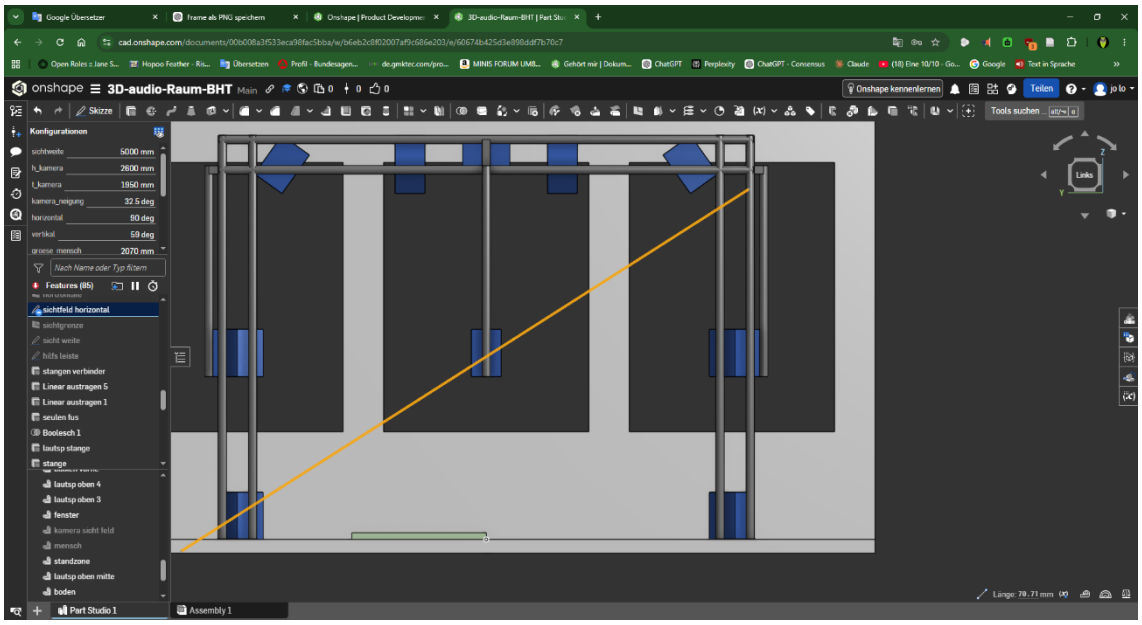



**Figure 66 – Änderung der Hand Geschwindigkeit in zwei Testvideos**


-90°
-60°
-45°
-30°




onshape
3D-audio-Raum-BHT
Main
Skizze
Konfigurationen
sichtfeld horizontal
Part Studio 1
Assembly 1


Hand and Pose Detection
FPS: 12.8
VIDEO PAUSED
Frame: 350/4092
closed
W/S=+/-1 frame | A/D=+/-200 frames
SPACE=Pause | ENTER=processing on/off